

Instituto Politécnico Nacional

Escuela Superior de Cómputo



Ingeniería en Sistemas Computacionales · Plan 2020

Unidad de aprendizaje: **Teoría de la Computación**

Práctica 1

Operaciones Básicas sobre Lenguajes

Alumno: Rico Espluga Juan Diego

Boleta: 2025630474

Grupo: 4CV4

Docente: Gabriel Hurtado Avilés

Fecha de entrega: 22 de septiembre de 2026

Índice

1. Objetivo **3**

2. Ejercicio 1. Entorno de trabajo: control de versiones y contenedores **3**

2.1. Parte A. Investigación . . . 3

2.1.1. Control de versiones . . . 3

2.1.2. Contenedores . . . 4

2.2. Parte B. Repositorio . . . 5

2.3. Parte C. Los tres entornos de Python . . . 6

2.3.1. Archivos versionados . . . 6

2.3.2. Justificación de las versiones . . . 7

2.3.3. Verificación . . . 7

3. Ejercicio 2. Investigación: qué es la Teoría de la Computación **8**

3.1. Objeto de estudio y preguntas centrales . . . 8

3.2. Orígenes . . . 9

3.3. Las tres ramas . . . 9

3.4. La tesis de Church-Turing . . . 10

3.5. Alfabetos, cadenas y lenguajes . . . 10

3.6. La jerarquía de Chomsky . . . 11

3.7. Autómatas finitos y expresiones regulares . . . 11

4. Ejercicio 3. Estado del arte: cinco artículos **12**

4.1. Texto 1. Gribkoff (2013) . . . 12

4.2. Texto 2. Luna-Benoso et al. (2022) . . . 14

4.3. Texto 3. Turing (1936), trabajo original . . . 15

4.4. Texto 4. Mavridou y Laszka (2018) . . . 16

4.5. Texto 5. Davis et al. (2019) . . . 16

4.6. Comparación 17

5. Ejercicio 4. Autómatas finitos en JFLAP 18

5.1. Método de diseño 18

5.2. Implementación y validación 19

5.3. Simulación paso a paso 19

5.4. Resumen de los autómatas 22

6. Ejercicio 5. Aplicación con interfaz gráfica 24

6.1. Arquitectura 24

6.2. Diseño de la interfaz 25

6.3. Operación 1. Prefijos, sufijos y subcadenas 26

6.4. Operación 2. Cerradura de Kleene y cerradura positiva 26

6.5. Pruebas automatizadas 27

7. Conclusiones 30

A. Anexo A. Fichas de diseño de los autómatas 34

1. Objetivo

Aplicar los conceptos de la Unidad Temática I al diseño de autómatas finitos en JFLAP y al desarrollo de una aplicación con interfaz gráfica que implemente las operaciones básicas sobre cadenas y lenguajes, y construir el entorno de trabajo, con control de versiones y contenedores, que se usará el resto del curso.

La práctica tiene cinco ejercicios: el primero prepara el entorno; el segundo y el tercero dan el sustento conceptual, y el cuarto y el quinto lo aplican. El código completo está en `src/` y `tests/`, los autómatas en `automatas/` y la versión en Markdown de este reporte en `docs/`.

2. Ejercicio 1. Entorno de trabajo: control de versiones y contenedores

2.1. Parte A. Investigación

2.1.1. Control de versiones

Un sistema de control de versiones registra cada estado de un conjunto de archivos junto con su autor, su fecha y una explicación del cambio, de modo que cualquier versión anterior puede recuperarse o compararse con la actual (Chacon & Straub, 2014, cap. 1). En un equipo resuelve tres problemas que aparecen en cuanto dos personas tocan el mismo proyecto: saber quién cambió qué y por qué, trabajar en paralelo sin sobrescribir el trabajo ajeno y volver atrás cuando un cambio introduce un error. Git es *distribuido*: cada copia del repositorio contiene la historia completa, por lo que se puede confirmar, crear ramas y consultar el historial sin conexión.

Git y GitHub. Git es el programa que administra la historia en la computadora de cada persona. GitHub es un servicio en la nube que aloja repositorios Git y añade lo que Git no trae: Pull Requests con revisión, integración continua (GitHub Actions), control de acceso e incidencias. Se puede usar Git sin GitHub, pero GitHub no tiene sentido sin Git.

Tabla 1. Términos de control de versiones.

Concepto	Definición	Ejemplo en esta práctica
Repositorio	Carpeta cuya historia completa guarda Git en el subdirectorio oculto `.git`.	`practica1-tc`, con `src/`, `tests/`, `docs/` y `entorno/`.
Confirmación (*commit*)	Fotografía de los archivos preparados, con un mensaje y un identificador único (hash SHA-1).	`Agrega el enumerador de` Σ^n `con límite de cadenas`.
Rama (*branch*)	Puntero móvil a una confirmación; permite desarrollar sin afectar la rama principal.	`feature/interfaz`, para la interfaz Flet.
Fusión (*merge*)	Integra en una rama los cambios de otra; si ambas avanzaron, crea una confirmación con dos padres.	`feature/interfaz` se fusionó en `main`.
Conflicto de fusión	Ocurre cuando las dos ramas modificaron las mismas líneas; Git no elige y marca el archivo con `<<<<<<<`, `=======` y `>>>>>>>` para que una persona decida.	Dos versiones distintas del mensaje de error del límite en `lenguajes.py`.
Pull Request	Solicitud en GitHub para fusionar una rama, con descripción, revisión, comentarios y resultado de las pruebas (GitHub Inc., 2026).	«Interfaz con menú lateral y ayuda por sección».
`.gitignore`	Lista de patrones de archivos que Git no debe rastrear, normalmente porque se regeneran.	`venv/`, `__pycache__/`, `*.pyc`, `salidas/`.
`README`	Documento de entrada del repositorio; GitHub lo muestra en la página principal.	Portada con datos del alumno, índice e instrucciones.

2.1.2. Contenedores

Un contenedor es un proceso aislado del sistema operativo anfitrión que ve su propio sistema de archivos, su propia red y sus propios procesos, pero **comparte el núcleo** del anfitrión. Una máquina virtual, en cambio, emula hardware completo y arranca su propio sistema operativo sobre un hipervisor. De ahí las tres diferencias que pide la práctica (Felter et al., 2015):

- **Arranque.** Un contenedor inicia en fracciones de segundo, porque sólo lanza un proceso; una máquina virtual tarda lo que tarda en arrancar un sistema operativo.
- **Tamaño.** Una imagen de contenedor contiene únicamente la aplicación y sus bibliotecas (decenas o cientos de MB); una máquina virtual incluye un sistema operativo completo (varios GB).
- **Aislamiento.** La máquina virtual aísla más, porque cada una tiene su propio núcleo; en el contenedor el aislamiento se logra con *namespaces* y *cgroups* del mismo núcleo, que es una frontera

más delgada.

Los cuatro términos de Docker que se usan en el ejercicio son (Docker Inc., 2026b): la **imagen**, plantilla de sólo lectura formada por capas (por ejemplo, `python:3.12-slim` más las dependencias); el **contenedor**, una instancia en ejecución de una imagen, con una capa escribible propia; el **volumen**, almacenamiento que vive fuera del ciclo de vida del contenedor y que aquí monta la carpeta del proyecto en `/app` para que los cambios al código se vean sin reconstruir; y el **puerto publicado**, que conecta un puerto del equipo con uno del contenedor (`8550:8550`), necesario para abrir la interfaz desde el navegador.

Entorno virtual de Python. Es una carpeta con su propio `site-packages` y enlaces al intérprete que lo creó; aísla las *bibliotecas* de cada proyecto, no el intérprete (Python Software Foundation, 2026b). Por eso un entorno virtual no cambia la versión de Python: `python -m venv` con Python 3.12 siempre produce un entorno 3.12. Para probar tres versiones hacen falta tres intérpretes, y cada contenedor aporta el suyo.

Por qué fijar la imagen. `python:latest` apunta a la versión más reciente en el momento de construir; un día es 3.13 y al siguiente 3.14, con lo que la misma receta produce entornos distintos y un fallo puede aparecer sin que nadie haya cambiado el código. `python:3.12-slim` fija la versión menor (y la variante ligera), de modo que la construcción es reproducible; las correcciones de la versión de parche siguen llegando.

2.2. Parte B. Repositorio

El repositorio `practica1-tc` contiene `README.md` (portada con nombre, boleta, grupo, carrera, unidad de aprendizaje, profesor y fecha, índice enlazado e instrucciones), un `.gitignore` que excluye `venv/`, `.venv/`, `__pycache__/`, `*.pyc`, `.pytest_cache/`, `salidas/` y los auxiliares de LaTeX, y que **no** excluye los `.jff`, y la historia de confirmaciones. Se hicieron más de cinco confirmaciones con mensajes en imperativo que dicen qué cambia; la interfaz se desarrolló en la rama `feature/interfaz` y entró a `main` mediante un Pull Request descrito y fusionado.

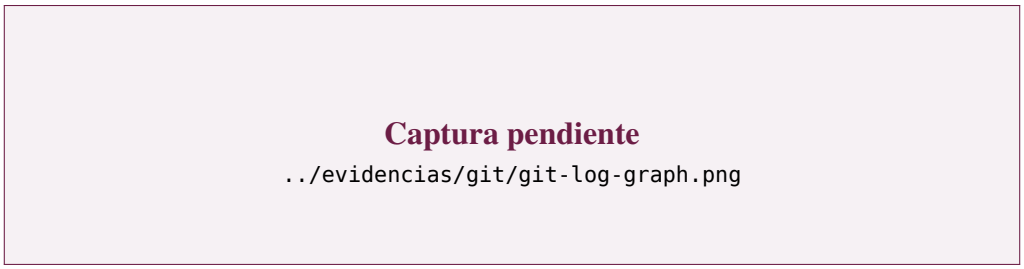


Figura 1. Salida de `git log --oneline --graph --all`.

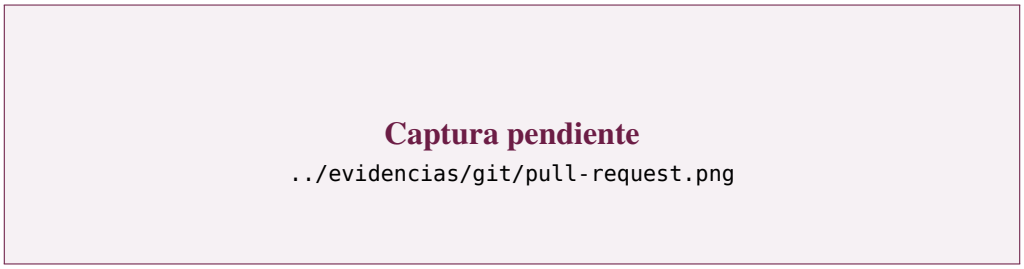


Figura 2. Pull Request de `feature/interfaz` fusionado en `main`.

2.3. Parte C. Los tres entornos de Python

2.3.1. Archivos versionados

El `Dockerfile` (carpeta `entorno/`) es único y recibe la versión como argumento. La tabla explica cada instrucción.

Tabla 2. Instrucciones del `Dockerfile`.

Instrucción	Función
`ARG PYTHON_VERSION=3.12`	Declara el argumento de construcción; `compose.yml` le da 3.11, 3.12 o 3.13. El valor por omisión permite construir la imagen sin Compose.
`FROM python:${PYTHON_VERSION}-slim`	Imagen base oficial en su variante ligera, con la versión elegida.
`RUN python -m venv /opt/venv`	Crea el entorno virtual fuera de `/app`, porque el volumen de Compose reemplaza `/app` con la carpeta del equipo y borraría un entorno creado ahí.
`ENV PATH="/opt/venv/bin:$PATH"`	Antepone el `bin/` del entorno al `PATH`; equivale a activarlo para todas las órdenes siguientes.
`WORKDIR /app`	Carpeta de trabajo dentro del contenedor.
`COPY requirements.txt .`	Copia sólo la lista de dependencias, para que la capa siguiente se reutilice mientras esa lista no cambie.

Instrucción	Función
`RUN pip install … -r requirements.txt`	Actualiza pip e instala las versiones fijadas; `--no-cache-dir` evita guardar los paquetes descargados en la imagen.
`COPY . .`	Copia el proyecto (útil si se ejecuta sin volumen).
`ENV FLET_FORCE_WEB_SERVER=true` y `FLET_SERVER_PORT=8550`	Sin escritorio, Flet se sirve como aplicación web en el puerto 8550.
`CMD ["python", "src/app.py"]`	Orden por omisión: abrir la interfaz.

`compose.yml` declara los servicios `py311`, `py312` y `py313` con el mismo `Dockerfile` (Docker Inc., 2026a) y distinto `PYTHON_VERSION`; los tres montan la carpeta del proyecto en `/app`, `py311` y `py313` ejecutan `pytest -q` por omisión y sólo `py312` publica el puerto 8550 para la interfaz. `requirements.txt` fija `flet[web]==0.86.5`, `fastapi==0.115.12` y `pytest==8.3.4`. FastAPI se fija porque `flet-web` lo requiere sin acotar su versión y las publicaciones de FastAPI posteriores a la 0.115 eliminaron un parámetro que `flet-web` 0.86.5 todavía usa; sin esa línea la interfaz no arranca en un entorno nuevo.

2.3.2. Justificación de las versiones

Tabla 3. Elección de versiones.

Servicio	Versión	Motivo
`py311`	3.11	Flet 0.86.5 exige Python $\geq$ 3.10 (Appveyor Systems Inc., 2026), pero 3.10 llegó al fin de su soporte en octubre de 2026 (Python Software Foundation, 2026a); 3.11 es la más antigua que conserva parches de seguridad (hasta octubre de 2027) y que Flet admite.
`py312`	3.12	Versión de referencia del curso; recibe parches de seguridad hasta octubre de 2028. Sirve la interfaz.
`py313`	3.13	Versión estable reciente (soporte hasta octubre de 2029); detecta dependencias de comportamientos que cambiaron.

2.3.3. Verificación

Cada contenedor se construye con `docker compose build` y se comprueba con `docker compose run --rm py311 python --version` y sus equivalentes; las salidas son `Python 3.11.x`, `Python`

`3.12.x` y `Python 3.13.x`.

Captura pendiente
`../evidencias/docker/python-version.png`

Figura 3. `python --version` en los contenedores `py311`, `py312` y `py313`.

3. Ejercicio 2. Investigación: qué es la Teoría de la Computación

3.1. Objeto de estudio y preguntas centrales

La Teoría de la Computación estudia, con modelos matemáticos, qué pueden calcular las máquinas y con qué recursos. Sipser (2013, cap. 0) la organiza alrededor de una sola pregunta: *cuáles son las capacidades y las limitaciones fundamentales de las computadoras*, y la descompone en tres áreas que responden a versiones cada vez más finas de esa pregunta. Hopcroft et al. (2007, cap. 1) parten de un punto distinto: definen la teoría de autómatas como el estudio de dispositivos de cómputo abstractos, y llegan a la computabilidad y a la complejidad como consecuencias de preguntar qué lenguajes reconoce cada dispositivo. Linz (2017, cap. 1) adopta un enfoque todavía más concreto: los lenguajes formales, las gramáticas y los autómatas son el lenguaje común con el que se describen y comparan los modelos.

Las definiciones coinciden en el objeto (modelos abstractos de cómputo) y en el método (definición formal y demostración), pero difieren en el orden y en el énfasis. Sipser parte de la pregunta filosófica y presenta los modelos como herramientas para contestarla; Hopcroft, Motwani y Ullman, y Linz, parten de los modelos y dejan que las preguntas surjan de sus límites. Para un curso que empieza por los autómatas, la segunda forma es la más práctica; la primera explica por qué vale la pena el esfuerzo.

Las preguntas que plantea la teoría pueden resumirse en tres: ¿qué problemas se pueden resolver con un algoritmo?, ¿cuáles se pueden resolver eficientemente? y ¿qué modelo mínimo basta para

un problema dado?

3.2. Orígenes

A principios del siglo XX la lógica matemática atravesaba una crisis: la paradoja de Russell (el conjunto de los conjuntos que no se contienen a sí mismos) mostró que la teoría intuitiva de conjuntos era contradictoria, y otras paradojas de autorreferencia, como la del mentiroso, apuntaban al mismo problema (Kleene, 1952, cap. 3). David Hilbert propuso entonces un programa para dar a la matemática una base segura: formalizarla por completo en sistemas axiomáticos y demostrar, con medios finitos, que esos sistemas son consistentes y completos.

Una pieza del programa era el *Entscheidungsproblem*, planteado por Hilbert y Ackermann en 1928: encontrar un procedimiento mecánico que, dada una fórmula de la lógica de primer orden, decida en un número finito de pasos si es válida. Para responder negativamente había que decir con precisión qué es un «procedimiento mecánico», y esa necesidad dio origen a la teoría. Gödel demostró en 1931 que todo sistema consistente suficientemente expresivo contiene enunciados verdaderos que no puede demostrar, y en 1936 Church (1936) y Turing (1936), de manera independiente, mostraron que el *Entscheidungsproblem* no tiene solución: Church con el cálculo λ y Turing con sus máquinas.

3.3. Las tres ramas

Tabla 4. Las tres ramas de la teoría (Sipser, 2013, cap. 0).

Rama	Pregunta	Ejemplo de resultado
Teoría de autómatas y lenguajes formales	¿Qué puede reconocer o generar cada modelo de cómputo restringido?	Los autómatas finitos reconocen exactamente los lenguajes regulares; $\{a^n b^n\}$ no es regular.
Computabilidad	¿Qué problemas se pueden resolver con un algoritmo?	El problema de la parada es indecidible.
Complejidad	¿Qué problemas se pueden resolver con recursos razonables de tiempo y memoria?	La pregunta P = NP sigue abierta.

3.4. La tesis de Church-Turing

La tesis afirma que **toda función que puede calcularse mediante un procedimiento efectivo, en el sentido intuitivo, puede calcularse con una máquina de Turing**. En la década de 1930 se propusieron varios modelos independientes: las funciones recursivas generales de Herbrand y Gödel, el cálculo λ de Church, las máquinas de Turing y los sistemas de Post; todos resultaron equivalentes, en el sentido de que calculan exactamente las mismas funciones (Kleene, 1952, cap. 13; Sipser, 2013, sec. 3.3). Turing lo demostró para su modelo y el de Church en el apéndice de su artículo.

Se llama *tesis* y no teorema porque uno de sus dos lados, «procedimiento efectivo», es una noción intuitiva y no un objeto matemático; no puede demostrarse una equivalencia entre algo definido y algo que no lo está. Lo que sí está demostrado es la equivalencia entre los modelos formales, y esa coincidencia es la mejor evidencia a favor de la tesis.

3.5. Alfabetos, cadenas y lenguajes

Un **alfabeto** Σ es un conjunto finito y no vacío de símbolos. Una **cadena** (o palabra) sobre Σ es una sucesión finita de símbolos de Σ; su longitud $|\omega|$ es el número de símbolos, y la **cadena vacía** λ es la única de longitud 0. Σ^n es el conjunto de las cadenas de longitud n, $\Sigma^* = \bigcup_{n \geq 0} \Sigma^n$ el de todas las cadenas y $\Sigma^+ = \bigcup_{n \geq 1} \Sigma^n$ el de las no vacías. Un **lenguaje** sobre Σ es cualquier subconjunto de Σ^* (Hopcroft et al., 2007, sec. 1.5).

Operaciones sobre cadenas. La *concatenación* $\omega\pi$ escribe π a continuación de ω, es asociativa y tiene a λ como neutro; $|\omega\pi| = |\omega| + |\pi|$. La *potencia* se define por recursión: $\omega^0 = \lambda$ y $\omega^{n+1} = \omega^n\omega$. La *reflexión* ω^R invierte el orden: $\lambda^R = \lambda$ y $(a\omega)^R = \omega^R a$, con $(\omega\pi)^R = \pi^R\omega^R$.

Operaciones sobre lenguajes. Como conjuntos, los lenguajes admiten *unión*, *intersección* y *diferencia*. La *concatenación* es $L_1L_2 = \{xy : x \in L_1, y \in L_2\}$; la *potencia*, $L^0 = \{\lambda\}$ y $L^{n+1} = L^nL$; la *reflexión*, $L^R = \{\omega^R : \omega \in L\}$. La **cerradura de Kleene** es $L^* = \bigcup_{n \geq 0} L^n$ y la **cerradura positiva**, $L^+ = \bigcup_{n \geq 1} L^n$ (Linz, 2017, sec. 1.2).

Por qué $\Sigma^0 = \{\lambda\}$. Σ^0 es el conjunto de las cadenas de longitud 0, y hay exactamente una: λ.

Además, $\{\lambda\}$ es el neutro de la concatenación de lenguajes ($L\{\lambda\} = L$), igual que λ lo es de la de cadenas; si Σ^0 fuera $\varnothing$, la identidad $\Sigma^{m+n} = \Sigma^m\Sigma^n$ fallaría para $m = 0$, porque $\varnothing L = \varnothing$. **$\Sigma^*$ frente a Σ^+.** La única diferencia es λ: $\Sigma^* = \Sigma^+ \cup \{\lambda\}$ y $\Sigma^+ = \Sigma\Sigma^*$. *En general* $L^+ = L$ sólo si $\lambda \in L$.

3.6. La jerarquía de Chomsky

Chomsky (1956) clasificó las gramáticas según la forma de sus producciones; cada tipo genera una clase de lenguajes que contiene a la siguiente, y a cada clase corresponde una máquina que la reconoce.

Tabla 5. Jerarquía de Chomsky (Hopcroft et al., 2007; Lewis & Papadimitriou, 1998; Sipser, 2013).

Tipo	Lenguajes	Forma de las producciones	Máquina reconocedora
0	Recursivamente enumerables	$\alpha \to \beta$, con α no vacía (sin restricción)	Máquina de Turing
1	Sensibles al contexto	$\alpha A \beta \to \alpha \gamma \beta$, con γ no vacía	Autómata linealmente acotado
2	Independientes del contexto	$A \to \gamma$	Autómata de pila (no determinista)
3	Regulares	$A \to aB$ o $A \to a$ (lineal por la derecha)	Autómata finito

Las inclusiones son estrictas: $\{a^n b^n : n \geq 0\}$ es independiente del contexto pero no regular, $\{a^n b^n c^n : n \geq 0\}$ es sensible al contexto pero no independiente del contexto, y el lenguaje de la parada es recursivamente enumerable pero no recursivo.

3.7. Autómatas finitos y expresiones regulares

Un **autómata finito determinista** es una quíntupla $M = (Q, \Sigma, \delta, q_0, F)$, con Q un conjunto finito de estados, Σ el alfabeto, $\delta: Q \times \Sigma \to Q$ la función de transición, $q_0 \in Q$ el estado inicial y $F \subseteq Q$ los de aceptación. δ se extiende a cadenas con $\hat{\delta}(q, \lambda) = q$ y $\hat{\delta}(q, \omega a) = \delta(\hat{\delta}(q, \omega), a)$; M acepta ω si $\hat{\delta}(q_0, \omega) \in F$, y $L(M)$ es el conjunto de las cadenas aceptadas (Hopcroft et al., 2007, sec. 2.2). Un **autómata finito no determinista** sólo cambia la función de transición: $\delta: Q \times \Sigma \to \mathcal{P}(Q)$ devuelve un conjunto de estados, posiblemente vacío, y la cadena se acepta si *alguna* de las

sucesiones posibles de estados termina en F.

Los dos modelos son **equivalentes**: para todo AFN existe un AFD que reconoce el mismo lenguaje (Kozen, 1997). Rabin y Scott (1959) lo demostraron con la *construcción de subconjuntos*, en la que cada estado del AFD es un conjunto de estados del AFN; el precio es que el AFD puede tener hasta 2^n estados. Kleene (1956) probó además que los lenguajes que reconocen los autómatas finitos son exactamente los que describen las **expresiones regulares**, construidas a partir de $\varnothing$, λ y los símbolos con unión, concatenación y estrella.

Tres problemas reales en los que se aplican expresiones regulares:

1. **Análisis léxico.** El primer paso de un compilador divide el código en fichas (identificadores, números, palabras reservadas); cada clase de ficha se describe con una expresión, como `[A-Za-z_][A-Za-z0-9_]*` para los identificadores, y el analizador se construye como un AFD.
2. **Validación de datos.** Un formulario rechaza un correo o un código postal mal escritos comparando el texto con una expresión antes de enviarlo.
3. **Búsqueda en texto y registros.** Herramientas como `grep` encuentran todas las líneas de un archivo de bitácora que contienen, por ejemplo, una dirección IP o un código de error, sin recorrer el archivo más de una vez.

Las expresiones de los lenguajes de programación añaden operadores que no son regulares, como las referencias hacia atrás, y sus motores pueden tardar un tiempo exponencial; el artículo de Davis et al. (2019) del Ejercicio 3 estudia esas diferencias.

4. Ejercicio 3. Estado del arte: cinco artículos

Cada texto se resume en una ficha de seis puntos; para los tres asignados se responden además las preguntas del enunciado.

4.1. Texto 1. Gribkoff (2013)

1. **Cita.** Gribkoff, E. (2013). *Applications of deterministic finite automata* [Documento de curso, ECS 120]. University of California, Davis. https://www.cs.ucdavis.edu/~rogaway/classes/120/

spring13/eric-dfa.pdf

2. **Problema.** Mostrar a estudiantes de un primer curso de teoría que el AFD no es sólo un objeto abstracto, sino un modelo que aparece en sistemas cotidianos.
3. **Método.** Tras repasar la definición formal del AFD y la de la máquina de Mealy, analiza cuatro ejemplos: una máquina expendedora, el comportamiento de los fantasmas de Pac-Man, el diagrama de estados de TCP y el autocompletado de Apache Lucene.
4. **Resultado.** Cada sistema se describe con pocos estados; en Lucene, el transductor comparte prefijos y sufijos entre palabras y así reduce la memoria del índice.
5. **Tema de la Unidad I.** Autómatas finitos: definición formal del AFD y aplicaciones.
6. **Aportación.** Ofrece ejemplos de modelado que sirven de punto de partida para diseñar autómatas propios, como el del fantasma de Pac-Man que se amplía en la Práctica 2.

AFD y máquina de Mealy. El AFD sólo produce un veredicto al final de la lectura: acepta o rechaza. La máquina de Mealy agrega un alfabeto de salida y una función que emite un símbolo en *cada* transición, de modo que convierte una cadena de entrada en una de salida. El autocompletado de Lucene necesita esa salida: no basta con saber que un prefijo pertenece al diccionario; hay que recuperar información de cada palabra completada (en el ejemplo, un número que se obtiene acumulando las salidas del camino).

Máquina expendedora con F = $\varnothing$. Si ningún estado es de aceptación, $\delta(q_0, \omega) \notin F$ para toda ω, así que el lenguaje del autómata es $\varnothing$: ninguna cadena se acepta. El modelo sigue siendo útil para describir cómo cambia el estado de la máquina con cada moneda, pero la noción de «cadena aceptada» deja de aportar información; se está usando el autómata como un sistema de transiciones y no como un reconocedor.

Afirmación por verificar. El texto afirma que, gracias al transductor, Lucene mantiene en memoria el índice completo del autocompletado y responde mucho más rápido. No da cifras, versiones ni una referencia; antes de citarla debería confirmarse en una publicación arbitrada o en la documentación oficial de Lucene.

4.2. Texto 2. Luna-Benoso et al. (2022)

1. **Cita.** Luna-Benoso, B., Martínez-Perales, J. C., Cortés-Galicia, J., Flores-Carapia, R., y Silva-García, V. M. (2022). Melanoma detection in dermoscopic images using a cellular automata classifier. *Computers*, *11*(1), 8. https://doi.org/10.3390/computers11010008
2. **Problema.** Distinguir el melanoma de lesiones benignas en imágenes dermatoscópicas como apoyo al diagnóstico temprano.
3. **Método.** Segmentan la lesión (filtro de mediana, umbral de Otsu y operaciones morfológicas sobre la vecindad de Moore), extraen 11 descriptores estadísticos de textura y clasifican con un autómata celular asociativo de estados binarios.
4. **Resultado.** Con 190 imágenes de la base PH2 obtienen exactitud de 0.978, sensibilidad de 0.944, especificidad de 0.987 y área bajo la curva ROC de 0.965.
5. **Tema de la Unidad I.** Modelos de cómputo; diferencia entre autómata finito y autómata celular.
6. **Aportación.** Un modelo de transiciones locales, barato de calcular, compite con clasificadores mucho más costosos en una aplicación médica.

Autómata celular y autómata finito. Se parecen en que ambos son discretos y evolucionan con una función de transición sobre un conjunto finito de estados. Difieren en dónde vive el estado y en qué se lee. En el autómata celular el **estado** es de cada celda de una retícula (en el artículo, 0 o 1); el **vecindario** es el conjunto de celdas que influyen en una celda (la vecindad de Moore: sus ocho vecinas); y la **regla local** calcula el nuevo estado de cada celda a partir de su vecindario, para todas las celdas a la vez. Un autómata finito tiene un único estado global que cambia al leer un símbolo de una cadena, uno por paso, y termina aceptando o rechazando.

Problema y elección del modelo. El melanoma es el cáncer de piel más agresivo y su pronóstico depende de detectarlo temprano. Los autores eligen autómatas celulares porque se implementan con facilidad y requieren pocos recursos, a diferencia de redes profundas como AlexNet o VGG16.

Datos y métricas. Usan la base PH2 del Hospital Pedro Hispano: 200 imágenes RGB de 768 $\times$ 560 píxeles, de las que descartan 10 por falta de nitidez. Reportan exactitud, sensibilidad, especificidad y AUC, con los valores del punto 4.

4.3. Texto 3. Turing (1936), trabajo original

1. **Cita.** Turing, A. M. (1936).[1] On computable numbers, with an application to the Entscheidungsproblem. *Proceedings of the London Mathematical Society*, *s2-42*(1), 230–265. https://doi.org/10.1112/plms/s2-42.1.230
2. **Problema.** Precisar qué significa que un número sea calculable por un procedimiento mecánico y, con esa definición, responder el *Entscheidungsproblem*.
3. **Método.** Introduce la *máquina automática*: una cinta dividida en casillas, una cabeza que lee y escribe un símbolo a la vez y un número finito de configuraciones. Un número es computable si una máquina lo escribe cifra por cifra. Construye una máquina *universal* que imita a cualquier otra a partir de su descripción.
4. **Resultado.** Ninguna máquina decide si una máquina arbitraria escribe infinitas cifras; de ahí se sigue que no existe un procedimiento general de decisión para la lógica de primer orden. En un apéndice muestra que su noción coincide con la λ-definibilidad de Church.
5. **Tema de la Unidad I.** Orígenes de la teoría, computabilidad y tesis de Church-Turing.
6. **Aportación.** Es el origen del modelo que ocupa el nivel más alto de la jerarquía de Chomsky estudiada en el curso.

Resultado y modelo. Se demuestra la indecidibilidad del *Entscheidungsproblem* y se introduce la máquina de Turing, incluida la universal.

Tesis de Church-Turing. Toda función efectivamente calculable, en el sentido intuitivo, es calculable por una máquina de Turing. Es una tesis porque «efectivamente calculable» no es un concepto formal; sólo se puede argumentar a su favor, como hace Turing al analizar lo que una persona hace al calcular con lápiz y papel, y con la equivalencia entre modelos independientes.

Dificultades de lectura. La notación es la mayor barrera: letras góticas para las configuraciones, máquinas descritas como tablas de «configuración, símbolo, operaciones, configuración final» y «tablas esqueleto» que funcionan como subrutinas. Se resolvió leyendo en paralelo el capítulo 3 de

[1] El artículo se publicó en las partes 3 y 4 del volumen 42 (serie 2), aparecidas a finales de 1936, pero el volumen se cerró en 1937. El registro del DOI usa la fecha del volumen y la literatura, la de las partes.

Sipser (2013) y reescribiendo cada tabla como δ(q, a) = (p, b, D), la notación del curso.

4.4. Texto 4. Mavridou y Laszka (2018)

1. **Cita.** Mavridou, A., y Laszka, A. (2018). Designing secure Ethereum smart contracts: A finite state machine based approach. En S. Meiklejohn y K. Sako (Eds.), *Financial Cryptography and Data Security* (pp. 523–540). Springer. https://doi.org/10.1007/978-3-662-58387-6_28
2. **Problema.** Los contratos inteligentes de Ethereum administran activos valiosos y no pueden corregirse una vez desplegados, pero muchos se escriben con errores que se han explotado para robar fondos.
3. **Método.** Proponen FSolidM: el desarrollador describe el contrato como una máquina de estados finitos (estados, transiciones y las acciones de cada transición) en una herramienta gráfica, y el marco genera el código Solidity. La semántica de la máquina se define formalmente y la seguridad se refuerza con patrones de diseño que se agregan como complementos.
4. **Resultado.** Muestran con casos como una subasta a ciegas que el modelo de estados evita por construcción errores frecuentes (llamadas en un estado indebido, reentrada) y que los complementos añaden protecciones sin reescribir la lógica.
5. **Tema de la Unidad I.** Autómatas finitos aplicados fuera de la teoría (seguridad y verificación de software).
6. **Aportación.** Ejemplo de que fijar el conjunto de estados y la función de transición antes de programar es una técnica de ingeniería, no sólo de teoría; es la misma separación entre modelo e implementación que sigue el software de esta práctica.

4.5. Texto 5. Davis et al. (2019)

1. **Cita.** Davis, J. C., Michael IV, L. G., Coghlan, C. A., Servant, F., y Lee, D. (2019). Why aren't regular expressions a lingua franca? An empirical study on the re-use and portability of regular expressions. En *Proceedings of the 27th ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering* (pp. 443–454). ACM. https://doi.org/10.1145/3338906.3338909

2. **Problema.** Los programadores copian expresiones regulares de un lenguaje a otro suponiendo que significan lo mismo; nadie había medido si esa suposición es cierta.
3. **Método.** Encuesta a 158 desarrolladores y análisis de 537 806 expresiones únicas extraídas de 193 524 proyectos en ocho lenguajes; cada expresión se ejecuta en todos los motores con unas 10 000 entradas generadas y se buscan diferencias de resultado y de tiempo.
4. **Resultado.** El 94 % de los encuestados reutiliza expresiones; alrededor del 15 % de las expresiones produce resultados distintos en algún lenguaje y alrededor del 10 % tiene un costo en tiempo distinto. Go y Rust no presentaron expresiones de tiempo exponencial; Java, JavaScript, Python y Ruby, más de mil cada uno.
5. **Tema de la Unidad I.** Expresiones regulares y su relación con los autómatas finitos.
6. **Aportación.** Explica por qué el curso distingue la expresión regular teórica de la de los lenguajes de programación y por qué conviene validar con un autómata cuyo tiempo es lineal.

4.6. Comparación

Tabla 6. Resumen de los cinco textos.

Texto	Arbitrado	Año	Modelo que usa	Campo de aplicación
1. Gribkoff	No	2013	AFD y máquina de Mealy	Docencia: máquinas, juegos, redes, búsqueda
2. Luna-Benoso et al.	Sí	2022	Autómata celular	Diagnóstico médico por imagen
3. Turing (original)	Sí	1936	Máquina de Turing	Fundamentos de la matemática
4. Mavridou y Laszka	Sí	2018	Máquina de estados finitos	Seguridad de contratos inteligentes
5. Davis et al.	Sí	2019	Expresiones regulares y sus motores	Ingeniería de software

Los cinco describen un sistema mediante estados y reglas que dicen cómo cambiar de uno a otro;

la diferencia está en el rigor y en el propósito. Turing demuestra un teorema de fundamentos con una definición nueva; Gribkoff, sin revisión ni bibliografía, divulga; Luna-Benoso et al. y Mavridou y Laszka construyen un sistema y lo evalúan; Davis et al. miden cómo se usa la teoría en la práctica. Los textos 4 y 5 se complementan: uno muestra el beneficio de fijar el modelo de estados antes de programar y el otro, el costo de no hacerlo, pues las expresiones de los lenguajes de programación ya no corresponden a un autómata finito. De ahí surge un problema abierto: cómo garantizar que una especificación escrita como autómata o expresión regular conserve su significado y su costo lineal cuando se traduce a la implementación concreta de cada lenguaje.

5. Ejercicio 4. Autómatas finitos en JFLAP

5.1. Método de diseño

Todos los autómatas se diseñaron con la misma pregunta: *¿qué es lo mínimo que hay que recordar del prefijo leído para decidir el resto?* La respuesta define los estados; la función de transición dice cómo cambia ese recuerdo al leer un símbolo, y los estados de aceptación son aquellos cuyo recuerdo ya garantiza que la cadena pertenece al lenguaje. Cuatro familias de enunciados cubren casi todas las listas:

Tabla 7. Familias de diseño.

Familia	Qué guarda cada estado	Ejemplos
Conteo módulo k	El residuo de una longitud, de un número de apariciones o del valor numérico leído	097, 139, 145, 147
Búsqueda de patrones	El sufijo más largo leído que es prefijo del patrón (construcción de Knuth-Morris-Pratt)	099, 102, 103, 106
Producto de autómatas	Un par de recuerdos, uno por condición, cuando el enunciado pide «y» o «o»	107, 109, 133
Prefijos y extremos	Cuánto del prefijo exigido se ha leído, o el primer símbolo y el último	120, 135, 137

Cuando una condición ya no puede cumplirse (por ejemplo, la cadena no empieza como se exige) se usa un estado **sumidero**: no es de aceptación y todas sus transiciones vuelven a él. En los

enunciados de la Lista 3 que piden un AFN, el AFD se obtuvo convirtiendo ese AFN y minimizando el resultado; su ficha indica qué estados del AFN representa cada estado del AFD.

5.2. Implementación y validación

Cada autómata se capturó en JFLAP 7.1 y se exportó a `automatas/lista2/` y `automatas/lista3/`; los archivos `.jff` se leen también con el software del Ejercicio 5 de la Práctica 2. Se resolvieron los 77 ejercicios con asterisco que piden un autómata; los ejercicios 116 (lenguaje no regular), 143 (figuras ausentes en la lista), 191 y 201–235 no piden un AFD y se resuelven en las tareas.

Cada ficha del Anexo A tiene el diagrama en JFLAP, la tabla de transiciones —en todos los autómatas, no sólo en los cinco que pide el enunciado—, tres cadenas válidas, tres inválidas y los casos frontera: la cadena vacía, la aceptada más corta y, cuando existe, una cadena justo por debajo de esa longitud. Las cadenas se eligieron de longitudes crecientes y se comprobaron dos veces: con *Input* → *Multiple Run* en JFLAP y con un programa que compara cada autómata contra la definición del lenguaje sobre todas las cadenas de longitud ≤ 14 (≤ 9 con tres símbolos). El archivo `automatas/pruebas.json` reúne las cadenas de todas las fichas.

5.3. Simulación paso a paso

La simulación paso a paso (*Input* → *Step with Closure*) se usó en los cinco autómatas de la tabla; cada captura muestra la configuración (estado y resto de la entrada) después de algunos pasos y el resultado final.

Tabla 8. Autómatas simulados paso a paso.

Ejercicio	Lenguaje	Cadena simulada	Resultado
097	Longitud múltiplo de 3	`102211`	aceptada en r0
113	Dos apariciones de 010, con traslape	`01010`	aceptada
120	Primer y último símbolo distintos	`2101`	aceptada
135	$b^m ab^n$ con m, n > 0	`bbab`	aceptada
147	Binario (izquierda a derecha) múltiplo de 6	`1100`	aceptada (12 = 2·6)

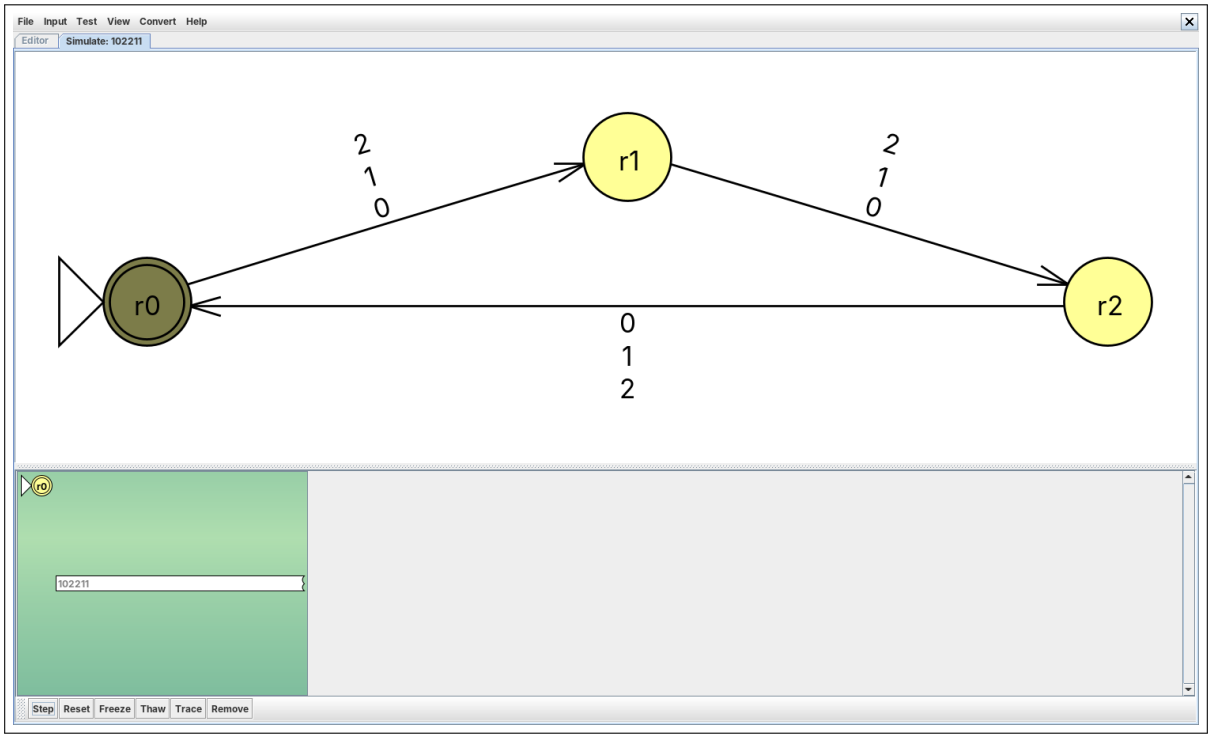


Figura 4. JFLAP, simulación paso a paso de `102211` en lista2-097.

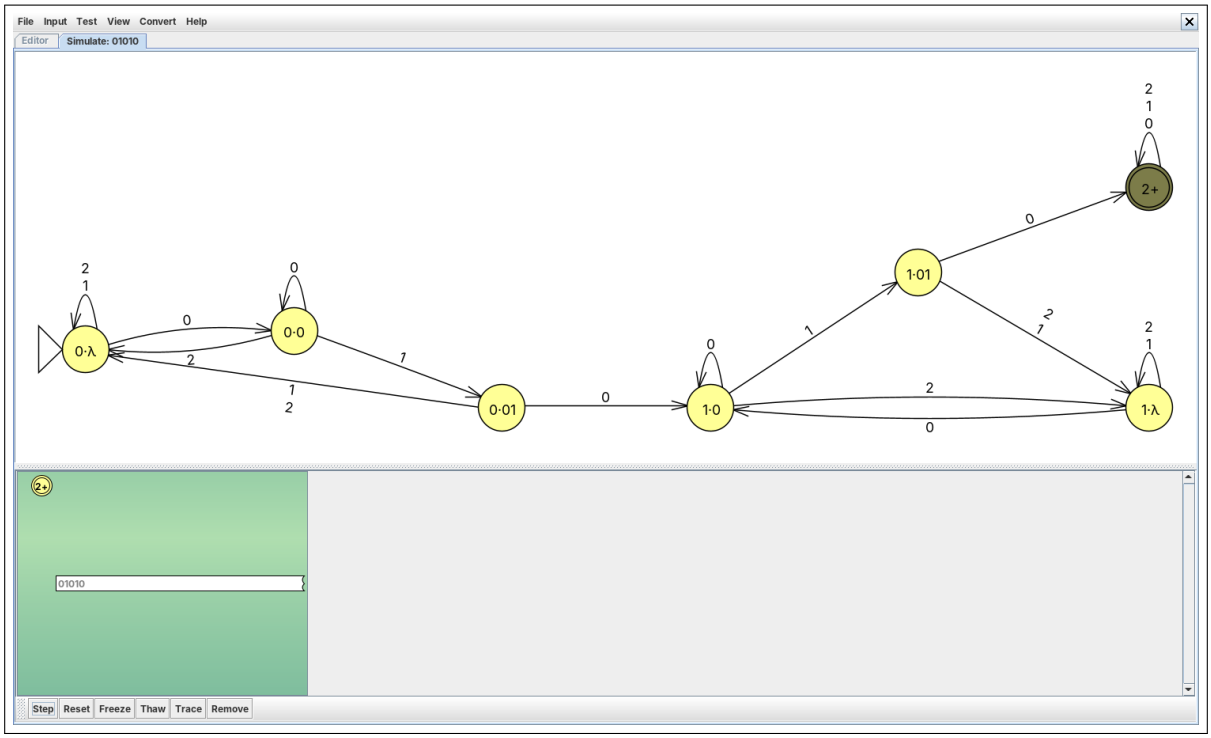


Figura 5. JFLAP, simulación paso a paso de `01010` en lista2-113.

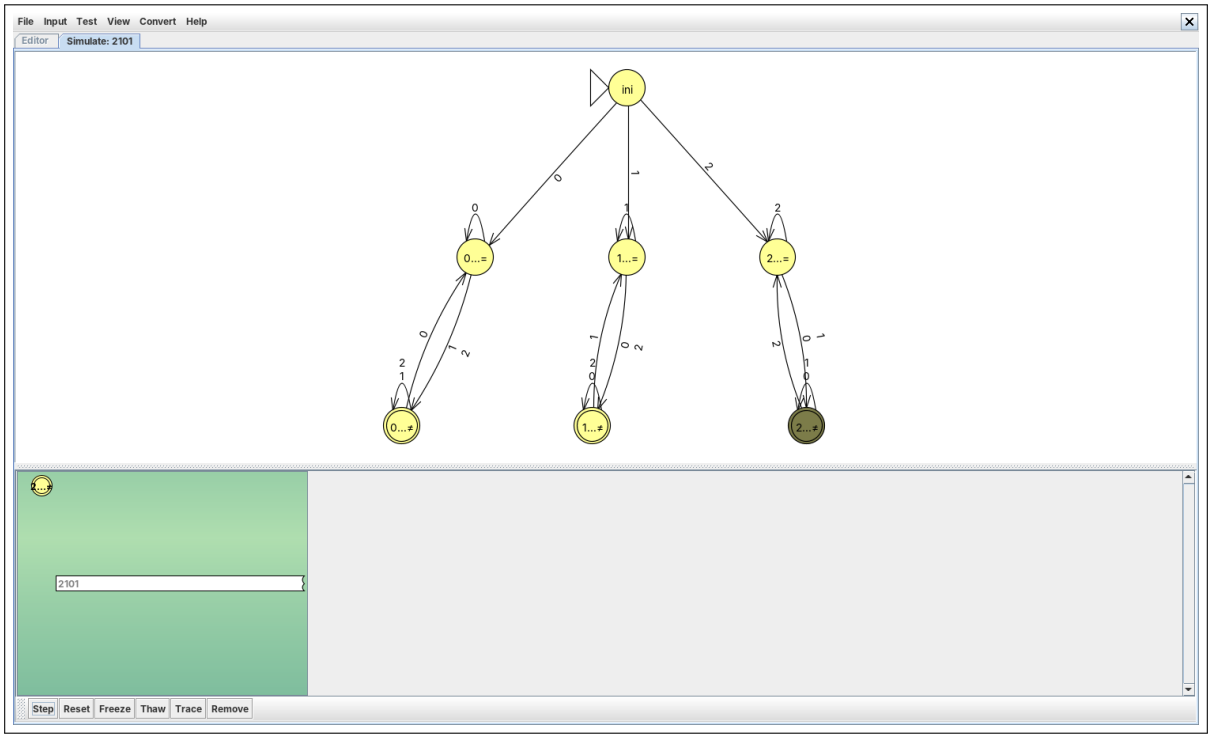


Figura 6. JFLAP, simulación paso a paso de `2101` en lista2-120.

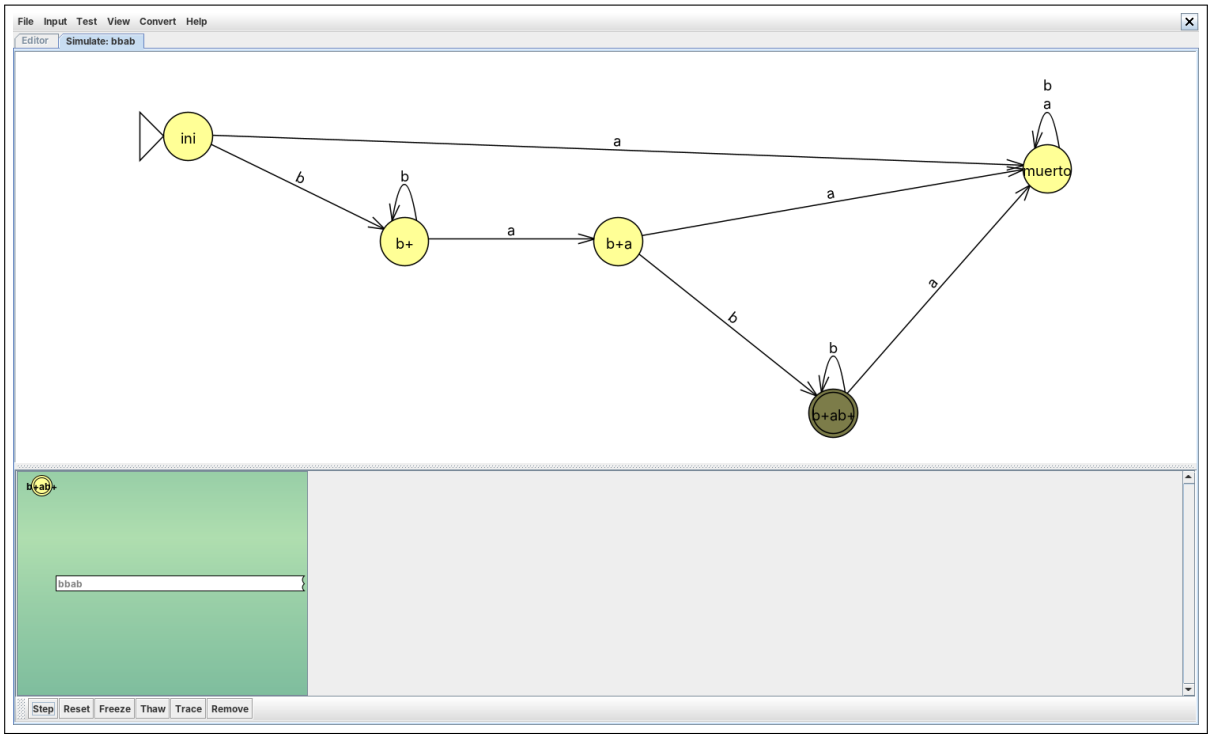


Figura 7. JFLAP, simulación paso a paso de `bbab` en lista2-135.

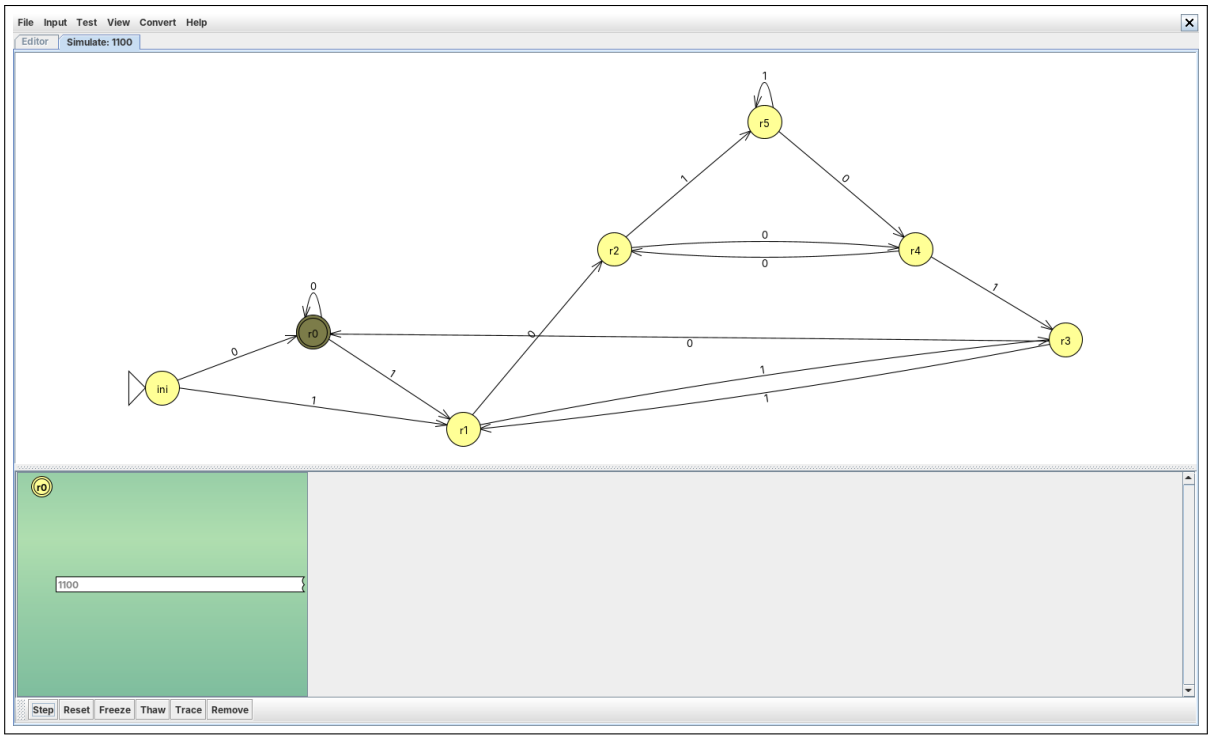


Figura 8. JFLAP, simulación paso a paso de `1100` en lista2-147.

5.4. Resumen de los autómatas

Tabla 9. Autómatas de los ejercicios con asterisco.

Ejercicio	Σ	Diseño original	Estados del AFD	Archivo
L2 · 097	{0, 1, 2}	AFD	3	`lista2-097.jff`
L2 · 099	{0, 1, 2}	AFD	7	`lista2-099.jff`
L2 · 102	{a, b}	AFD	7	`lista2-102.jff`
L2 · 103	{a, b}	AFD	7	`lista2-103.jff`
L2 · 106	{0, 1, 2}	AFD	8	`lista2-106.jff`
L2 · 107	{0, 1, 2}	AFD	13	`lista2-107.jff`
L2 · 109	{0, 1, a, b, c}	AFD	22	`lista2-109.jff`
L2 · 111	{0, 1, 2}	AFD	21	`lista2-111.jff`
L2 · 112	{0, 1, 2}	AFD	7	`lista2-112.jff`
L2 · 113	{0, 1, 2}	AFD	7	`lista2-113.jff`
L2 · 118	{a, b}	AFD	6	`lista2-118.jff`
L2 · 120	{0, 1, 2}	AFD	7	`lista2-120.jff`
L2 · 121	{0, 1, 2}	AFD	6	`lista2-121.jff`

Ejercicio	Σ	Diseño original	Estados del AFD	Archivo
L2 · 122	{0, 1, 2}	AFD	4	`lista2-122.jff`
L2 · 123	{a, b}	AFD	15	`lista2-123.jff`
L2 · 124	{0, 1}	AFD	13	`lista2-124.jff`
L2 · 125	{0, 1}	AFD	2	`lista2-125.jff`
L2 · 126	{0, 1}	AFD	2	`lista2-126.jff`
L2 · 127	{0, 1}	AFD	2	`lista2-127.jff`
L2 · 128	{0, 1}	AFD	6	`lista2-128.jff`
L2 · 131	{0, 1}	AFD	2	`lista2-131.jff`
L2 · 133	{a, b}	AFD	4	`lista2-133.jff`
L2 · 135	{a, b}	AFD	5	`lista2-135.jff`
L2 · 137	{a, b}	AFD	4	`lista2-137.jff`
L2 · 139	{0, 1}	AFD	2	`lista2-139.jff`
L2 · 141	{0, 1}	AFD	3	`lista2-141.jff`
L2 · 145	{0, 1}	AFD	6	`lista2-145.jff`
L2 · 147	{0, 1}	AFD	7	`lista2-147.jff`
L2 · 148	{A, N, P, S}	AFD	13	`lista2-148.jff`
L2 · 149	{0, 1}	AFD	3	`lista2-149.jff`
L2 · 150	{0, 1, a}	AFD	4	`lista2-150.jff`
L3 · 151	{0, 1}	AFD	10	`lista3-151.jff`
L3 · 152	{0, 1, 2}	AFD	6	`lista3-152.jff`
L3 · 153	{0, 1}	AFD	7	`lista3-153.jff`
L3 · 154	{0, 1}	AFD	11	`lista3-154.jff`
L3 · 155	{0, 1}	AFD	12	`lista3-155.jff`
L3 · 156	{0, 1}	AFD	5	`lista3-156.jff`
L3 · 157	{0, 1}	AFD	6	`lista3-157.jff`
L3 · 158	{0, 1}	AFD	3	`lista3-158.jff`
L3 · 159	{0, 1}	AFD	3	`lista3-159.jff`
L3 · 160	{0, 1}	AFD	3	`lista3-160.jff`
L3 · 161	{0, 1}	AFD	3	`lista3-161.jff`
L3 · AFN del 097	{0, 1, 2}	AFD	3	`lista3-afn-097.jff`
L3 · AFN del 099	{0, 1, 2}	AFN	7	`lista3-afn-099.jff`
L3 · AFN del 102	{a, b}	AFN	7	`lista3-afn-102.jff`
L3 · AFN del 103	{a, b}	AFN	7	`lista3-afn-103.jff`
L3 · AFN del 106	{0, 1, 2}	AFN	8	`lista3-afn-106.jff`
L3 · AFN del 107	{0, 1, 2}	AFN	13	`lista3-afn-107.jff`
L3 · AFN del 109	{0, 1, a, b, c}	AFN	22	`lista3-afn-109.jff`

Ejercicio	Σ	Diseño original	Estados del AFD	Archivo
L3 · AFN del 111	{0, 1, 2}	AFD	20	`lista3-afn-111.jff`
L3 · AFN del 112	{0, 1, 2}	AFN	7	`lista3-afn-112.jff`
L3 · AFN del 113	{0, 1, 2}	AFN	7	`lista3-afn-113.jff`
L3 · AFN del 118	{a, b}	AFN	5	`lista3-afn-118.jff`
L3 · AFN del 120	{0, 1, 2}	AFN	7	`lista3-afn-120.jff`
L3 · AFN del 121	{0, 1, 2}	AFD	5	`lista3-afn-121.jff`
L3 · AFN del 122	{0, 1, 2}	AFD	3	`lista3-afn-122.jff`
L3 · AFN del 123	{a, b}	AFN	15	`lista3-afn-123.jff`
L3 · 189	{0, 1}	AFN	7	`lista3-189.jff`
L3 · 190	{0, 1}	AFN	19	`lista3-190.jff`
L3 · 192	{0, 1}	AFN	5	`lista3-192.jff`
L3 · 193	{0, 1}	AFN	6	`lista3-193.jff`
L3 · 194	{0, 1, 2}	AFN	2	`lista3-194.jff`
L3 · 195	{0, 1}	AFN	7	`lista3-195.jff`
L3 · 236	{0, 1}	AFN-λ	4	`lista3-236.jff`
L3 · 237	{0, 1}	AFN-λ	4	`lista3-237.jff`
L3 · 238	{0, 1}	AFN-λ	6	`lista3-238.jff`
L3 · 239	{0, 1}	AFN-λ	6	`lista3-239.jff`
L3 · 240	{0, 1}	AFN-λ	6	`lista3-240.jff`
L3 · 254	{0, 1}	AFD	2	`lista3-254.jff`
L3 · 255	{0, 1}	AFD	2	`lista3-255.jff`
L3 · 256	{0, 1, 2}	AFD	2	`lista3-256.jff`
L3 · 257	{0, 1}	AFD	3	`lista3-257.jff`
L3 · 258	{0, 1}	AFD	3	`lista3-258.jff`
L3 · 260	{0, 1}	AFN	6	`lista3-260.jff`
L3 · 261	{0, 1}	AFN	3	`lista3-261.jff`
L3 · 262	{a, b}	AFD	5	`lista3-262.jff`
L3 · 263	{0, 1}	AFN	3	`lista3-263.jff`

6. Ejercicio 5. Aplicación con interfaz gráfica

6.1. Arquitectura

La aplicación separa el **núcleo**, que no importa Flet, de la **interfaz**, que sólo traduce clics en llamadas al núcleo. Así las pruebas ejercitan toda la lógica sin abrir una ventana.

```
src/
├── app.py                  # punto de entrada: lista de secciones y arranque
├── nucleo/
│   ├── errores.py          # ErrorTC y sus subclases (mensajes para el usuario)
│   └── lenguajes.py        # Alfabeto, Palabra, Lenguaje, Enumerador, Informe
└── interfaz/
    ├── aplicacion.py       # ventana: menú lateral, barra superior, área de trabajo
    ├── componentes.py      # tarjetas, avisos, pastillas, métricas, ayuda numerada
    ├── tema.py · sesion.py # paleta institucional y estado compartido
    └── vistas/             # Inicio, Palabras, Cerraduras, Lenguajes
```

Tabla 10. Clases principales.

Clase	Responsabilidad
`Alfabeto`	Conjunto finito y no vacío de símbolos de un carácter; rechaza λ, comas, llaves y espacios. Responde si una palabra está en Σ, *cuenta* Σ^n *sin generarlo y construye* Σ^n, Σ y Σ^+ acotados.
`Palabra`	Cadena inmutable: longitud, concatenación (+), potencia, reflexión, prefijos, sufijos y subcadenas.
`Lenguaje`	Conjunto finito de palabras con `|`, `&`, `-`, concatenación, potencia, reflexión y cerraduras acotadas.
`Enumerador`	Recorre $\Sigma^{min} \cup \ldots \cup \Sigma^{max}$ en orden canónico y verifica el límite *antes* de generar.
`Informe`	Texto exportable con encabezado, datos de la operación, total y resultados.
`Vista` (interfaz)	Base de cada sección: encabezado, ayuda «¿Cómo se usa?», avisos y manejo uniforme de errores.

Todas las operaciones del núcleo devuelven objetos nuevos; ninguna modifica sus argumentos. Los errores de dominio (símbolo inválido, palabra fuera de Σ, límite excedido) son subclases de `ErrorTC` y la interfaz los muestra en una franja roja con el mensaje, sin interrumpir el programa.

6.2. Diseño de la interfaz

La ventana tiene un menú lateral con las secciones y un área de trabajo. Cada sección empieza con un encabezado que dice qué hace y un botón **¿Cómo se usa?** que despliega los pasos numerados. Los resultados se calculan **mientras se escribe**: no hay que pulsar un botón para ver los prefijos de una palabra, y un símbolo que no pertenece a Σ se señala en el propio campo. Los resultados se muestran como pastillas (la palabra vacía, resaltada) acompañadas de métricas: la longitud, el

número de resultados y el máximo teórico.

6.3. Operación 1. Prefijos, sufijos y subcadenas

`Palabra.prefijos()` devuelve `[ω[:0], ω[:1], …, ω[:n]]`; `sufijos()`, los n + 1 finales; y `subcadenas()` recorre todas las longitudes y posiciones y conserva cada subcadena la primera vez que aparece. La sección **Palabras** permite además la reflexión y la potencia, y guarda cualquier resultado en `salidas/palabras-<operación>.txt`.

Conteos. Una palabra de longitud n tiene exactamente **n + 1 prefijos** (uno por cada punto de corte, de 0 a n) y n + 1 sufijos. Las subcadenas no vacías se determinan por un par de posiciones 0 $\leq$ i < j $\leq$ n, de modo que hay a lo sumo n(n + 1)/2; con λ, el máximo es **n(n + 1)/2 + 1**, y se alcanza sólo cuando todos los símbolos son distintos. Para `abcb` hay 5 prefijos y 10 subcadenas distintas, una menos que la cota 11 porque `b` se repite.

Convención sobre λ. El programa incluye λ y la propia palabra entre los prefijos y los sufijos. Se sigue la definición: x es prefijo de ω si existe y tal que xy = ω; con y = ω se obtiene x = λ, y con y = λ, x = ω. Excluirlas haría que una palabra de longitud n tuviera n − 1 prefijos *propios*, que es otro concepto; el núcleo ofrece esa variante con `prefijos(propios=True)`.

6.4. Operación 2. Cerradura de Kleene y cerradura positiva

La sección **Cerraduras** genera Σ^*, Σ^+ o Σ^n hasta la longitud que el usuario elige con un control deslizante. Antes de generar, `Alfabeto.cuantas` calcula el total con la fórmula

$$|\Sigma^0 \cup \Sigma^1 \cup \cdots \cup \Sigma^n| = \sum_{i=0}^{n} |\Sigma|^i = \frac{|\Sigma|^{n+1} - 1}{|\Sigma| - 1} \quad (|\Sigma| > 1),$$

y la interfaz muestra ese número y una barra respecto del límite **mientras se mueve el control**. Si el total supera 200 000, el botón *Generar* se desactiva y el mensaje indica la longitud máxima permitida para ese alfabeto; el núcleo repite la verificación y lanza `LimiteExcedido` aunque se le llame desde otro programa.

```
self.total = alfabeto.cuantas(maximo, minimo)
if self.total > limite:
    raise LimiteExcedido(f"... produce {self.total} cadenas; el límite es {limite}.")
```

Relación con la unidad. Σ^* es infinito para todo alfabeto, así que ningún programa puede «calcularlo»; lo que se genera es un truncamiento finito. Cada nivel tiene $|\Sigma^n| = |\Sigma|^n$ cadenas, de modo que el truncamiento crece exponencialmente en la longitud: con $|\Sigma| = 2$ se llega a n = 16 (131 071 cadenas), pero con $|\Sigma| = 10$ sólo a n = 5. El límite convierte esa propiedad teórica en una regla que el usuario ve antes de esperar. Σ^+ con longitud 0 es vacío: la única cadena de longitud 0 es λ, y Σ^+ la excluye.

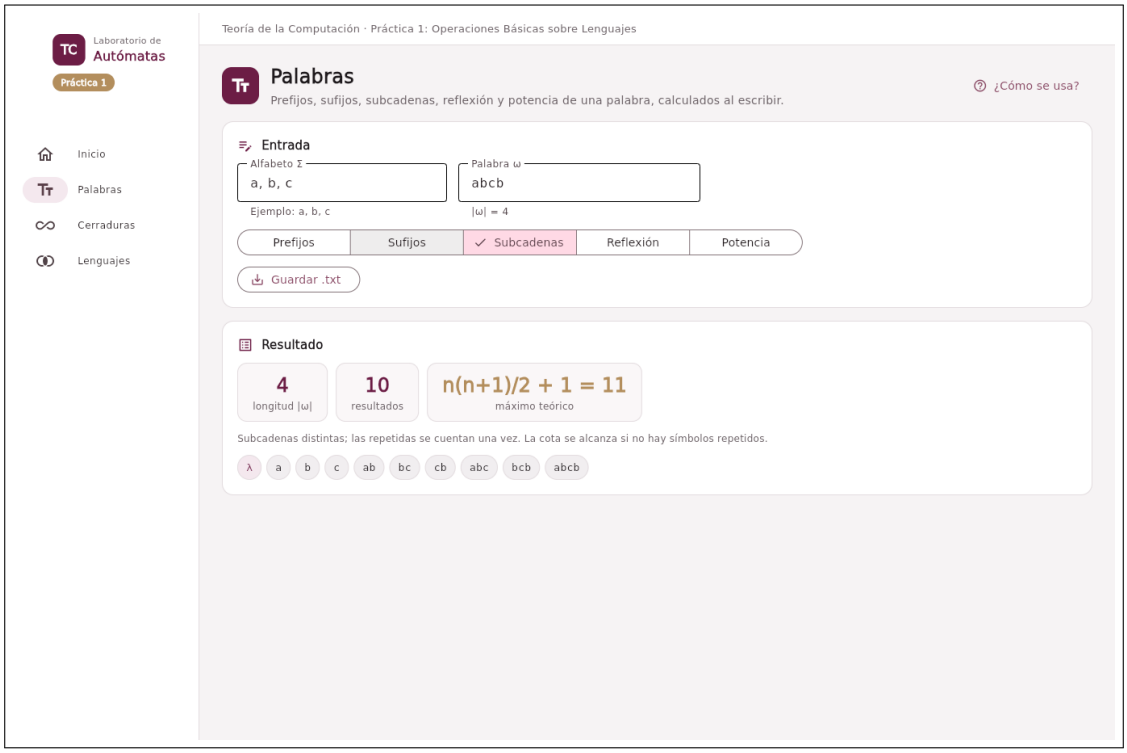


Figura 9. Sección Palabras: subcadenas de `abcb` con las métricas y la cota teórica.

6.5. Pruebas automatizadas

`tests/test_lenguajes.py` (34 pruebas) cubre lo que pide el enunciado y algo más: la palabra vacía (λ = «» = «ε», sus prefijos, sufijos y subcadenas), un alfabeto de un solo símbolo, los prefijos y sufijos de una palabra de longitud 1, la diferencia $\Sigma^* - \Sigma^+ = \{\lambda\}$ y $\Sigma^+ = \varnothing$ para longitud 0, los

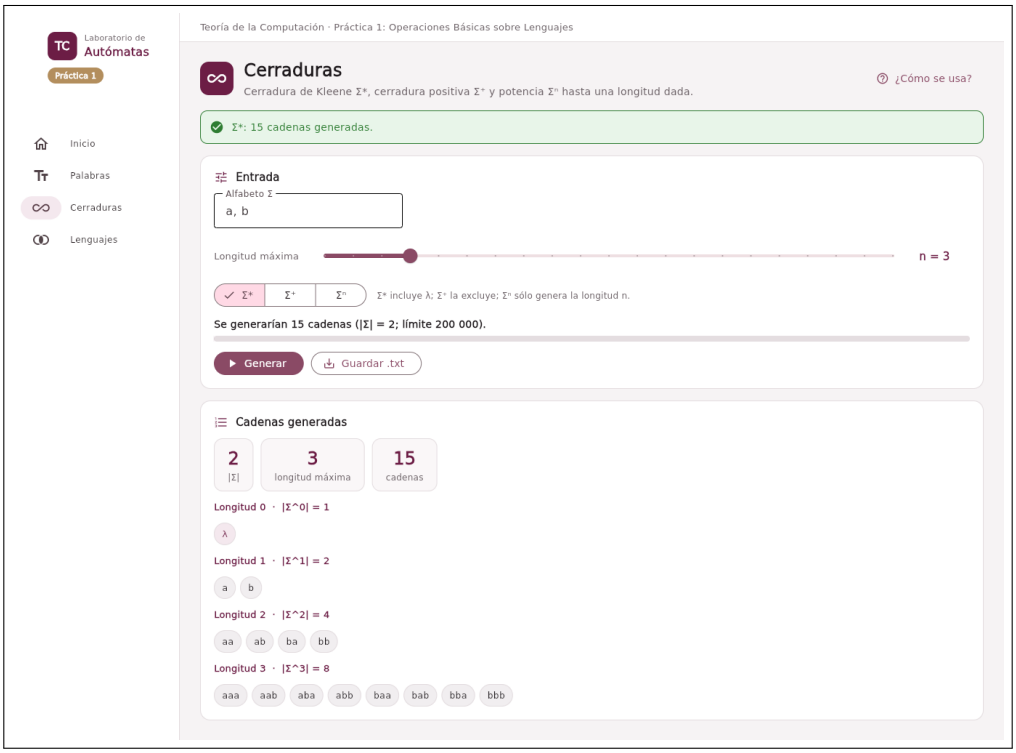


Figura 10. Sección Cerraduras: Σ^* con $\Sigma = \{a, b\}$ y n = 3, agrupada por longitud.

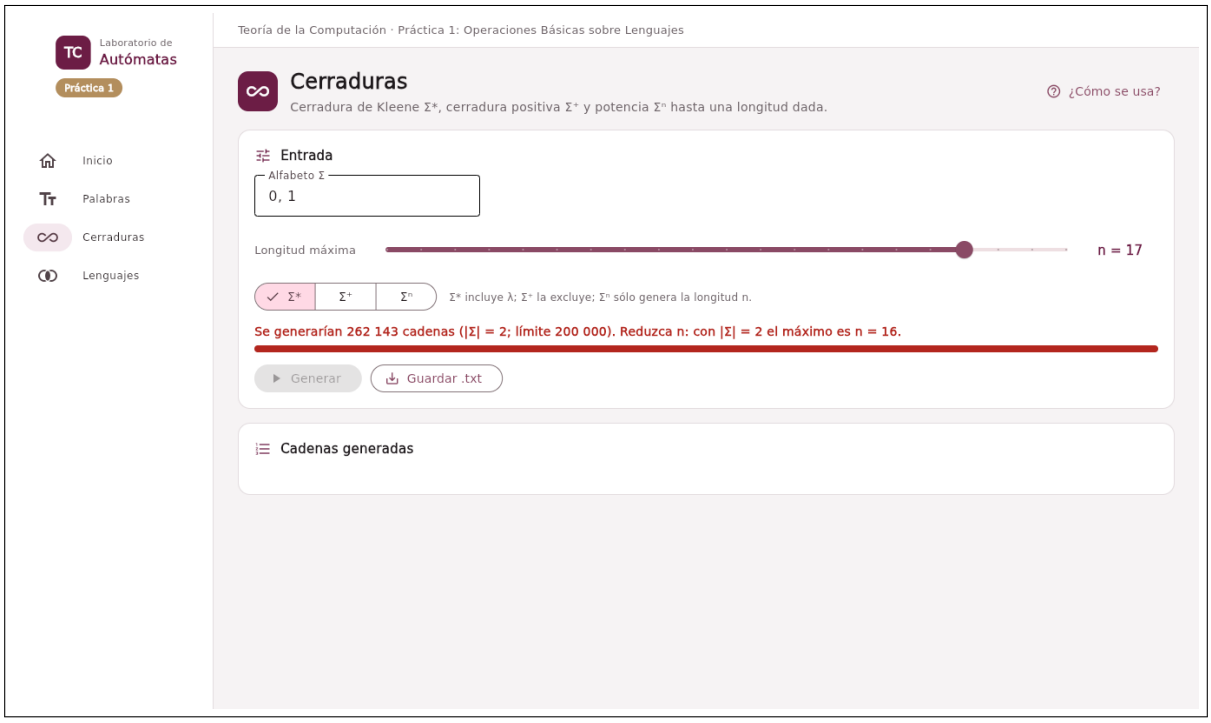


Figura 11. Con $\Sigma = \{0, 1\}$ y n = 17 el contador supera 200 000 y el botón Generar se desactiva.

conteos n + 1 y n(n + 1)/2 + 1, el orden canónico, el límite exacto (n = 16 se permite y n = 17 no, con |Σ| = 2) y las operaciones entre lenguajes. `tests/test_interfaz.py` construye cada sección sin ventana y ejecuta sus acciones principales, por ejemplo escribir una palabra con un símbolo ajeno y comprobar que el campo muestra el error.

Tabla 11. Resultados de la suite en los tres entornos.

Servicio	Intérprete	Resultado de `pytest -q`
`py311`	Python 3.11	41 aprobadas, 3 omitidas
`py312`	Python 3.12	41 aprobadas, 3 omitidas
`py313`	Python 3.13	41 aprobadas, 3 omitidas

Las tres pruebas omitidas pertenecen a secciones de prácticas posteriores (editor, algoritmos y expresiones) y se saltan porque esta práctica no incluye esos módulos. No se observaron diferencias de comportamiento entre versiones.

Captura pendiente
`../evidencias/docker/pytest-tres-versiones.png`

Figura 12. `docker compose run --rm py311 pytest -q` y sus equivalentes en 3.12 y 3.13.

Captura pendiente
`../evidencias/app/04-contenedor.png`

Figura 13. Interfaz servida por el contenedor `py312` en http://localhost:8550.

7. Conclusiones

El entorno de trabajo quedó resuelto antes de escribir la primera línea de lógica: un repositorio con historia, ramas y Pull Requests, y tres contenedores construidos con un único `Dockerfile` que difieren sólo en la versión del intérprete. Fijar la imagen y las dependencias tuvo un efecto concreto: la incompatibilidad entre `flet-web` y las versiones nuevas de FastAPI apareció una sola vez, se resolvió con una línea en `requirements.txt` y no volvió a afectar a ninguno de los tres entornos.

La investigación mostró que las definiciones de la unidad no son convenciones arbitrarias. Que Σ^0 sea $\{\lambda\}$ y no $\varnothing$ es lo que hace funcionar las leyes de las potencias, y la diferencia entre Σ^* y Σ^+ se reduce a λ. La lectura de Turing explicó por qué la tesis de Church-Turing no puede demostrarse, y los artículos recientes, por qué la distinción entre la expresión regular de la teoría y la de los lenguajes de programación tiene consecuencias de seguridad.

En JFLAP, preguntarse qué hay que recordar del prefijo leído redujo casi todos los enunciados a cuatro familias de diseño (conteo modular, búsqueda de patrones, producto y extremos), y comparar cada autómata contra la definición del lenguaje sobre todas las cadenas cortas detectó errores que tres cadenas de prueba no habrían mostrado.

En la aplicación, separar el núcleo de la interfaz permitió probar toda la lógica con 34 pruebas en tres versiones de Python sin abrir una ventana, y calcular el número de cadenas con la fórmula antes de generarlas convirtió el crecimiento exponencial de $|\Sigma|^n$ en un aviso que el usuario ve mientras elige la longitud.

Referencias

Appveyor Systems Inc. (2026). *flet 0.86.5. Metadatos del paquete (Requires-Python ≥ 3.10)*. Consultado el 6 de noviembre de 2026, desde https://pypi.org/project/flet/0.86.5/

Chacon, S., & Straub, B. (2014). *Pro Git* (2.ª ed.). Apress. https://doi.org/10.1007/978-1-4842-0076-6

Chomsky, N. (1956). Three models for the description of language. *IRE Transactions on Information Theory*, *2*(3), 113-124. https://doi.org/10.1109/TIT.1956.1056813

Church, A. (1936). An unsolvable problem of elementary number theory. *American Journal of Mathematics*, *58*(2), 345-363. https://doi.org/10.2307/2371045

Davis, J. C., Michael IV, L. G., Coghlan, C. A., Servant, F., & Lee, D. (2019). Why aren't regular expressions a lingua franca? An empirical study on the re-use and portability of regular expressions. *Proceedings of the 27th ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering*, 443-454. https://doi.org/10.1145/3338906.3338909

Docker Inc. (2026a). *Docker Compose*. Consultado el 6 de noviembre de 2026, desde https://docs.docker.com/compose/

Docker Inc. (2026b). *Docker overview*. Consultado el 6 de noviembre de 2026, desde https://docs.docker.com/get-started/docker-overview/

Felter, W., Ferreira, A., Rajamony, R., & Rubio, J. (2015). An updated performance comparison of virtual machines and Linux containers. *2015 IEEE International Symposium on Performance Analysis of Systems and Software (ISPASS)*, 171-172. https://doi.org/10.1109/ISPASS.2015.7095802

GitHub Inc. (2026). *About pull requests. GitHub Docs*. Consultado el 6 de noviembre de 2026, desde https://docs.github.com/en/pull-requests/collaborating-with-pull-requests/proposing-changes-to-your-work-with-pull-requests/about-pull-requests

Gribkoff, E. (2013). *Applications of deterministic finite automata* [Documento de curso, ECS 120]. https://www.cs.ucdavis.edu/~rogaway/classes/120/spring13/eric-dfa.pdf

Hopcroft, J. E., Motwani, R., & Ullman, J. D. (2007). *Introduction to automata theory, languages, and computation* (3.ª ed.). Pearson Addison-Wesley.

Kleene, S. C. (1952). *Introduction to metamathematics*. North-Holland.

Kleene, S. C. (1956). Representation of events in nerve nets and finite automata. En C. E. Shannon & J. McCarthy (Eds.), *Automata Studies* (pp. 3-41). Princeton University Press. https://doi.org/10.1515/9781400882618-002

Kozen, D. C. (1997). *Automata and computability*. Springer. https://doi.org/10.1007/978-1-4612-1844-9

Lewis, H. R., & Papadimitriou, C. H. (1998). *Elements of the theory of computation* (2.ª ed.). Prentice Hall.

Linz, P. (2017). *An introduction to formal languages and automata* (6.ª ed.). Jones & Bartlett Learning.

Luna-Benoso, B., Martínez-Perales, J. C., Cortés-Galicia, J., Flores-Carapia, R., & Silva-García, V. M. (2022). Melanoma detection in dermoscopic images using a cellular automata classifier. *Computers*, *11*(1), 8. https://doi.org/10.3390/computers11010008

Mavridou, A., & Laszka, A. (2018). Designing secure Ethereum smart contracts: A finite state machine based approach. En S. Meiklejohn & K. Sako (Eds.), *Financial Cryptography and Data Security* (pp. 523-540, Vol. 10957). Springer. https://doi.org/10.1007/978-3-662-58387-6_28

Python Software Foundation. (2026a). *Status of Python versions. Python Developer's Guide*. Consultado el 6 de noviembre de 2026, desde https://devguide.python.org/versions/

Python Software Foundation. (2026b). *venv: Creation of virtual environments (Python 3.12 documentation)*. Consultado el 6 de noviembre de 2026, desde https://docs.python.org/3.12/library/venv.html

Rabin, M. O., & Scott, D. (1959). Finite automata and their decision problems. *IBM Journal of Research and Development*, *3*(2), 114-125. https://doi.org/10.1147/rd.32.0114

Sipser, M. (2013). *Introduction to the theory of computation* (3.ª ed.). Cengage Learning.

Turing, A. M. (1936). On computable numbers, with an application to the Entscheidungsproblem. *Proceedings of the London Mathematical Society*, *s2-42*(1), 230-265. https://doi.org/10.1112/plms/s2-42.1.230

A. Anexo A. Fichas de diseño de los autómatas

Cada ficha reúne el enunciado, la idea del diseño, lo que recuerda cada estado, la tabla de transiciones, las cadenas de prueba (tres válidas, tres inválidas y los casos frontera) y la captura de JFLAP con el *Multiple Run* de las tres válidas y las tres inválidas. Cuando el diseño natural del enunciado es no determinista, la ficha presenta el AFD que se obtuvo con la construcción de subconjuntos y la minimización, e indica qué estados del AFN representa cada estado del AFD. El archivo de cada autómata está en `automatas/lista2/` o `automatas/lista3/` con el nombre de la ficha.

A.1. Lista 2

A.1.1. Lista 2 · Ejercicio 097

Enunciado. Las palabras de $\Sigma 1$ cuya longitud es un múltiplo de 3.
Diseño. Basta recordar la longitud módulo 3; λ tiene longitud 0, que es múltiplo de 3.
Archivo. `automatas/lista2/lista2-097.jff` · $\Sigma = \{0, 1, 2\}$ · $|Q| = 3$

Tabla 12. Tabla de transiciones ($\rightarrow$ inicial, * aceptación, — sin transición).

δ	0	1	2	Qué recuerda del prefijo leído
$\rightarrow$* r0	r1	r1	r1	la longitud leída es $\equiv 0 \pmod 3$
r1	r2	r2	r2	la longitud leída es $\equiv 1 \pmod 3$
r2	r0	r0	r0	la longitud leída es $\equiv 2 \pmod 3$

Tabla 13. Cadenas de prueba.

Cadena	Longitud	Esperado	Obtenido	Tipo de caso
λ	0	acepta	acepta	válida
`212200`	6	acepta	acepta	válida
`100022002`	9	acepta	acepta	válida
`1`	1	rechaza	rechaza	inválida
`20`	2	rechaza	rechaza	inválida
`0100`	4	rechaza	rechaza	inválida

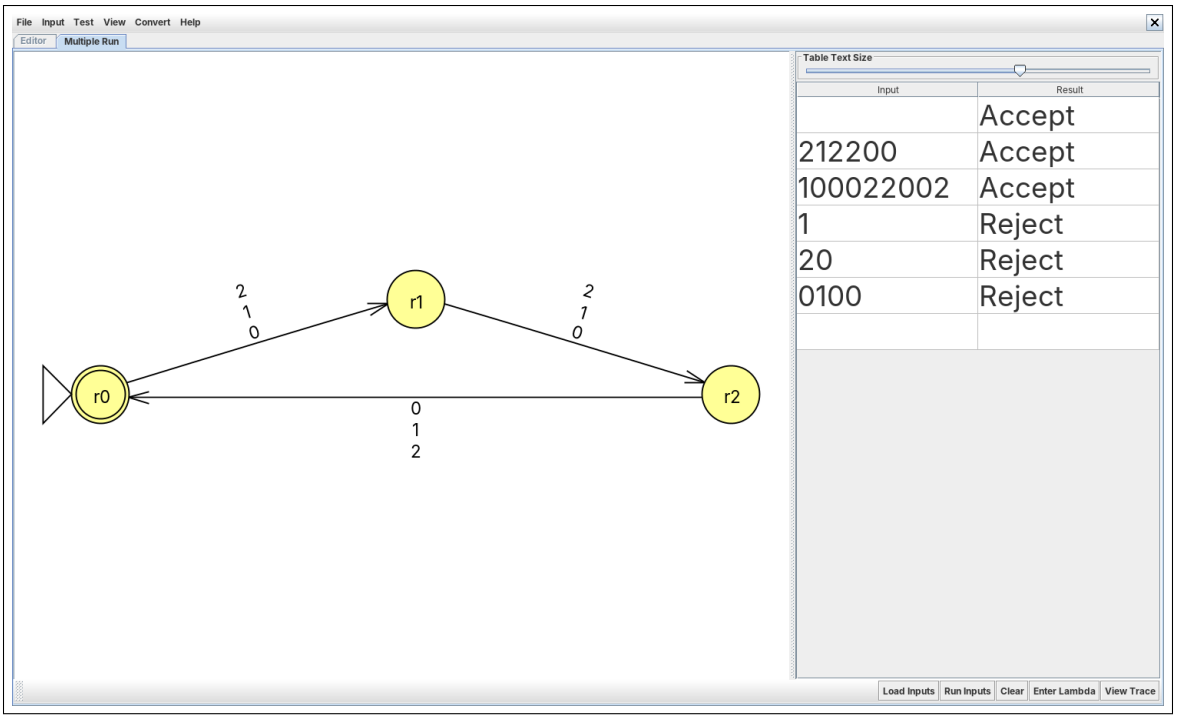


Figura 14. lista2-097 en JFLAP: Multiple Run con las tres cadenas válidas y las tres inválidas de la tabla anterior.

A.1.2. Lista 2 · Ejercicio 099

Enunciado. Las palabras de $\Sigma 1$ que terminan con la cadena 001001.

Diseño. Autómata de búsqueda de patrones (Knuth-Morris-Pratt): cada estado guarda el sufijo más largo leído que es prefijo de 001001; se acepta cuando ese sufijo es el patrón completo.

Archivo. `automatas/lista2/lista2-099.jff` · $\Sigma = \{0, 1, 2\}$ · $|Q| = 7$

Tabla 14. Tabla de transiciones ($\rightarrow$ inicial, * aceptación, — sin transición).

δ	0	1	2	Qué recuerda del prefijo leído
$\rightarrow \lambda$	0	λ	λ	el sufijo más largo leído que es prefijo de 001001 es λ
0	00	λ	λ	el sufijo más largo leído que es prefijo de 001001 es 0
00	00	001	λ	el sufijo más largo leído que es prefijo de 001001 es 00
001	0010	λ	λ	el sufijo más largo leído que es prefijo de 001001 es 001
0010	00100	λ	λ	el sufijo más largo leído que es prefijo de 001001 es 0010
00100	00	001001	λ	el sufijo más largo leído que es prefijo de 001001 es 00100

δ	0	1	2	Qué recuerda del prefijo leído
* 001001	0010	λ	λ	el sufijo más largo leído que es prefijo de 001001 es 001001 (la cadena termina en el patrón)

Tabla 15. Cadenas de prueba.

Cadena	Longitud	Esperado	Obtenido	Tipo de caso
`001001`	6	acepta	acepta	válida
`1001001`	7	acepta	acepta	válida
`120001001`	9	acepta	acepta	válida
λ	0	rechaza	rechaza	inválida
`1`	1	rechaza	rechaza	inválida
`002`	3	rechaza	rechaza	inválida
`11111`	5	rechaza	rechaza	frontera: justo por debajo de la longitud mínima

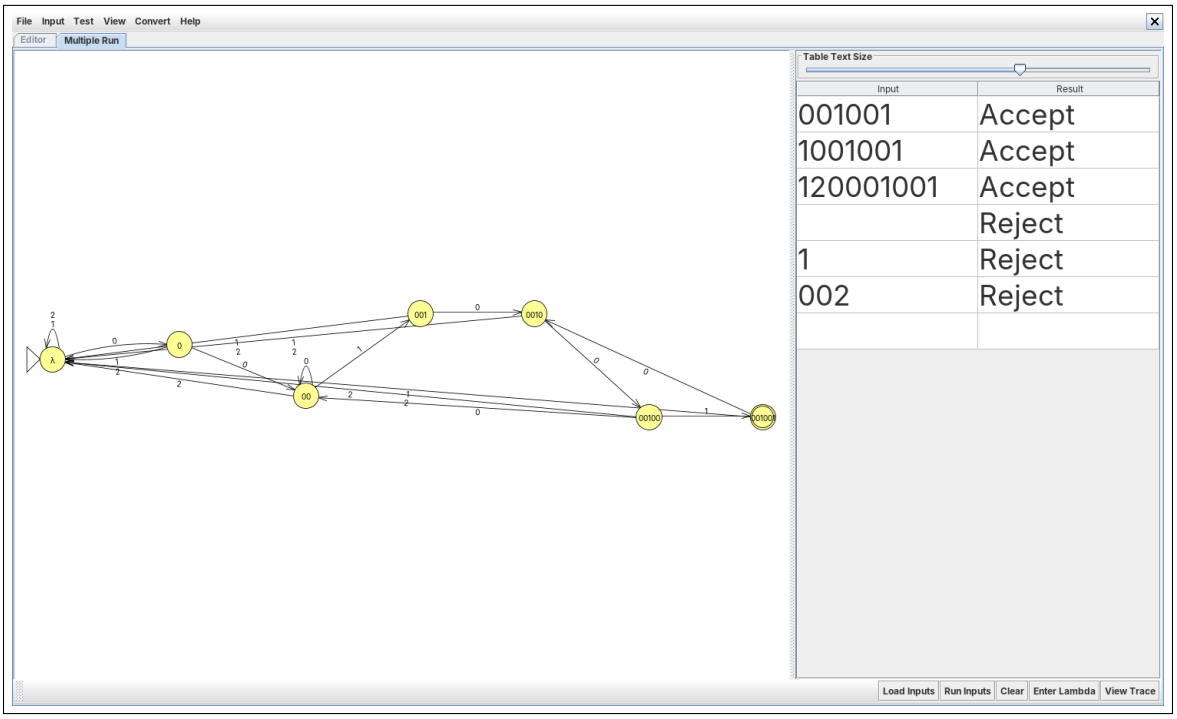


Figura 15. lista2-099 en JFLAP: Multiple Run con las tres cadenas válidas y las tres inválidas de la tabla anterior.

A.1.3. Lista 2 · Ejercicio 102

Enunciado. Las palabras de $\Sigma 3$ que contienen la cadena abbbab.

Diseño. Búsqueda de abbbab con el autómata de Knuth-Morris-Pratt; al completar el patrón el estado es absorbente.

Archivo. `automatas/lista2/lista2-102.jff` · $\Sigma = \{a, b\}$ · $|Q| = 7$

Tabla 16. Tabla de transiciones ($\rightarrow$ inicial, * aceptación, — sin transición).

δ	a	b	Qué recuerda del prefijo leído
$\rightarrow \lambda$	a	λ	aún no aparece abbbab; el sufijo más largo leído que es prefijo de abbbab es λ
a	a	ab	aún no aparece abbbab; el sufijo más largo leído que es prefijo de abbbab es a
ab	a	abb	aún no aparece abbbab; el sufijo más largo leído que es prefijo de abbbab es ab
abb	a	abbb	aún no aparece abbbab; el sufijo más largo leído que es prefijo de abbbab es abb
abbb	abbba	λ	aún no aparece abbbab; el sufijo más largo leído que es prefijo de abbbab es abbb
abbba	a	abbbab	aún no aparece abbbab; el sufijo más largo leído que es prefijo de abbbab es abbba
* abbbab	abbbab	abbbab	ya apareció abbbab

Tabla 17. Cadenas de prueba.

Cadena	Longitud	Esperado	Obtenido	Tipo de caso
`abbbab`	6	acepta	acepta	válida
`abbbaba`	7	acepta	acepta	válida
`aabbbabab`	9	acepta	acepta	válida
λ	0	rechaza	rechaza	inválida
`b`	1	rechaza	rechaza	inválida
`aab`	3	rechaza	rechaza	inválida
`baaab`	5	rechaza	rechaza	frontera: justo por debajo de la longitud mínima

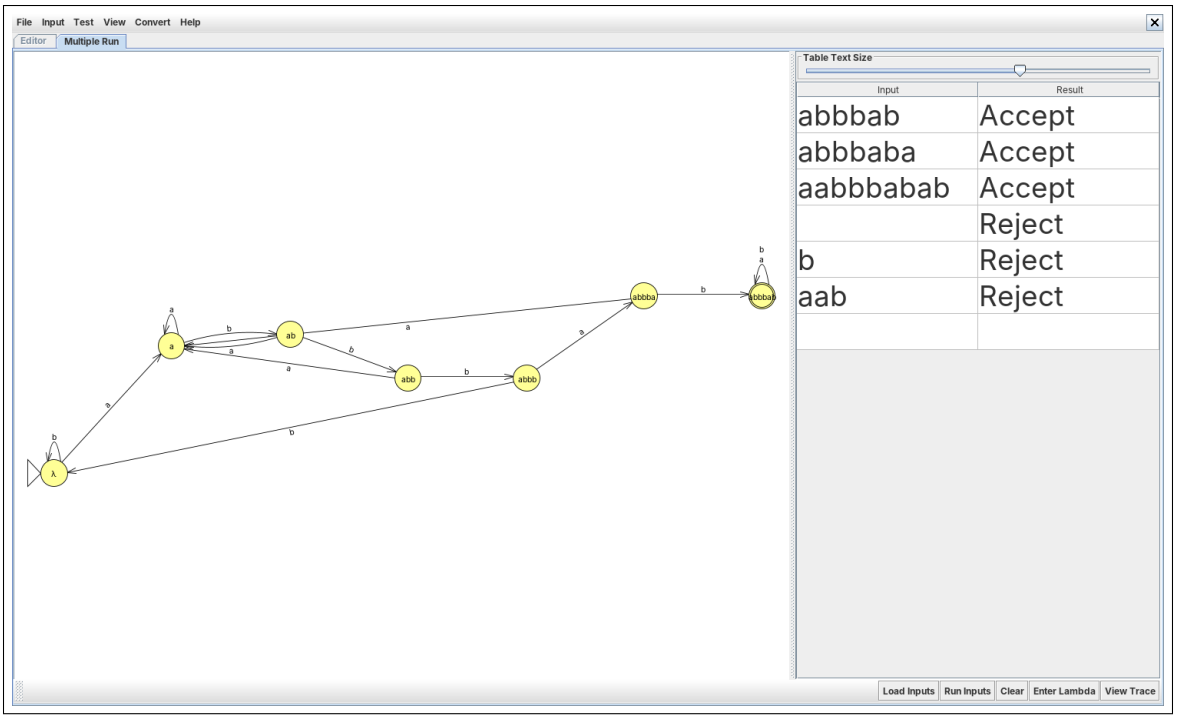


Figura 16. lista2-102 en JFLAP: Multiple Run con las tres cadenas válidas y las tres inválidas de la tabla anterior.

A.1.4. Lista 2 · Ejercicio 103

Enunciado. Las palabras de $\Sigma 3$ que contienen la cadena aabbba.

Diseño. Búsqueda de aabbba con el autómata de Knuth-Morris-Pratt; al completar el patrón el estado es absorbente.

Archivo. `automatas/lista2/lista2-103.jff` · $\Sigma = \{a, b\}$ · $|Q| = 7$

Tabla 18. Tabla de transiciones ($\rightarrow$ inicial, * aceptación, — sin transición).

δ	a	b	Qué recuerda del prefijo leído
$\rightarrow \lambda$	a	λ	aún no aparece aabbba; el sufijo más largo leído que es prefijo de aabbba es λ
a	aa	λ	aún no aparece aabbba; el sufijo más largo leído que es prefijo de aabbba es a
aa	aa	aab	aún no aparece aabbba; el sufijo más largo leído que es prefijo de aabbba es aa
aab	a	aabb	aún no aparece aabbba; el sufijo más largo leído que es prefijo de aabbba es aab
aabb	a	aabbb	aún no aparece aabbba; el sufijo más largo leído que es prefijo de aabbba es aabb
aabbb	aabbba	λ	aún no aparece aabbba; el sufijo más largo leído que es prefijo de aabbba es aabbb

δ	a	b	Qué recuerda del prefijo leído
* aabbba	aabbba	aabbba	ya apareció aabbba

Tabla 19. Cadenas de prueba.

Cadena	Longitud	Esperado	Obtenido	Tipo de caso
`aabbba`	6	acepta	acepta	válida
`aabbbab`	7	acepta	acepta	válida
`aaaabbbab`	9	acepta	acepta	válida
λ	0	rechaza	rechaza	inválida
`a`	1	rechaza	rechaza	inválida
`aab`	3	rechaza	rechaza	inválida
`aabab`	5	rechaza	rechaza	frontera: justo por debajo de la longitud mínima

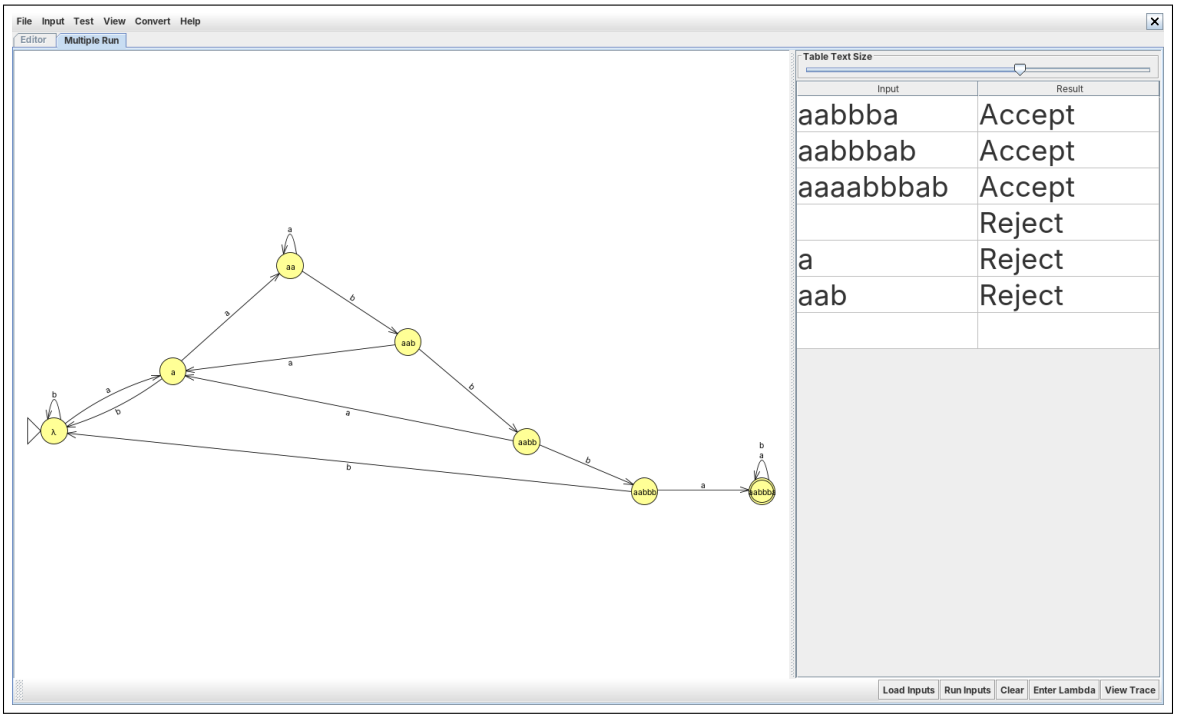


Figura 17. lista2-103 en JFLAP: Multiple Run con las tres cadenas válidas y las tres inválidas de la tabla anterior.

A.1.5. Lista 2 · Ejercicio 106

Enunciado. Las palabras de $\Sigma 1$ que incluyen la subcadena 1101001.

Diseño. Búsqueda de 1101001 con el autómata de Knuth-Morris-Pratt; al completar el patrón el estado es absorbente.

Archivo. `automatas/lista2/lista2-106.jff` · $\Sigma = \{0, 1, 2\}$ · $|Q| = 8$

Tabla 20. Tabla de transiciones ($\rightarrow$ inicial, * aceptación, — sin transición).

δ	0	1	2	Qué recuerda del prefijo leído
$\rightarrow \lambda$	λ	1	λ	aún no aparece 1101001; el sufijo más largo leído que es prefijo de 1101001 es λ
1	λ	11	λ	aún no aparece 1101001; el sufijo más largo leído que es prefijo de 1101001 es 1
11	110	11	λ	aún no aparece 1101001; el sufijo más largo leído que es prefijo de 1101001 es 11
110	λ	1101	λ	aún no aparece 1101001; el sufijo más largo leído que es prefijo de 1101001 es 110
1101	11010	11	λ	aún no aparece 1101001; el sufijo más largo leído que es prefijo de 1101001 es 1101
11010	110100	1	λ	aún no aparece 1101001; el sufijo más largo leído que es prefijo de 1101001 es 11010
110100	λ	1101001	λ	aún no aparece 1101001; el sufijo más largo leído que es prefijo de 1101001 es 110100
* 1101001	1101001	1101001	1101001	ya apareció 1101001

Tabla 21. Cadenas de prueba.

Cadena	Longitud	Esperado	Obtenido	Tipo de caso
`1101001`	7	acepta	acepta	válida
`11010010`	8	acepta	acepta	válida
`0021101001`	10	acepta	acepta	válida
λ	0	rechaza	rechaza	inválida
`0`	1	rechaza	rechaza	inválida
`122`	3	rechaza	rechaza	inválida
`101012`	6	rechaza	rechaza	frontera: justo por debajo de la longitud mínima

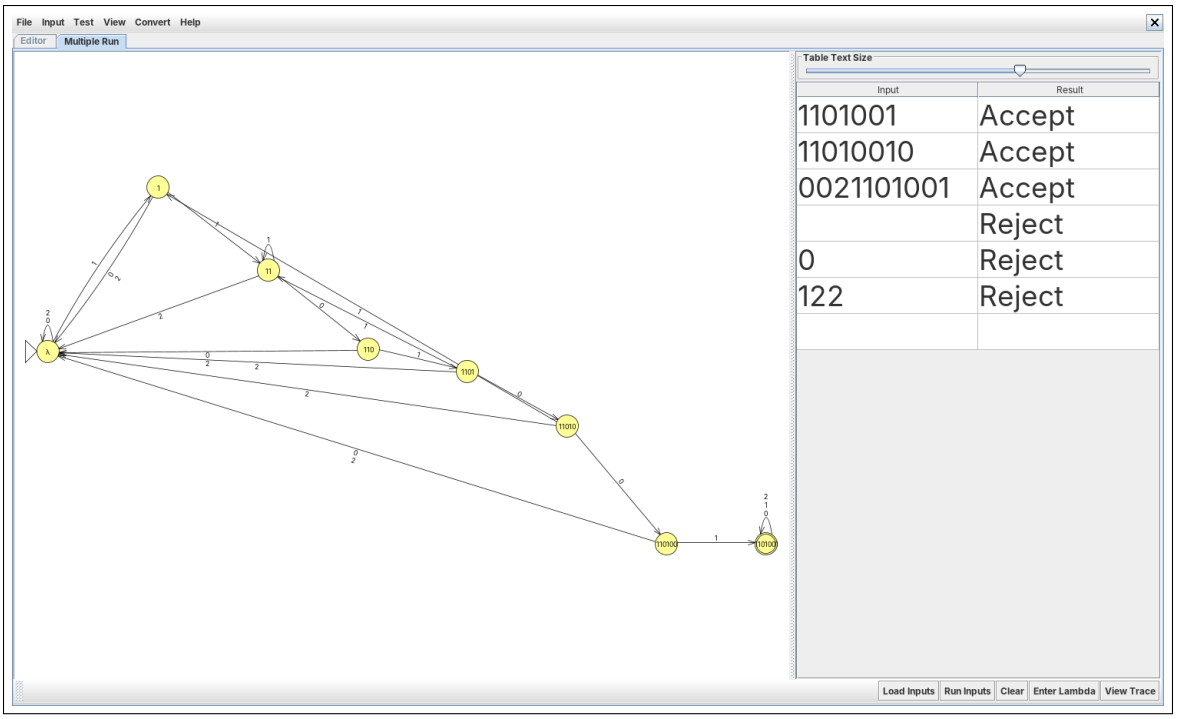


Figura 18. lista2-106 en JFLAP: Multiple Run con las tres cadenas válidas y las tres inválidas de la tabla anterior.

A.1.6. Lista 2 · Ejercicio 107

Enunciado. Las palabras de $\Sigma 1$ que incluyen al menos una vez 0101 y al menos una vez 021.

Diseño. Producto de los autómatas que buscan 0101 y 021: cada estado es un par (avance en 0101, avance en 021).

Archivo. `automatas/lista2/lista2-107.jff` · $\Sigma = \{0, 1, 2\}$ · $|Q| = 13$

Tabla 22. Tabla de transiciones ($\rightarrow$ inicial, * aceptación, — sin transición).

δ	0	1	2	Qué recuerda del prefijo leído
$\rightarrow (\lambda, \lambda)$	$(0, 0)$	(λ, λ)	(λ, λ)	en el primer autómata λ, en el segundo λ
$(0, 0)$	$(0, 0)$	$(01, \lambda)$	$(\lambda, 02)$	en el primer autómata 0, en el segundo 0
$(01, \lambda)$	$(010, 0)$	(λ, λ)	(λ, λ)	en el primer autómata 01, en el segundo λ
$(\lambda, 02)$	$(0, 0)$	$(\lambda, 021)$	(λ, λ)	en el primer autómata λ, en el segundo 02
$(010, 0)$	$(0, 0)$	$(0101, \lambda)$	$(\lambda, 02)$	en el primer autómata 010, en el segundo 0
$(\lambda, 021)$	$(0, 021)$	$(\lambda, 021)$	$(\lambda, 021)$	en el primer autómata λ, en el segundo 021
$(0101, \lambda)$	$(0101, 0)$	$(0101, \lambda)$	$(0101, \lambda)$	en el primer autómata 0101, en el segundo λ
$(0, 021)$	$(0, 021)$	$(01, 021)$	$(\lambda, 021)$	en el primer autómata 0, en el segundo 021
$(0101, 0)$	$(0101, 0)$	$(0101, \lambda)$	$(0101, 02)$	en el primer autómata 0101, en el segundo 0

δ	0	1	2	Qué recuerda del prefijo leído
(01, 021)	(010, 021)	(λ, 021)	(λ, 021)	en el primer autómata 01, en el segundo 021
(0101, 02)	(0101, 0)	(0101, 021)	(0101, λ)	en el primer autómata 0101, en el segundo 02
(010, 021)	(0, 021)	(0101, 021)	(λ, 021)	en el primer autómata 010, en el segundo 021
* (0101, 021)	(0101, 021)	(0101, 021)	(0101, 021)	en el primer autómata 0101, en el segundo 021

Tabla 23. Cadenas de prueba.

Cadena	Longitud	Esperado	Obtenido	Tipo de caso
`0101021`	7	acepta	acepta	válida
`01012021`	8	acepta	acepta	válida
`0101002120`	10	acepta	acepta	válida
λ	0	rechaza	rechaza	inválida
`1`	1	rechaza	rechaza	inválida
`010`	3	rechaza	rechaza	inválida
`0210101`	7	acepta	acepta	frontera: aceptada más corta
`111100`	6	rechaza	rechaza	frontera: justo por debajo de la longitud mínima

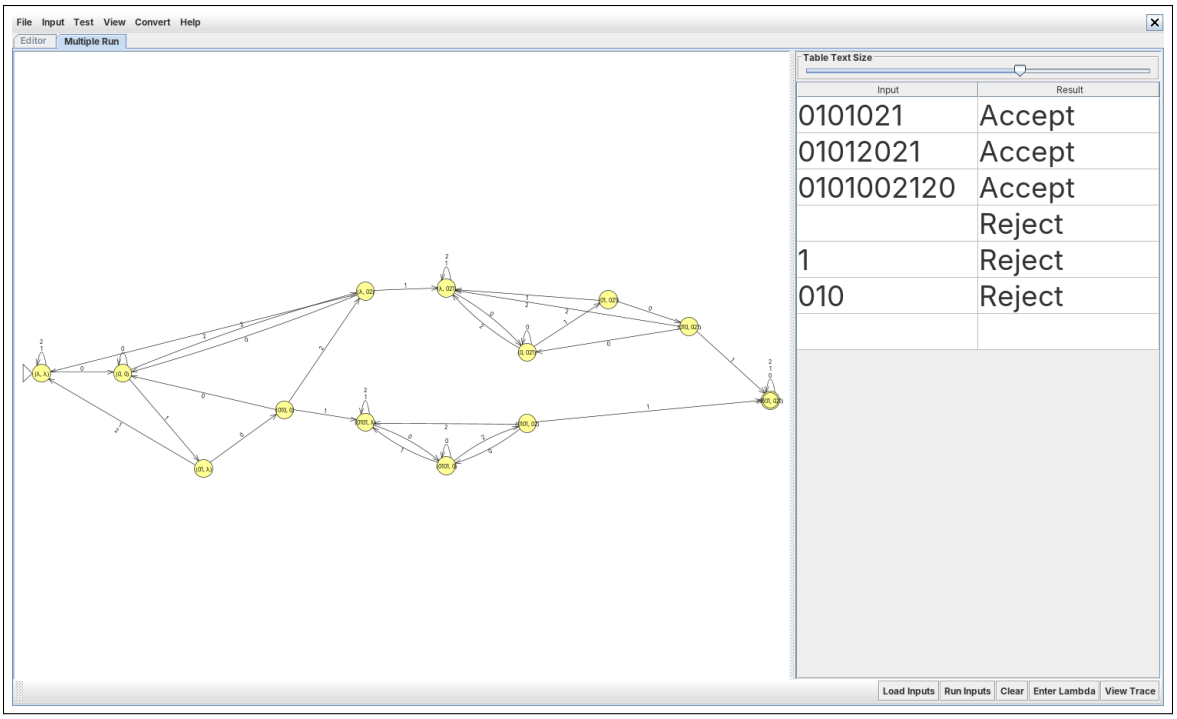


Figura 19. lista2-107 en JFLAP: Multiple Run con las tres cadenas válidas y las tres inválidas de la tabla anterior.

A.1.7. Lista 2 · Ejercicio 109

Enunciado. Las palabras de $\Sigma 2$ que incluyen abbcb y cbaabb, que sí se pueden traslapar.

Diseño. Producto de los dos buscadores de patrones: como pueden traslaparse, cada patrón se busca de forma independiente.

Archivo. `automatas/lista2/lista2-109.jff` · $\Sigma = \{0, 1, a, b, c\}$ · $|Q| = 22$

Tabla 24. Tabla de transiciones ($\rightarrow$ inicial, * aceptación, — sin transición).

δ	0	1	a	b	c	Qué recuerda del prefijo leído
$\rightarrow (\lambda, \lambda)$	(λ, λ)	(λ, λ)	(a, λ)	(λ, λ)	(λ, c)	en el primer autómata λ, en el segundo λ
(a, λ)	(λ, λ)	(λ, λ)	(a, λ)	(ab, λ)	(λ, c)	en el primer autómata a, en el segundo λ
(λ, c)	(λ, λ)	(λ, λ)	(a, λ)	(λ, cb)	(λ, c)	en el primer autómata λ, en el segundo c
(ab, λ)	(λ, λ)	(λ, λ)	(a, λ)	(abb, λ)	(λ, c)	en el primer autómata ab, en el segundo λ
(λ, cb)	(λ, λ)	(λ, λ)	(a, cba)	(λ, λ)	(λ, c)	en el primer autómata λ, en el segundo cb
(abb, λ)	(λ, λ)	(λ, λ)	(a, λ)	(λ, λ)	$(abbc, c)$	en el primer autómata abb, en el segundo λ

δ	0	1	a	b	c	Qué recuerda del prefijo leído
(a, cba)	(λ, λ)	(λ, λ)	(a, cbaa)	(ab, λ)	(λ, c)	en el primer autómata a, en el segundo cba
(abbc, c)	(λ, λ)	(λ, λ)	(a, λ)	(abbcb, cb)	(λ, c)	en el primer autómata abbc, en el segundo c
(a, cbaa)	(λ, λ)	(λ, λ)	(a, λ)	(ab, cbaab)	(λ, c)	en el primer autómata a, en el segundo cbaa
(abbcb, cb)	(abbcb, λ)	(abbcb, λ)	(abbcb, cba)	(abbcb, λ)	(abbcb, c)	en el primer autómata abbcb, en el segundo cb
(ab, cbaab)	(λ, λ)	(λ, λ)	(a, λ)	(abb, cbaabb)	(λ, c)	en el primer autómata ab, en el segundo cbaab
(abbcb, cba)	(abbcb, λ)	(abbcb, λ)	(abbcb, cbaa)	(abbcb, λ)	(abbcb, c)	en el primer autómata abbcb, en el segundo cba
(abbcb, λ)	(abbcb, λ)	(abbcb, λ)	(abbcb, λ)	(abbcb, λ)	(abbcb, c)	en el primer autómata abbcb, en el segundo λ
(abbcb, c)	(abbcb, λ)	(abbcb, λ)	(abbcb, λ)	(abbcb, cb)	(abbcb, c)	en el primer autómata abbcb, en el segundo c
(abb, cbaabb)	(λ, cbaabb)	(λ, cbaabb)	(a, cbaabb)	(λ, cbaabb)	(abbc, cbaabb)	en el primer autómata abb, en el segundo cbaabb
(abbcb, cbaa)	(abbcb, λ)	(abbcb, λ)	(abbcb, λ)	(abbcb, cbaab)	(abbcb, c)	en el primer autómata abbcb, en el segundo cbaa
(a, cbaabb)	(λ, cbaabb)	(λ, cbaabb)	(a, cbaabb)	(ab, cbaabb)	(λ, cbaabb)	en el primer autómata a, en el segundo cbaabb
(λ, cbaabb)	(λ, cbaabb)	(λ, cbaabb)	(a, cbaabb)	(λ, cbaabb)	(λ, cbaabb)	en el primer autómata λ, en el segundo cbaabb
(abbc, cbaabb)	(λ, cbaabb)	(λ, cbaabb)	(a, cbaabb)	(abbcb, cbaabb)	(λ, cbaabb)	en el primer autómata abbc, en el segundo cbaabb
(abbcb, cbaab)	(abbcb, λ)	(abbcb, λ)	(abbcb, λ)	(abbcb, cbaabb)	(abbcb, c)	en el primer autómata abbcb, en el segundo cbaab
(ab, cbaabb)	(λ, cbaabb)	(λ, cbaabb)	(a, cbaabb)	(abb, cbaabb)	(λ, cbaabb)	en el primer autómata ab, en el segundo cbaabb
* (abbcb, cbaabb)	(abbcb, cbaabb)	(abbcb, cbaabb)	(abbcb, cbaabb)	(abbcb, cbaabb)	(abbcb, cbaabb)	en el primer autómata abbcb, en el segundo cbaabb

Tabla 25. Cadenas de prueba.

Cadena	Longitud	Esperado	Obtenido	Tipo de caso
`cbaabbcb`	8	acepta	acepta	válida
`0cbaabbcb`	9	acepta	acepta	válida
`acacbaabbcb`	11	acepta	acepta	válida

Cadena	Longitud	Esperado	Obtenido	Tipo de caso
λ	0	rechaza	rechaza	inválida
`1`	1	rechaza	rechaza	inválida
`b0c`	3	rechaza	rechaza	inválida
`cbbcccb`	7	rechaza	rechaza	frontera: justo por debajo de la longitud mínima

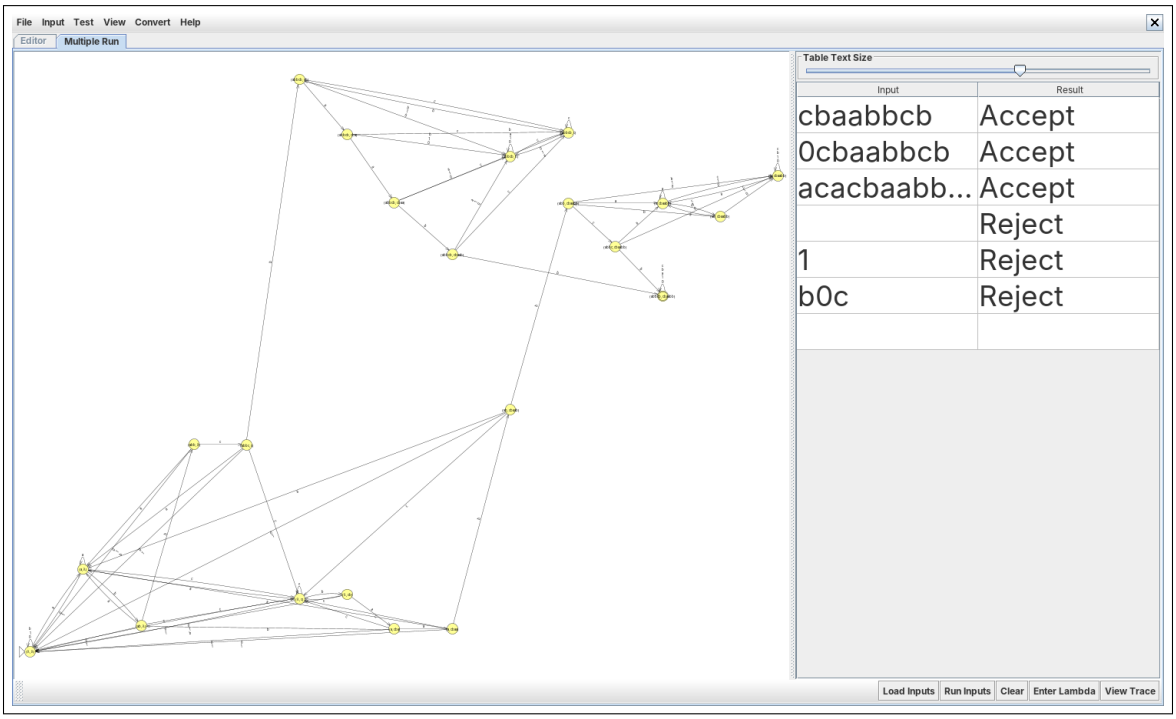


Figura 20. lista2-109 en JFLAP: Multiple Run con las tres cadenas válidas y las tres inválidas de la tabla anterior.

A.1.8. Lista 2 · Ejercicio 111

Enunciado. Las palabras de $\Sigma 1$ que contienen exactamente tres veces la subcadena 01002.

Diseño. Cada estado combina el número de apariciones de 01002 (0 a 3, y muerto si pasa de 3) con el avance en el patrón; 01002 no puede traslaparse consigo mismo.

Archivo. `automatas/lista2/lista2-111.jff` · $\Sigma = \{0, 1, 2\}$ · $|Q| = 21$

Tabla 26. Tabla de transiciones ($\rightarrow$ inicial, * aceptación, — sin transición).

δ	0	1	2	Qué recuerda del prefijo leído
$\rightarrow 0{\cdot}\lambda$	$0{\cdot}0$	$0{\cdot}\lambda$	$0{\cdot}\lambda$	van 0 apariciones; el sufijo útil leído es λ
$0{\cdot}0$	$0{\cdot}0$	$0{\cdot}01$	$0{\cdot}\lambda$	van 0 apariciones; el sufijo útil leído es 0
$0{\cdot}01$	$0{\cdot}010$	$0{\cdot}\lambda$	$0{\cdot}\lambda$	van 0 apariciones; el sufijo útil leído es 01

δ	0	1	2	Qué recuerda del prefijo leído
0·010	0·0100	0·01	0·λ	van 0 apariciones; el sufijo útil leído es 010
0·0100	0·0	0·01	1·λ	van 0 apariciones; el sufijo útil leído es 0100
1·λ	1·0	1·λ	1·λ	van 1 apariciones; el sufijo útil leído es λ
1·0	1·0	1·01	1·λ	van 1 apariciones; el sufijo útil leído es 0
1·01	1·010	1·λ	1·λ	van 1 apariciones; el sufijo útil leído es 01
1·010	1·0100	1·01	1·λ	van 1 apariciones; el sufijo útil leído es 010
1·0100	1·0	1·01	2·λ	van 1 apariciones; el sufijo útil leído es 0100
2·λ	2·0	2·λ	2·λ	van 2 apariciones; el sufijo útil leído es λ
2·0	2·0	2·01	2·λ	van 2 apariciones; el sufijo útil leído es 0
2·01	2·010	2·λ	2·λ	van 2 apariciones; el sufijo útil leído es 01
2·010	2·0100	2·01	2·λ	van 2 apariciones; el sufijo útil leído es 010
2·0100	2·0	2·01	3·λ	van 2 apariciones; el sufijo útil leído es 0100
* 3·λ	3·0	3·λ	3·λ	van 3 apariciones; el sufijo útil leído es λ
* 3·0	3·0	3·01	3·λ	van 3 apariciones; el sufijo útil leído es 0
* 3·01	3·010	3·λ	3·λ	van 3 apariciones; el sufijo útil leído es 01
* 3·010	3·0100	3·01	3·λ	van 3 apariciones; el sufijo útil leído es 010
* 3·0100	3·0	3·01	muerto	van 3 apariciones; el sufijo útil leído es 0100
muerto	muerto	muerto	muerto	ya hay más de 3 apariciones de 01002

Tabla 27. Cadenas de prueba.

Cadena	Longitud	Esperado	Obtenido	Tipo de caso
`010020100201002`	15	acepta	acepta	válida
`0100201002001002`	16	acepta	acepta	válida
`010020010020101002`	18	acepta	acepta	válida
λ	0	rechaza	rechaza	inválida
`0`	1	rechaza	rechaza	inválida
`111`	3	rechaza	rechaza	inválida
`01122210022010`	14	rechaza	rechaza	frontera: justo por debajo de la longitud mínima

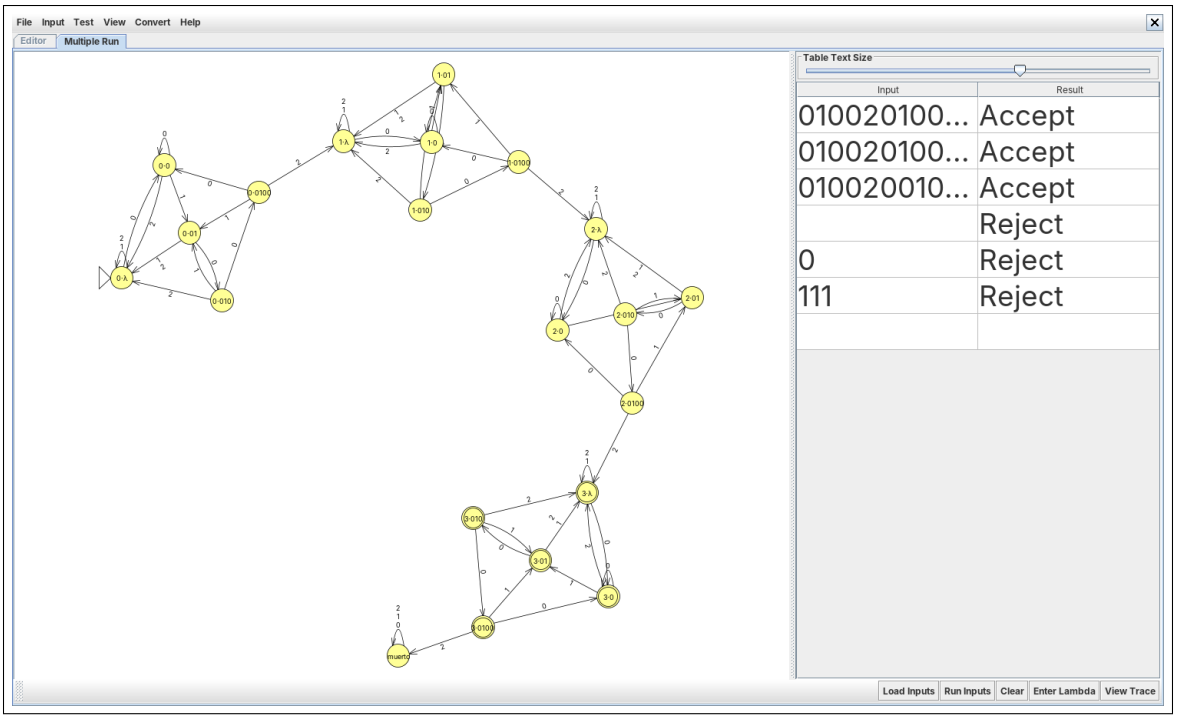


Figura 21. lista2-111 en JFLAP: Multiple Run con las tres cadenas válidas y las tres inválidas de la tabla anterior.

A.1.9. Lista 2 · Ejercicio 112

Enunciado. Las palabras de $\Sigma 1$ que contienen dos veces 010 sin traslaparse (01010 no pertenece, 010010 sí).

Diseño. Se interpreta «al menos dos apariciones disjuntas». Tras completar 010 la búsqueda reinicia desde λ, de modo que la siguiente aparición no reutiliza el último 0.

Archivo. `automatas/lista2/lista2-112.jff` · $\Sigma = \{0, 1, 2\}$ · $|Q| = 7$

Tabla 28. Tabla de transiciones ($\rightarrow$ inicial, * aceptación, — sin transición).

δ	0	1	2	Qué recuerda del prefijo leído
$\rightarrow 0{\cdot}\lambda$	$0{\cdot}0$	$0{\cdot}\lambda$	$0{\cdot}\lambda$	van 0 apariciones; el sufijo útil leído es λ
$0{\cdot}0$	$0{\cdot}0$	$0{\cdot}01$	$0{\cdot}\lambda$	van 0 apariciones; el sufijo útil leído es 0
$0{\cdot}01$	$1{\cdot}\lambda$	$0{\cdot}\lambda$	$0{\cdot}\lambda$	van 0 apariciones; el sufijo útil leído es 01
$1{\cdot}\lambda$	$1{\cdot}0$	$1{\cdot}\lambda$	$1{\cdot}\lambda$	van 1 apariciones; el sufijo útil leído es λ
$1{\cdot}0$	$1{\cdot}0$	$1{\cdot}01$	$1{\cdot}\lambda$	van 1 apariciones; el sufijo útil leído es 0
$1{\cdot}01$	2+	$1{\cdot}\lambda$	$1{\cdot}\lambda$	van 1 apariciones; el sufijo útil leído es 01
* 2+	2+	2+	2+	ya hay 2 apariciones de 010

Tabla 29. Cadenas de prueba.

Cadena	Longitud	Esperado	Obtenido	Tipo de caso
`010010`	6	acepta	acepta	válida
`0100010`	7	acepta	acepta	válida
`001001012`	9	acepta	acepta	válida
λ	0	rechaza	rechaza	inválida
`2`	1	rechaza	rechaza	inválida
`122`	3	rechaza	rechaza	inválida
`20101`	5	rechaza	rechaza	frontera: justo por debajo de la longitud mínima

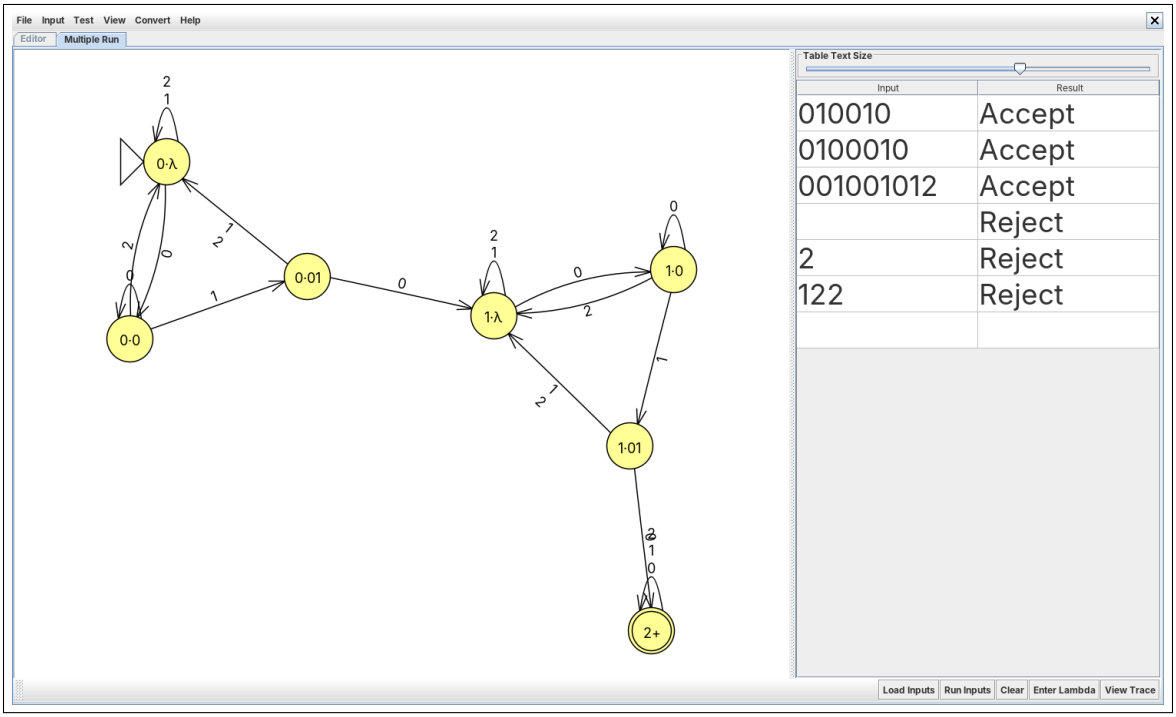


Figura 22. lista2-112 en JFLAP: Multiple Run con las tres cadenas válidas y las tres inválidas de la tabla anterior.

A.1.10. Lista 2 · Ejercicio 113

Enunciado. Las palabras de $\Sigma 1$ que contienen dos veces 010, que sí se pueden traslapar.

Diseño. Igual que 112, pero tras completar 010 el autómata conserva el borde 0, que puede iniciar la siguiente aparición (01010).

Archivo. `automatas/lista2/lista2-113.jff` · $\Sigma = \{0, 1, 2\}$ · $|Q| = 7$

Tabla 30. Tabla de transiciones ($\rightarrow$ inicial, * aceptación, — sin transición).

δ	0	1	2	Qué recuerda del prefijo leído
$\rightarrow 0{\cdot}\lambda$	$0{\cdot}0$	$0{\cdot}\lambda$	$0{\cdot}\lambda$	van 0 apariciones; el sufijo útil leído es λ
$0{\cdot}0$	$0{\cdot}0$	$0{\cdot}01$	$0{\cdot}\lambda$	van 0 apariciones; el sufijo útil leído es 0
$0{\cdot}01$	$1{\cdot}0$	$0{\cdot}\lambda$	$0{\cdot}\lambda$	van 0 apariciones; el sufijo útil leído es 01
$1{\cdot}0$	$1{\cdot}0$	$1{\cdot}01$	$1{\cdot}\lambda$	van 1 apariciones; el sufijo útil leído es 0
$1{\cdot}01$	2+	$1{\cdot}\lambda$	$1{\cdot}\lambda$	van 1 apariciones; el sufijo útil leído es 01
$1{\cdot}\lambda$	$1{\cdot}0$	$1{\cdot}\lambda$	$1{\cdot}\lambda$	van 1 apariciones; el sufijo útil leído es λ
* 2+	2+	2+	2+	ya hay 2 apariciones de 010

Tabla 31. Cadenas de prueba.

Cadena	Longitud	Esperado	Obtenido	Tipo de caso
`01010`	5	acepta	acepta	válida
`010010`	6	acepta	acepta	válida
`01010101`	8	acepta	acepta	válida
λ	0	rechaza	rechaza	inválida
`1`	1	rechaza	rechaza	inválida
`012`	3	rechaza	rechaza	inválida
`0100`	4	rechaza	rechaza	frontera: justo por debajo de la longitud mínima

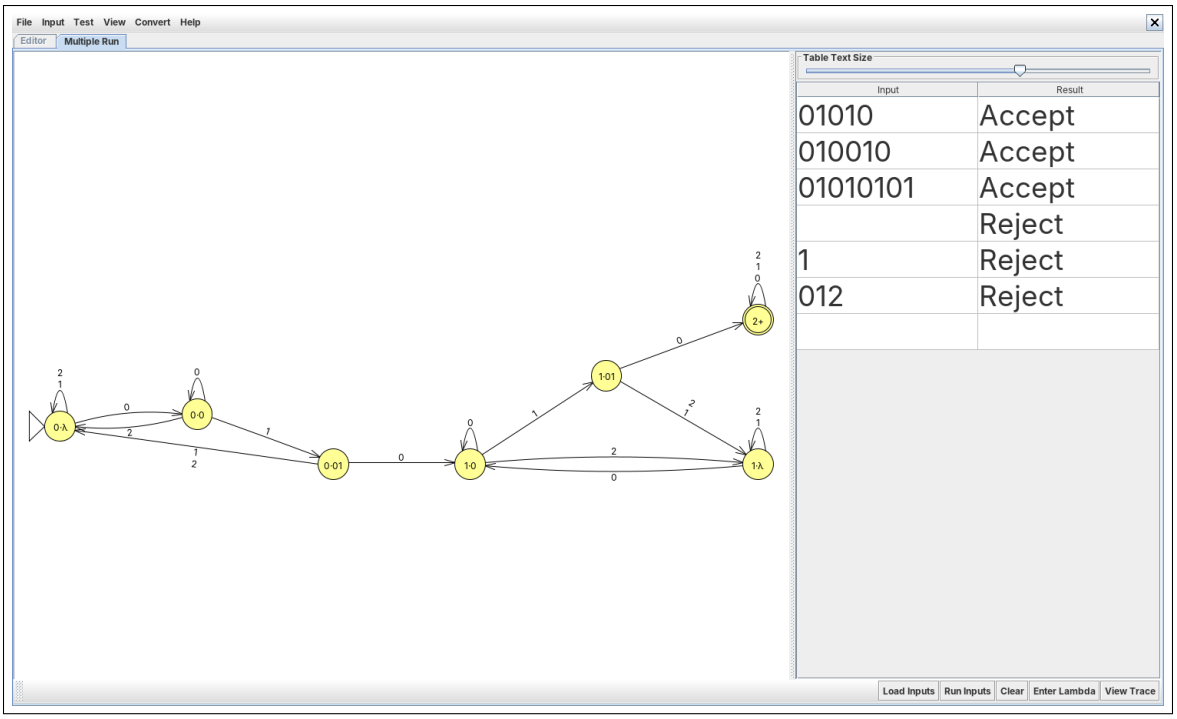


Figura 23. lista2-113 en JFLAP: Multiple Run con las tres cadenas válidas y las tres inválidas de la tabla anterior.

A.1.11. Lista 2 · Ejercicio 116

Enunciado. L = {w ∈ Σ2* : w = $a^n b^m c^{n+1}$, m, n ∈ N}.

El lenguaje no es regular (lema de bombeo); no existe AFD.

A.1.12. Lista 2 · Ejercicio 118

Enunciado. L = {w ∈ Σ3* : w = axabya, x, y ∈ Σ3*}.

Diseño. La primera a no puede ser la del ab interior (debe haber una a inicial propia), y la a final debe ir después de la b.

Archivo. `automatas/lista2/lista2-118.jff` · Σ = {a, b} · |Q| = 6

Tabla 32. Tabla de transiciones (→ inicial, * aceptación, — sin transición).

δ	a	b	Qué recuerda del prefijo leído
→ ini	a	muerto	no se ha leído nada
a	a·a	a	empieza con a; aún no hay una a utilizable para el ab interior
muerto	muerto	muerto	la cadena no empieza con a
a·a	a·a	ab	el último símbolo es una a que puede iniciar el ab interior
ab	ab·a	ab	ya apareció el ab interior; el último símbolo no es a
* ab·a	ab·a	ab	ya apareció el ab interior y el último símbolo es a

Tabla 33. Cadenas de prueba.

Cadena	Longitud	Esperado	Obtenido	Tipo de caso
`aaba`	4	acepta	acepta	válida
`aabba`	5	acepta	acepta	válida
`aabbaaa`	7	acepta	acepta	válida
λ	0	rechaza	rechaza	inválida
`a`	1	rechaza	rechaza	inválida
`bbb`	3	rechaza	rechaza	inválida
`bab`	3	rechaza	rechaza	frontera: justo por debajo de la longitud mínima

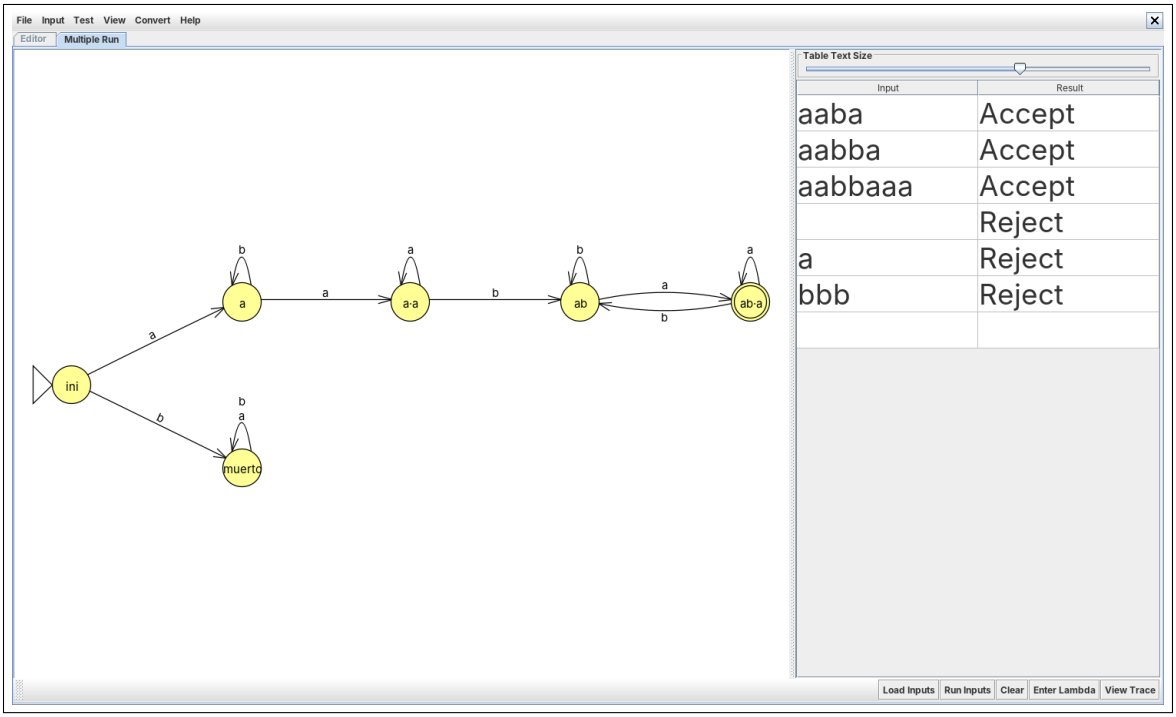


Figura 24. lista2-118 en JFLAP: Multiple Run con las tres cadenas válidas y las tres inválidas de la tabla anterior.

A.1.13. Lista 2 · Ejercicio 120

Enunciado. Las palabras de $\Sigma 1$ tales que el primer carácter y el último son distintos.
Diseño. Se recuerda el primer símbolo y si el último coincide con él.
Archivo. `automatas/lista2/lista2-120.jff` · $\Sigma = \{0, 1, 2\}$ · $|Q| = 7$

Tabla 34. Tabla de transiciones (→ inicial, * aceptación, — sin transición).

δ	0	1	2	Qué recuerda del prefijo leído
→ ini	0…=	1…=	2…=	no se ha leído nada
0…=	0…=	0…≠	0…≠	empieza con 0 y el último símbolo es igual
1…=	1…≠	1…=	1…≠	empieza con 1 y el último símbolo es igual
2…=	2…≠	2…≠	2…=	empieza con 2 y el último símbolo es igual
* 0…≠	0…=	0…≠	0…≠	empieza con 0 y el último símbolo es distinto
* 1…≠	1…≠	1…=	1…≠	empieza con 1 y el último símbolo es distinto
* 2…≠	2…≠	2…≠	2…=	empieza con 2 y el último símbolo es distinto

Tabla 35. Cadenas de prueba.

Cadena	Longitud	Esperado	Obtenido	Tipo de caso
`21`	2	acepta	acepta	válida
`012`	3	acepta	acepta	válida
`20001`	5	acepta	acepta	válida
λ	0	rechaza	rechaza	inválida
`0`	1	rechaza	rechaza	inválida
`222`	3	rechaza	rechaza	inválida
`01`	2	acepta	acepta	frontera: aceptada más corta
`1`	1	rechaza	rechaza	frontera: justo por debajo de la longitud mínima

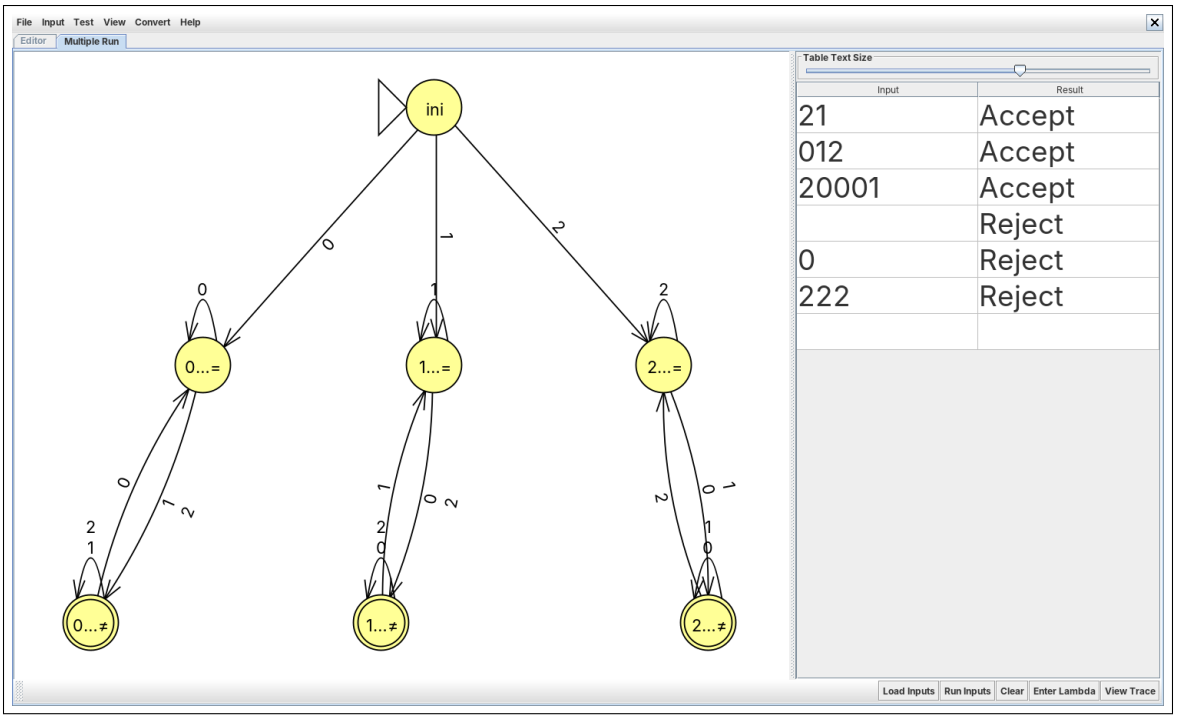


Figura 25. lista2-120 en JFLAP: Multiple Run con las tres cadenas válidas y las tres inválidas de la tabla anterior.

A.1.14. Lista 2 · Ejercicio 121

Enunciado. Las palabras de $\Sigma 1$ de longitud 3n tales que cada bloque de longitud 3 tiene al menos un 0.
Diseño. Se lleva la posición dentro del bloque y si el bloque ya contiene un 0. λ (n = 0) se acepta.
Archivo. `automatas/lista2/lista2-121.jff` · $\Sigma = \{0, 1, 2\}$ · $|Q| = 6$

Tabla 36. Tabla de transiciones ($\rightarrow$ inicial, * aceptación, — sin transición).

δ	0	1	2	Qué recuerda del prefijo leído
$\rightarrow$* b0	b1+0	b1	b1	posición 0 dentro del bloque; aún no hay un 0 en el bloque
b1+0	b2+0	b2+0	b2+0	posición 1 dentro del bloque; ya hay un 0 en el bloque
b1	b2+0	b2	b2	posición 1 dentro del bloque; aún no hay un 0 en el bloque
b2+0	b0	b0	b0	posición 2 dentro del bloque; ya hay un 0 en el bloque
b2	b0	muerto	muerto	posición 2 dentro del bloque; aún no hay un 0 en el bloque
muerto	muerto	muerto	muerto	un bloque terminó sin 0

Tabla 37. Cadenas de prueba.

Cadena	Longitud	Esperado	Obtenido	Tipo de caso
λ	0	acepta	acepta	válida
`011`	3	acepta	acepta	válida
`012220`	6	acepta	acepta	válida
`2`	1	rechaza	rechaza	inválida
`02`	2	rechaza	rechaza	inválida
`0202`	4	rechaza	rechaza	inválida

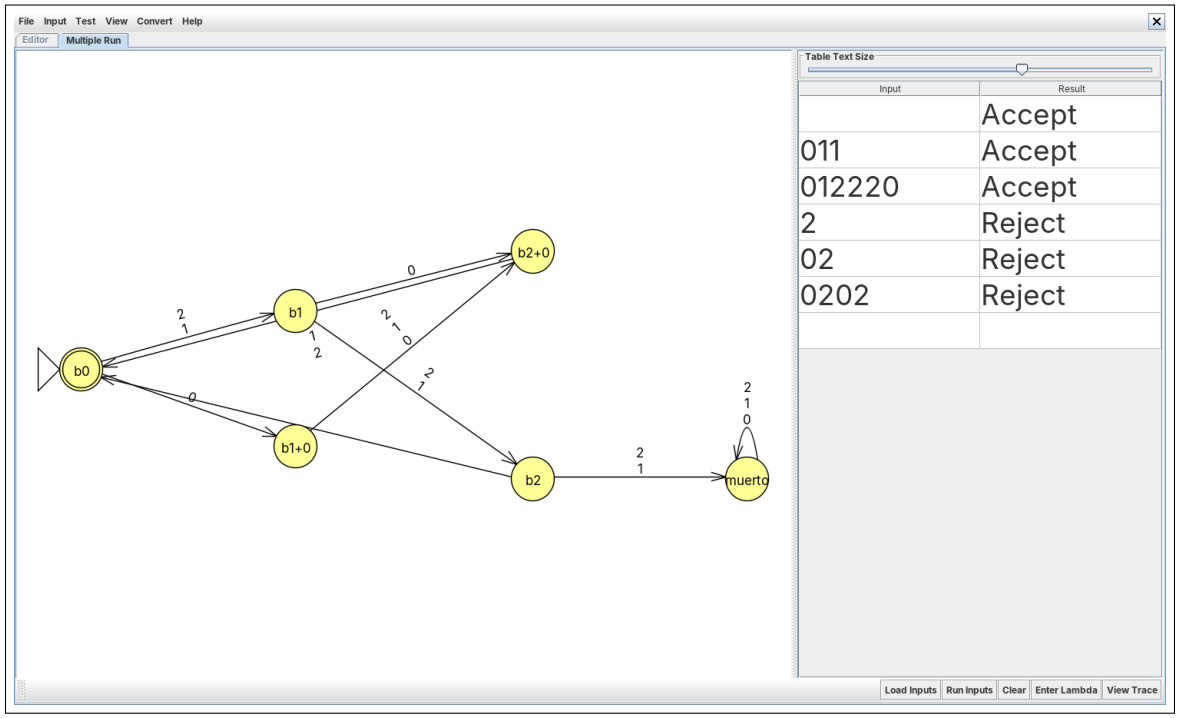


Figura 26. lista2-121 en JFLAP: Multiple Run con las tres cadenas válidas y las tres inválidas de la tabla anterior.

A.1.15. Lista 2 · Ejercicio 122

Enunciado. Las palabras de $\Sigma 1$ tales que cualquier cadena de tres símbolos consecutivos tiene un 0.
Diseño. Basta contar cuántos símbolos distintos de 0 hay al final; tres seguidos invalidan la cadena.
Archivo. `automatas/lista2/lista2-122.jff` · $\Sigma = \{0, 1, 2\}$ · $|Q| = 4$

Tabla 38. Tabla de transiciones ($\rightarrow$ inicial, * aceptación, — sin transición).

δ	0	1	2	Qué recuerda del prefijo leído
$\rightarrow$* 0 sin 0	0 sin 0	1 sin 0	1 sin 0	los últimos 0 símbolos son distintos de 0
* 1 sin 0	0 sin 0	2 sin 0	2 sin 0	los últimos 1 símbolos son distintos de 0

δ	0	1	2	Qué recuerda del prefijo leído
* 2 sin 0	0 sin 0	muerto	muerto	los últimos 2 símbolos son distintos de 0
muerto	muerto	muerto	muerto	hubo tres símbolos seguidos sin 0

Tabla 39. Cadenas de prueba.

Cadena	Longitud	Esperado	Obtenido	Tipo de caso
λ	0	acepta	acepta	válida
`1`	1	acepta	acepta	válida
`100`	3	acepta	acepta	válida
`112`	3	rechaza	rechaza	inválida
`0122`	4	rechaza	rechaza	inválida
`122010`	6	rechaza	rechaza	inválida

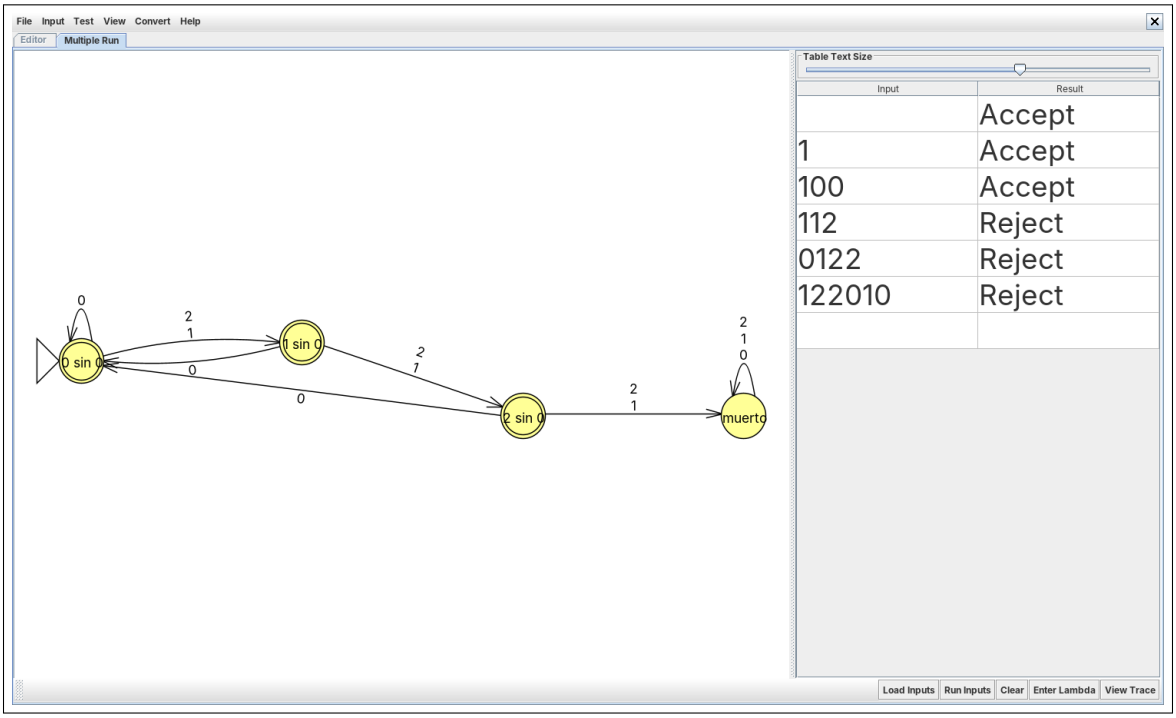


Figura 27. lista2-122 en JFLAP: Multiple Run con las tres cadenas válidas y las tres inválidas de la tabla anterior.

A.1.16. Lista 2 · Ejercicio 123

Enunciado. Las palabras de $\Sigma 3$ de la forma wxw donde w $\in \Sigma 3^2$ y x $\in \Sigma 3^*$.

Diseño. Se memoriza w (los dos primeros símbolos) y, para cada w, tres situaciones: la cadena termina en w, el último símbolo es la primera letra de w, u otra. Tras los dos primeros símbolos el estado es «otra» aunque la cadena termine en w, porque las dos copias de w no pueden traslaparse (se exige longitud ≥ 4).

Archivo. `automatas/lista2/lista2-123.jff` · $\Sigma = \{a, b\}$ · $|Q| = 15$

Tabla 40. Tabla de transiciones (→ inicial, * aceptación, — sin transición).

δ	a	b	Qué recuerda del prefijo leído
→ ini	a	b	no se ha leído nada
a	aa:∅	ab:∅	el primer símbolo es a
b	ba:∅	bb:∅	el primer símbolo es b
aa:∅	aa:a…	aa:∅	w = aa; el final leído no sirve para la copia final de w (o la longitud aún es 2)
ab:∅	ab:a…	ab:∅	w = ab; el final leído no sirve para la copia final de w (o la longitud aún es 2)
ba:∅	ba:∅	ba:b…	w = ba; el final leído no sirve para la copia final de w (o la longitud aún es 2)
bb:∅	bb:∅	bb:b…	w = bb; el final leído no sirve para la copia final de w (o la longitud aún es 2)
aa:a…	aa:aa	aa:∅	w = aa; el último símbolo es a, que puede iniciar la copia final de w
ab:a…	ab:a…	ab:ab	w = ab; el último símbolo es a, que puede iniciar la copia final de w
ba:b…	ba:ba	ba:b…	w = ba; el último símbolo es b, que puede iniciar la copia final de w
bb:b…	bb:∅	bb:bb	w = bb; el último símbolo es b, que puede iniciar la copia final de w
* aa:aa	aa:aa	aa:∅	w = aa y la cadena (de longitud ≥ 4) termina en aa
* ab:ab	ab:a…	ab:∅	w = ab y la cadena (de longitud ≥ 4) termina en ab
* ba:ba	ba:∅	ba:b…	w = ba y la cadena (de longitud ≥ 4) termina en ba
* bb:bb	bb:∅	bb:bb	w = bb y la cadena (de longitud ≥ 4) termina en bb

Tabla 41. Cadenas de prueba.

Cadena	Longitud	Esperado	Obtenido	Tipo de caso
`bbbb`	4	acepta	acepta	válida
`abbab`	5	acepta	acepta	válida
`bbbbabb`	7	acepta	acepta	válida
λ	0	rechaza	rechaza	inválida
`a`	1	rechaza	rechaza	inválida
`bab`	3	rechaza	rechaza	inválida
`abab`	4	acepta	acepta	frontera: aceptada más corta

Cadena	Longitud	Esperado	Obtenido	Tipo de caso
`aaa`	3	rechaza	rechaza	frontera: justo por debajo de la longitud mínima

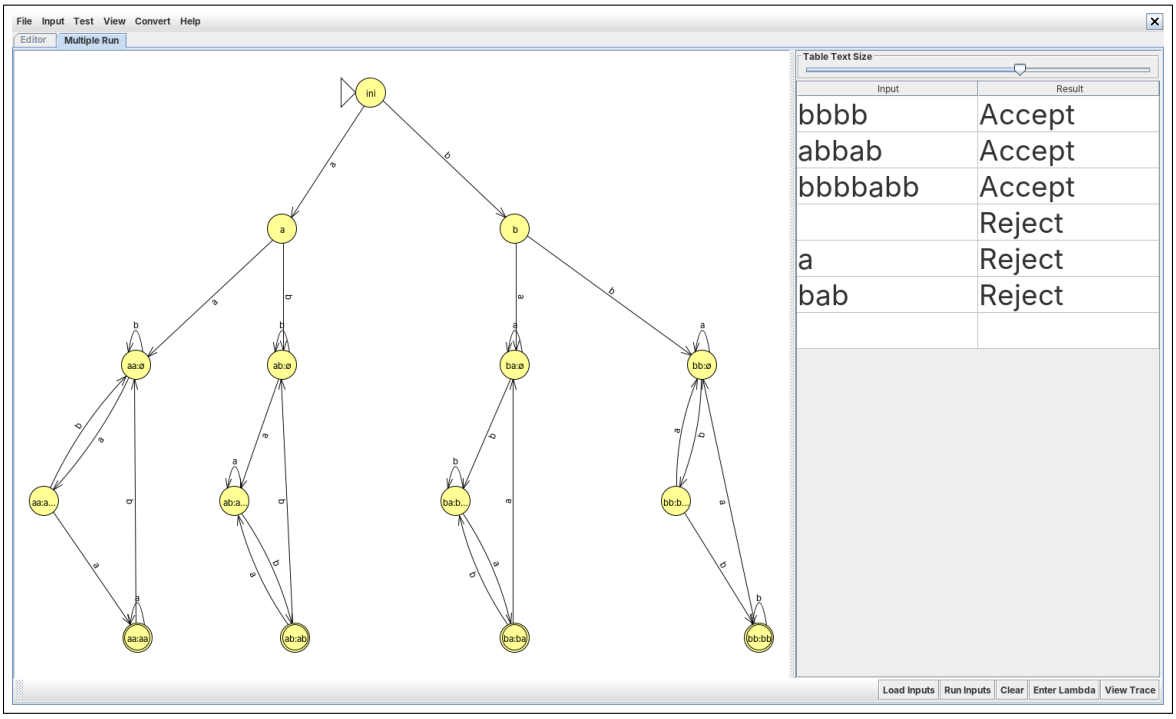


Figura 28. lista2-123 en JFLAP: Multiple Run con las tres cadenas válidas y las tres inválidas de la tabla anterior.

A.1.17. Lista 2 · Ejercicio 124

Enunciado. Mecanismo de tres palancas con canicas ($\Sigma = \{0, 1\}$); se acepta si la última canica sale por A.

Diseño. Ver el análisis del mecanismo.

Archivo. `automatas/lista2/lista2-124.jff` · $\Sigma = \{0, 1\}$ · $|Q| = 13$

Tabla 42. Tabla de transiciones ($\rightarrow$ inicial, * aceptación, — sin transición).

δ	0	1	Qué recuerda del prefijo leído
$\rightarrow$ III	DIIa	IDDa	palancas izquierda=I, derecha=I, central=I (I: desvía a la izquierda, D: a la derecha); aún no cae ninguna canica
* DIIa	IIDa	DDDa	palancas izquierda=D, derecha=I, central=I (I: desvía a la izquierda, D: a la derecha); la última canica salió por A

δ	0	1	Qué recuerda del prefijo leído
* IDDa	DDDa	IIDr	palancas izquierda=I, derecha=D, central=D (I: desvía a la izquierda, D: a la derecha); la última canica salió por A
* IIDa	DIDa	IDIr	palancas izquierda=I, derecha=I, central=D (I: desvía a la izquierda, D: a la derecha); la última canica salió por A
* DDDa	IDIr	DIDr	palancas izquierda=D, derecha=D, central=D (I: desvía a la izquierda, D: a la derecha); la última canica salió por A
IIDr	DIDa	IDIr	palancas izquierda=I, derecha=I, central=D (I: desvía a la izquierda, D: a la derecha); la última canica salió por R
* DIDa	IIIr	DDIr	palancas izquierda=D, derecha=I, central=D (I: desvía a la izquierda, D: a la derecha); la última canica salió por A
IDIr	DDIa	IIIr	palancas izquierda=I, derecha=D, central=I (I: desvía a la izquierda, D: a la derecha); la última canica salió por R
DIDr	IIIr	DDIr	palancas izquierda=D, derecha=I, central=D (I: desvía a la izquierda, D: a la derecha); la última canica salió por R
IIIr	DIIa	IDDa	palancas izquierda=I, derecha=I, central=I (I: desvía a la izquierda, D: a la derecha); la última canica salió por R
DDIr	IDDa	DIIr	palancas izquierda=D, derecha=D, central=I (I: desvía a la izquierda, D: a la derecha); la última canica salió por R
* DDIa	IDDa	DIIr	palancas izquierda=D, derecha=D, central=I (I: desvía a la izquierda, D: a la derecha); la última canica salió por A
DIIr	IIDa	DDDa	palancas izquierda=D, derecha=I, central=I (I: desvía a la izquierda, D: a la derecha); la última canica salió por R

Tabla 43. Cadenas de prueba.

Cadena	Longitud	Esperado	Obtenido	Tipo de caso
`0`	1	acepta	acepta	válida
`01`	2	acepta	acepta	válida
`0010`	4	acepta	acepta	válida

Cadena	Longitud	Esperado	Obtenido	Tipo de caso
λ	0	rechaza	rechaza	inválida
`1010`	4	rechaza	rechaza	inválida
`00101`	5	rechaza	rechaza	inválida

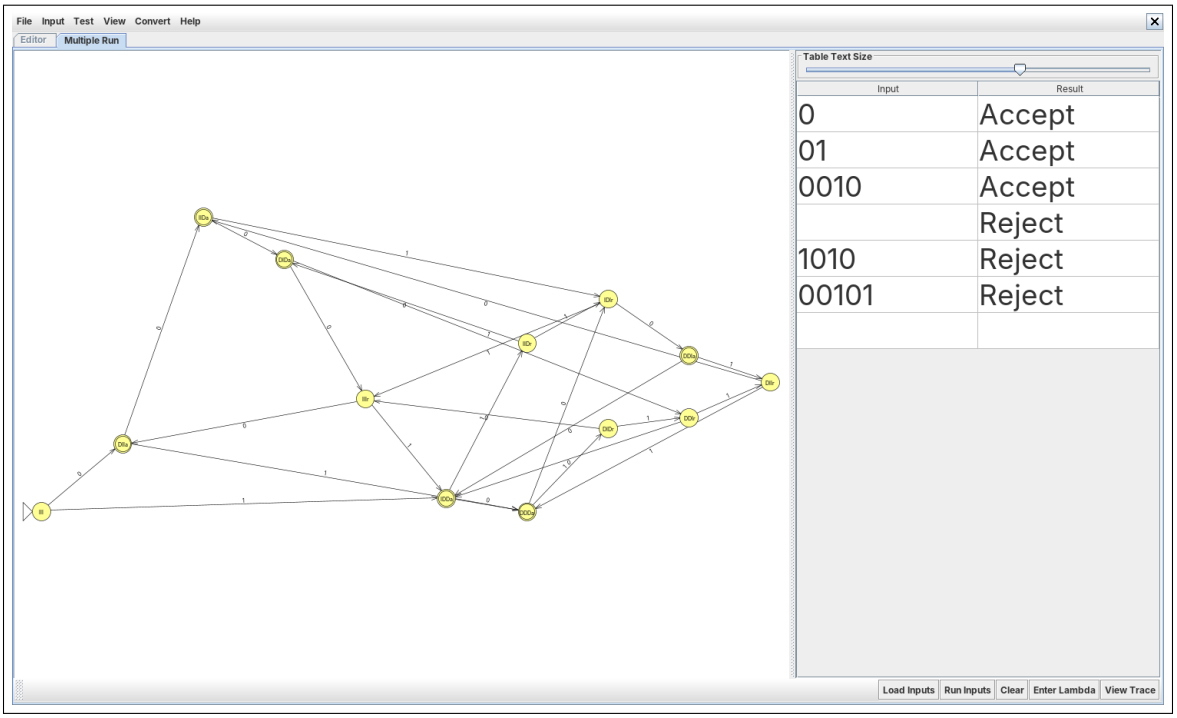


Figura 29. lista2-124 en JFLAP: Multiple Run con las tres cadenas válidas y las tres inválidas de la tabla anterior.

A.1.18. Lista 2 · Ejercicio 125

Enunciado. Sea A = (Q, Σ, δ, A0, F) un AFD. ¿Cuál es el lenguaje de A' = (Q, Σ, δ, A0, Q F)?
Diseño. Como ejemplo se complementa el AFD de 139 (número par de ceros).
Archivo. `automatas/lista2/lista2-125.jff` · Σ = {0, 1} · |Q| = 2

Tabla 44. Tabla de transiciones (→ inicial, * aceptación, — sin transición).

δ	0	1	Qué recuerda del prefijo leído
→ par	impar	par	se ha leído un número par de ceros
* impar	par	impar	se ha leído un número impar de ceros

Tabla 45. Cadenas de prueba.

Cadena	Longitud	Esperado	Obtenido	Tipo de caso
`0`	1	acepta	acepta	válida
`01`	2	acepta	acepta	válida
`1101`	4	acepta	acepta	válida
λ	0	rechaza	rechaza	inválida
`1`	1	rechaza	rechaza	inválida
`100`	3	rechaza	rechaza	inválida

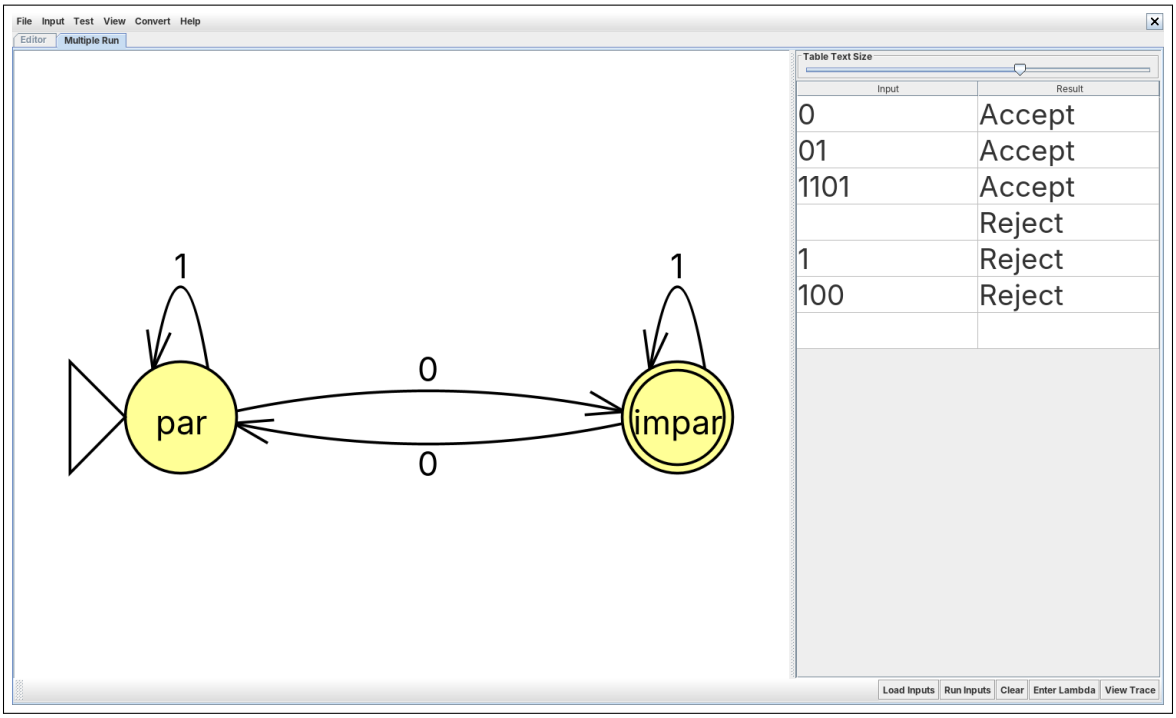


Figura 30. lista2-125 en JFLAP: Multiple Run con las tres cadenas válidas y las tres inválidas de la tabla anterior.

A.1.19. Lista 2 · Ejercicio 126

Enunciado. Sea A un AFD. ¿Cuál es el lenguaje del autómata si F = $\varnothing$?
Diseño. Ejemplo: el AFD de 139 con F = $\varnothing$.
Archivo. `automatas/lista2/lista2-126.jff` · $\Sigma = \{0, 1\}$ · $|Q| = 2$

Tabla 46. Tabla de transiciones ($\rightarrow$ inicial, * aceptación, — sin transición).

δ	0	1	Qué recuerda del prefijo leído
$\rightarrow$ par	impar	par	número par de ceros
impar	par	impar	número impar de ceros

Tabla 47. Cadenas de prueba.

Cadena	Longitud	Esperado	Obtenido	Tipo de caso
λ	0	rechaza	rechaza	inválida
`1`	1	rechaza	rechaza	inválida
`001`	3	rechaza	rechaza	inválida

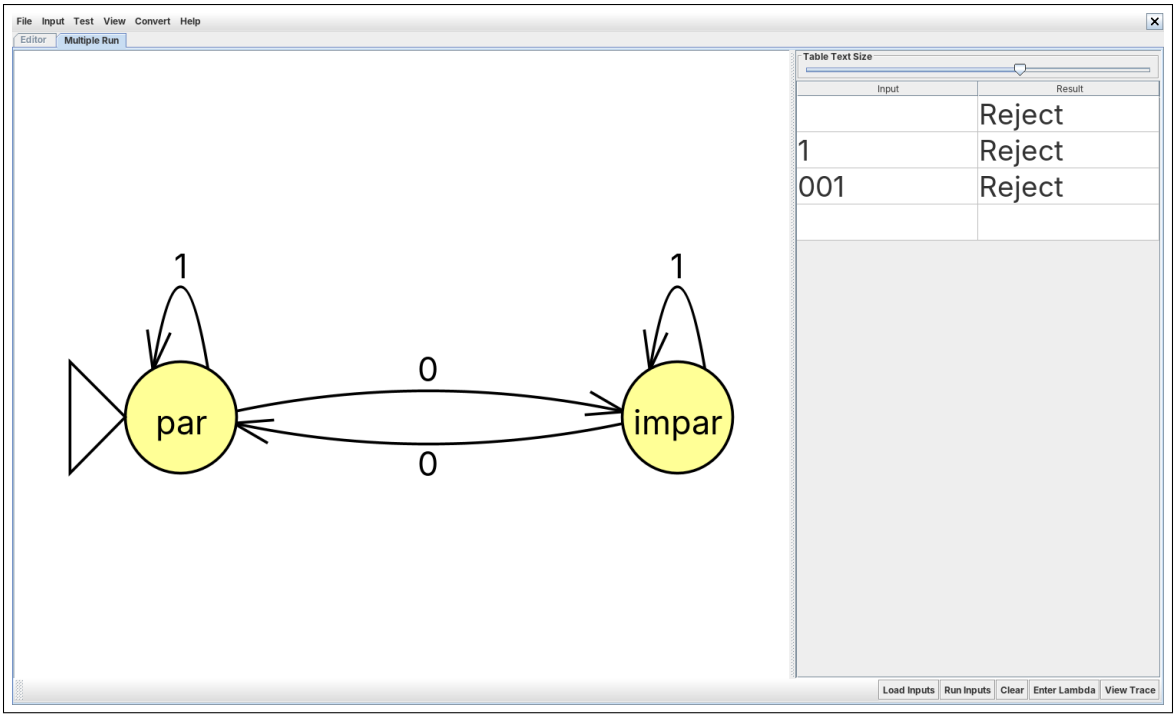


Figura 31. lista2-126 en JFLAP: Multiple Run con las tres cadenas válidas y las tres inválidas de la tabla anterior.

A.1.20. Lista 2 · Ejercicio 127

Enunciado. Sea A un AFD. ¿Cuál es el lenguaje del autómata si F = Q?
Diseño. Ejemplo: el AFD de 139 con F = Q.
Archivo. `automatas/lista2/lista2-127.jff` · $\Sigma = \{0, 1\}$ · $|Q| = 2$

Tabla 48. Tabla de transiciones ($\rightarrow$ inicial, * aceptación, — sin transición).

δ	0	1	Qué recuerda del prefijo leído
$\rightarrow$* par	impar	par	número par de ceros
* impar	par	impar	número impar de ceros

Tabla 49. Cadenas de prueba.

Cadena	Longitud	Esperado	Obtenido	Tipo de caso
λ	0	acepta	acepta	válida
`1`	1	acepta	acepta	válida
`100`	3	acepta	acepta	válida

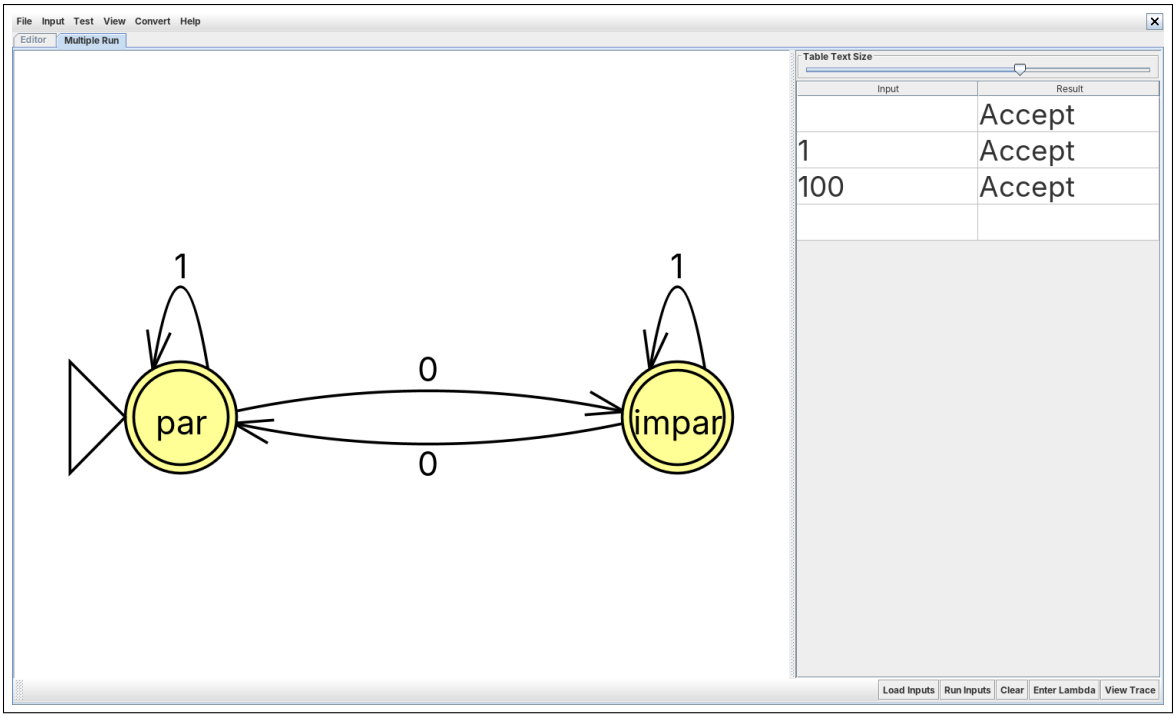


Figura 32. lista2-127 en JFLAP: Multiple Run con las tres cadenas válidas y las tres inválidas de la tabla anterior.

A.1.21. Lista 2 · Ejercicio 128

Enunciado. Sea L el lenguaje del autómata del diagrama. Encuentre un AFD para L2 = {w : wa ∈ L para algún símbolo a}.

Diseño. Se conservan los estados y las transiciones; un estado q es de aceptación en el nuevo autómata si algún símbolo lo lleva a F. Como δ(D, 0) = F y δ(E, 1) = F, F' = {D, E}.

Archivo. `automatas/lista2/lista2-128.jff` · $\Sigma = \{0, 1\}$ · |Q| = 6

Tabla 50. Tabla de transiciones (→ inicial, * aceptación, — sin transición).

δ	0	1	Qué recuerda del prefijo leído
→ A	B	C	mismo estado que en el autómata original
B	C	D	mismo estado que en el autómata original
C	D	E	mismo estado que en el autómata original

δ	0	1	Qué recuerda del prefijo leído
* D	F	E	mismo estado que en el autómata original
* E	E	F	mismo estado que en el autómata original
F	D	D	mismo estado que en el autómata original

Tabla 51. Cadenas de prueba.

Cadena	Longitud	Esperado	Obtenido	Tipo de caso
`10`	2	acepta	acepta	válida
`001`	3	acepta	acepta	válida
`10100`	5	acepta	acepta	válida
λ	0	rechaza	rechaza	inválida
`1`	1	rechaza	rechaza	inválida
`111`	3	rechaza	rechaza	inválida

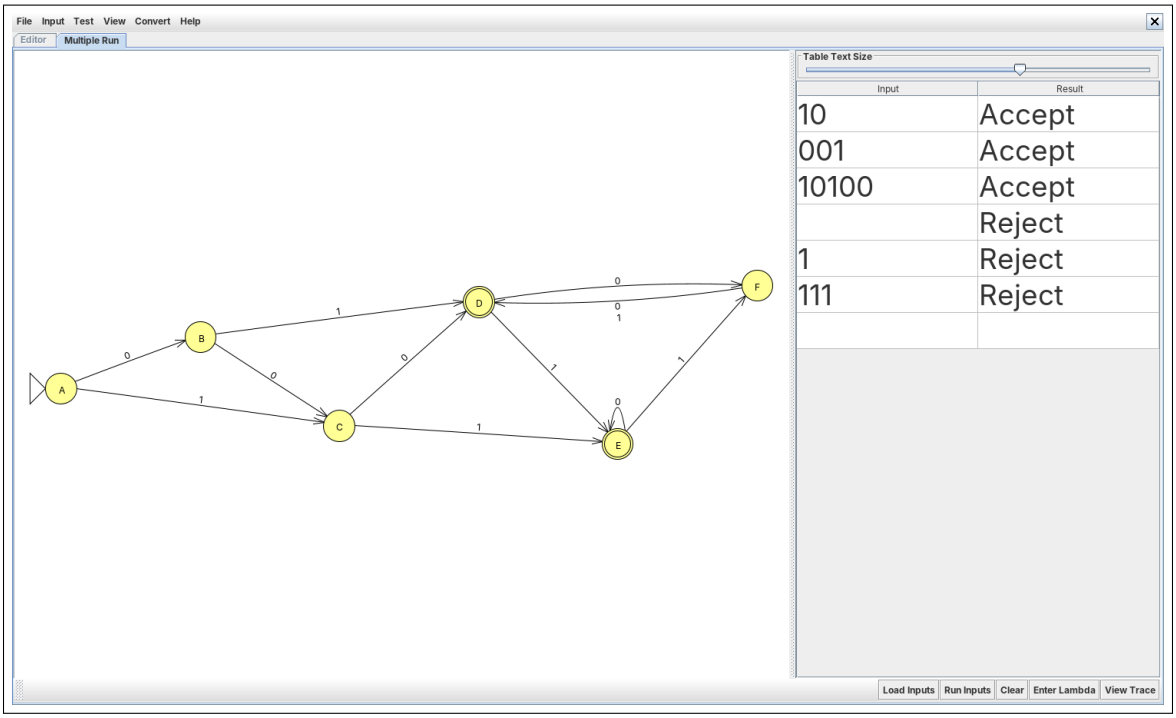


Figura 33. lista2-128 en JFLAP: Multiple Run con las tres cadenas válidas y las tres inválidas de la tabla anterior.

A.1.22. Lista 2 · Ejercicio 131

Enunciado. AFD dado M = ({q1, q2}, {0, 1}, δ, q1, {q2}).
Diseño. q2 significa que el último símbolo leído es 1.
Archivo. `automatas/lista2/lista2-131.jff` · $\Sigma = \{0, 1\}$ · $|Q| = 2$

Tabla 52. Tabla de transiciones (→ inicial, * aceptación, — sin transición).

δ	0	1
→ q1	q1	q2
* q2	q1	q2

Tabla 53. Cadenas de prueba.

Cadena	Longitud	Esperado	Obtenido	Tipo de caso
`1`	1	acepta	acepta	válida
`11`	2	acepta	acepta	válida
`0111`	4	acepta	acepta	válida
λ	0	rechaza	rechaza	inválida
`0`	1	rechaza	rechaza	inválida
`010`	3	rechaza	rechaza	inválida

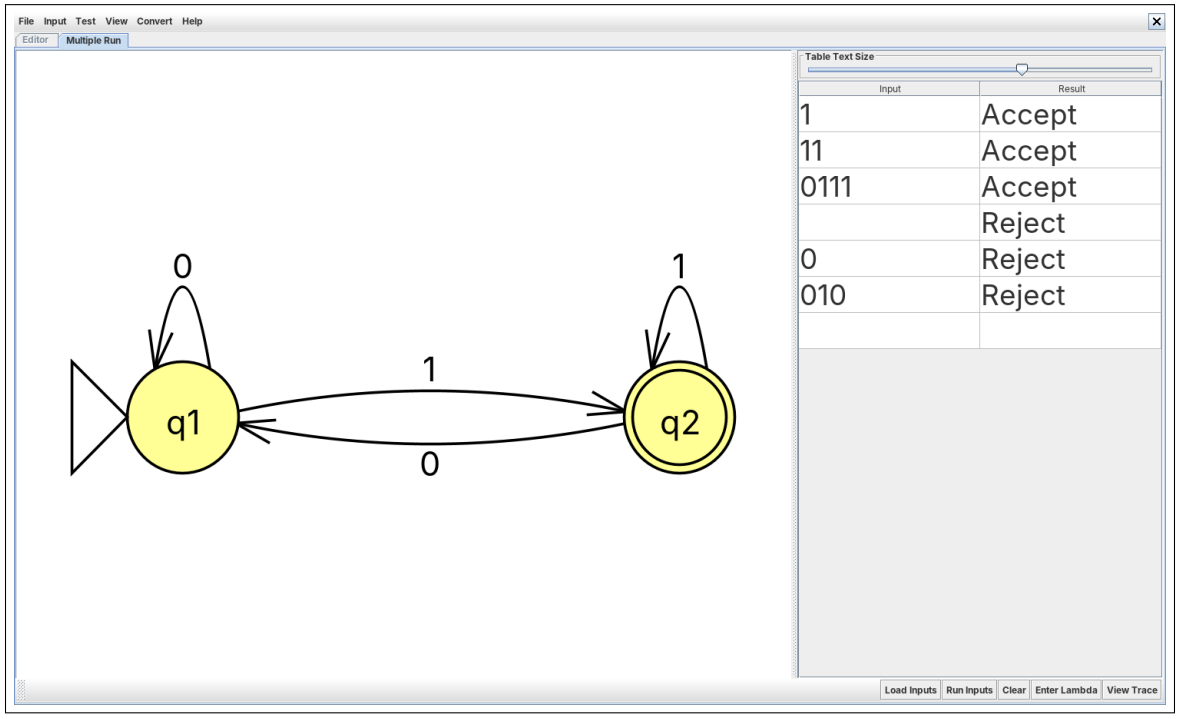


Figura 34. lista2-131 en JFLAP: Multiple Run con las tres cadenas válidas y las tres inválidas de la tabla anterior.

A.1.23. Lista 2 · Ejercicio 133

Enunciado. AFD dado con Q = {q0, q1, q2, q3}, Σ = {a, b}, F = {q0}.
Diseño. q0 = (a par, b par), q1 = (a par, b impar), q2 = (a impar, b par), q3 = (a impar, b impar).
Archivo. `automatas/lista2/lista2-133.jff` · Σ = {a, b} · |Q| = 4

Tabla 54. Tabla de transiciones (→ inicial, * aceptación, — sin transición).

δ	a	b
→* q0	q2	q1
q1	q3	q0
q2	q0	q3
q3	q1	q2

Tabla 55. Cadenas de prueba.

Cadena	Longitud	Esperado	Obtenido	Tipo de caso
λ	0	acepta	acepta	válida
`baab`	4	acepta	acepta	válida
`bbaa`	4	acepta	acepta	válida
`a`	1	rechaza	rechaza	inválida
`ab`	2	rechaza	rechaza	inválida
`baaa`	4	rechaza	rechaza	inválida

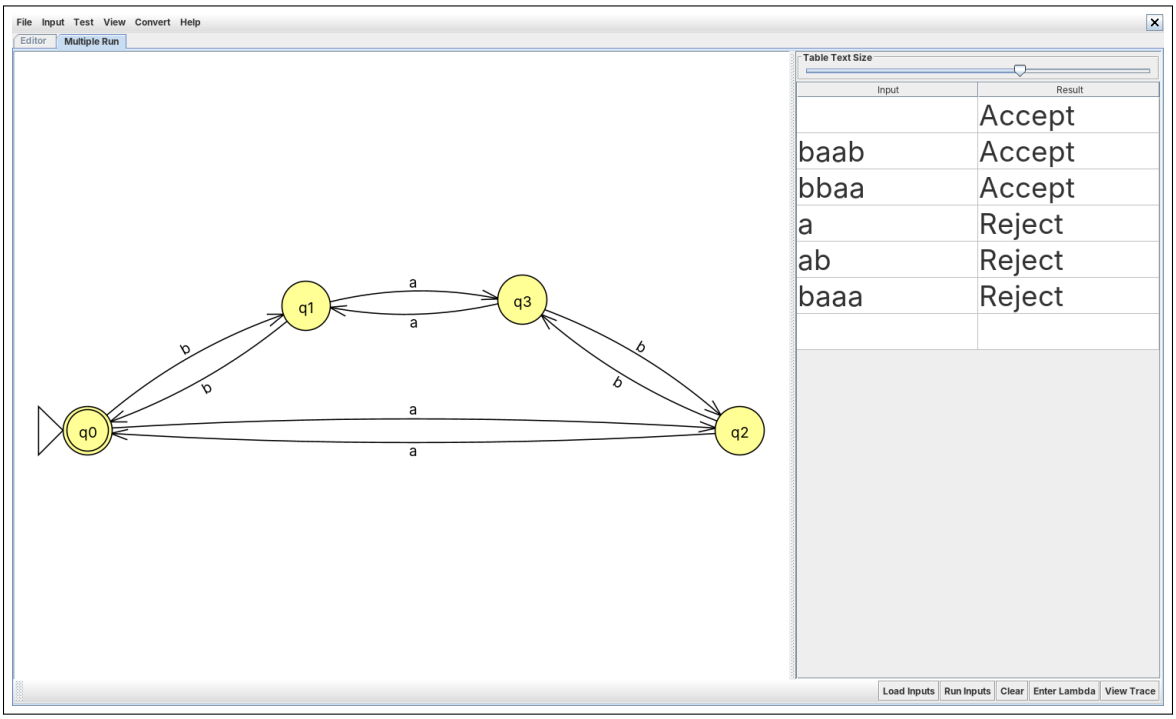


Figura 35. lista2-133 en JFLAP: Multiple Run con las tres cadenas válidas y las tres inválidas de la tabla anterior.

A.1.24. Lista 2 · Ejercicio 135

Enunciado. Dado $\Sigma = \{a, b\}$, AFD para $L = \{b^m a b^n : m, n > 0\}$.

Diseño. Cuatro fases; m > 0 y n > 0 exigen al menos una b en cada lado.
Archivo. `automatas/lista2/lista2-135.jff` · $\Sigma = \{a, b\}$ · $|Q| = 5$

Tabla 56. Tabla de transiciones ($\rightarrow$ inicial, * aceptación, — sin transición).

δ	a	b	Qué recuerda del prefijo leído
$\rightarrow$ ini	muerto	b+	no se ha leído nada
muerto	muerto	muerto	la cadena no tiene la forma
b+	b+a	b+	una o más bes iniciales
b+a	muerto	b+ab+	bes y la a
* b+ab+	muerto	b+ab+	bes, la a y una o más bes

Tabla 57. Cadenas de prueba.

Cadena	Longitud	Esperado	Obtenido	Tipo de caso
`bab`	3	acepta	acepta	válida
`babb`	4	acepta	acepta	válida
`bbbbab`	6	acepta	acepta	válida
λ	0	rechaza	rechaza	inválida
`b`	1	rechaza	rechaza	inválida
`bba`	3	rechaza	rechaza	inválida
`aa`	2	rechaza	rechaza	frontera: justo por debajo de la longitud mínima

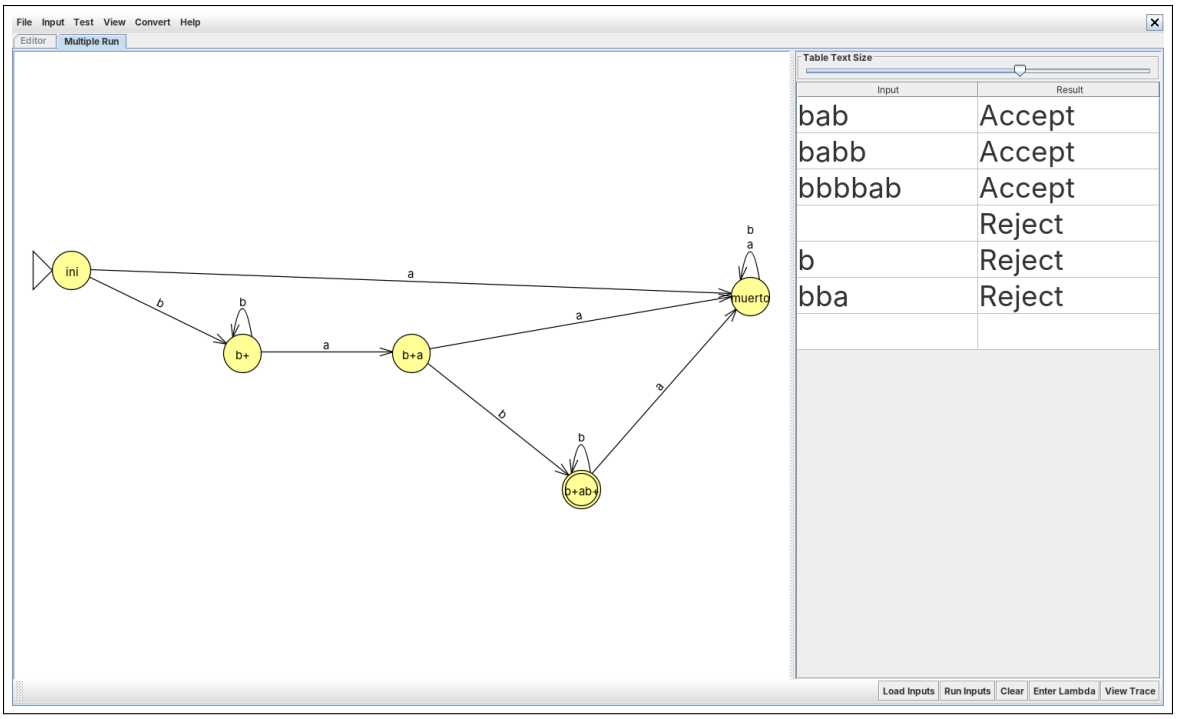


Figura 36. lista2-135 en JFLAP: Multiple Run con las tres cadenas válidas y las tres inválidas de la tabla anterior.

A.1.25. Lista 2 · Ejercicio 137

Enunciado. AFD que reconozca las cadenas sobre {a, b} que comienzan con el prefijo ab.
Diseño. Se verifica el prefijo y después se acepta todo.
Archivo. `automatas/lista2/lista2-137.jff` · $\Sigma = \{a, b\}$ · $|Q| = 4$

Tabla 58. Tabla de transiciones ($\rightarrow$ inicial, * aceptación, — sin transición).

δ	a	b	Qué recuerda del prefijo leído
$\rightarrow \lambda$	a	muerto	se han leído los primeros 0 símbolos de ab
a	muerto	ab	se han leído los primeros 1 símbolos de ab
muerto	muerto	muerto	el prefijo leído no coincide con ab
* ab	ab	ab	ya se leyó el prefijo ab

Tabla 59. Cadenas de prueba.

Cadena	Longitud	Esperado	Obtenido	Tipo de caso
`ab`	2	acepta	acepta	válida
`aba`	3	acepta	acepta	válida
`abbbb`	5	acepta	acepta	válida
λ	0	rechaza	rechaza	inválida

Cadena	Longitud	Esperado	Obtenido	Tipo de caso
`b`	1	rechaza	rechaza	inválida
`bab`	3	rechaza	rechaza	inválida
`a`	1	rechaza	rechaza	frontera: justo por debajo de la longitud mínima

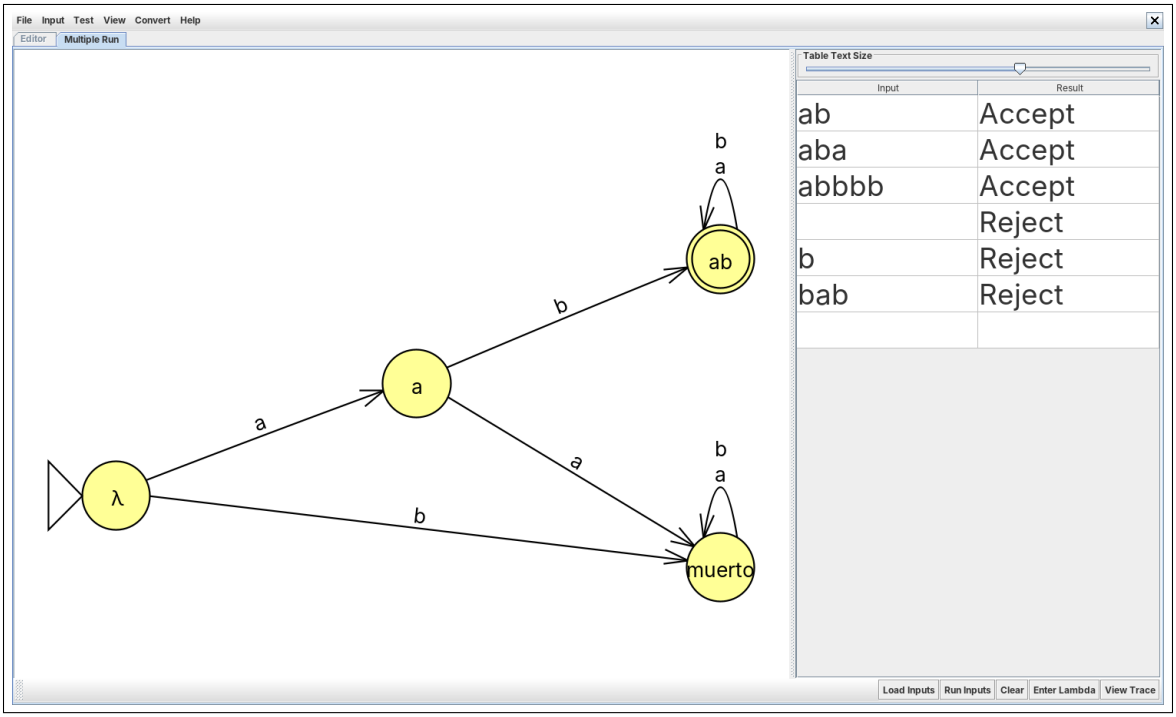


Figura 37. lista2-137 en JFLAP: Multiple Run con las tres cadenas válidas y las tres inválidas de la tabla anterior.

A.1.26. Lista 2 · Ejercicio 139

Enunciado. Autómata que acepte las cadenas en $\{0, 1\}^*$ con un número par de ceros.
Diseño. Paridad de ceros.
Archivo. `automatas/lista2/lista2-139.jff` · $\Sigma = \{0, 1\}$ · $|Q| = 2$

Tabla 60. Tabla de transiciones ($\rightarrow$ inicial, * aceptación, — sin transición).

δ	0	1	Qué recuerda del prefijo leído
$\rightarrow$* par	impar	par	se ha leído un número par de ceros
impar	par	impar	se ha leído un número impar de ceros

Tabla 61. Cadenas de prueba.

Cadena	Longitud	Esperado	Obtenido	Tipo de caso
λ	0	acepta	acepta	válida
`1`	1	acepta	acepta	válida
`010`	3	acepta	acepta	válida
`0`	1	rechaza	rechaza	inválida
`01`	2	rechaza	rechaza	inválida
`0100`	4	rechaza	rechaza	inválida

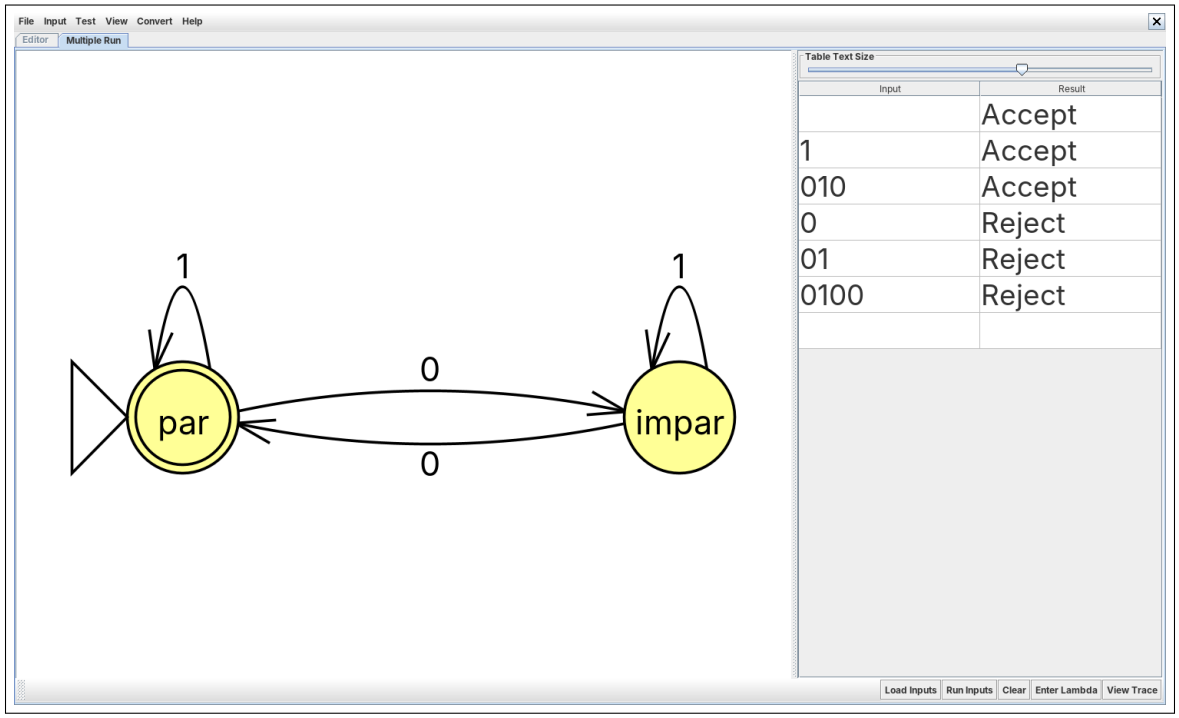


Figura 38. lista2-139 en JFLAP: Multiple Run con las tres cadenas válidas y las tres inválidas de la tabla anterior.

A.1.27. Lista 2 · Ejercicio 141

Enunciado. L = {w ∈ {0, 1}* : cada 0 en w tiene un 1 inmediatamente a su derecha}.
Diseño. Un estado de espera después de cada 0.
Archivo. `automatas/lista2/lista2-141.jff` · Σ = {0, 1} · |Q| = 3

Tabla 62. Tabla de transiciones (→ inicial, * aceptación, — sin transición).

δ	0	1	Qué recuerda del prefijo leído
→* ok	0?	ok	todo 0 leído ya tiene su 1
0?	muerto	ok	el último 0 aún espera su 1

δ	0	1	Qué recuerda del prefijo leído
muerto	muerto	muerto	un 0 no fue seguido de 1

Tabla 63. Cadenas de prueba.

Cadena	Longitud	Esperado	Obtenido	Tipo de caso
λ	0	acepta	acepta	válida
`1`	1	acepta	acepta	válida
`111`	3	acepta	acepta	válida
`0`	1	rechaza	rechaza	inválida
`00`	2	rechaza	rechaza	inválida
`0100`	4	rechaza	rechaza	inválida

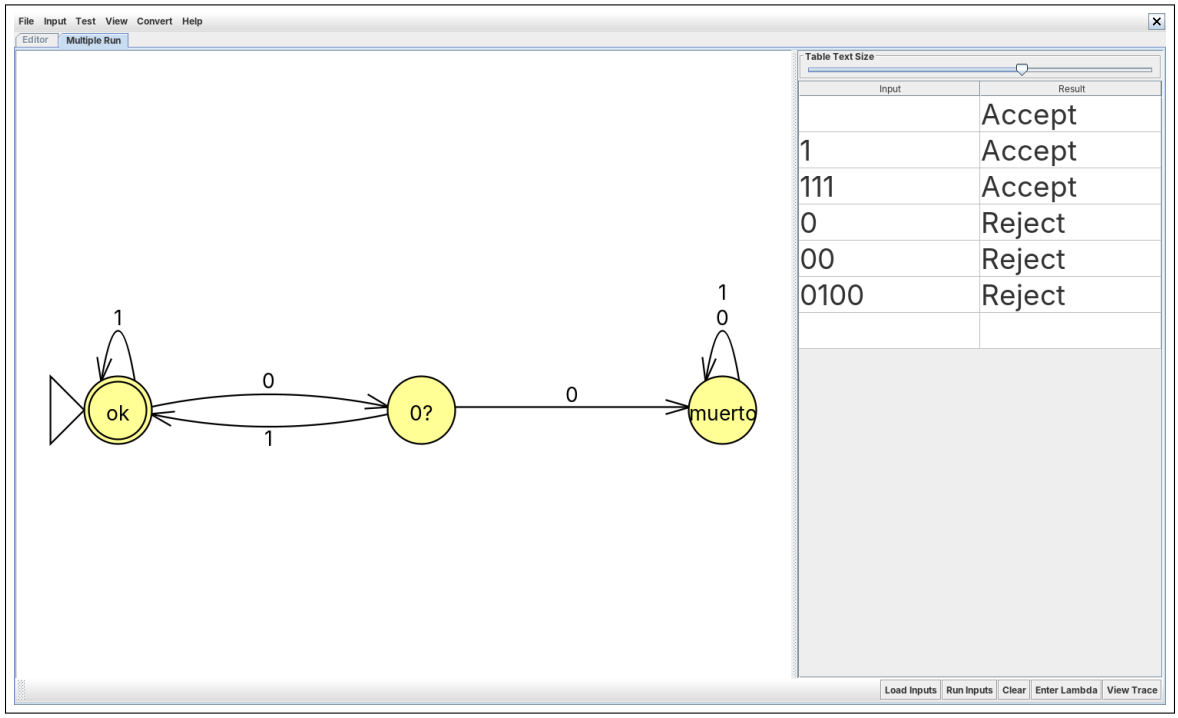


Figura 39. lista2-141 en JFLAP: Multiple Run con las tres cadenas válidas y las tres inválidas de la tabla anterior.

A.1.28. Lista 2 · Ejercicio 143

Enunciado. Determine los lenguajes de los autómatas de las Figuras (a) y (b).
Las figuras del enunciado no aparecen en el PDF de la lista.

A.1.29. Lista 2 · Ejercicio 145

Enunciado. AFD que lee un número binario de derecha a izquierda e identifica los múltiplos de 5.

Diseño. El primer bit leído es el menos significativo. Si los bits restantes forman (de derecha a izquierda) el número V, leer el bit b hace que el valor total sea b + 2V; así, si se necesita que el total sea $\equiv$ t, el resto

debe valer $V \equiv (t - b) \cdot 2^{-1} = 3(t - b) \pmod 5$. El estado guarda ese valor t pendiente: se empieza pidiendo t = 0 y se acepta si, al terminar, lo pendiente es 0. La cadena vacía no es un número y se rechaza.

Archivo. `automatas/lista2/lista2-145.jff` · $\Sigma = \{0, 1\}$ · $|Q| = 6$

Tabla 64. Tabla de transiciones ($\rightarrow$ inicial, * aceptación, — sin transición).

δ	0	1	Qué recuerda del prefijo leído
$\rightarrow$ ini	t0	t2	no se ha leído ningún bit
* t0	t0	t2	los bits que faltan, leídos como número desde su bit menos significativo, deben valer $\equiv 0 \pmod 5$
t2	t1	t3	los bits que faltan, leídos como número desde su bit menos significativo, deben valer $\equiv 2 \pmod 5$
t1	t3	t0	los bits que faltan, leídos como número desde su bit menos significativo, deben valer $\equiv 1 \pmod 5$
t3	t4	t1	los bits que faltan, leídos como número desde su bit menos significativo, deben valer $\equiv 3 \pmod 5$
t4	t2	t4	los bits que faltan, leídos como número desde su bit menos significativo, deben valer $\equiv 4 \pmod 5$

Tabla 65. Cadenas de prueba.

Cadena	Longitud	Esperado	Obtenido	Tipo de caso
`0`	1	acepta	acepta	válida
`00`	2	acepta	acepta	válida
`1111`	4	acepta	acepta	válida
λ	0	rechaza	rechaza	inválida
`1`	1	rechaza	rechaza	inválida
`111`	3	rechaza	rechaza	inválida

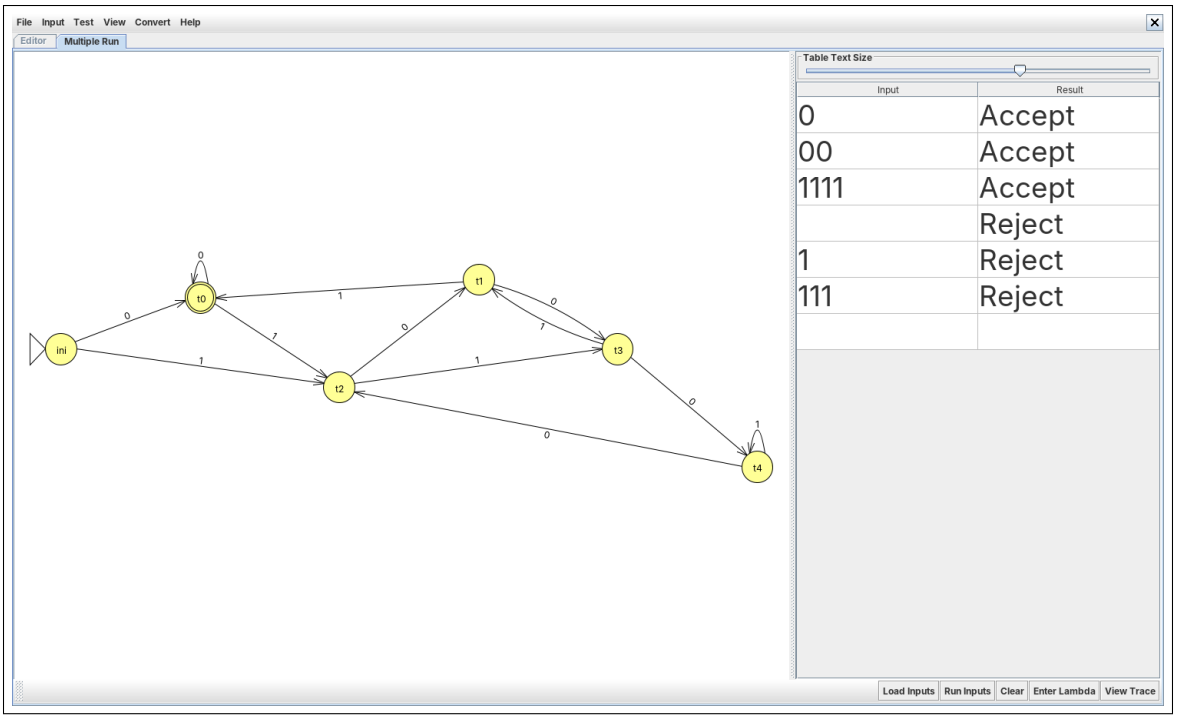


Figura 40. lista2-145 en JFLAP: Multiple Run con las tres cadenas válidas y las tres inválidas de la tabla anterior.

A.1.30. Lista 2 · Ejercicio 147

Enunciado. AFD que lee un número binario de izquierda a derecha y lo acepta si es múltiplo de 6.

Diseño. Leer un bit b convierte el valor v en 2v + b, así que basta el residuo mod 6. La cadena vacía no representa un número y se rechaza. Este AFD de 7 estados se puede minimizar a 4: r y r + 3 tienen el mismo sucesor ($2(r + 3) \equiv 2r$), de modo que {r1, r4}, {r2, r5} e {ini, r3} son equivalentes; es la caracterización «múltiplo de 3 y par».

Archivo. `automatas/lista2/lista2-147.jff` · $\Sigma = \{0, 1\}$ · $|Q| = 7$

Tabla 66. Tabla de transiciones ($\rightarrow$ inicial, * aceptación, — sin transición).

δ	0	1	Qué recuerda del prefijo leído
$\rightarrow$ ini	r0	r1	no se ha leído ningún bit
* r0	r0	r1	el número leído es $\equiv 0 \pmod 6$
r1	r2	r3	el número leído es $\equiv 1 \pmod 6$
r2	r4	r5	el número leído es $\equiv 2 \pmod 6$
r3	r0	r1	el número leído es $\equiv 3 \pmod 6$
r4	r2	r3	el número leído es $\equiv 4 \pmod 6$
r5	r4	r5	el número leído es $\equiv 5 \pmod 6$

Tabla 67. Cadenas de prueba.

Cadena	Longitud	Esperado	Obtenido	Tipo de caso
`0`	1	acepta	acepta	válida
`00`	2	acepta	acepta	válida
`0000`	4	acepta	acepta	válida
λ	0	rechaza	rechaza	inválida
`1`	1	rechaza	rechaza	inválida
`010`	3	rechaza	rechaza	inválida

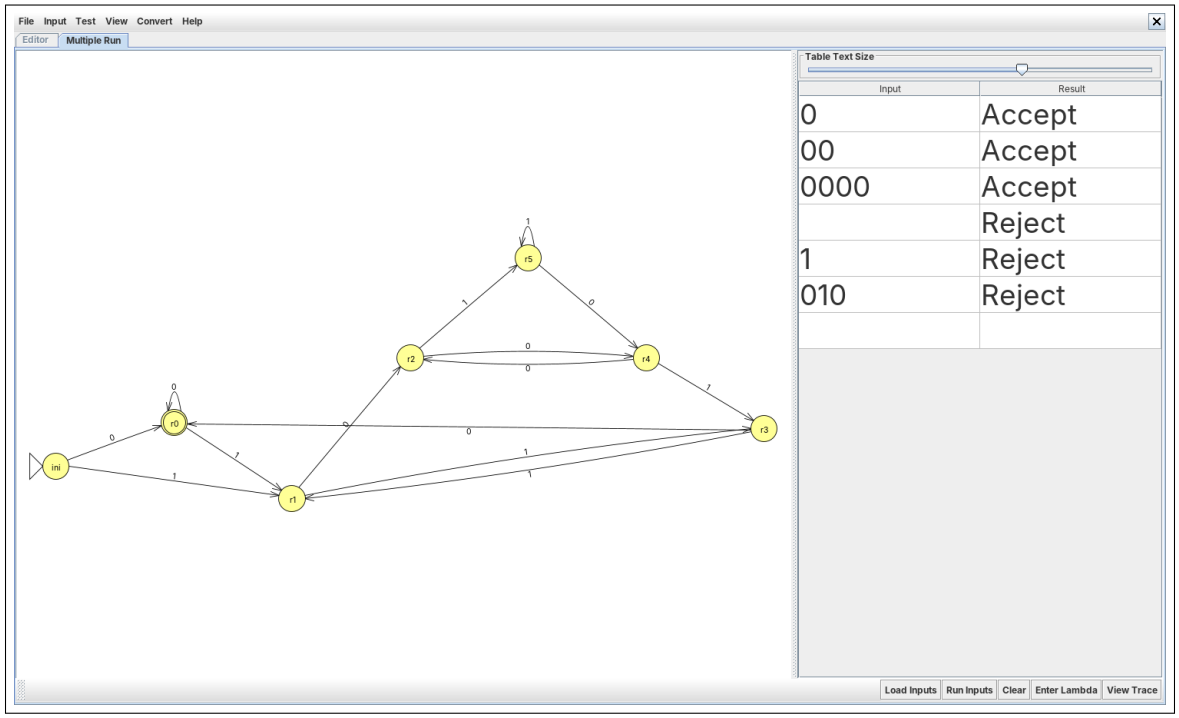


Figura 41. lista2-147 en JFLAP: Multiple Run con las tres cadenas válidas y las tres inválidas de la tabla anterior.

A.1.31. Lista 2 · Ejercicio 148

Enunciado. AFD del progreso de un alumno en Teoría de la Computación (convocatorias de febrero, septiembre y diciembre; $\Sigma = \{P, N, A, S\}$; no más de dos presentaciones por año; termina al aprobar).

Diseño. Interpretación: en cada convocatoria el alumno no se presenta (N) o se presenta (P) y después aprueba (A) o suspende (S). Se cuentan las presentaciones del año; con dos, en la convocatoria restante sólo cabe N. Después de diciembre empieza un año nuevo con el contador en cero. APROBADO es el único estado de aceptación y cualquier símbolo posterior lleva a muerto.

Archivo. `automatas/lista2/lista2-148.jff` · $\Sigma = \{A, N, P, S\}$ · $|Q| = 13$

Tabla 68. Tabla de transiciones (→ inicial, * aceptación, — sin transición).

δ	A	N	P	S	Qué recuerda del prefijo leído
→ feb0	muerto	sep0	feb0?	muerto	convocatoria de feb, 0 presentaciones usadas en el año; decide si se presenta
muerto	muerto	muerto	muerto	muerto	secuencia inválida
sep0	muerto	dic0	sep0?	muerto	convocatoria de sep, 0 presentaciones usadas en el año; decide si se presenta
feb0?	APROBADO	muerto	muerto	sep1	convocatoria de feb, 0 presentaciones usadas en el año; se presentó y espera el resultado
dic0	muerto	feb0	dic0?	muerto	convocatoria de dic, 0 presentaciones usadas en el año; decide si se presenta
sep0?	APROBADO	muerto	muerto	dic1	convocatoria de sep, 0 presentaciones usadas en el año; se presentó y espera el resultado
* APROBADO	muerto	muerto	muerto	muerto	aprobó el curso
sep1	muerto	dic1	sep1?	muerto	convocatoria de sep, 1 presentaciones usadas en el año; decide si se presenta
dic0?	APROBADO	muerto	muerto	feb0	convocatoria de dic, 0 presentaciones usadas en el año; se presentó y espera el resultado
dic1	muerto	feb0	dic1?	muerto	convocatoria de dic, 1 presentaciones usadas en el año; decide si se presenta
sep1?	APROBADO	muerto	muerto	dic2	convocatoria de sep, 1 presentaciones usadas en el año; se presentó y espera el resultado
dic1?	APROBADO	muerto	muerto	feb0	convocatoria de dic, 1 presentaciones usadas en el año; se presentó y espera el resultado
dic2	muerto	feb0	muerto	muerto	convocatoria de dic, 2 presentaciones usadas en el año; decide si se presenta

Tabla 69. Cadenas de prueba.

Cadena	Longitud	Esperado	Obtenido	Tipo de caso
`PA`	2	acepta	acepta	válida
`NPA`	3	acepta	acepta	válida
`NPSPA`	5	acepta	acepta	válida
λ	0	rechaza	rechaza	inválida
`N`	1	rechaza	rechaza	inválida
`PAS`	3	rechaza	rechaza	inválida
`P`	1	rechaza	rechaza	frontera: justo por debajo de la longitud mínima

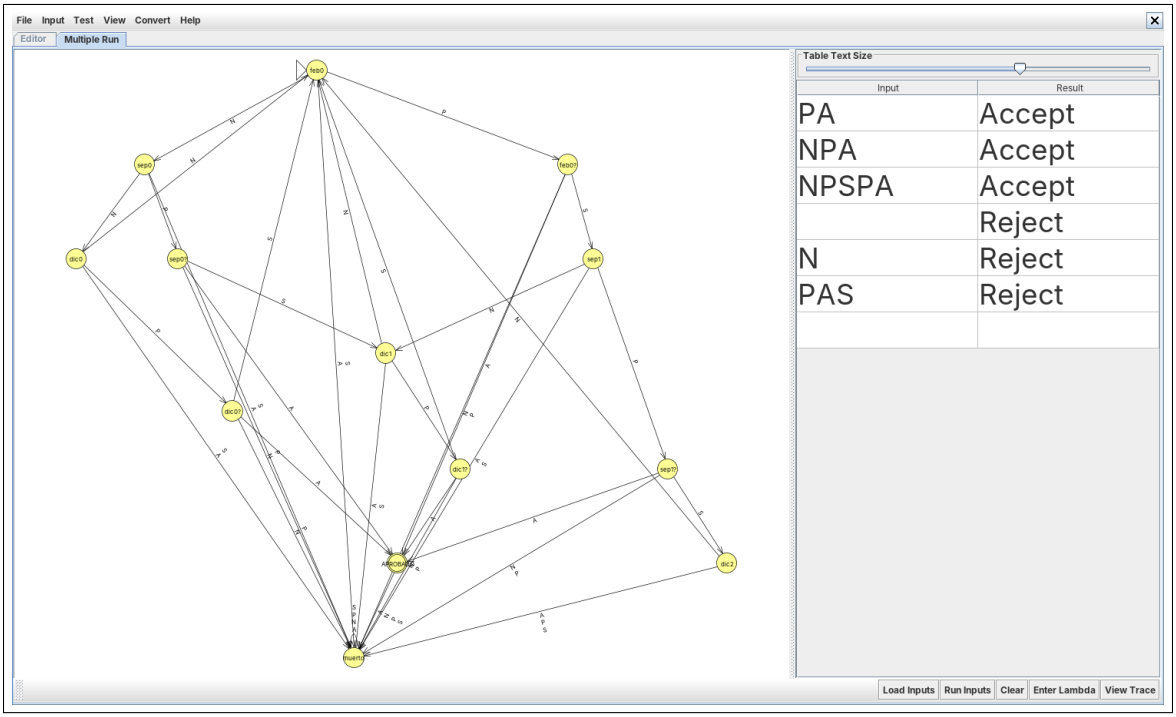


Figura 42. lista2-148 en JFLAP: Multiple Run con las tres cadenas válidas y las tres inválidas de la tabla anterior.

A.1.32. Lista 2 · Ejercicio 149

Enunciado. AFD de la tabla (→A, C final). Halle δ(B, 0), δ(C, 1), $\hat{\delta}$(A, 1101), $\hat{\delta}$(A, 01001).
Diseño. Q = {A, B, C}, Σ = {0, 1}, A0 = A, F = {C}.
Archivo. `automatas/lista2/lista2-149.jff` · Σ = {0, 1} · |Q| = 3

Tabla 70. Tabla de transiciones (→ inicial, * aceptación, — sin transición).

δ	0	1
→ A	B	C
B	A	C
* C	C	A

Tabla 71. Cadenas de prueba.

Cadena	Longitud	Esperado	Obtenido	Tipo de caso
`1`	1	acepta	acepta	válida
`10`	2	acepta	acepta	válida
`1110`	4	acepta	acepta	válida
λ	0	rechaza	rechaza	inválida
`0`	1	rechaza	rechaza	inválida
`011`	3	rechaza	rechaza	inválida

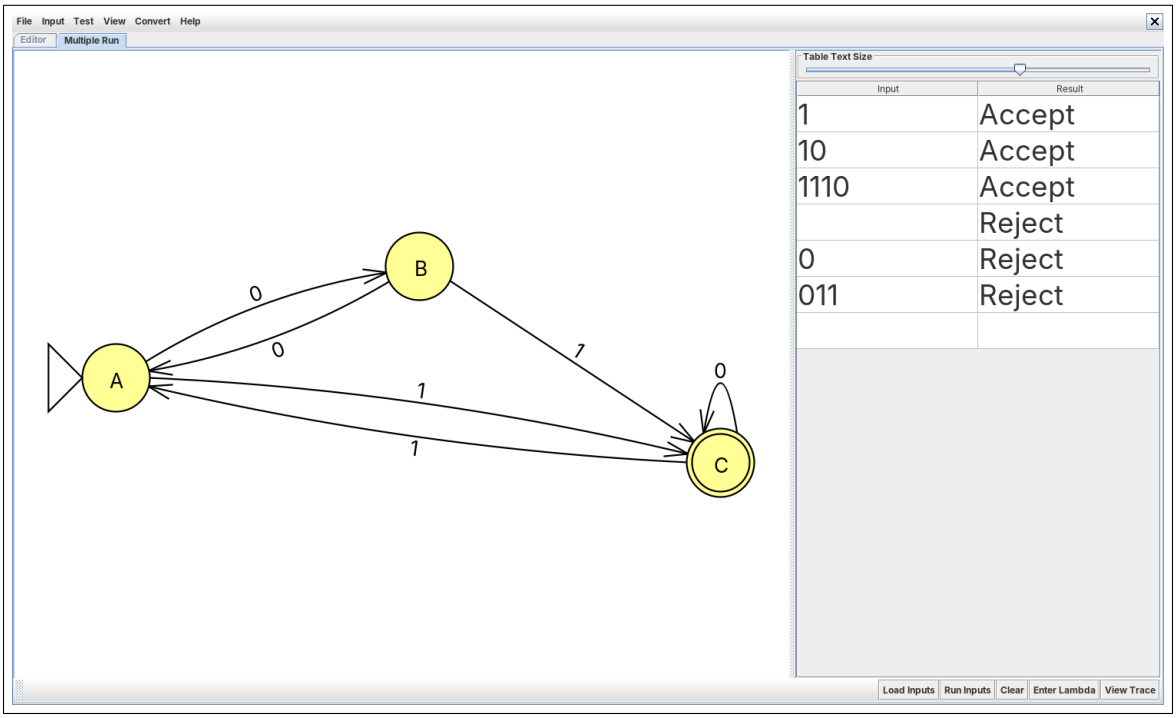


Figura 43. lista2-149 en JFLAP: Multiple Run con las tres cadenas válidas y las tres inválidas de la tabla anterior.

A.1.33. Lista 2 · Ejercicio 150

Enunciado. AFD de la tabla (→A; B, C, D finales). Halle δ(B, 10a11), δ(B, aa1100), δ(A, a01a01), δ(C, a11a00).

Diseño. Q = {A, B, C, D}, Σ = {0, 1, a}, A0 = A, F = {B, C, D}.
Archivo. `automatas/lista2/lista2-150.jff` · Σ = {0, 1, a} · |Q| = 4

Tabla 72. Tabla de transiciones (→ inicial, * aceptación, — sin transición).

δ	0	1	a
→ A	B	C	D
* B	B	C	C
* C	A	A	D
* D	B	B	B

Tabla 73. Cadenas de prueba.

Cadena	Longitud	Esperado	Obtenido	Tipo de caso
`1`	1	acepta	acepta	válida
`aa`	2	acepta	acepta	válida
`1a01`	4	acepta	acepta	válida
λ	0	rechaza	rechaza	inválida
`01011`	5	rechaza	rechaza	inválida
`0aa10a0`	7	rechaza	rechaza	inválida

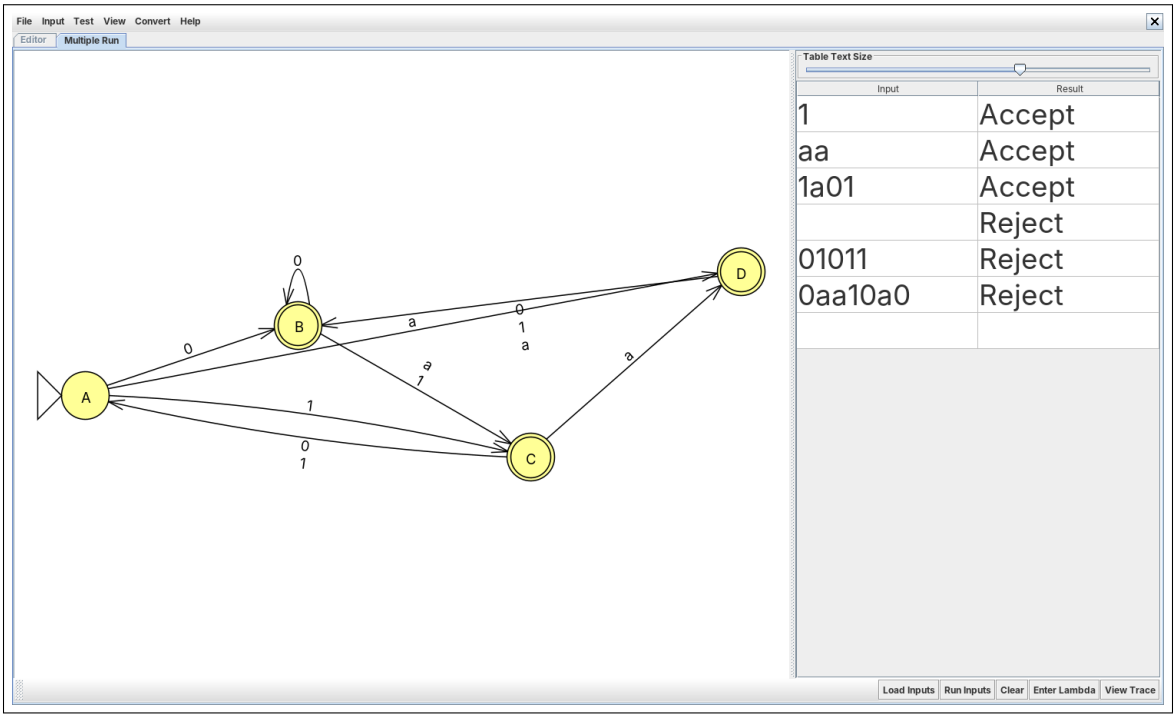


Figura 44. lista2-150 en JFLAP: Multiple Run con las tres cadenas válidas y las tres inválidas de la tabla anterior.

A.2. Lista 3

A.2.1. Lista 3 · Ejercicio 151

Enunciado. Diagrama y simplificación del AFD de la tabla 151.

Diseño. Primero se eliminan los estados que no se alcanzan desde el inicial; después se refina la partición {F, Q − F} hasta que se estabiliza.

Archivo. `automatas/lista3/lista3-151.jff` · $\Sigma = \{0, 1\}$ · $|Q| = 10$

Tabla 74. Tabla de transiciones ($\rightarrow$ inicial, * aceptación, — sin transición).

δ	0	1
$\rightarrow$ A	C	E
B	F	E
* C	A	G
D	C	E
* E	I	B
* F	D	H
G	F	I
H	C	J
I	J	E
J	I	E

Tabla 75. Cadenas de prueba.

Cadena	Longitud	Esperado	Obtenido	Tipo de caso
`1`	1	acepta	acepta	válida
`0111`	4	acepta	acepta	válida
`1001`	4	acepta	acepta	válida
λ	0	rechaza	rechaza	inválida
`001001000`	9	rechaza	rechaza	inválida
`100100000`	9	rechaza	rechaza	inválida

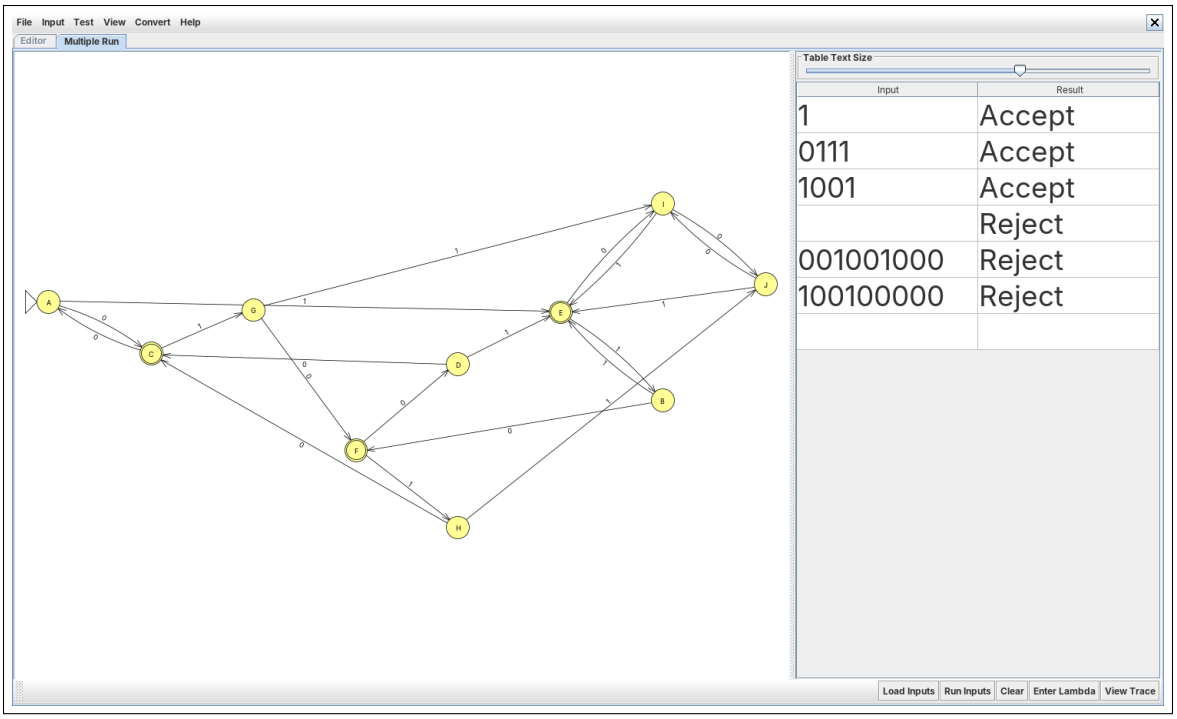


Figura 45. lista3-151 en JFLAP: Multiple Run con las tres cadenas válidas y las tres inválidas de la tabla anterior.

A.2.2. Lista 3 · Ejercicio 152

Enunciado. Diagrama y simplificación del AFD de la tabla 152.

Diseño. Primero se eliminan los estados que no se alcanzan desde el inicial; después se refina la partición {F, Q – F} hasta que se estabiliza.

Archivo. `automatas/lista3/lista3-152.jff` · $\Sigma = \{0, 1, 2\}$ · $|Q| = 6$

Tabla 76. Tabla de transiciones ($\rightarrow$ inicial, * aceptación, — sin transición).

δ	0	1	2
$\rightarrow$ A	C	E	B
B	D	E	B
C	A	D	F
* D	F	B	C
* E	F	B	A
* F	D	B	C

Tabla 77. Cadenas de prueba.

Cadena	Longitud	Esperado	Obtenido	Tipo de caso
`1`	1	acepta	acepta	válida
`02`	2	acepta	acepta	válida
`2021`	4	acepta	acepta	válida
λ	0	rechaza	rechaza	inválida
`0`	1	rechaza	rechaza	inválida
`211`	3	rechaza	rechaza	inválida

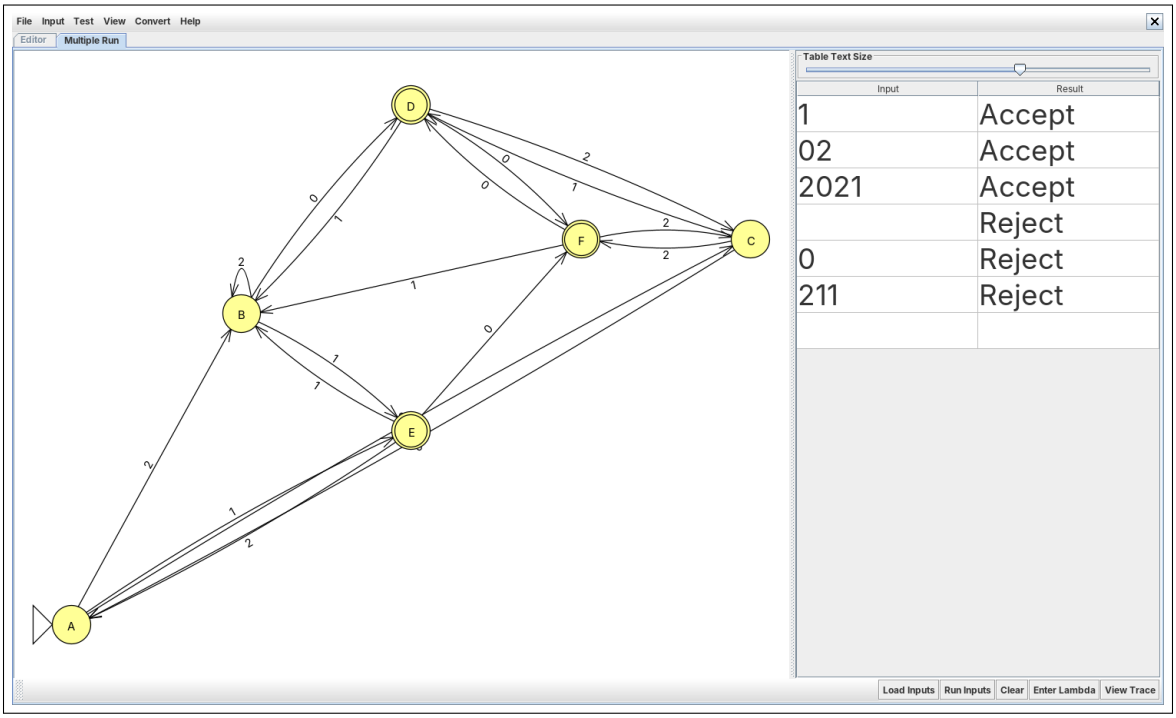


Figura 46. lista3-152 en JFLAP: Multiple Run con las tres cadenas válidas y las tres inválidas de la tabla anterior.

A.2.3. Lista 3 · Ejercicio 153

Enunciado. Diagrama y simplificación del AFD de la tabla 153.

Diseño. Primero se eliminan los estados que no se alcanzan desde el inicial; después se refina la partición {F, Q – F} hasta que se estabiliza.

Archivo. `automatas/lista3/lista3-153.jff` · $\Sigma = \{0, 1\}$ · $|Q| = 7$

Tabla 78. Tabla de transiciones ($\rightarrow$ inicial, * aceptación, — sin transición).

δ	0	1
$\rightarrow$ A	B	C

δ	0	1
B	D	E
C	F	G
* D	D	E
E	F	G
* F	E	D
* G	F	G

Tabla 79. Cadenas de prueba.

Cadena	Longitud	Esperado	Obtenido	Tipo de caso
`10`	2	acepta	acepta	válida
`101`	3	acepta	acepta	válida
`01101`	5	acepta	acepta	válida
λ	0	rechaza	rechaza	inválida
`1`	1	rechaza	rechaza	inválida
`100`	3	rechaza	rechaza	inválida
`0`	1	rechaza	rechaza	frontera: justo por debajo de la longitud mínima

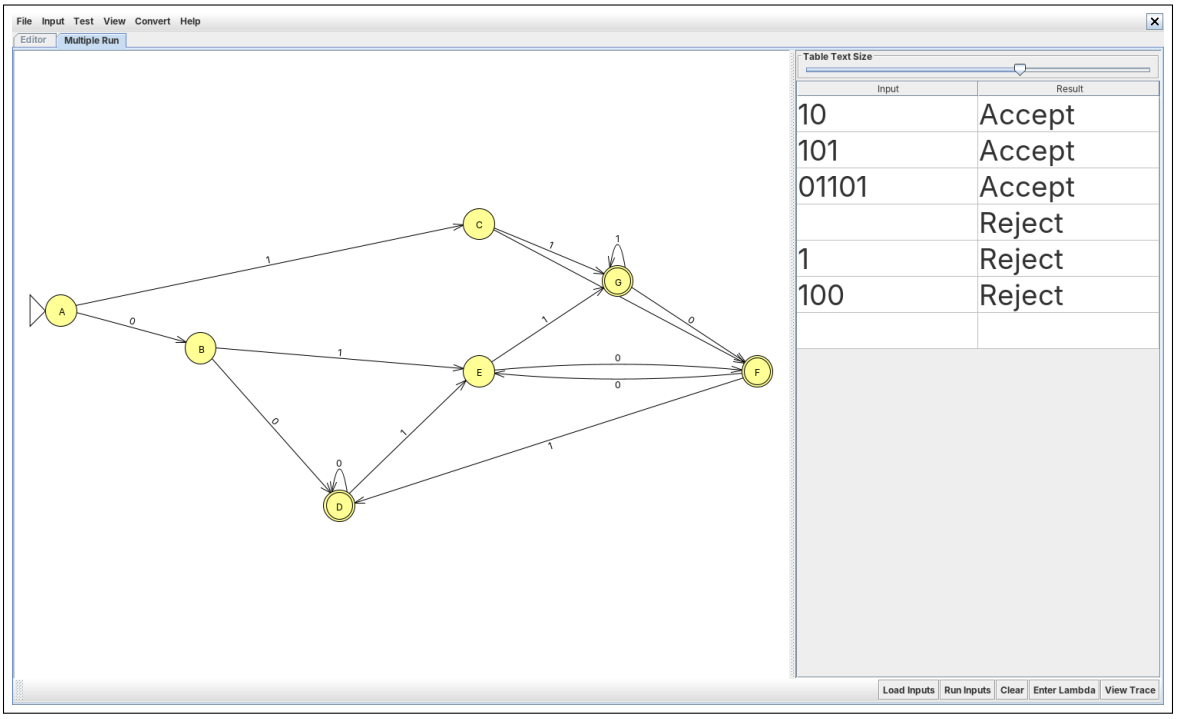


Figura 47. lista3-153 en JFLAP: Multiple Run con las tres cadenas válidas y las tres inválidas de la tabla anterior.

A.2.4. Lista 3 · Ejercicio 154

Enunciado. Diagrama y simplificación del AFD de la tabla 154.

Diseño. Primero se eliminan los estados que no se alcanzan desde el inicial; después se refina la partición {F, Q – F} hasta que se estabiliza.

Archivo. `automatas/lista3/lista3-154.jff` · $\Sigma = \{0, 1\}$ · $|Q| = 11$

Tabla 80. Tabla de transiciones ($\rightarrow$ inicial, * aceptación, — sin transición).

δ	0	1
$\rightarrow$ A	D	K
B	H	K
C	I	E
D	C	G
* E	K	I
F	B	E
* G	C	I
* H	C	B
I	G	K
* J	F	B
K	C	F

Tabla 81. Cadenas de prueba.

Cadena	Longitud	Esperado	Obtenido	Tipo de caso
`01`	2	acepta	acepta	válida
`001`	3	acepta	acepta	válida
`10110`	5	acepta	acepta	válida
λ	0	rechaza	rechaza	inválida
`1`	1	rechaza	rechaza	inválida
`100`	3	rechaza	rechaza	inválida

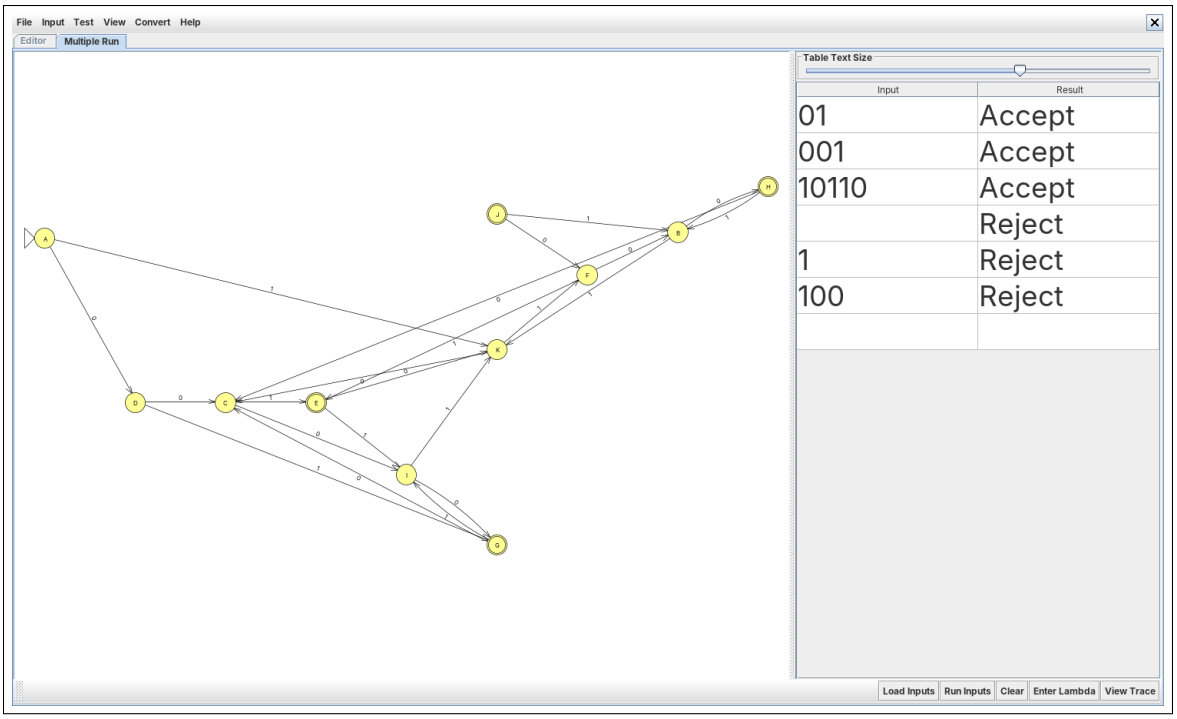


Figura 48. lista3-154 en JFLAP: Multiple Run con las tres cadenas válidas y las tres inválidas de la tabla anterior.

A.2.5. Lista 3 · Ejercicio 155

Enunciado. Diagrama y simplificación del AFD de la tabla 155.

Diseño. Primero se eliminan los estados que no se alcanzan desde el inicial; después se refina la partición {F, Q − F} hasta que se estabiliza.

Archivo. `automatas/lista3/lista3-155.jff` · $\Sigma = \{0, 1\}$ · $|Q| = 12$

Tabla 82. Tabla de transiciones ($\rightarrow$ inicial, * aceptación, — sin transición).

δ	0	1
$\rightarrow$ A	B	C
B	D	E
C	K	G
D	H	E
E	F	D
* F	J	G
G	E	G
H	B	K
I	C	A
J	D	G
K	F	D

δ	0	1
L	C	J

Tabla 83. Cadenas de prueba.

Cadena	Longitud	Esperado	Obtenido	Tipo de caso
`010`	3	acepta	acepta	válida
`1100`	4	acepta	acepta	válida
`101010`	6	acepta	acepta	válida
λ	0	rechaza	rechaza	inválida
`0`	1	rechaza	rechaza	inválida
`110`	3	rechaza	rechaza	inválida
`01`	2	rechaza	rechaza	frontera: justo por debajo de la longitud mínima

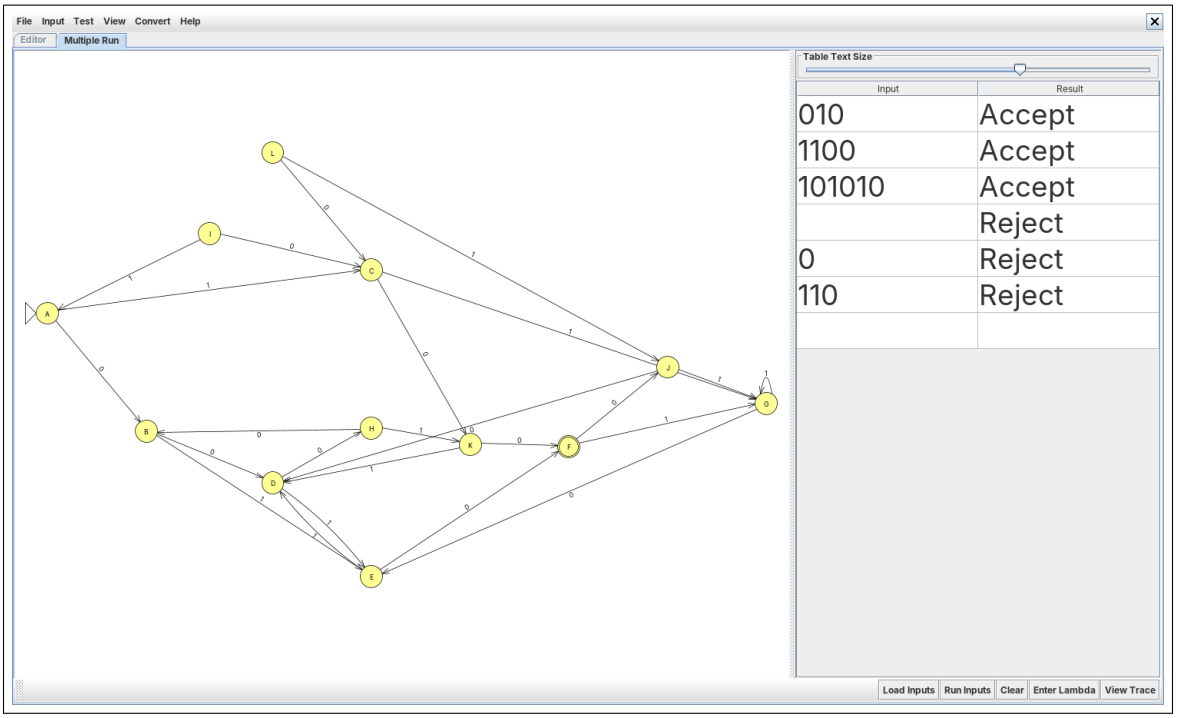


Figura 49. lista3-155 en JFLAP: Multiple Run con las tres cadenas válidas y las tres inválidas de la tabla anterior.

A.2.6. Lista 3 · Ejercicio 156

Enunciado. Diagrama y simplificación del AFD de la tabla 156 (tabla transcrita del diagrama).

Diseño. Primero se eliminan los estados que no se alcanzan desde el inicial; después se refina la partición {F, Q – F} hasta que se estabiliza.

Archivo. `automatas/lista3/lista3-156.jff` · $\Sigma = \{0, 1\}$ · $|Q| = 5$

Tabla 84. Tabla de transiciones ($\rightarrow$ inicial, * aceptación, — sin transición).

δ	0	1
$\rightarrow$ A	B	C
B	C	E
C	D	C
D	C	E
* E	B	E

Tabla 85. Cadenas de prueba.

Cadena	Longitud	Esperado	Obtenido	Tipo de caso
`01`	2	acepta	acepta	válida
`011`	3	acepta	acepta	válida
`10001`	5	acepta	acepta	válida
λ	0	rechaza	rechaza	inválida
`1`	1	rechaza	rechaza	inválida
`010`	3	rechaza	rechaza	inválida

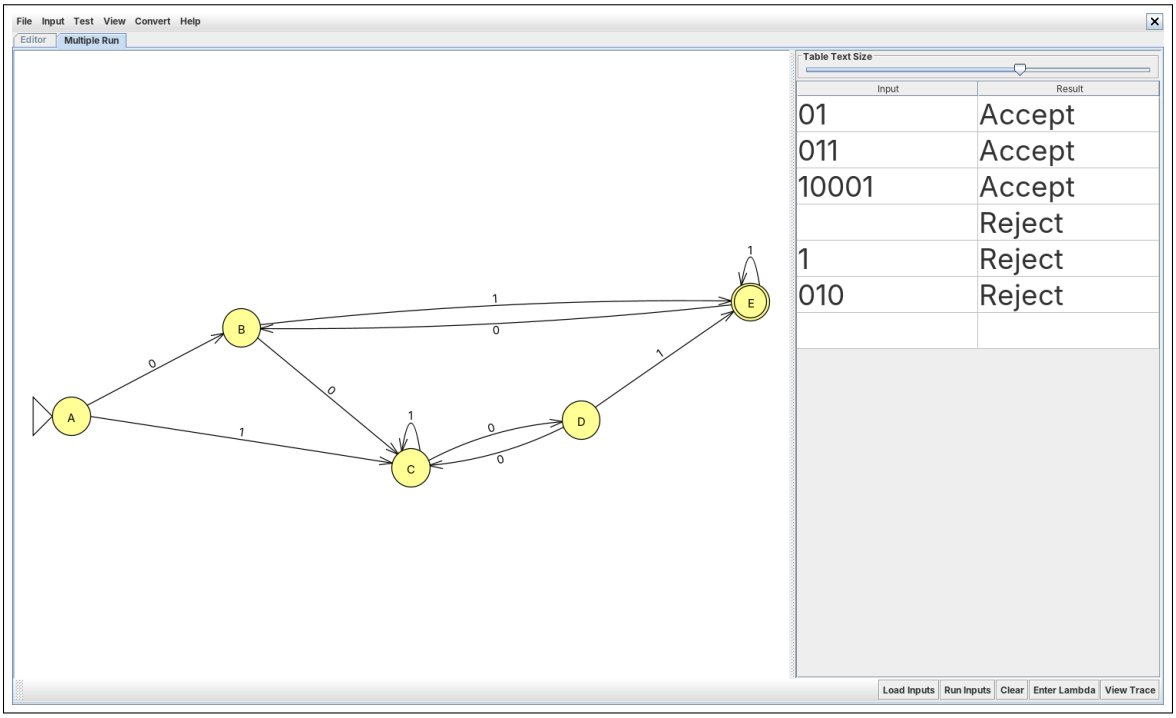


Figura 50. lista3-156 en JFLAP: Multiple Run con las tres cadenas válidas y las tres inválidas de la tabla anterior.

A.2.7. Lista 3 · Ejercicio 157

Enunciado. Diagrama y simplificación del AFD de la tabla 157 (tabla transcrita del diagrama).

Diseño. Primero se eliminan los estados que no se alcanzan desde el inicial; después se refina la partición {F, Q – F} hasta que se estabiliza.

Archivo. `automatas/lista3/lista3-157.jff` · $\Sigma = \{0, 1\}$ · $|Q| = 6$

Tabla 86. Tabla de transiciones ($\rightarrow$ inicial, * aceptación, — sin transición).

δ	0	1
$\rightarrow$ A	B	F
B	C	E
C	D	E
D	B	E
E	F	D
* F	F	B

Tabla 87. Cadenas de prueba.

Cadena	Longitud	Esperado	Obtenido	Tipo de caso
`1`	1	acepta	acepta	válida
`10`	2	acepta	acepta	válida
`0100`	4	acepta	acepta	válida
λ	0	rechaza	rechaza	inválida
`0`	1	rechaza	rechaza	inválida
`001`	3	rechaza	rechaza	inválida

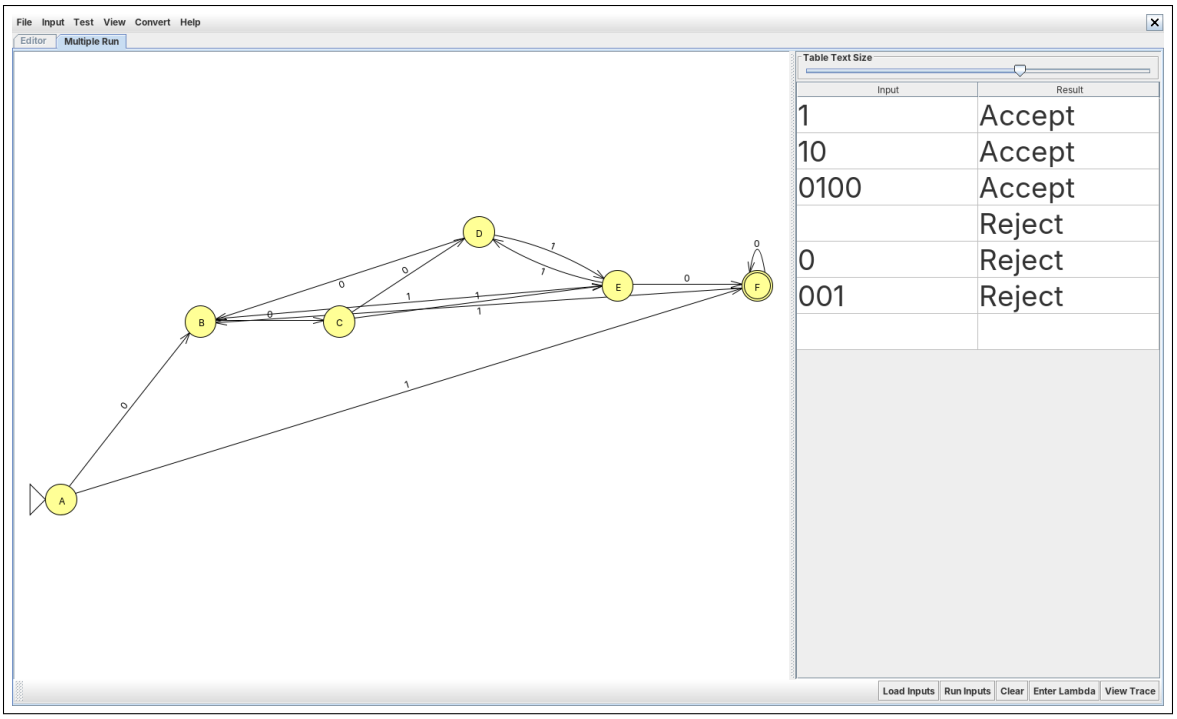


Figura 51. lista3-157 en JFLAP: Multiple Run con las tres cadenas válidas y las tres inválidas de la tabla anterior.

A.2.8. Lista 3 · Ejercicio 158

Enunciado. Describa lo que hace el AFN del diagrama 158.

Diseño. El autómata acepta únicamente la cadena 01: L = {01}. Las transiciones que faltan llevan a un rechazo implícito.

Archivo. `automatas/lista3/lista3-158.jff` · $\Sigma = \{0, 1\}$ · $|Q| = 3$

Tabla 88. Tabla de transiciones ($\rightarrow$ inicial, * aceptación, — sin transición).

δ	0	1
$\rightarrow$ q1	q2	—
q2	—	q3
* q3	—	—

Tabla 89. Cadenas de prueba.

Cadena	Longitud	Esperado	Obtenido	Tipo de caso
`01`	2	acepta	acepta	válida
λ	0	rechaza	rechaza	inválida
`1`	1	rechaza	rechaza	inválida
`111`	3	rechaza	rechaza	inválida

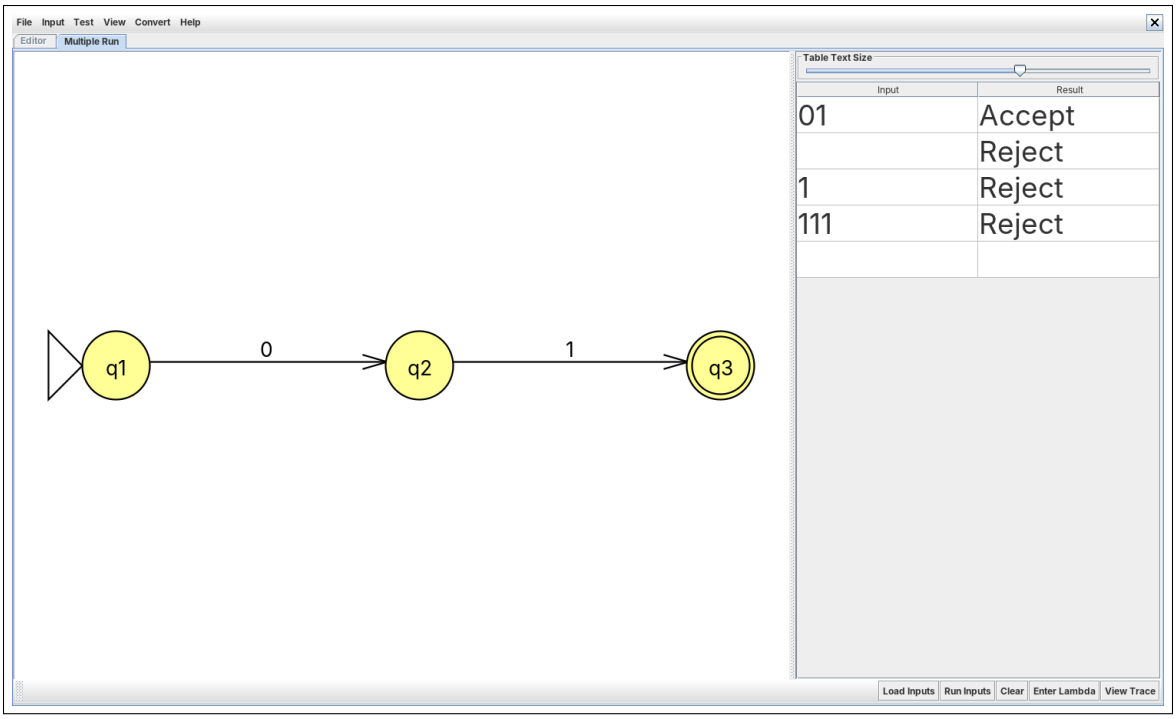


Figura 52. lista3-158 en JFLAP: Multiple Run con las tres cadenas válidas y las tres inválidas de la tabla anterior.

A.2.9. Lista 3 · Ejercicio 159

Enunciado. Describa lo que hace el AFN del diagrama 159.

Diseño. El autómata acepta las cadenas formadas por unos seguidos de 01: L = 1*01. Las transiciones que faltan llevan a un rechazo implícito.

Archivo. `automatas/lista3/lista3-159.jff` · $\Sigma = \{0, 1\}$ · $|Q| = 3$

Tabla 90. Tabla de transiciones ($\rightarrow$ inicial, * aceptación, — sin transición).

δ	0	1
$\rightarrow$ q1	q2	q1
q2	—	q3
* q3	—	—

Tabla 91. Cadenas de prueba.

Cadena	Longitud	Esperado	Obtenido	Tipo de caso
`01`	2	acepta	acepta	válida
`101`	3	acepta	acepta	válida
`11101`	5	acepta	acepta	válida
λ	0	rechaza	rechaza	inválida

Cadena	Longitud	Esperado	Obtenido	Tipo de caso
`0`	1	rechaza	rechaza	inválida
`001`	3	rechaza	rechaza	inválida
`1`	1	rechaza	rechaza	frontera: justo por debajo de la longitud mínima

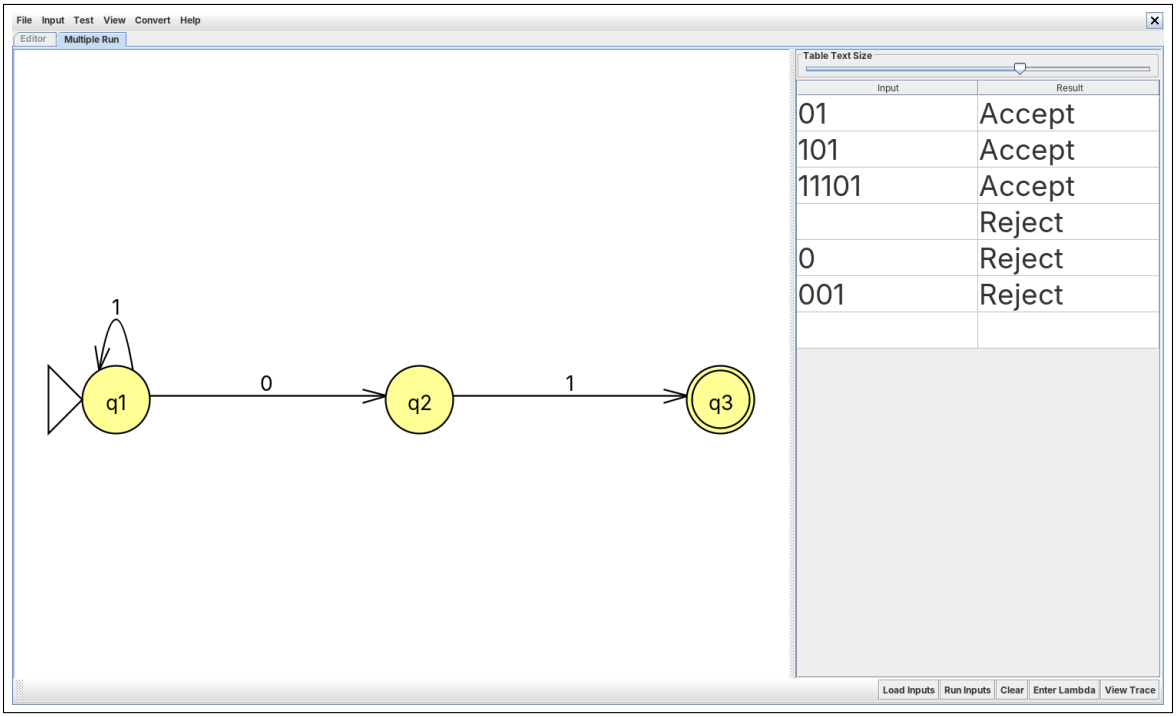


Figura 53. lista3-159 en JFLAP: Multiple Run con las tres cadenas válidas y las tres inválidas de la tabla anterior.

A.2.10. Lista 3 · Ejercicio 160

Enunciado. Describa lo que hace el AFN del diagrama 160.

Diseño. El autómata acepta las cadenas que empiezan con 01: L = 01(0 + 1)*. Las transiciones que faltan llevan a un rechazo implícito.

Archivo. `automatas/lista3/lista3-160.jff` · $\Sigma = \{0, 1\}$ · $|Q| = 3$

Tabla 92. Tabla de transiciones ($\rightarrow$ inicial, * aceptación, — sin transición).

δ	0	1
$\rightarrow$ q1	q2	—
q2	—	q3
* q3	q3	q3

Tabla 93. Cadenas de prueba.

Cadena	Longitud	Esperado	Obtenido	Tipo de caso
`01`	2	acepta	acepta	válida
`011`	3	acepta	acepta	válida
`01100`	5	acepta	acepta	válida
λ	0	rechaza	rechaza	inválida
`1`	1	rechaza	rechaza	inválida
`101`	3	rechaza	rechaza	inválida

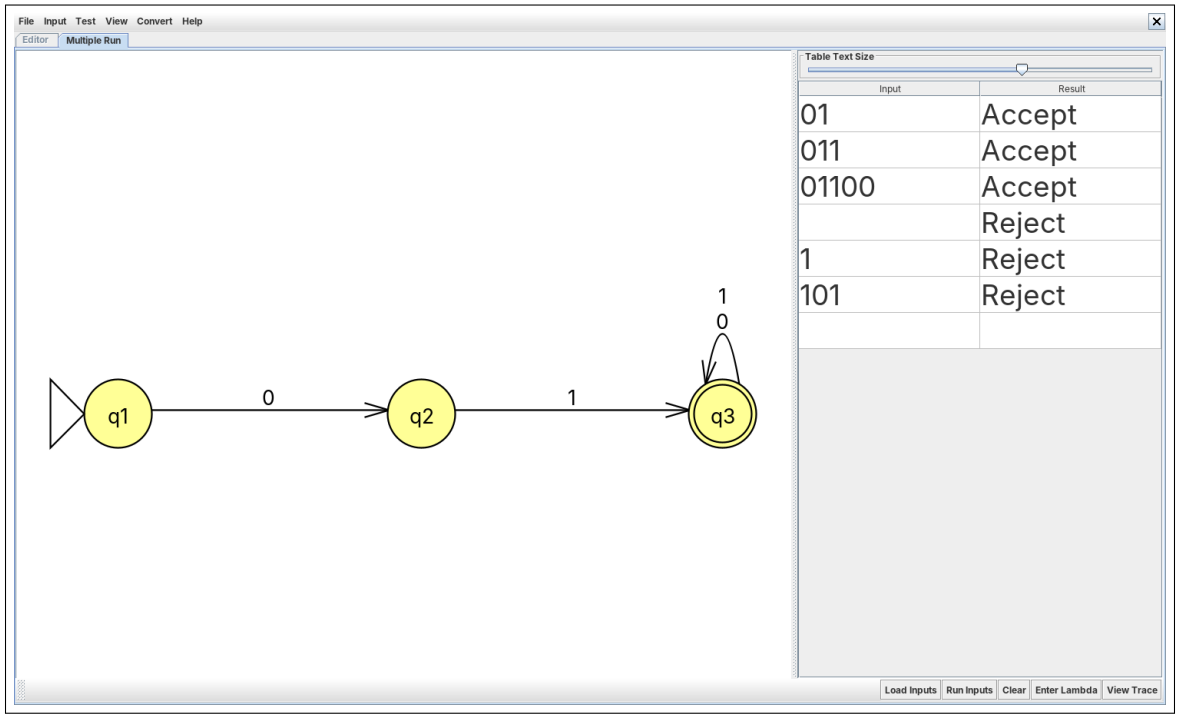


Figura 54. lista3-160 en JFLAP: Multiple Run con las tres cadenas válidas y las tres inválidas de la tabla anterior.

A.2.11. Lista 3 · Ejercicio 161

Enunciado. Describa lo que hace el AFN del diagrama 161.

Diseño. El autómata acepta las repeticiones del bloque 010, incluida λ: L = (010)*. Las transiciones que faltan llevan a un rechazo implícito.

Archivo. `automatas/lista3/lista3-161.jff` · $\Sigma = \{0, 1\}$ · $|Q| = 3$

Tabla 94. Tabla de transiciones ($\rightarrow$ inicial, * aceptación, — sin transición).

δ	0	1
$\rightarrow$* q1	q2	—

δ	0	1
q2	—	q3
q3	q1	—

Tabla 95. Cadenas de prueba.

Cadena	Longitud	Esperado	Obtenido	Tipo de caso
λ	0	acepta	acepta	válida
`010`	3	acepta	acepta	válida
`010010`	6	acepta	acepta	válida
`0`	1	rechaza	rechaza	inválida
`01`	2	rechaza	rechaza	inválida
`0100`	4	rechaza	rechaza	inválida

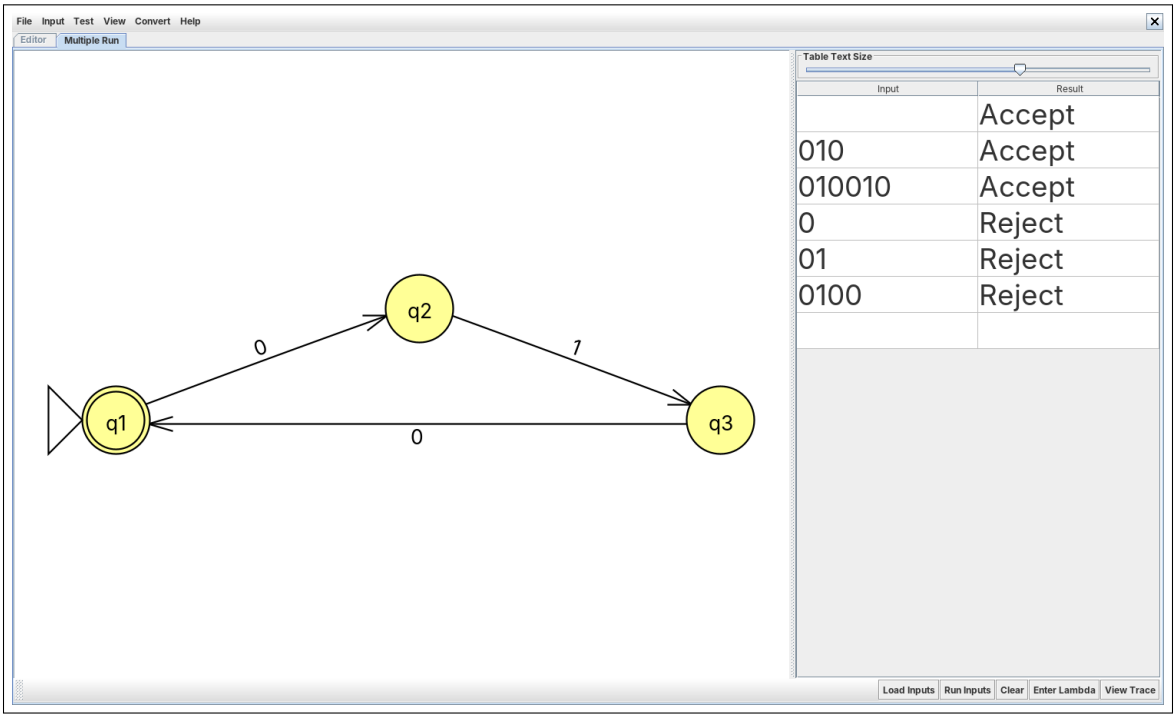


Figura 55. lista3-161 en JFLAP: Multiple Run con las tres cadenas válidas y las tres inválidas de la tabla anterior.

A.2.12. Lista 3 · AFN del problema 097

Enunciado. AFN para el problema 97: Las palabras de $\Sigma 1$ cuya longitud es un múltiplo de 3.

Diseño. No hay una versión no determinista más simple: el lenguaje depende de contar (longitud, apariciones o fases). Se presenta el AFD del problema sin el estado muerto; una función de transición parcial es un caso particular de AFN.

Archivo. `automatas/lista3/lista3-afn-097.jff` · $\Sigma = \{0, 1, 2\}$ · $|Q| = 3$

Tabla 96. Tabla de transiciones (→ inicial, * aceptación, — sin transición).

δ	0	1	2
→* r0	r1	r1	r1
r1	r2	r2	r2
r2	r0	r0	r0

Tabla 97. Cadenas de prueba.

Cadena	Longitud	Esperado	Obtenido	Tipo de caso
λ	0	acepta	acepta	válida
`120`	3	acepta	acepta	válida
`200`	3	acepta	acepta	válida
`1`	1	rechaza	rechaza	inválida
`21`	2	rechaza	rechaza	inválida
`2102`	4	rechaza	rechaza	inválida

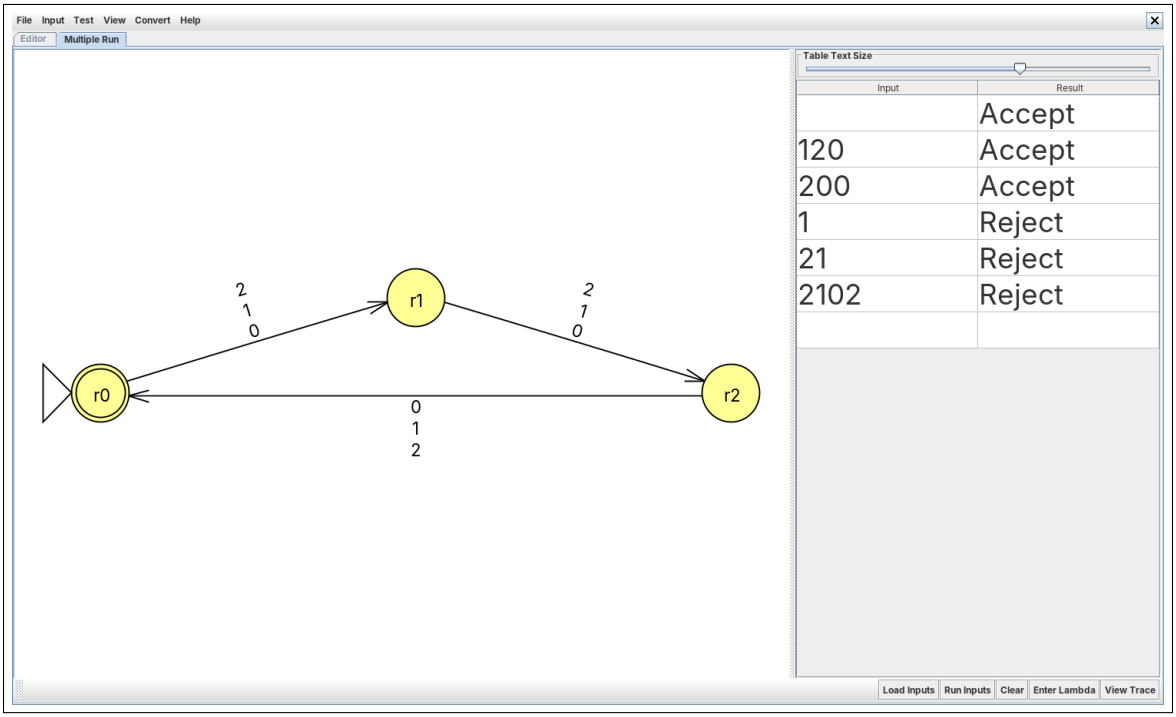


Figura 56. lista3-afn-097 en JFLAP: Multiple Run con las tres cadenas válidas y las tres inválidas de la tabla anterior.

A.2.13. Lista 3 · AFN del problema 099

Enunciado. AFN para el problema 99: Las palabras de $\Sigma 1$ que terminan con la cadena 001001.

Diseño. Ciclo en c0 y una cadena que adivina dónde empieza el sufijo 001001. El enunciado se resuelve de forma natural con un AFN de 7 estados; el AFD equivalente se obtuvo con la construcción de subconjuntos y la minimización y tiene 7 estados.

Archivo. `automatas/lista3/lista3-afn-099.jff` · $\Sigma = \{0, 1, 2\}$ · $|Q| = 7$

Tabla 98. Tabla de transiciones ($\rightarrow$ inicial, * aceptación, — sin transición).

δ	0	1	2	Qué recuerda del prefijo leído
$\rightarrow$ q0	q1	q0	q0	activos del AFN: {c0}
q1	q2	q0	q0	activos del AFN: {c0,c1}
q2	q2	q3	q0	activos del AFN: {c0,c1,c2}
q3	q4	q0	q0	activos del AFN: {c0,c3}
q4	q5	q0	q0	activos del AFN: {c0,c1,c4}
q5	q2	q6	q0	activos del AFN: {c0,c1,c2,c5}
* q6	q4	q0	q0	activos del AFN: {c0,c3,c6}

Tabla 99. Cadenas de prueba.

Cadena	Longitud	Esperado	Obtenido	Tipo de caso
`001001`	6	acepta	acepta	válida
`1001001`	7	acepta	acepta	válida
`121001001`	9	acepta	acepta	válida
λ	0	rechaza	rechaza	inválida
`2`	1	rechaza	rechaza	inválida
`120`	3	rechaza	rechaza	inválida
`02210`	5	rechaza	rechaza	frontera: justo por debajo de la longitud mínima

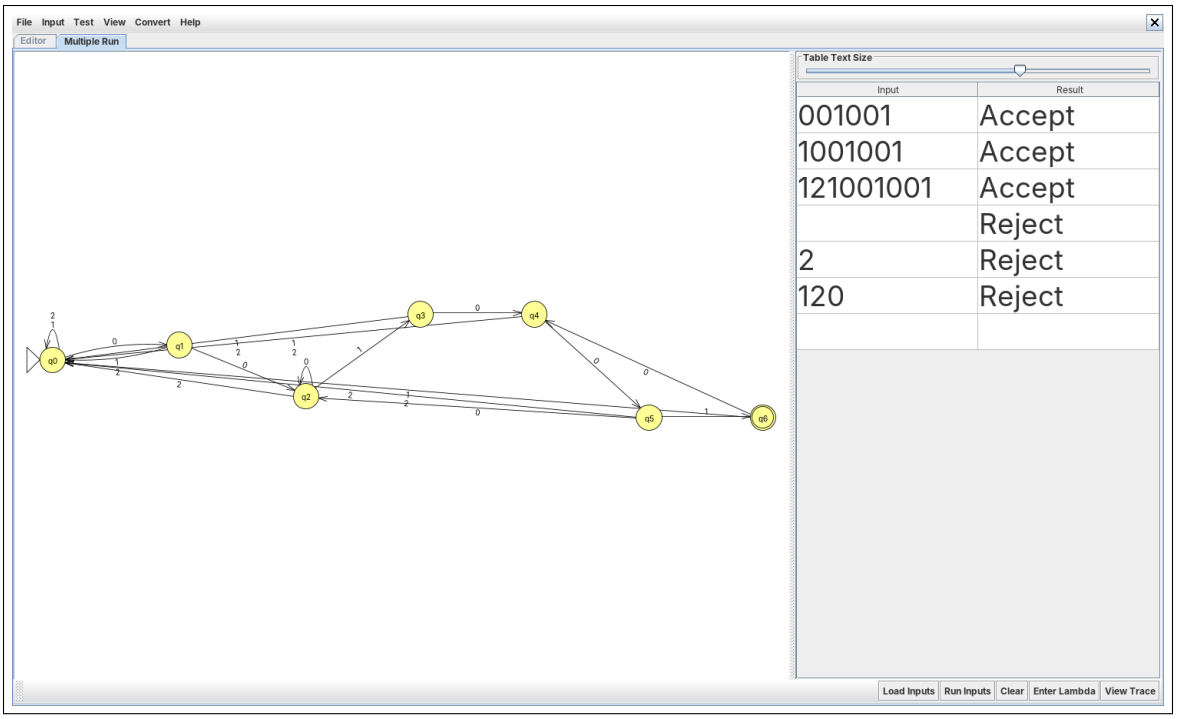


Figura 57. lista3-afn-099 en JFLAP: Multiple Run con las tres cadenas válidas y las tres inválidas de la tabla anterior.

A.2.14. Lista 3 · AFN del problema 102

Enunciado. AFN para el problema 102: Las palabras de $\Sigma 3$ que contienen la cadena abbbab.

Diseño. El AFN adivina dónde empieza abbbab: ciclo inicial, el patrón y ciclo final. No necesita los retrocesos del autómata de Knuth-Morris-Pratt. El enunciado se resuelve de forma natural con un AFN de 7 estados; el AFD equivalente se obtuvo con la construcción de subconjuntos y la minimización y tiene 7 estados.

Archivo. `automatas/lista3/lista3-afn-102.jff` · $\Sigma = \{a, b\}$ · $|Q| = 7$

Tabla 100. Tabla de transiciones ($\rightarrow$ inicial, * aceptación, — sin transición).

δ	a	b	Qué recuerda del prefijo leído
$\rightarrow$ q0	q1	q0	activos del AFN: {c0}
q1	q1	q2	activos del AFN: {c0,c1}
q2	q1	q3	activos del AFN: {c0,c2}
q3	q1	q4	activos del AFN: {c0,c3}
q4	q5	q0	activos del AFN: {c0,c4}
q5	q1	q6	activos del AFN: {c0,c1,c5}
* q6	q6	q6	activos del AFN: {c0,c2,c6} o {c0,c1,c6} o {c0,c3,c6} o {c0,c4,c6} o {c0,c1,c5,c6} o {c0,c6}

Tabla 101. Cadenas de prueba.

Cadena	Longitud	Esperado	Obtenido	Tipo de caso
`abbbab`	6	acepta	acepta	válida
`abbbabb`	7	acepta	acepta	válida
`abbbabaaa`	9	acepta	acepta	válida
λ	0	rechaza	rechaza	inválida
`a`	1	rechaza	rechaza	inválida
`baa`	3	rechaza	rechaza	inválida
`aabab`	5	rechaza	rechaza	frontera: justo por debajo de la longitud mínima

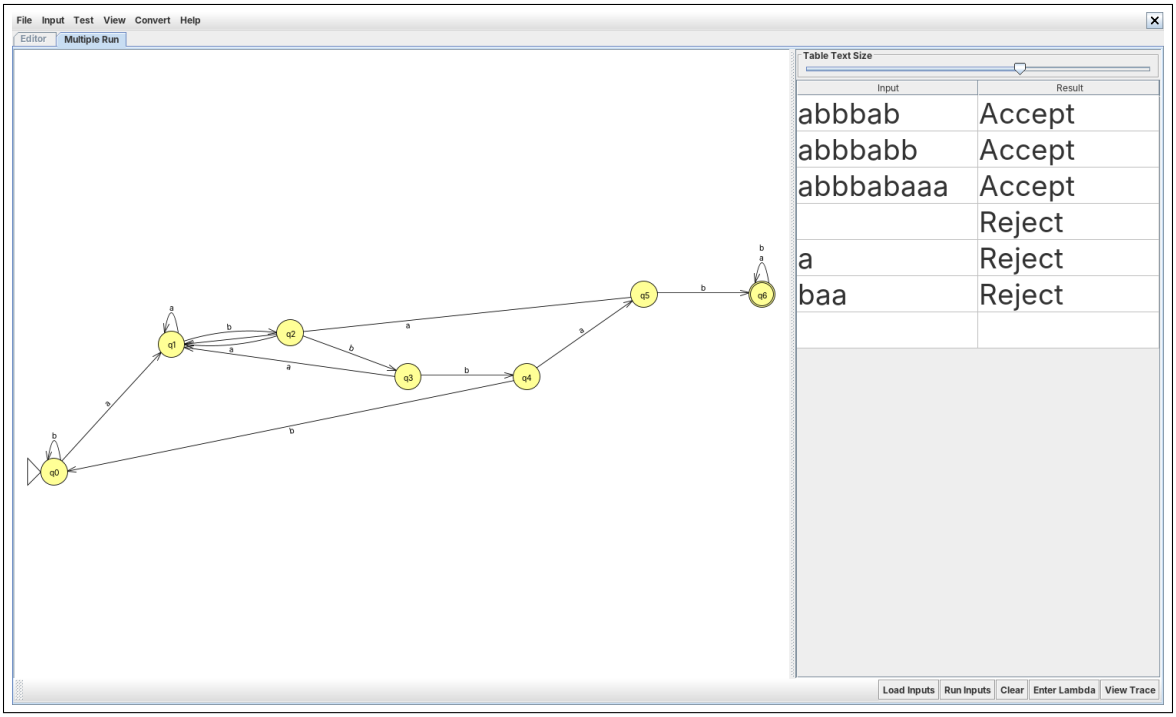


Figura 58. lista3-afn-102 en JFLAP: Multiple Run con las tres cadenas válidas y las tres inválidas de la tabla anterior.

A.2.15. Lista 3 · AFN del problema 103

Enunciado. AFN para el problema 103: Las palabras de $\Sigma 3$ que contienen la cadena aabbba.

Diseño. El AFN adivina dónde empieza aabbba: ciclo inicial, el patrón y ciclo final. No necesita los retrocesos del autómata de Knuth-Morris-Pratt. El enunciado se resuelve de forma natural con un AFN de 7 estados; el AFD equivalente se obtuvo con la construcción de subconjuntos y la minimización y tiene 7 estados.

Archivo. `automatas/lista3/lista3-afn-103.jff` · $\Sigma = \{a, b\}$ · $|Q| = 7$

Tabla 102. Tabla de transiciones (→ inicial, * aceptación, — sin transición).

δ	a	b	Qué recuerda del prefijo leído
→ q0	q1	q0	activos del AFN: {c0}
q1	q2	q0	activos del AFN: {c0,c1}
q2	q2	q3	activos del AFN: {c0,c1,c2}
q3	q1	q4	activos del AFN: {c0,c3}
q4	q1	q5	activos del AFN: {c0,c4}
q5	q6	q0	activos del AFN: {c0,c5}
* q6	q6	q6	activos del AFN: {c0,c1,c6} o {c0,c1,c2,c6} o {c0,c6} o {c0,c3,c6} o {c0,c4,c6} o {c0,c5,c6}

Tabla 103. Cadenas de prueba.

Cadena	Longitud	Esperado	Obtenido	Tipo de caso
`aabbba`	6	acepta	acepta	válida
`baabbba`	7	acepta	acepta	válida
`babaabbba`	9	acepta	acepta	válida
λ	0	rechaza	rechaza	inválida
`a`	1	rechaza	rechaza	inválida
`aaa`	3	rechaza	rechaza	inválida
`ababb`	5	rechaza	rechaza	frontera: justo por debajo de la longitud mínima

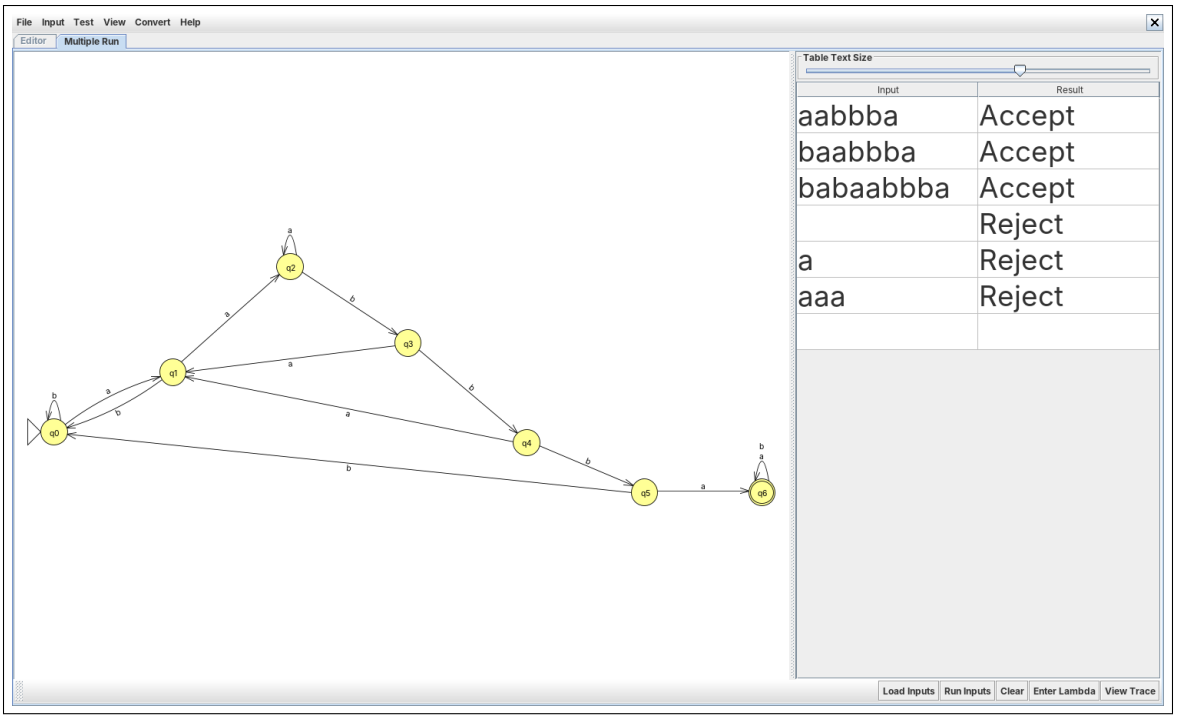


Figura 59. lista3-afn-103 en JFLAP: Multiple Run con las tres cadenas válidas y las tres inválidas de la tabla anterior.

A.2.16. Lista 3 · AFN del problema 106

Enunciado. AFN para el problema 106: Las palabras de $\Sigma 1$ que incluyen la subcadena 1101001.

Diseño. El AFN adivina dónde empieza 1101001: ciclo inicial, el patrón y ciclo final. No necesita los retrocesos del autómata de Knuth-Morris-Pratt. El enunciado se resuelve de forma natural con un AFN de 8 estados; el AFD equivalente se obtuvo con la construcción de subconjuntos y la minimización y tiene 8 estados.

Archivo. `automatas/lista3/lista3-afn-106.jff` · $\Sigma = \{0, 1, 2\}$ · $|Q| = 8$

Tabla 104. Tabla de transiciones ($\rightarrow$ inicial, * aceptación, — sin transición).

δ	0	1	2	Qué recuerda del prefijo leído
$\rightarrow$ q0	q0	q1	q0	activos del AFN: {c0}
q1	q0	q2	q0	activos del AFN: {c0,c1}
q2	q3	q2	q0	activos del AFN: {c0,c1,c2}
q3	q0	q4	q0	activos del AFN: {c0,c3}
q4	q5	q2	q0	activos del AFN: {c0,c1,c4}
q5	q6	q1	q0	activos del AFN: {c0,c5}
q6	q0	q7	q0	activos del AFN: {c0,c6}
* q7	q7	q7	q7	activos del AFN: {c0,c1,c7} o {c0,c7} o {c0,c1,c2,c7} o {c0,c3,c7} o {c0,c1,c4,c7} o {c0,c5,c7} o {c0,c6,c7}

Tabla 105. Cadenas de prueba.

Cadena	Longitud	Esperado	Obtenido	Tipo de caso
`1101001`	7	acepta	acepta	válida
`11010010`	8	acepta	acepta	válida
`1101001110`	10	acepta	acepta	válida
λ	0	rechaza	rechaza	inválida
`2`	1	rechaza	rechaza	inválida
`210`	3	rechaza	rechaza	inválida
`221200`	6	rechaza	rechaza	frontera: justo por debajo de la longitud mínima

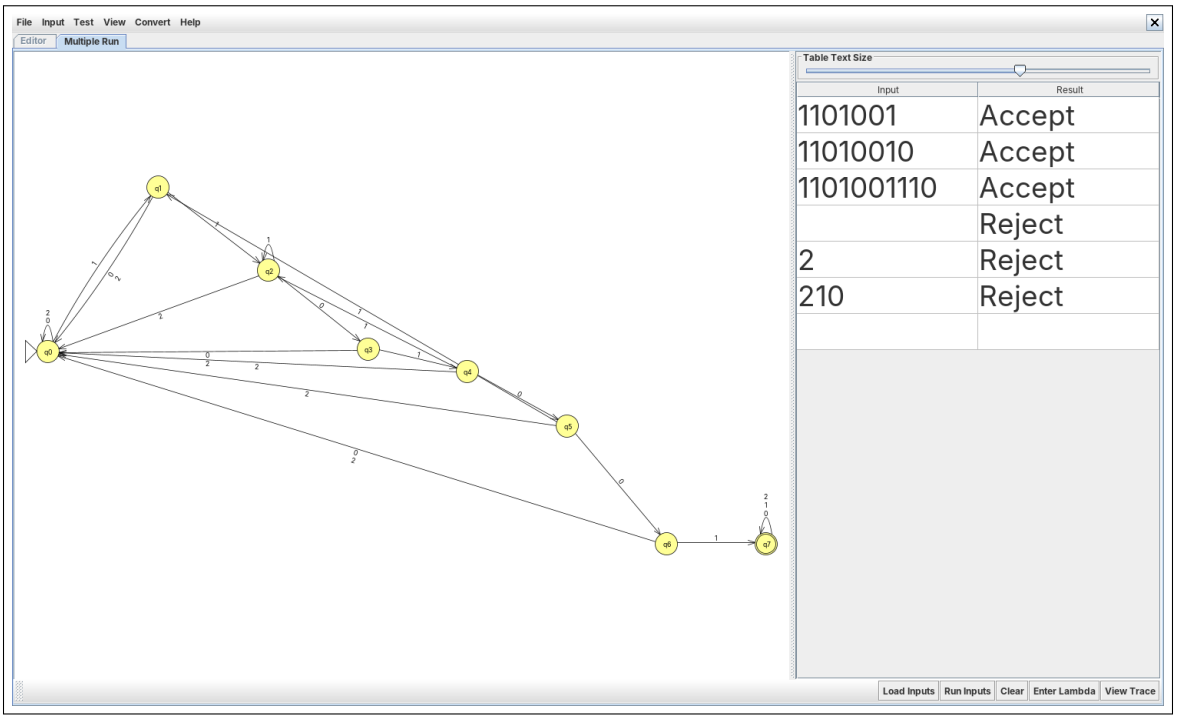


Figura 60. lista3-afn-106 en JFLAP: Multiple Run con las tres cadenas válidas y las tres inválidas de la tabla anterior.

A.2.17. Lista 3 · AFN del problema 107

Enunciado. AFN para el problema 107: Las palabras de $\Sigma 1$ que incluyen al menos una vez 0101 y al menos una vez 021.

Diseño. Intersección de los AFN de «contiene 0101» y «contiene 021» con la construcción de producto; cada estado es un par $c_i c_j$. Como las dos apariciones pueden traslaparse, no basta con concatenar los patrones. El enunciado se resuelve de forma natural con un AFN de 16 estados; el AFD equivalente se obtuvo con la construcción de subconjuntos y la minimización y tiene 13 estados.

Archivo. `automatas/lista3/lista3-afn-107.jff` · $\Sigma = \{0, 1, 2\}$ · $|Q| = 13$

Tabla 106. Tabla de transiciones (→ inicial, * aceptación, — sin transición).

δ	0	1	2	Qué recuerda del prefijo leído
→ q0	q1	q0	q0	activos del AFN: {c0c0}
q1	q1	q2	q3	activos del AFN: {c0c0,c0c1,c1c0,c1c1}
q2	q4	q0	q0	activos del AFN: {c0c0,c2c0}
q3	q1	q5	q0	activos del AFN: {c0c0,c0c2}
q4	q1	q6	q3	activos del AFN: {c0c0,c0c1,c1c0,c1c1,c3c0,c3c1}
q5	q7	q5	q5	activos del AFN: {c0c0,c0c3} o {c0c0,c0c2,c0c3}
q6	q8	q6	q6	activos del AFN: {c0c0,c2c0,c4c0} o {c0c0,c4c0}
q7	q7	q9	q5	activos del AFN: {c0c0,c0c1,c1c0,c1c1,c0c3,c1c3}
q8	q8	q6	q10	activos del AFN: {c0c0,c0c1,c1c0,c1c1,c3c0,c3c1,c4c0,c4c1} o {c0c0,c0c1,c1c0,c1c1,c4c0,c4c1}
q9	q11	q5	q5	activos del AFN: {c0c0,c2c0,c0c3,c2c3}
q10	q8	q12	q6	activos del AFN: {c0c0,c0c2,c4c0,c4c2}
q11	q7	q12	q5	activos del AFN: {c0c0,c0c1,c1c0,c1c1,c0c3,c3c0,c3c1,c1c3,c3c3}
* q12	q12	q12	q12	activos del AFN: {c0c0,c0c3,c4c0,c4c3} o {c0c0,c2c0,c0c3,c4c0,c2c3,c4c3} o {c0c0,c0c1,c1c0,c1c1,c0c3,c1c3,c4c0,c4c1,c4c3} o {c0c0,c0c1,c1c0,c1c1,c0c3,c3c0,c3c1,c1c3,c4c0,c4c1,c3c3, o {c0c0,c0c2,c0c3,c4c0,c4c2,c4c3}

Tabla 107. Cadenas de prueba.

Cadena	Longitud	Esperado	Obtenido	Tipo de caso
`0101021`	7	acepta	acepta	válida
`20210101`	8	acepta	acepta	válida
`0212100101`	10	acepta	acepta	válida
λ	0	rechaza	rechaza	inválida
`0`	1	rechaza	rechaza	inválida
`201`	3	rechaza	rechaza	inválida
`0210101`	7	acepta	acepta	frontera: aceptada más corta

Cadena	Longitud	Esperado	Obtenido	Tipo de caso
`101002`	6	rechaza	rechaza	frontera: justo por debajo de la longitud mínima

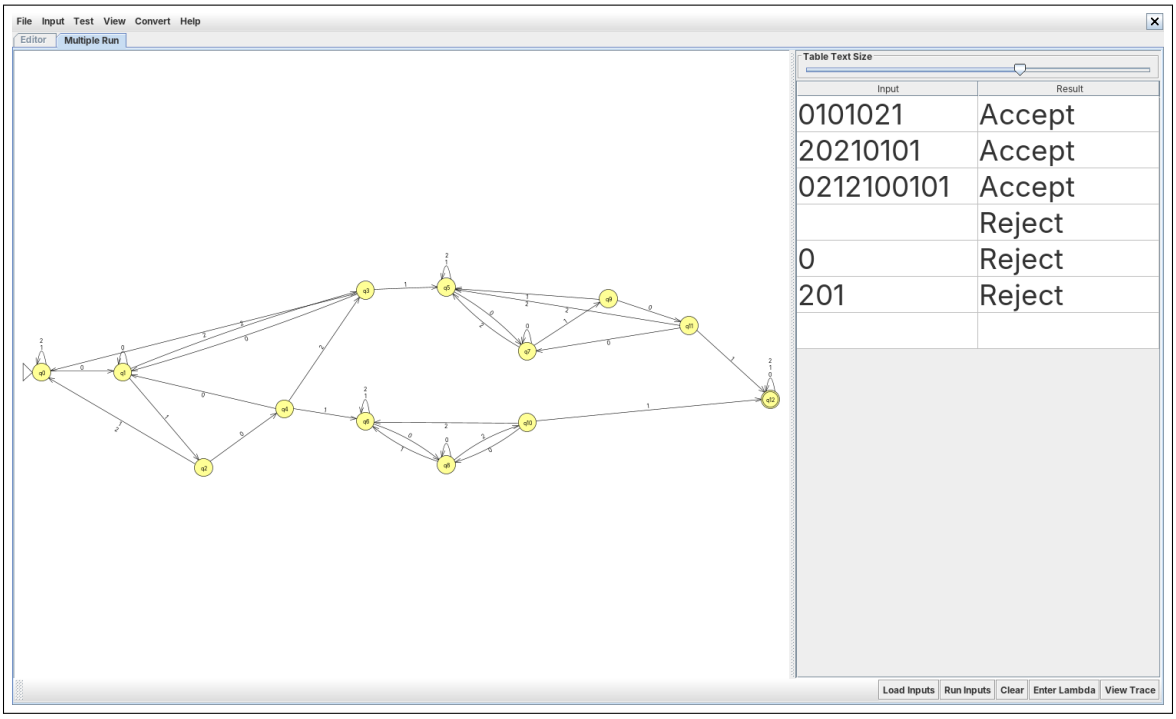


Figura 61. lista3-afn-107 en JFLAP: Multiple Run con las tres cadenas válidas y las tres inválidas de la tabla anterior.

A.2.18. Lista 3 · AFN del problema 109

Enunciado. AFN para el problema 109: Las palabras de Σ2 que incluyen abbcb y cbaabb, que sí se pueden traslapar.

Diseño. Intersección de los dos AFN «contiene» por la construcción de producto, que admite traslapes. El enunciado se resuelve de forma natural con un AFN de 26 estados; el AFD equivalente se obtuvo con la construcción de subconjuntos y la minimización y tiene 22 estados.

Archivo. `automatas/lista3/lista3-afn-109.jff` · Σ = {0, 1, a, b, c} · |Q| = 22

Tabla 108. Tabla de transiciones (→ inicial, * aceptación, — sin transición).

δ	0	1	a	b	c	Qué recuerda del prefijo leído
→ q0	q0	q0	q1	q0	q2	activos del AFN: {c0c0}
q1	q0	q0	q1	q3	q2	activos del AFN: {c0c0,c1c0}
q2	q0	q0	q1	q4	q2	activos del AFN: {c0c0,c0c1}
q3	q0	q0	q1	q5	q2	activos del AFN: {c0c0,c2c0}

δ	0	1	a	b	c	Qué recuerda del prefijo leído
q4	q0	q0	q6	q0	q2	activos del AFN: {c0c0,c0c2}
q5	q0	q0	q1	q0	q7	activos del AFN: {c0c0,c3c0}
q6	q0	q0	q8	q3	q2	activos del AFN: {c0c0,c1c0,c0c3,c1c3}
q7	q0	q0	q1	q9	q2	activos del AFN: {c0c0,c0c1,c4c0,c4c1}
q8	q0	q0	q1	q10	q2	activos del AFN: {c0c0,c1c0,c0c4,c1c4}
q9	q11	q11	q12	q11	q13	activos del AFN: {c0c0,c0c2,c5c0,c5c2}
q10	q0	q0	q1	q14	q2	activos del AFN: {c0c0,c2c0,c0c5,c2c5}
q11	q11	q11	q11	q11	q13	activos del AFN: {c0c0,c5c0} o {c0c0,c1c0,c5c0} o {c0c0,c2c0,c5c0} o {c0c0,c3c0,c5c0}
q12	q11	q11	q15	q11	q13	activos del AFN: {c0c0,c1c0,c0c3,c1c3,c5c0,c5c3}
q13	q11	q11	q11	q9	q13	activos del AFN: {c0c0,c0c1,c5c0,c5c1} o {c0c0,c0c1,c4c0,c4c1,c5c0,c5c1}
q14	q16	q16	q17	q16	q18	activos del AFN: {c0c0,c3c0,c0c6,c3c6}
q15	q11	q11	q11	q19	q13	activos del AFN: {c0c0,c1c0,c0c4,c1c4,c5c0,c5c4}
q16	q16	q16	q17	q16	q16	activos del AFN: {c0c0,c0c6} o {c0c0,c0c1,c0c6} o {c0c0,c0c2,c0c6}
q17	q16	q16	q17	q20	q16	activos del AFN: {c0c0,c1c0,c0c6,c1c6} o {c0c0,c1c0,c0c3,c1c3,c0c6,c1c6} o {c0c0,c1c0,c0c4,c1c4,c0c6,c1c6}
q18	q16	q16	q17	q21	q16	activos del AFN: {c0c0,c0c1,c4c0,c4c1,c0c6,c4c6}
q19	q11	q11	q11	q21	q13	activos del AFN: {c0c0,c2c0,c5c0,c0c5,c2c5,c5c5}
q20	q16	q16	q17	q14	q16	activos del AFN: {c0c0,c2c0,c0c6,c2c6} o {c0c0,c2c0,c0c5,c2c5,c0c6,c2c6}

δ	0	1	a	b	c	Qué recuerda del prefijo leído
* q21	q21	q21	q21	q21	q21	activos del AFN: {c0c0,c0c2,c5c0,c5c2,c0c6,c5c6} o {c0c0,c3c0,c5c0,c0c6,c3c6,c5c6} o {c0c0,c5c0,c0c6,c5c6} o {c0c0,c1c0,c0c3,c1c3,c5c0,c5c3,c0c6,c1c6,c5c6 o {c0c0,c0c1,c5c0,c5c1,c0c6,c5c6} o {c0c0,c1c0,c5c0,c0c6,c1c6,c5c6} o {c0c0,c0c1,c4c0,c4c1,c5c0,c5c1,c0c6,c4c6,c5c6 o {c0c0,c1c0,c0c4,c1c4,c5c0,c0c6,c5c4,c1c6,c5c6 o {c0c0,c2c0,c5c0,c0c6,c2c6,c5c6} o {c0c0,c2c0,c5c0,c0c5,c2c5,c0c6,c5c5,c2c6,c5c6

Tabla 109. Cadenas de prueba.

Cadena	Longitud	Esperado	Obtenido	Tipo de caso
`cbaabbcb`	8	acepta	acepta	válida
`acbaabbcb`	9	acepta	acepta	válida
`1acbaabbcba`	11	acepta	acepta	válida
λ	0	rechaza	rechaza	inválida
`a`	1	rechaza	rechaza	inválida
`1ac`	3	rechaza	rechaza	inválida
`abc0bca`	7	rechaza	rechaza	frontera: justo por debajo de la longitud mínima

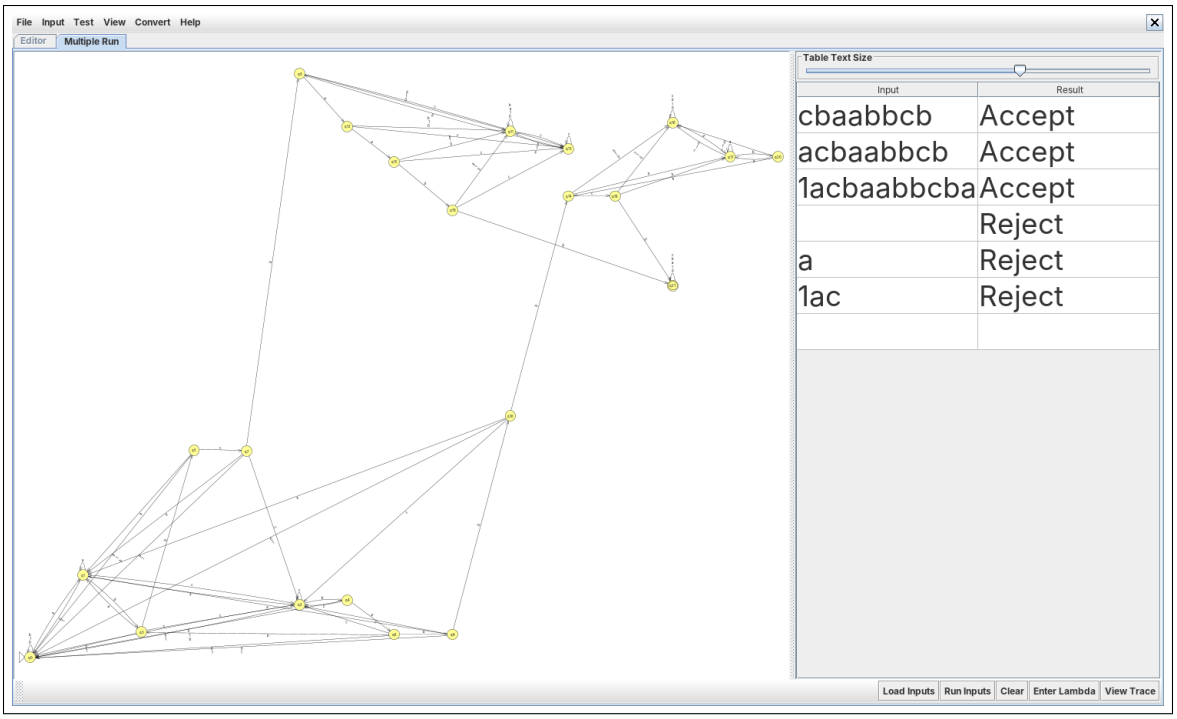


Figura 62. lista3-afn-109 en JFLAP: Multiple Run con las tres cadenas válidas y las tres inválidas de la tabla anterior.

A.2.19. Lista 3 · AFN del problema 111

Enunciado. AFN para el problema 111: Las palabras de $\Sigma 1$ que contienen exactamente tres veces la subcadena 01002.

Diseño. No hay una versión no determinista más simple: el lenguaje depende de contar (longitud, apariciones o fases). Se presenta el AFD del problema sin el estado muerto; una función de transición parcial es un caso particular de AFN.

Archivo. `automatas/lista3/lista3-afn-111.jff` · $\Sigma = \{0, 1, 2\}$ · $|Q| = 20$

Tabla 110. Tabla de transiciones ($\rightarrow$ inicial, * aceptación, — sin transición).

δ	0	1	2
$\rightarrow 0{\cdot}\lambda$	$0{\cdot}0$	$0{\cdot}\lambda$	$0{\cdot}\lambda$
$0{\cdot}0$	$0{\cdot}0$	$0{\cdot}01$	$0{\cdot}\lambda$
$0{\cdot}01$	$0{\cdot}010$	$0{\cdot}\lambda$	$0{\cdot}\lambda$
$0{\cdot}010$	$0{\cdot}0100$	$0{\cdot}01$	$0{\cdot}\lambda$
$0{\cdot}0100$	$0{\cdot}0$	$0{\cdot}01$	$1{\cdot}\lambda$
$1{\cdot}\lambda$	$1{\cdot}0$	$1{\cdot}\lambda$	$1{\cdot}\lambda$
$1{\cdot}0$	$1{\cdot}0$	$1{\cdot}01$	$1{\cdot}\lambda$
$1{\cdot}01$	$1{\cdot}010$	$1{\cdot}\lambda$	$1{\cdot}\lambda$
$1{\cdot}010$	$1{\cdot}0100$	$1{\cdot}01$	$1{\cdot}\lambda$
$1{\cdot}0100$	$1{\cdot}0$	$1{\cdot}01$	$2{\cdot}\lambda$

δ	0	1	2
2·λ	2·0	2·λ	2·λ
2·0	2·0	2·01	2·λ
2·01	2·010	2·λ	2·λ
2·010	2·0100	2·01	2·λ
2·0100	2·0	2·01	3·λ
* 3·λ	3·0	3·λ	3·λ
* 3·0	3·0	3·01	3·λ
* 3·01	3·010	3·λ	3·λ
* 3·010	3·0100	3·01	3·λ
* 3·0100	3·0	3·01	—

Tabla 111. Cadenas de prueba.

Cadena	Longitud	Esperado	Obtenido	Tipo de caso
`010020100201002`	15	acepta	acepta	válida
`0100200100201002`	16	acepta	acepta	válida
`010020201002010021`	18	acepta	acepta	válida
λ	0	rechaza	rechaza	inválida
`1`	1	rechaza	rechaza	inválida
`022`	3	rechaza	rechaza	inválida
`21120022200202`	14	rechaza	rechaza	frontera: justo por debajo de la longitud mínima

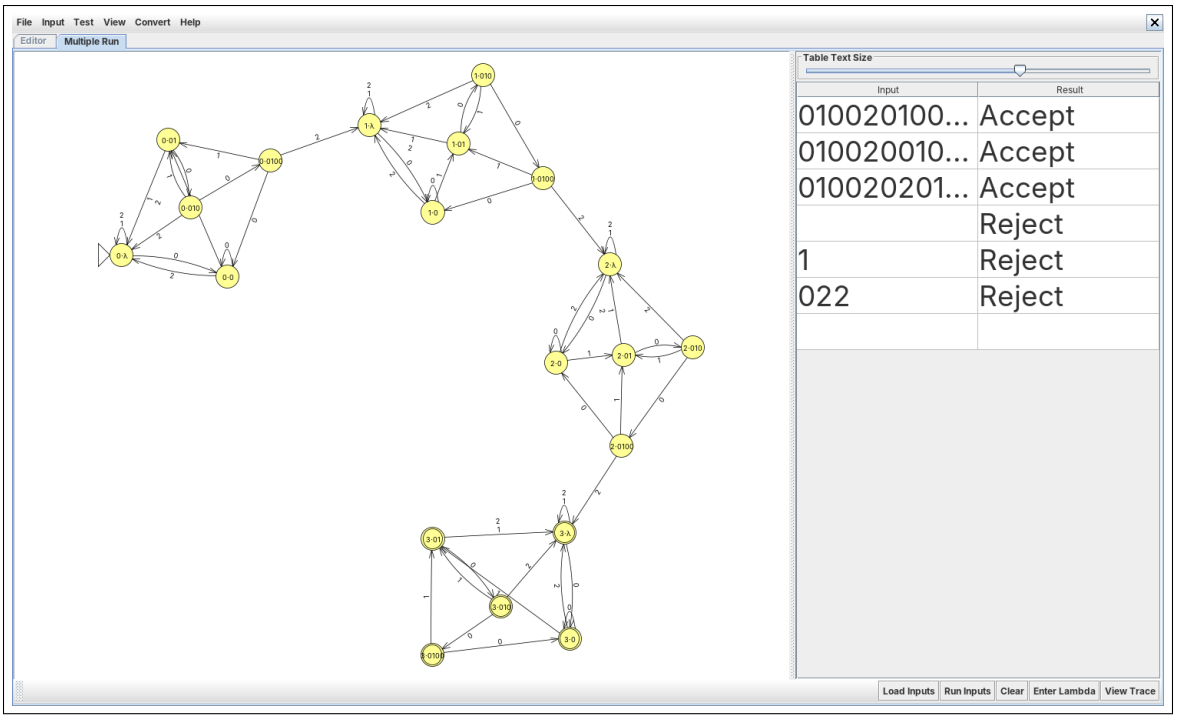


Figura 63. lista3-afn-111 en JFLAP: Multiple Run con las tres cadenas válidas y las tres inválidas de la tabla anterior.

A.2.20. Lista 3 · AFN del problema 112

Enunciado. AFN para el problema 112: Las palabras de $\Sigma 1$ que contienen dos veces 010 sin traslaparse (01010 no pertenece, 010010 sí).

Diseño. $\Sigma 010 \Sigma 010 \Sigma^*$: dos copias de 010 separadas por un ciclo; la segunda empieza después de que termina la primera. El enunciado se resuelve de forma natural con un AFN de 7 estados; el AFD equivalente se obtuvo con la construcción de subconjuntos y la minimización y tiene 7 estados.

Archivo. `automatas/lista3/lista3-afn-112.jff` · $\Sigma = \{0, 1, 2\}$ · $|Q| = 7$

Tabla 112. Tabla de transiciones ($\rightarrow$ inicial, * aceptación, — sin transición).

δ	0	1	2	Qué recuerda del prefijo leído
$\rightarrow$ q0	q1	q0	q0	activos del AFN: {i}
q1	q1	q2	q0	activos del AFN: {i,a1}
q2	q3	q0	q0	activos del AFN: {i,a2}
q3	q4	q3	q3	activos del AFN: {i,a1,a3} o {i,a2,a3} o {i,a3}
q4	q4	q5	q3	activos del AFN: {i,a1,a3,b1}
q5	q6	q3	q3	activos del AFN: {i,a2,a3,b2}
* q6	q6	q6	q6	activos del AFN: {i,a1,a3,b1,b3} o {i,a2,a3,b2,b3} o {i,a3,b3}

Tabla 113. Cadenas de prueba.

Cadena	Longitud	Esperado	Obtenido	Tipo de caso
`010010`	6	acepta	acepta	válida
`0100010`	7	acepta	acepta	válida
`100100100`	9	acepta	acepta	válida
λ	0	rechaza	rechaza	inválida
`1`	1	rechaza	rechaza	inválida
`122`	3	rechaza	rechaza	inválida
`10002`	5	rechaza	rechaza	frontera: justo por debajo de la longitud mínima

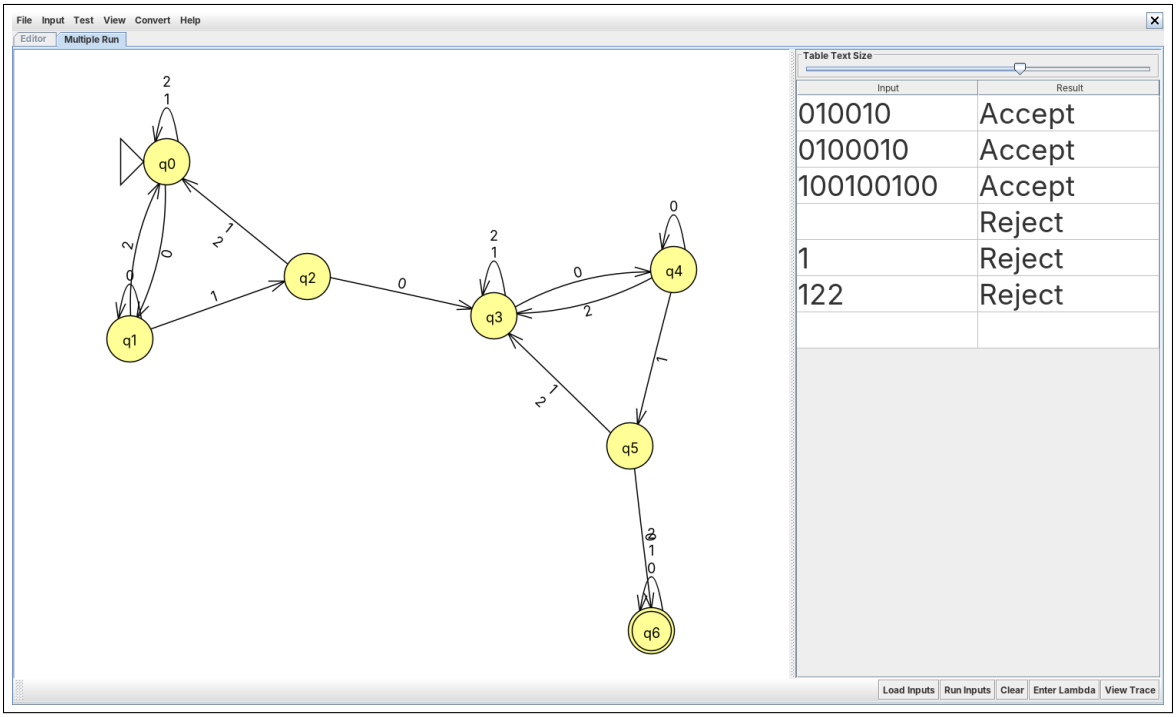


Figura 64. lista3-afn-112 en JFLAP: Multiple Run con las tres cadenas válidas y las tres inválidas de la tabla anterior.

A.2.21. Lista 3 · AFN del problema 113

Enunciado. AFN para el problema 113: Las palabras de $\Sigma 1$ que contienen dos veces 010, que sí se pueden traslapar.

Diseño. $\Sigma(010\Sigma 010 + 01010)\Sigma^*$: dos apariciones disjuntas o una sola cadena 01010 en la que comparten el 0 central. El enunciado se resuelve de forma natural con un AFN de 11 estados; el AFD equivalente se obtuvo con la construcción de subconjuntos y la minimización y tiene 7 estados.

Archivo. `automatas/lista3/lista3-afn-113.jff` · $\Sigma = \{0, 1, 2\}$ · $|Q| = 7$

Tabla 114. Tabla de transiciones (→ inicial, * aceptación, — sin transición).

δ	0	1	2	Qué recuerda del prefijo leído
→ q0	q1	q0	q0	activos del AFN: {i}
q1	q1	q2	q0	activos del AFN: {i,a1,o1}
q2	q3	q0	q0	activos del AFN: {i,a2,o2}
q3	q3	q4	q5	activos del AFN: {i,a1,a3,o1,o3} o {i,a1,a3,b1,o1}
q4	q6	q5	q5	activos del AFN: {i,a2,a3,o2,o4} o {i,a2,a3,b2,o2}
q5	q3	q5	q5	activos del AFN: {i,a3}
* q6	q6	q6	q6	activos del AFN: {i,a1,a3,b1,b3,o1,o3} o {i,a1,a3,b1,b3,o1} o {i,a2,a3,b2,b3,o2,o4} o {i,a3,b3} o {i,a2,a3,b2,b3,o2}

Tabla 115. Cadenas de prueba.

Cadena	Longitud	Esperado	Obtenido	Tipo de caso
`01010`	5	acepta	acepta	válida
`010101`	6	acepta	acepta	válida
`20101001`	8	acepta	acepta	válida
λ	0	rechaza	rechaza	inválida
`0`	1	rechaza	rechaza	inválida
`222`	3	rechaza	rechaza	inválida
`0111`	4	rechaza	rechaza	frontera: justo por debajo de la longitud mínima

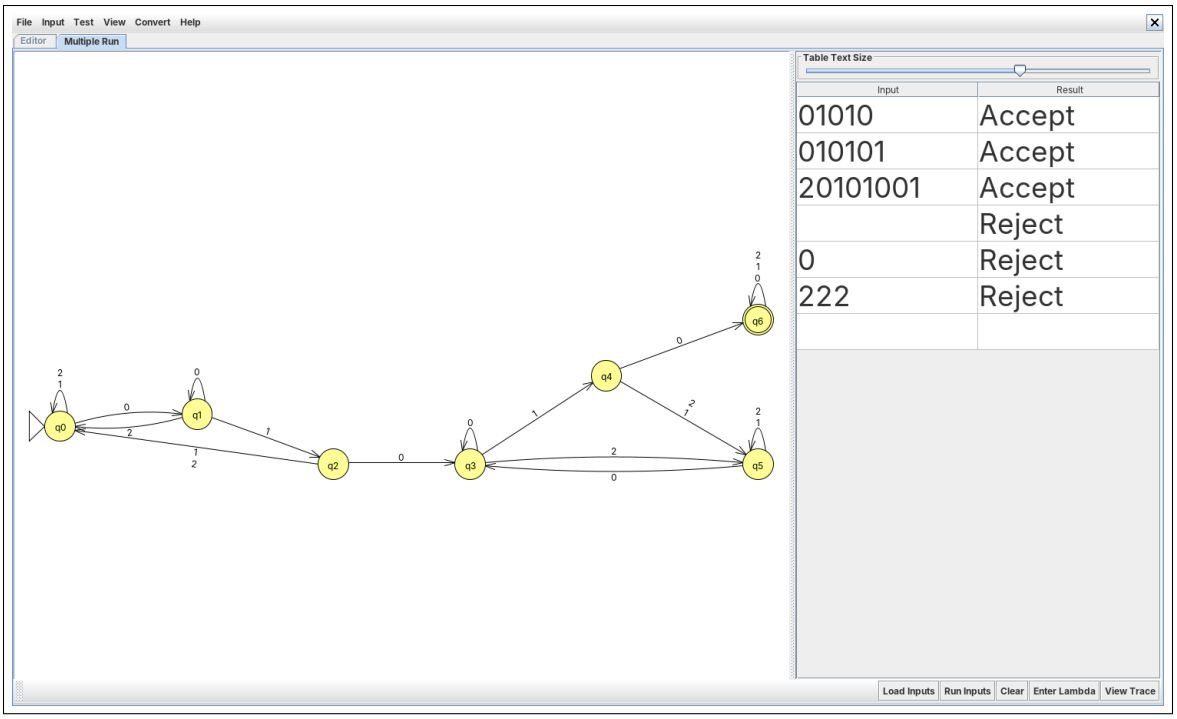


Figura 65. lista3-afn-113 en JFLAP: Multiple Run con las tres cadenas válidas y las tres inválidas de la tabla anterior.

A.2.22. Lista 3 · AFN del problema 116

Enunciado. AFN para el problema 116: L = {w ∈ Σ2* : w = $a^n b^m c^{n+1}$, m, n ∈ N}.

El lenguaje no es regular; tampoco existe AFN.

A.2.23. Lista 3 · AFN del problema 118

Enunciado. AFN para el problema 118: L = {w ∈ Σ3* : w = axabya, x, y ∈ Σ3*}.

Diseño. Sigue literalmente axabya: a, ciclo x, ab, ciclo y, a final. El enunciado se resuelve de forma natural con un AFN de 5 estados; el AFD equivalente se obtuvo con la construcción de subconjuntos y la minimización y tiene 5 estados.

Archivo. `automatas/lista3/lista3-afn-118.jff` · Σ = {a, b} · |Q| = 5

Tabla 116. Tabla de transiciones (→ inicial, * aceptación, — sin transición).

δ	a	b	Qué recuerda del prefijo leído
→ q0	q1	—	activos del AFN: {q0}
q1	q2	q1	activos del AFN: {q1}
q2	q2	q3	activos del AFN: {q1,q2}
q3	q4	q3	activos del AFN: {q1,q3}
* q4	q4	q3	activos del AFN: {q1,q2,q3,q4}

Tabla 117. Cadenas de prueba.

Cadena	Longitud	Esperado	Obtenido	Tipo de caso
`aaba`	4	acepta	acepta	válida
`aabba`	5	acepta	acepta	válida
`abababaaa`	7	acepta	acepta	válida
λ	0	rechaza	rechaza	inválida
`a`	1	rechaza	rechaza	inválida
`baa`	3	rechaza	rechaza	inválida

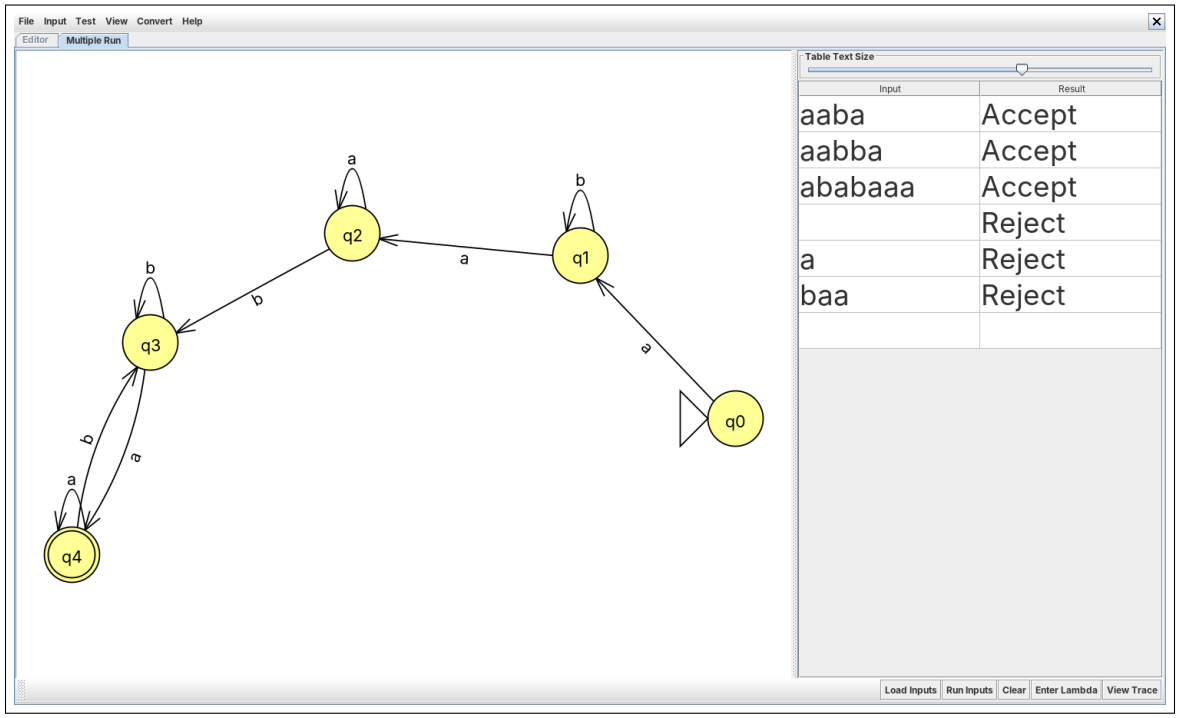


Figura 66. lista3-afn-118 en JFLAP: Multiple Run con las tres cadenas válidas y las tres inválidas de la tabla anterior.

A.2.24. Lista 3 · AFN del problema 120

Enunciado. AFN para el problema 120: Las palabras de $\Sigma 1$ tales que el primer carácter y el último son distintos.

Diseño. El estado s_x recuerda el primer símbolo x; el AFN adivina el último símbolo, que debe ser distinto de x. El enunciado se resuelve de forma natural con un AFN de 5 estados; el AFD equivalente se obtuvo con la construcción de subconjuntos y la minimización y tiene 7 estados.

Archivo. `automatas/lista3/lista3-afn-120.jff` · $\Sigma = \{0, 1, 2\}$ · $|Q| = 7$

Tabla 118. Tabla de transiciones (→ inicial, * aceptación, — sin transición).

δ	0	1	2	Qué recuerda del prefijo leído
→ q0	q1	q2	q3	activos del AFN: {q0}
q1	q1	q4	q4	activos del AFN: {s0}
q2	q5	q2	q5	activos del AFN: {s1}
q3	q6	q6	q3	activos del AFN: {s2}
* q4	q1	q4	q4	activos del AFN: {s0,f}
* q5	q5	q2	q5	activos del AFN: {s1,f}
* q6	q6	q6	q3	activos del AFN: {s2,f}

Tabla 119. Cadenas de prueba.

Cadena	Longitud	Esperado	Obtenido	Tipo de caso
`12`	2	acepta	acepta	válida
`211`	3	acepta	acepta	válida
`12002`	5	acepta	acepta	válida
λ	0	rechaza	rechaza	inválida
`1`	1	rechaza	rechaza	inválida
`222`	3	rechaza	rechaza	inválida
`02`	2	acepta	acepta	frontera: aceptada más corta
`2`	1	rechaza	rechaza	frontera: justo por debajo de la longitud mínima

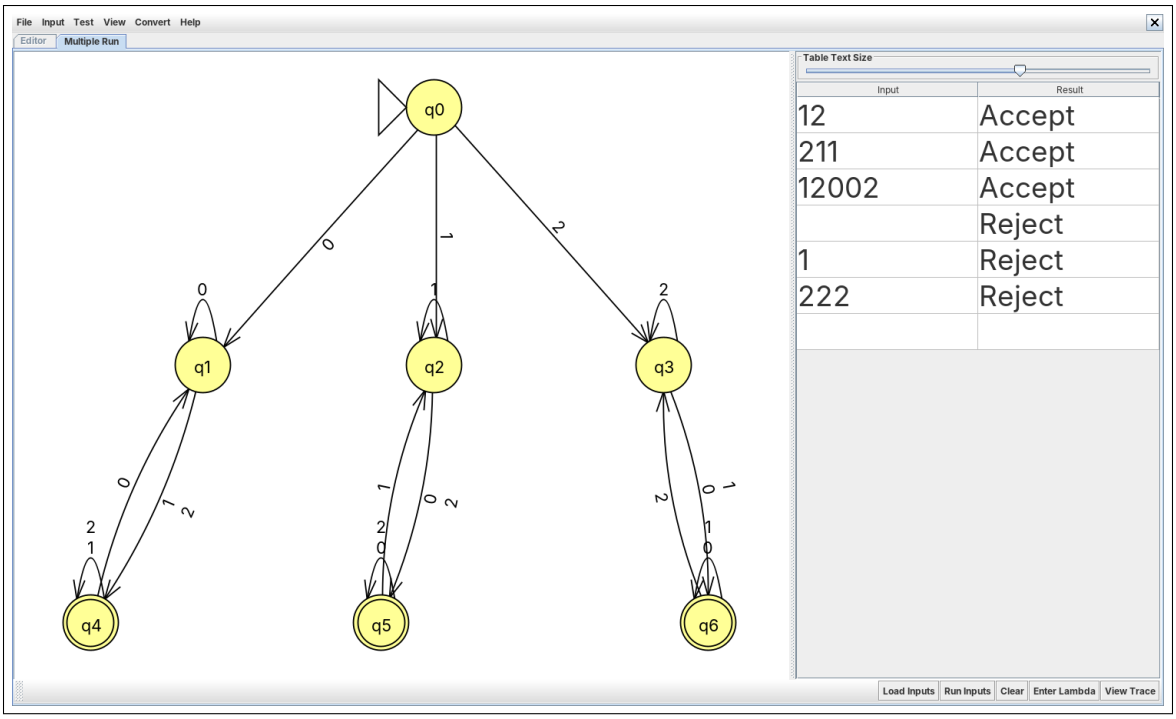


Figura 67. lista3-afn-120 en JFLAP: Multiple Run con las tres cadenas válidas y las tres inválidas de la tabla anterior.

A.2.25. Lista 3 · AFN del problema 121

Enunciado. AFN para el problema 121: Las palabras de $\Sigma 1$ de longitud 3n tales que cada bloque de longitud 3 tiene al menos un 0.

Diseño. No hay una versión no determinista más simple: el lenguaje depende de contar (longitud, apariciones o fases). Se presenta el AFD del problema sin el estado muerto; una función de transición parcial es un caso particular de AFN.

Archivo. `automatas/lista3/lista3-afn-121.jff` · $\Sigma = \{0, 1, 2\}$ · $|Q| = 5$

Tabla 120. Tabla de transiciones ($\rightarrow$ inicial, * aceptación, — sin transición).

δ	0	1	2
$\rightarrow$* b0	b1+0	b1	b1
b1+0	b2+0	b2+0	b2+0
b1	b2+0	b2	b2
b2+0	b0	b0	b0
b2	b0	—	—

Tabla 121. Cadenas de prueba.

Cadena	Longitud	Esperado	Obtenido	Tipo de caso
λ	0	acepta	acepta	válida
`021`	3	acepta	acepta	válida
`020001`	6	acepta	acepta	válida
`2`	1	rechaza	rechaza	inválida
`01`	2	rechaza	rechaza	inválida
`0022`	4	rechaza	rechaza	inválida

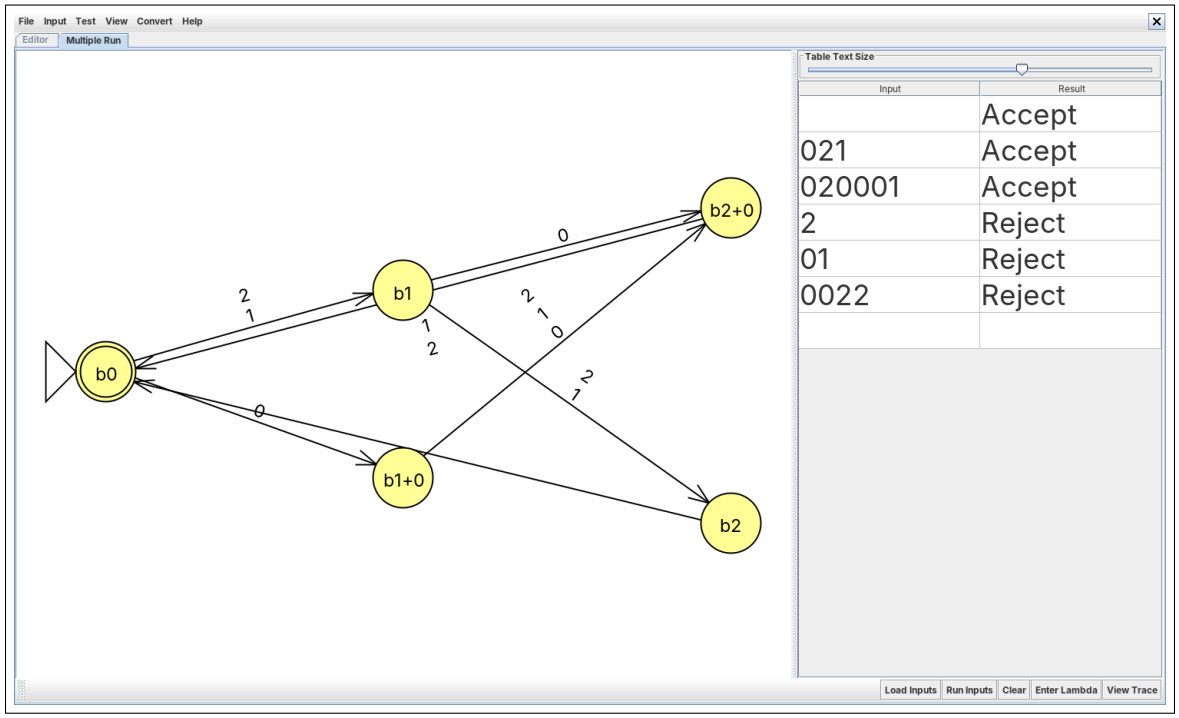


Figura 68. lista3-afn-121 en JFLAP: Multiple Run con las tres cadenas válidas y las tres inválidas de la tabla anterior.

A.2.26. Lista 3 · AFN del problema 122

Enunciado. AFN para el problema 122: Las palabras de $\Sigma 1$ tales que cualquier cadena de tres símbolos consecutivos tiene un 0.

Diseño. No hay una versión no determinista más simple: el lenguaje depende de contar (longitud, apariciones o fases). Se presenta el AFD del problema sin el estado muerto; una función de transición parcial es un caso particular de AFN.

Archivo. `automatas/lista3/lista3-afn-122.jff` · $\Sigma = \{0, 1, 2\}$ · $|Q| = 3$

Tabla 122. Tabla de transiciones (→ inicial, * aceptación, — sin transición).

δ	0	1	2
→* 0 sin 0	0 sin 0	1 sin 0	1 sin 0
* 1 sin 0	0 sin 0	2 sin 0	2 sin 0
* 2 sin 0	0 sin 0	—	—

Tabla 123. Cadenas de prueba.

Cadena	Longitud	Esperado	Obtenido	Tipo de caso
λ	0	acepta	acepta	válida
`1`	1	acepta	acepta	válida
`012`	3	acepta	acepta	válida
`112`	3	rechaza	rechaza	inválida
`1120`	4	rechaza	rechaza	inválida
`202211`	6	rechaza	rechaza	inválida

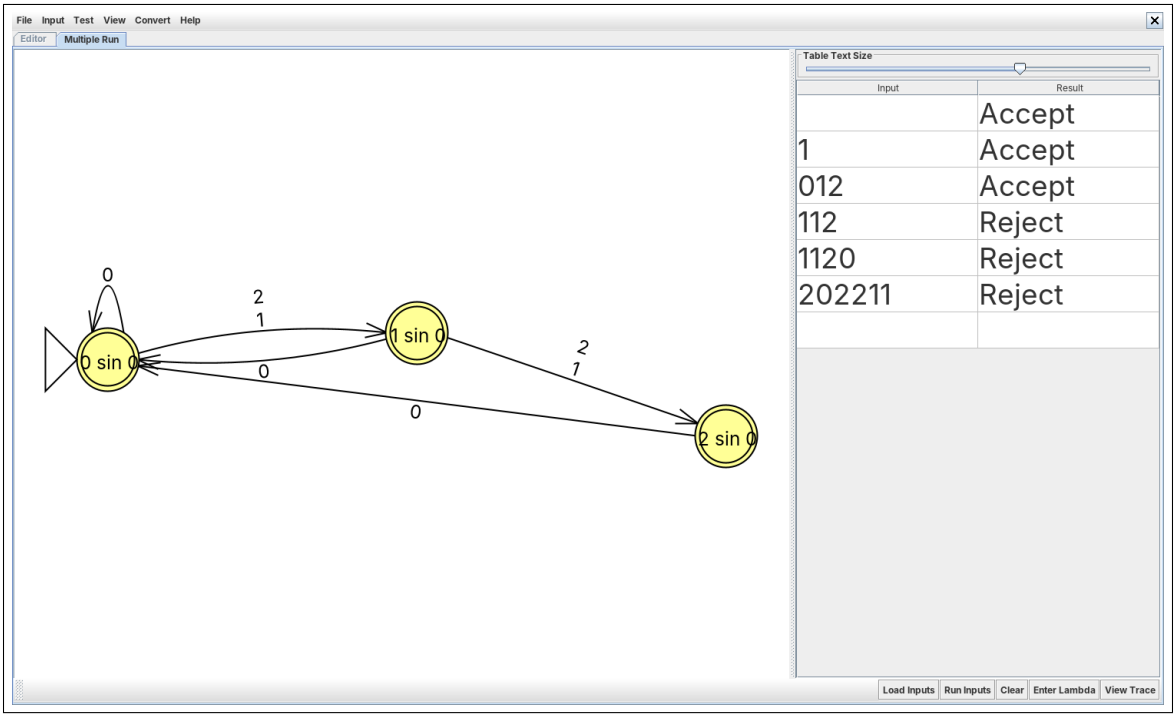


Figura 69. lista3-afn-122 en JFLAP: Multiple Run con las tres cadenas válidas y las tres inválidas de la tabla anterior.

A.2.27. Lista 3 · AFN del problema 123

Enunciado. AFN para el problema 123: Las palabras de Σ3 de la forma wxw donde w ∈ Σ3² y x ∈ Σ3*.

Diseño. Una rama por cada $w \in \Sigma 3^2$: se lee w, un ciclo para x y otra vez w. El AFN adivina cuándo empieza la copia final. El enunciado se resuelve de forma natural con un AFN de 14 estados; el AFD equivalente se obtuvo con la construcción de subconjuntos y la minimización y tiene 15 estados.

Archivo. `automatas/lista3/lista3-afn-123.jff` · $\Sigma = \{a, b\}$ · $|Q| = 15$

Tabla 124. Tabla de transiciones ($\rightarrow$ inicial, * aceptación, — sin transición).

δ	a	b	Qué recuerda del prefijo leído
$\rightarrow$ q0	q1	q2	activos del AFN: {q0}
q1	q3	q4	activos del AFN: {aa1,ab1}
q2	q5	q6	activos del AFN: {ba1,bb1}
q3	q7	q3	activos del AFN: {aa2}
q4	q8	q4	activos del AFN: {ab2}
q5	q5	q9	activos del AFN: {ba2}
q6	q6	q10	activos del AFN: {bb2}
q7	q11	q3	activos del AFN: {aa2,aa3}
q8	q8	q12	activos del AFN: {ab2,ab3}
q9	q13	q9	activos del AFN: {ba2,ba3}
q10	q6	q14	activos del AFN: {bb2,bb3}
* q11	q11	q3	activos del AFN: {aa2,aa3,F}
* q12	q8	q4	activos del AFN: {ab2,F}
* q13	q5	q9	activos del AFN: {ba2,F}
* q14	q6	q14	activos del AFN: {bb2,bb3,F}

Tabla 125. Cadenas de prueba.

Cadena	Longitud	Esperado	Obtenido	Tipo de caso
`abab`	4	acepta	acepta	válida
`aaaaa`	5	acepta	acepta	válida
`aababaa`	7	acepta	acepta	válida
λ	0	rechaza	rechaza	inválida
`a`	1	rechaza	rechaza	inválida
`bba`	3	rechaza	rechaza	inválida
`aaaa`	4	acepta	acepta	frontera: aceptada más corta
`aab`	3	rechaza	rechaza	frontera: justo por debajo de la longitud mínima

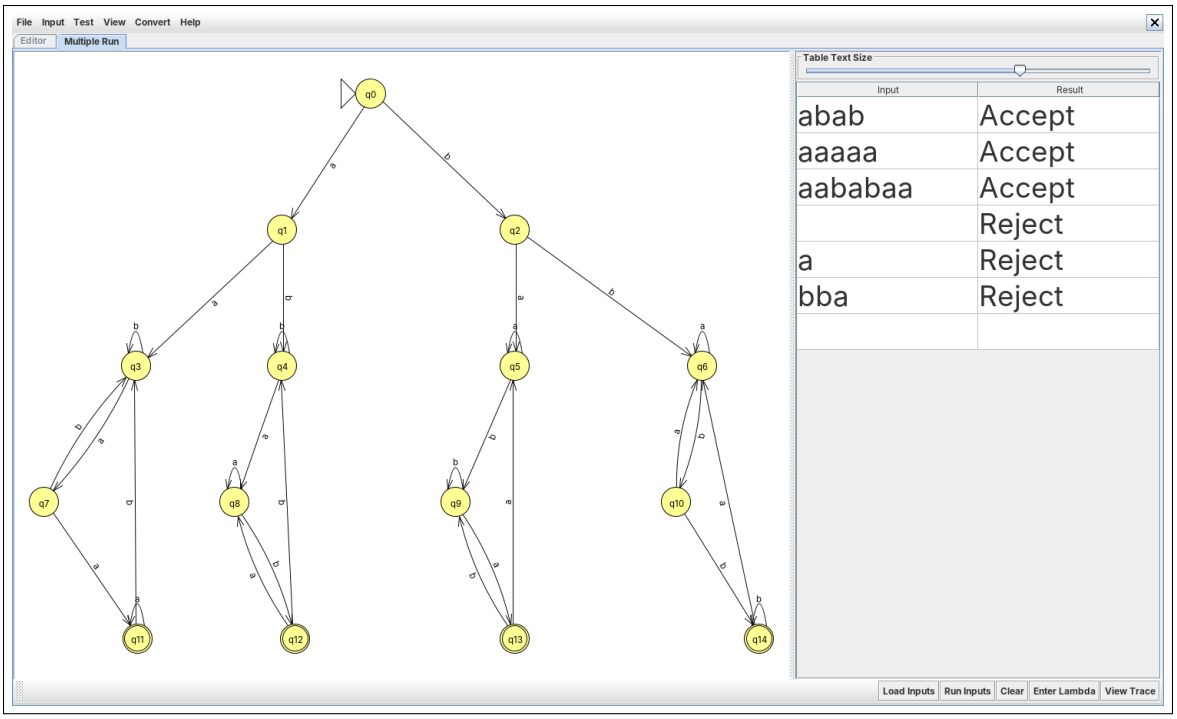


Figura 70. lista3-afn-123 en JFLAP: Multiple Run con las tres cadenas válidas y las tres inválidas de la tabla anterior.

A.2.28. Lista 3 · Ejercicio 189

Enunciado. AFN sobre {0, 1} para las palabras que contienen 00101 o 00110; recorrido de 1001011.

Diseño. Los dos patrones comparten el prefijo 001, así que se escribe una sola vez y se bifurca en 01 y 10. El enunciado se resuelve de forma natural con un AFN de 7 estados; el AFD equivalente se obtuvo con la construcción de subconjuntos y la minimización y tiene 7 estados.

Archivo. `automatas/lista3/lista3-189.jff` · $\Sigma = \{0, 1\}$ · $|Q| = 7$

Tabla 126. Tabla de transiciones ($\rightarrow$ inicial, * aceptación, — sin transición).

δ	0	1	Qué recuerda del prefijo leído
$\rightarrow$ q0	q1	q0	activos del AFN: {i}
q1	q2	q0	activos del AFN: {i,c1}
q2	q2	q3	activos del AFN: {i,c1,c2}
q3	q4	q5	activos del AFN: {i,c3}
q4	q2	q6	activos del AFN: {i,c1,u4}
q5	q6	q0	activos del AFN: {i,d4}
* q6	q6	q6	activos del AFN: {i,f} o {i,c1,f} o {i,c1,c2,f} o {i,c3,f} o {i,c1,u4,f} o {i,f,d4}

Tabla 127. Cadenas de prueba.

Cadena	Longitud	Esperado	Obtenido	Tipo de caso
`00110`	5	acepta	acepta	válida
`100110`	6	acepta	acepta	válida
`00101100`	8	acepta	acepta	válida
λ	0	rechaza	rechaza	inválida
`0`	1	rechaza	rechaza	inválida
`101`	3	rechaza	rechaza	inválida
`00101`	5	acepta	acepta	frontera: aceptada más corta
`0110`	4	rechaza	rechaza	frontera: justo por debajo de la longitud mínima

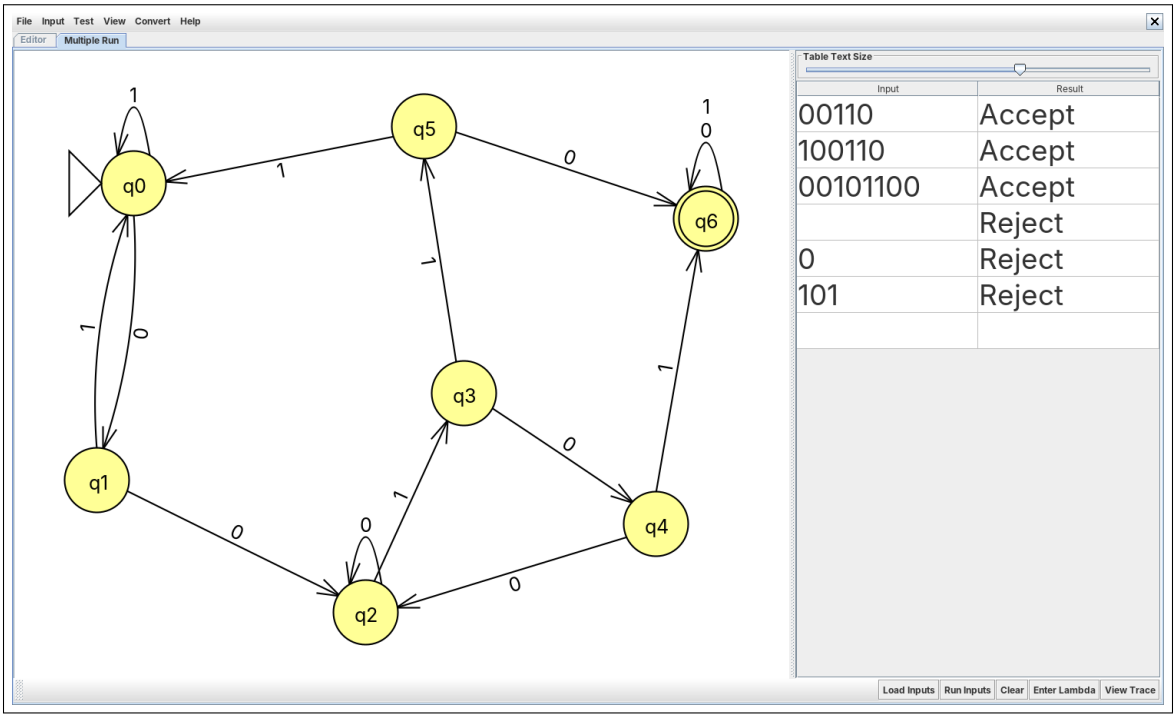


Figura 71. lista3-189 en JFLAP: Multiple Run con las tres cadenas válidas y las tres inválidas de la tabla anterior.

A.2.29. Lista 3 · Ejercicio 190

Enunciado. AFN sobre $\{0, 1\}$ para las palabras que contienen 11010 y 10100; recorrido de 01101010100.

Diseño. Producto de los AFN «contiene 11010» y «contiene 10100»; los patrones pueden traslaparse (110100 contiene ambos), por eso no basta concatenarlos. El enunciado se resuelve de forma natural con un AFN de 26 estados; el AFD equivalente se obtuvo con la construcción de subconjuntos y la minimización y tiene 19 estados.

Archivo. `automatas/lista3/lista3-190.jff` · $\Sigma = \{0, 1\}$ · $|Q| = 19$

Tabla 128. Tabla de transiciones (→ inicial, * aceptación, — sin transición).

δ	0	1	Qué recuerda del prefijo leído
→ q0	q0	q1	activos del AFN: {c0c0}
q1	q2	q3	activos del AFN: {c0c0,c0c1,c1c0,c1c1}
q2	q0	q4	activos del AFN: {c0c0,c0c2}
q3	q5	q3	activos del AFN: {c0c0,c0c1,c1c0,c1c1,c2c0,c2c1}
q4	q6	q3	activos del AFN: {c0c0,c0c1,c1c0,c1c1,c0c3,c1c3}
q5	q0	q7	activos del AFN: {c0c0,c0c2,c3c0,c3c2}
q6	q8	q4	activos del AFN: {c0c0,c0c2,c0c4}
q7	q9	q3	activos del AFN: {c0c0,c0c1,c1c0,c1c1,c0c3,c1c3,c4c0,c4c1,c4c3}
q8	q8	q10	activos del AFN: {c0c0,c0c5} o {c0c0,c0c2,c0c5} o {c0c0,c0c2,c0c4,c0c5}
q9	q11	q12	activos del AFN: {c0c0,c0c2,c0c4,c5c0,c5c2,c5c4}
q10	q8	q13	activos del AFN: {c0c0,c0c1,c1c0,c1c1,c0c5,c1c5} o {c0c0,c0c1,c1c0,c1c1,c0c3,c1c3,c0c5,c1c5}
* q11	q11	q11	activos del AFN: {c0c0,c0c5,c5c0,c5c5} o {c0c0,c0c1,c1c0,c1c1,c0c5,c5c0,c1c5,c5c1,c5c5} o {c0c0,c0c2,c0c5,c5c0,c5c2,c5c5} o {c0c0,c0c1,c1c0,c1c1,c2c0,c2c1,c0c5,c5c0,c1c5,c5c1,c5c5,c2c5} o {c0c0,c0c1,c1c0,c1c1,c0c3,c1c3,c0c5,c5c0,c1c5,c5c1,c5c3,c5c5} o {c0c0,c0c2,c3c0,c3c2,c0c5,c5c0,c5c2,c5c5,c3c5} o {c0c0,c0c2,c0c4,c0c5,c5c0,c5c2,c5c4,c5c5} o {c0c0,c0c1,c1c0,c1c1,c0c3,c1c3,c4c0,c4c1,c4c3,c0c5,c5c0,c1c5,c5c
q12	q9	q14	activos del AFN: {c0c0,c0c1,c1c0,c1c1,c0c3,c1c3,c5c0,c5c1,c5c3} o {c0c0,c0c1,c1c0,c1c1,c0c3,c1c3,c4c0,c4c1,c4c3,c5c0,c5c1,c5c3}
q13	q15	q13	activos del AFN: {c0c0,c0c1,c1c0,c1c1,c2c0,c2c1,c0c5,c1c5,c2c5}
q14	q16	q14	activos del AFN: {c0c0,c0c1,c1c0,c1c1,c2c0,c2c1,c5c0,c5c1} o {c0c0,c0c1,c1c0,c1c1,c5c0,c5c1}
q15	q8	q17	activos del AFN: {c0c0,c0c2,c3c0,c3c2,c0c5,c3c5}
q16	q18	q12	activos del AFN: {c0c0,c0c2,c3c0,c3c2,c5c0,c5c2} o {c0c0,c0c2,c5c0,c5c2}
q17	q11	q13	activos del AFN: {c0c0,c0c1,c1c0,c1c1,c0c3,c1c3,c4c0,c4c1,c4c3,c0c5,c1c5,c4c5}

q18	q18	q14	activos del AFN: {c0c0,c5c0}

Tabla 129. Cadenas de prueba.

Cadena	Longitud	Esperado	Obtenido	Tipo de caso
`110100`	6	acepta	acepta	válida
`1101001`	7	acepta	acepta	válida
`110100010`	9	acepta	acepta	válida
λ	0	rechaza	rechaza	inválida
`0`	1	rechaza	rechaza	inválida
`001`	3	rechaza	rechaza	inválida
`00010`	5	rechaza	rechaza	frontera: justo por debajo de la longitud mínima

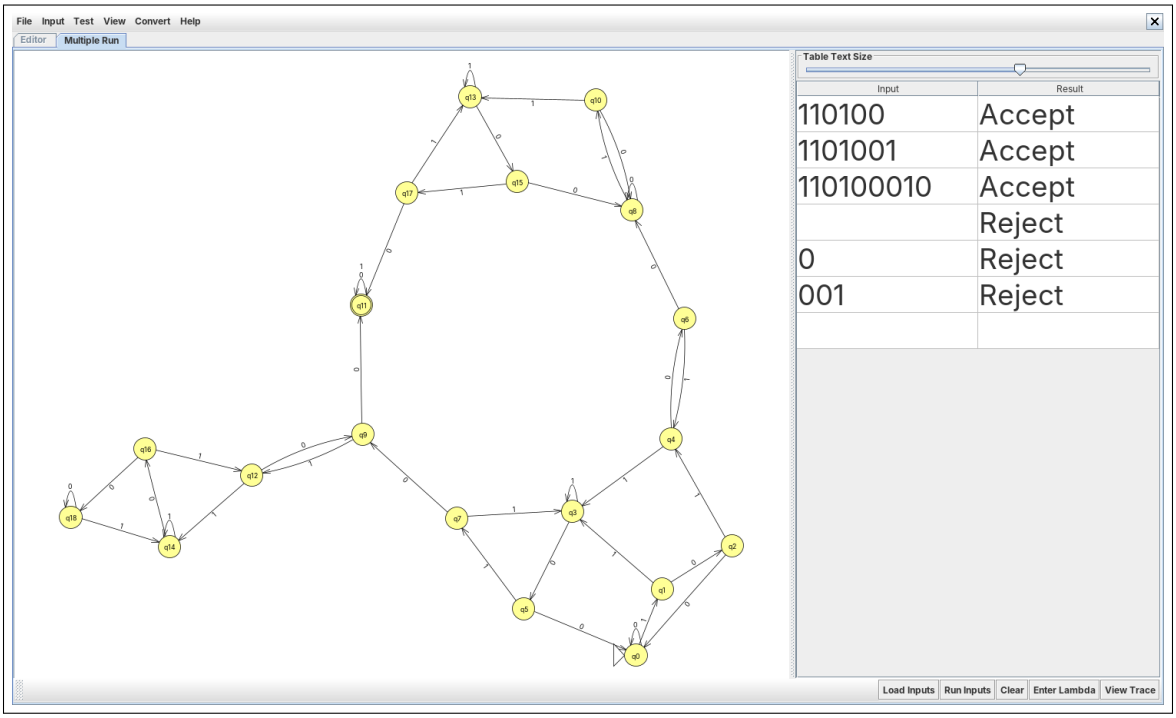


Figura 72. lista3-190 en JFLAP: Multiple Run con las tres cadenas válidas y las tres inválidas de la tabla anterior.

A.2.30. Lista 3 · Ejercicio 191

Enunciado. Instrucciones de 192-195: AFD equivalente, δ(Z, ω) y δ_D(Z, ω).
Ejercicio de cálculo sobre un AFN dado; se resuelve en la Tarea 3.

A.2.31. Lista 3 · Ejercicio 192

Enunciado. AFN de la tabla 192: AFD equivalente y δ(Z, ω).

Diseño. Construcción de subconjuntos (sólo alcanzables). El AFD asociado tiene un estado por subconjunto, así que δ_D(Z, ω) es el subconjunto δ(Z, ω). El enunciado se resuelve de forma natural con un AFN de 3 estados; el AFD equivalente se obtuvo con la construcción de subconjuntos y la minimización y tiene 5 estados.

Archivo. `automatas/lista3/lista3-192.jff` · $\Sigma = \{0, 1\}$ · $|Q| = 5$

Tabla 130. Tabla de transiciones (→ inicial, * aceptación, — sin transición).

δ	0	1	Qué recuerda del prefijo leído
→ q0	q1	q2	activos del AFN: {A}
q1	q3	—	activos del AFN: {B}
* q2	q4	—	activos del AFN: {B,C}
q3	q3	q2	activos del AFN: {A,B}
* q4	q4	q2	activos del AFN: {A,B,C}

Tabla 131. Cadenas de prueba.

Cadena	Longitud	Esperado	Obtenido	Tipo de caso
`1`	1	acepta	acepta	válida
`10`	2	acepta	acepta	válida
`1010`	4	acepta	acepta	válida
λ	0	rechaza	rechaza	inválida
`0`	1	rechaza	rechaza	inválida
`111`	3	rechaza	rechaza	inválida

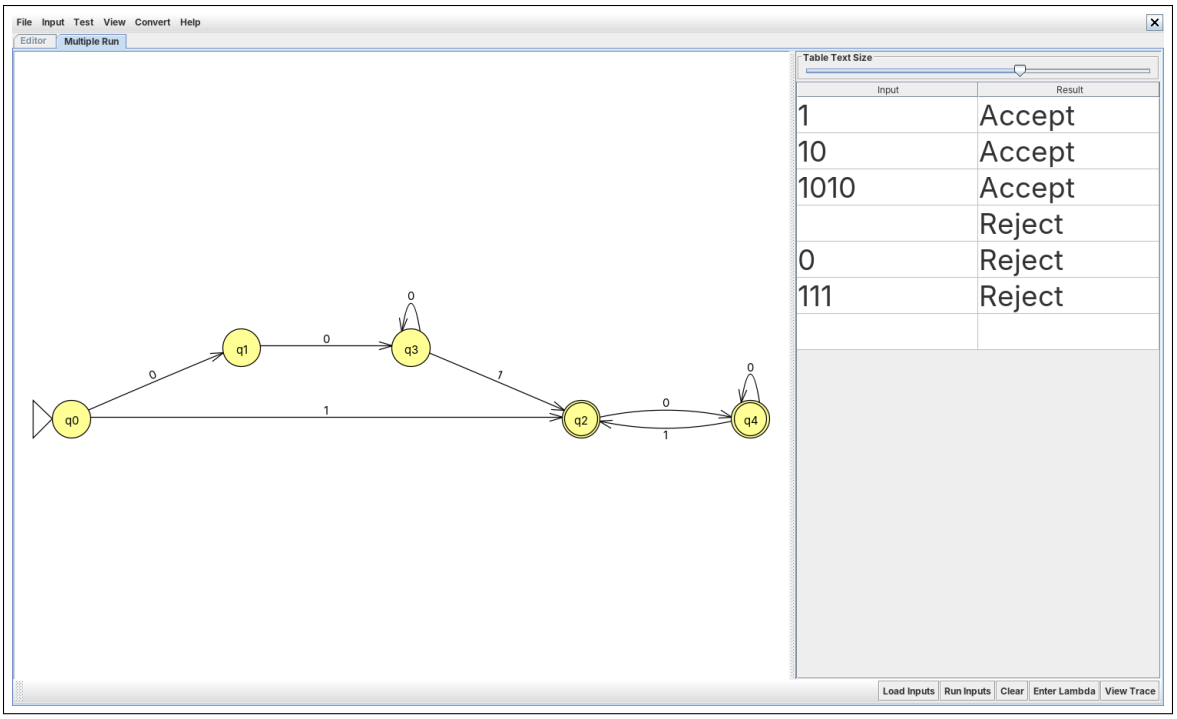


Figura 73. lista3-192 en JFLAP: Multiple Run con las tres cadenas válidas y las tres inválidas de la tabla anterior.

A.2.32. Lista 3 · Ejercicio 193

Enunciado. AFN de la tabla 193: AFD equivalente y δ(Z, ω).

Diseño. Construcción de subconjuntos (sólo alcanzables). El AFD asociado tiene un estado por subconjunto, así que δ_D(Z, ω) es el subconjunto δ(Z, ω). El enunciado se resuelve de forma natural con un AFN de 4 estados; el AFD equivalente se obtuvo con la construcción de subconjuntos y la minimización y tiene 6 estados.

Archivo. `automatas/lista3/lista3-193.jff` · $\Sigma = \{0, 1\}$ · $|Q| = 6$

Tabla 132. Tabla de transiciones (→ inicial, * aceptación, — sin transición).

δ	0	1	Qué recuerda del prefijo leído
→ q0	—	q1	activos del AFN: {A}
q1	q2	q3	activos del AFN: {B,C} o {A,B,C}
* q2	q4	q3	activos del AFN: {A,C,D}
* q3	q2	q3	activos del AFN: {A,B,C,D}
q4	q4	q5	activos del AFN: {C}
q5	q4	q1	activos del AFN: {A,C}

Tabla 133. Cadenas de prueba.

Cadena	Longitud	Esperado	Obtenido	Tipo de caso
`11`	2	acepta	acepta	válida
`110`	3	acepta	acepta	válida
`10101`	5	acepta	acepta	válida
λ	0	rechaza	rechaza	inválida
`1`	1	rechaza	rechaza	inválida
`000`	3	rechaza	rechaza	inválida
`10`	2	acepta	acepta	frontera: aceptada más corta
`0`	1	rechaza	rechaza	frontera: justo por debajo de la longitud mínima

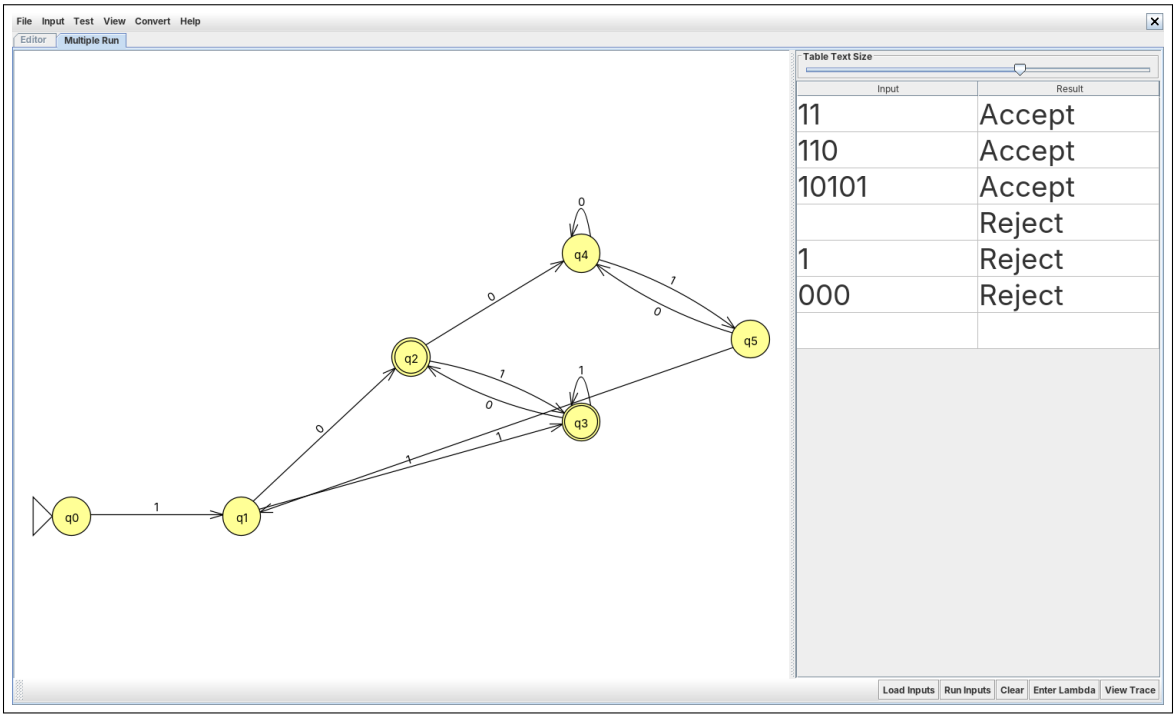


Figura 74. lista3-193 en JFLAP: Multiple Run con las tres cadenas válidas y las tres inválidas de la tabla anterior.

A.2.33. Lista 3 · Ejercicio 194

Enunciado. AFN de la tabla 194: AFD equivalente y δ(Z, ω).

Diseño. Construcción de subconjuntos (sólo alcanzables). El AFD asociado tiene un estado por subconjunto, así que δ_D(Z, ω) es el subconjunto δ(Z, ω). El enunciado se resuelve de forma natural con un AFN de 4 estados; el AFD equivalente se obtuvo con la construcción de subconjuntos y la minimización y tiene 2 estados.

Archivo. `automatas/lista3/lista3-194.jff` · $\Sigma = \{0, 1, 2\}$ · $|Q| = 2$

Tabla 134. Tabla de transiciones ($\rightarrow$ inicial, * aceptación, — sin transición).

δ	0	1	2	Qué recuerda del prefijo leído
$\rightarrow$ q0	q0	q1	q0	activos del AFN: {A} o {A,B}
* q1	q0	q1	q0	activos del AFN: {A,C} o {A,B,C,D}

Tabla 135. Cadenas de prueba.

Cadena	Longitud	Esperado	Obtenido	Tipo de caso
`1`	1	acepta	acepta	válida
`11`	2	acepta	acepta	válida
`0011`	4	acepta	acepta	válida
λ	0	rechaza	rechaza	inválida
`2`	1	rechaza	rechaza	inválida
`220`	3	rechaza	rechaza	inválida

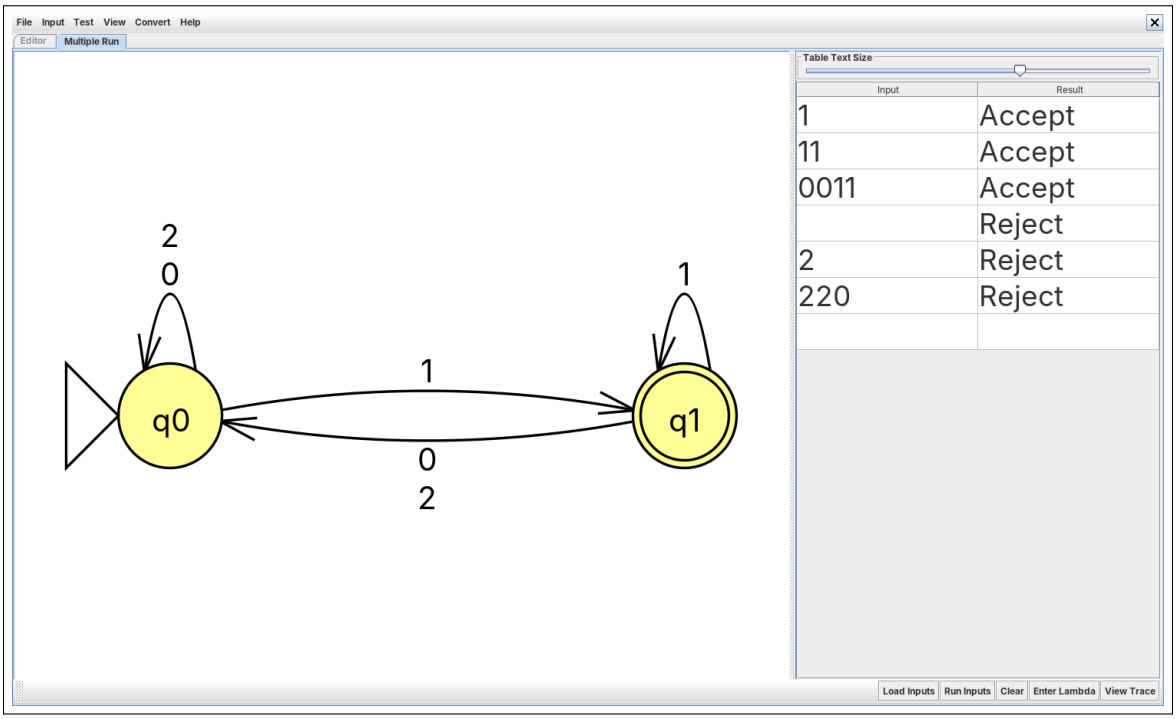


Figura 75. lista3-194 en JFLAP: Multiple Run con las tres cadenas válidas y las tres inválidas de la tabla anterior.

A.2.34. Lista 3 · Ejercicio 195

Enunciado. AFN de la tabla 195: AFD equivalente y $\delta(Z, \omega)$.

Diseño. Construcción de subconjuntos (sólo alcanzables). El AFD asociado tiene un estado por subconjunto, así que δ_D(Z, ω) es el subconjunto δ(Z, ω). El enunciado se resuelve de forma natural con un AFN de 5 estados; el AFD equivalente se obtuvo con la construcción de subconjuntos y la minimización y tiene 7 estados.

Archivo. `automatas/lista3/lista3-195.jff` · $\Sigma = \{0, 1\}$ · $|Q| = 7$

Tabla 136. Tabla de transiciones (→ inicial, * aceptación, — sin transición).

δ	0	1	Qué recuerda del prefijo leído
→ q0	q1	q0	activos del AFN: {A}
q1	q1	q2	activos del AFN: {A,B}
* q2	q1	q3	activos del AFN: {A,C}
q3	q3	q4	activos del AFN: {A,D} o {A,B,D}
* q4	q4	q5	activos del AFN: {A,E} o {A,C,E} o {A,B,E}
* q5	q3	q6	activos del AFN: {A,C,D}
* q6	q6	q6	activos del AFN: {A,D,E} o {A,B,D,E} o {A,C,D,E}

Tabla 137. Cadenas de prueba.

Cadena	Longitud	Esperado	Obtenido	Tipo de caso
`01`	2	acepta	acepta	válida
`101`	3	acepta	acepta	válida
`01110`	5	acepta	acepta	válida
λ	0	rechaza	rechaza	inválida
`1`	1	rechaza	rechaza	inválida
`011`	3	rechaza	rechaza	inválida

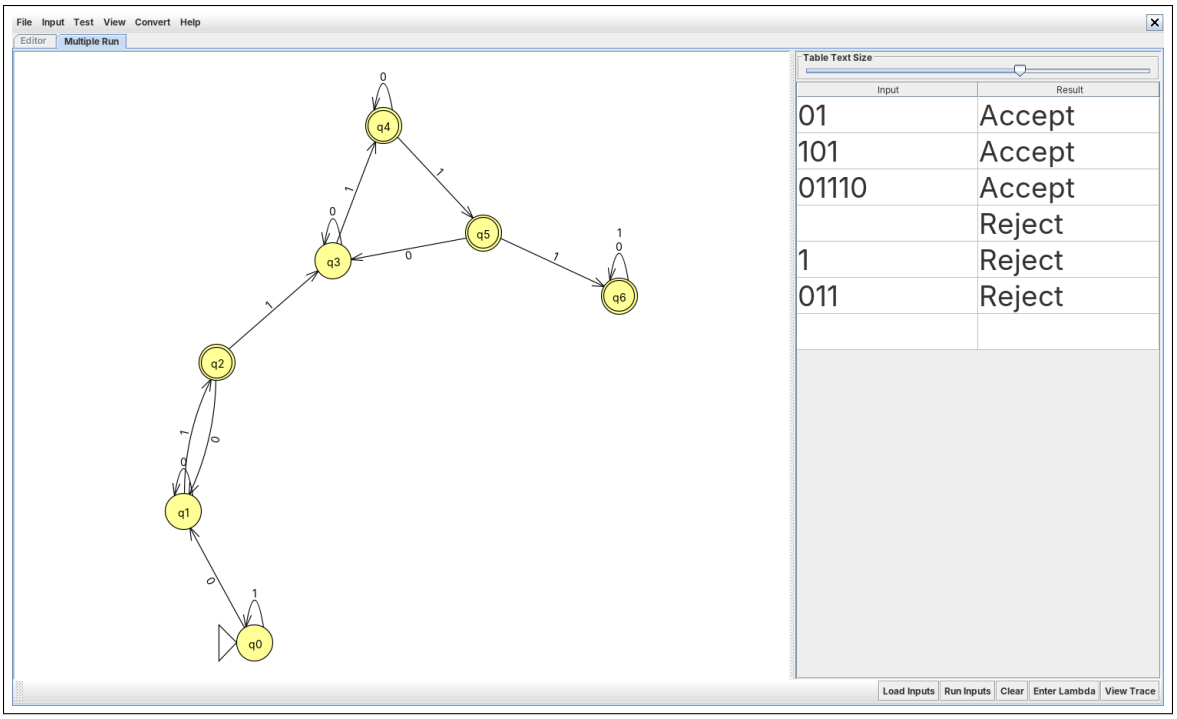


Figura 76. lista3-195 en JFLAP: Multiple Run con las tres cadenas válidas y las tres inválidas de la tabla anterior.

A.2.35. Lista 3 · Ejercicio 201-235

Enunciado. Indique si las afirmaciones 201-235 son verdaderas (V) o falsas (F).

Afirmaciones verdadero/falso sobre expresiones regulares; no piden autómata.

A.2.36. Lista 3 · Ejercicio 236

Enunciado. Expresión regular sobre $\{0, 1\}$ para: $L = \{x001y : x, y \in \Sigma^*\}$.

Diseño. La expresión se convierte en AFN-λ con la construcción de Thompson y se verifica contra la definición del lenguaje. El enunciado se resuelve de forma natural con un AFN-λ de 22 estados; el AFD equivalente se obtuvo con la construcción de subconjuntos y la minimización y tiene 4 estados.

Archivo. `automatas/lista3/lista3-236.jff` · $\Sigma = \{0, 1\}$ · $|Q| = 4$

Tabla 138. Tabla de transiciones ($\rightarrow$ inicial, * aceptación, — sin transición).

δ	0	1	Qué recuerda del prefijo leído
$\rightarrow$ q0	q1	q0	activos del AFN: {q6} o {q4,q0,q2,q3,q5,q7,q8}
q1	q2	q0	activos del AFN: {q4,q0,q1,q2,q5,q7,q8,q9,q10}
q2	q2	q3	activos del AFN: {q4,q0,q1,q2,q5,q7,q8,q9,q10,q11,q12}

δ	0	1	Qué recuerda del prefijo leído
* q3	q3	q3	activos del AFN: {q4,q0,q2,q3,q5,q7,q8,q13,q20,q18,q14,q16,q21} o {q4,q0,q1,q2,q5,q7,q8,q9,q10,q18,q14,q15,q16,q19,q21} o {q4,q0,q2,q3,q5,q7,q8,q18,q14,q16,q17,q19,q21} o {q4,q0,q1,q2,q5,q7,q8,q9,q10,q11,q12,q18,q14,q15,q16,q19,q21} o {q4,q0,q2,q3,q5,q7,q8,q13,q20,q18,q14,q16,q17,q19,q21}

Tabla 139. Cadenas de prueba.

Cadena	Longitud	Esperado	Obtenido	Tipo de caso
`001`	3	acepta	acepta	válida
`0001`	4	acepta	acepta	válida
`001010`	6	acepta	acepta	válida
λ	0	rechaza	rechaza	inválida
`0`	1	rechaza	rechaza	inválida
`100`	3	rechaza	rechaza	inválida
`00`	2	rechaza	rechaza	frontera: justo por debajo de la longitud mínima

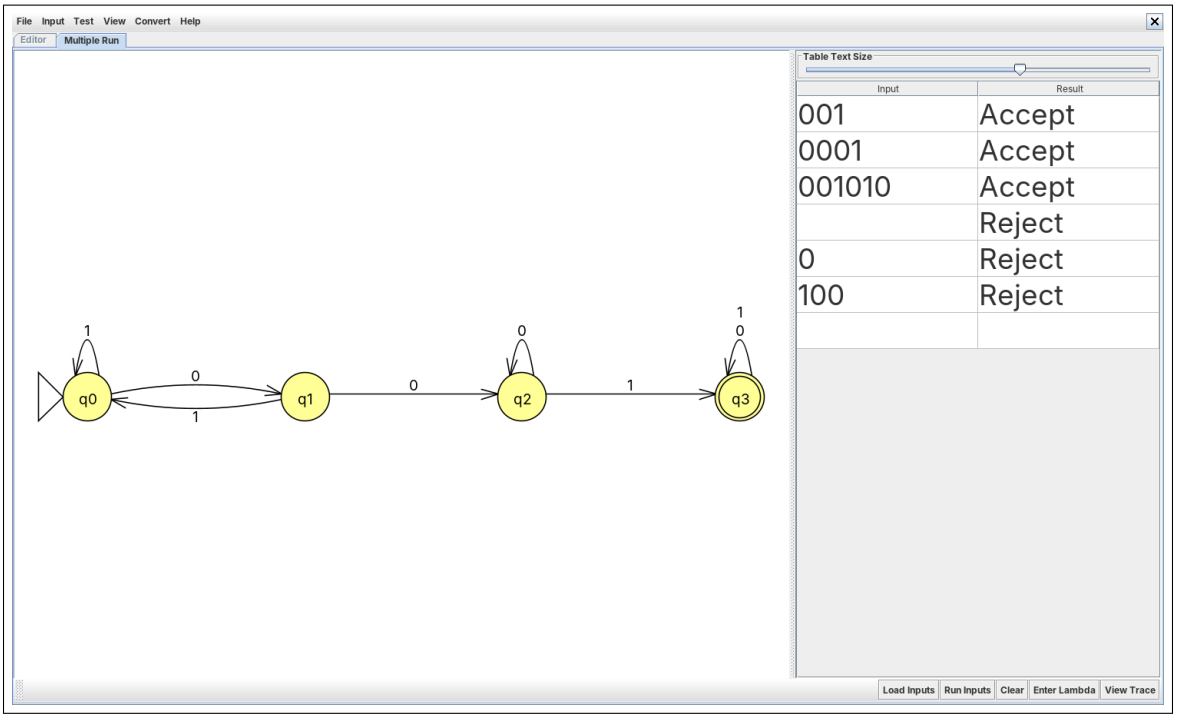


Figura 77. lista3-236 en JFLAP: Multiple Run con las tres cadenas válidas y las tres inválidas de la tabla anterior.

A.2.37. Lista 3 · Ejercicio 237

Enunciado. Expresión regular sobre {0, 1} para: L = {x010 : x ∈ Σ*}.

Diseño. La expresión se convierte en AFN-λ con la construcción de Thompson y se verifica contra la definición del lenguaje. El enunciado se resuelve de forma natural con un AFN-λ de 14 estados; el AFD equivalente se obtuvo con la construcción de subconjuntos y la minimización y tiene 4 estados.

Archivo. `automatas/lista3/lista3-237.jff` · Σ = {0, 1} · |Q| = 4

Tabla 140. Tabla de transiciones (→ inicial, * aceptación, — sin transición).

δ	0	1	Qué recuerda del prefijo leído
→ q0	q1	q0	activos del AFN: {q6} o {q4,q0,q2,q3,q5,q7,q8}
q1	q1	q2	activos del AFN: {q4,q0,q1,q2,q5,q7,q8,q9,q10}
q2	q3	q0	activos del AFN: {q4,q0,q2,q3,q5,q7,q8,q11,q12}
* q3	q1	q2	activos del AFN: {q4,q0,q1,q2,q5,q7,q8,q9,q10,q13}

Tabla 141. Cadenas de prueba.

Cadena	Longitud	Esperado	Obtenido	Tipo de caso
`010`	3	acepta	acepta	válida
`0010`	4	acepta	acepta	válida
`111010`	6	acepta	acepta	válida
λ	0	rechaza	rechaza	inválida
`0`	1	rechaza	rechaza	inválida
`110`	3	rechaza	rechaza	inválida
`11`	2	rechaza	rechaza	frontera: justo por debajo de la longitud mínima

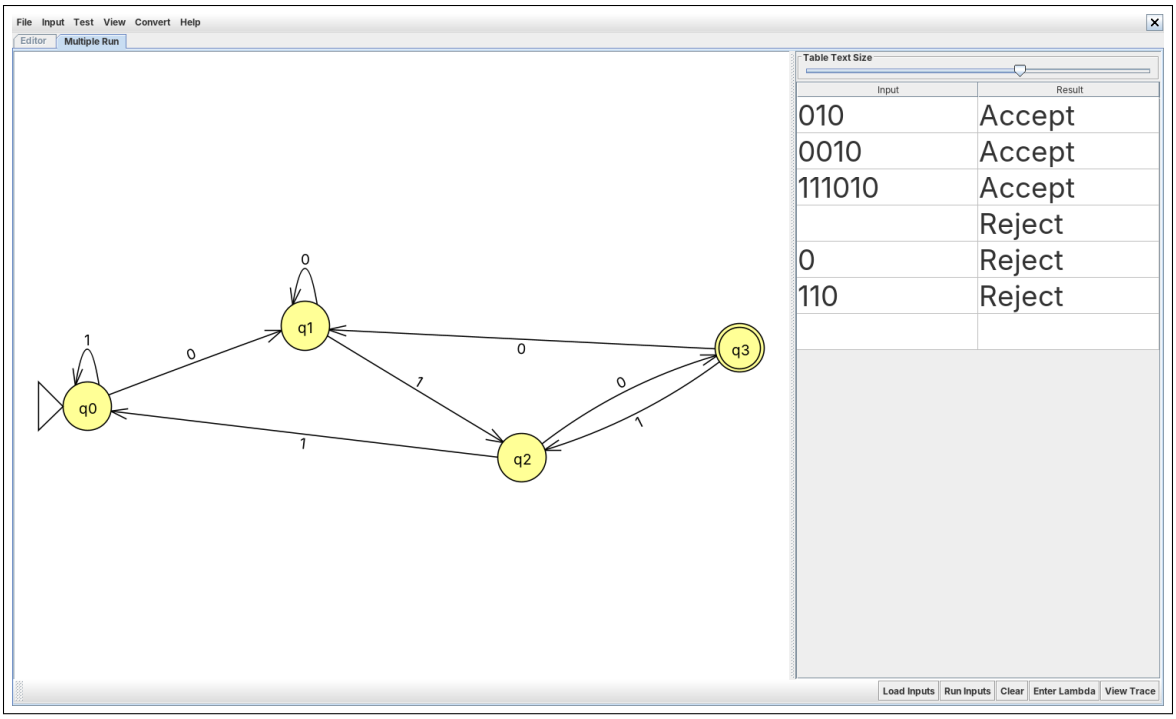


Figura 78. lista3-237 en JFLAP: Multiple Run con las tres cadenas válidas y las tres inválidas de la tabla anterior.

A.2.38. Lista 3 · Ejercicio 238

Enunciado. Expresión regular sobre $\{0, 1\}$ para: $L = \Sigma^3 \cup \Sigma^5$.

Diseño. La expresión se convierte en AFN-λ con la construcción de Thompson y se verifica contra la definición del lenguaje. El enunciado se resuelve de forma natural con un AFN-λ de 34 estados; el AFD equivalente se obtuvo con la construcción de subconjuntos y la minimización y tiene 6 estados.

Archivo. `automatas/lista3/lista3-238.jff` · $\Sigma = \{0, 1\}$ · $|Q| = 6$

Tabla 142. Tabla de transiciones ($\rightarrow$ inicial, * aceptación, — sin transición).

δ	0	1	Qué recuerda del prefijo leído
$\rightarrow$ q0	q1	q1	activos del AFN: {q4}
q1	q2	q2	activos del AFN: {q1,q5,q10,q6,q8} o {q3,q5,q10,q6,q8}
q2	q3	q3	activos del AFN: {q7,q11,q16,q12,q14} o {q9,q11,q16,q12,q14}
* q3	q4	q4	activos del AFN: {q13,q17,q32,q18,q19,q24,q20,q22,q33} o {q15,q17,q32,q18,q19,q24,q20,q22,q33}
q4	q5	q5	activos del AFN: {q21,q25,q30,q26,q28} o {q23,q25,q30,q26,q28}
* q5	—	—	activos del AFN: {q27,q31,q33} o {q29,q31,q33}

Tabla 143. Cadenas de prueba.

Cadena	Longitud	Esperado	Obtenido	Tipo de caso
`000`	3	acepta	acepta	válida
`001`	3	acepta	acepta	válida
`101`	3	acepta	acepta	válida
λ	0	rechaza	rechaza	inválida
`0`	1	rechaza	rechaza	inválida
`0100`	4	rechaza	rechaza	inválida
`00`	2	rechaza	rechaza	frontera: justo por debajo de la longitud mínima

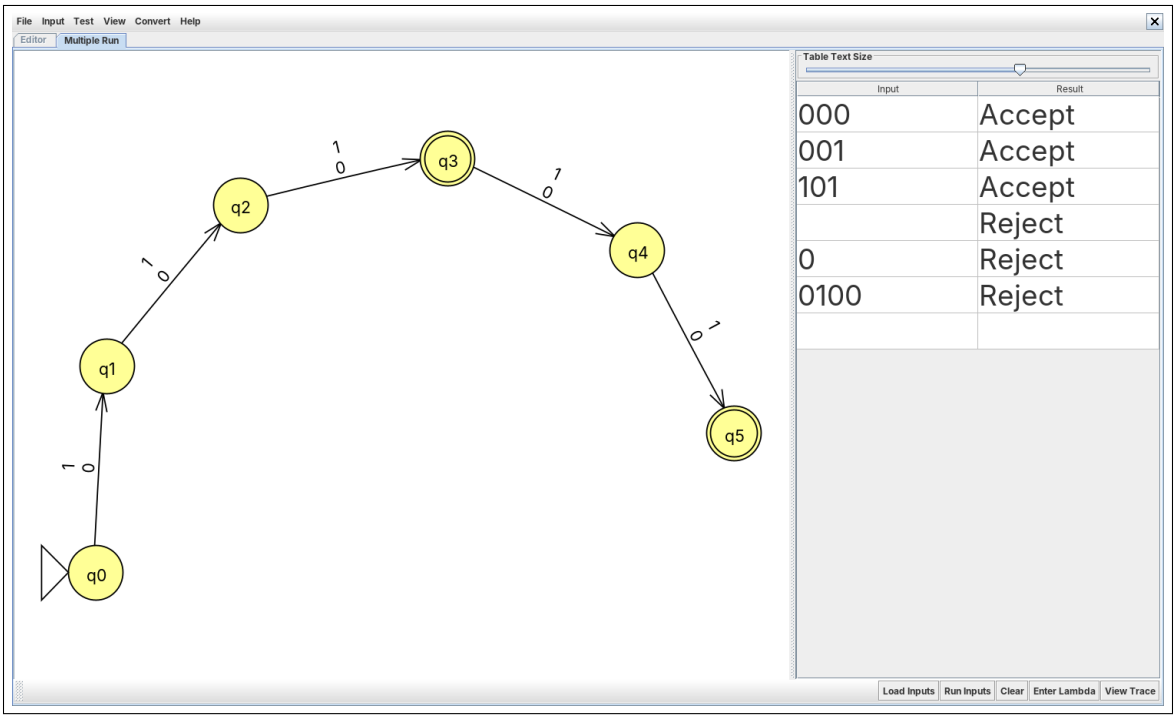


Figura 79. lista3-238 en JFLAP: Multiple Run con las tres cadenas válidas y las tres inválidas de la tabla anterior.

A.2.39. Lista 3 · Ejercicio 239

Enunciado. Expresión regular sobre $\{0, 1\}$ para: Cadenas que comienzan con 01 o terminan con 10.

Diseño. La expresión se convierte en AFN-λ con la construcción de Thompson y se verifica contra la definición del lenguaje. El enunciado se resuelve de forma natural con un AFN-λ de 26 estados; el AFD equivalente se obtuvo con la construcción de subconjuntos y la minimización y tiene 6 estados.

Archivo. `automatas/lista3/lista3-239.jff` · $\Sigma = \{0, 1\}$ · $|Q| = 6$

Tabla 144. Tabla de transiciones (→ inicial, * aceptación, — sin transición).

δ	0	1	Qué recuerda del prefijo leído
→ q0	q1	q2	activos del AFN: {q24}
q1	q3	q4	activos del AFN: {q1,q2,q16,q12,q13,q14,q17,q19,q20}
q2	q5	q2	activos del AFN: {q16,q12,q14,q15,q17,q19,q20,q21,q22}
q3	q3	q2	activos del AFN: {q16,q12,q13,q14,q17,q19,q20}
* q4	q4	q4	activos del AFN: {q3,q10,q8,q4,q6,q11,q16,q12,q14,q15,q17,q19,q20,q21,q22,q25} o {q8,q4,q5,q6,q9,q11,q16,q12,q13,q14,q17,q19,q20,q23,q25} o {q8,q4,q6,q7,q9,q11,q16,q12,q14,q15,q17,q19,q20,q21,q22,q25} o {q8,q4,q5,q6,q9,q11,q16,q12,q13,q14,q17,q19,q20,q25}
* q5	q3	q2	activos del AFN: {q16,q12,q13,q14,q17,q19,q20,q23,q25}

Tabla 145. Cadenas de prueba.

Cadena	Longitud	Esperado	Obtenido	Tipo de caso
`01`	2	acepta	acepta	válida
`010`	3	acepta	acepta	válida
`10010`	5	acepta	acepta	válida
λ	0	rechaza	rechaza	inválida
`0`	1	rechaza	rechaza	inválida
`001`	3	rechaza	rechaza	inválida

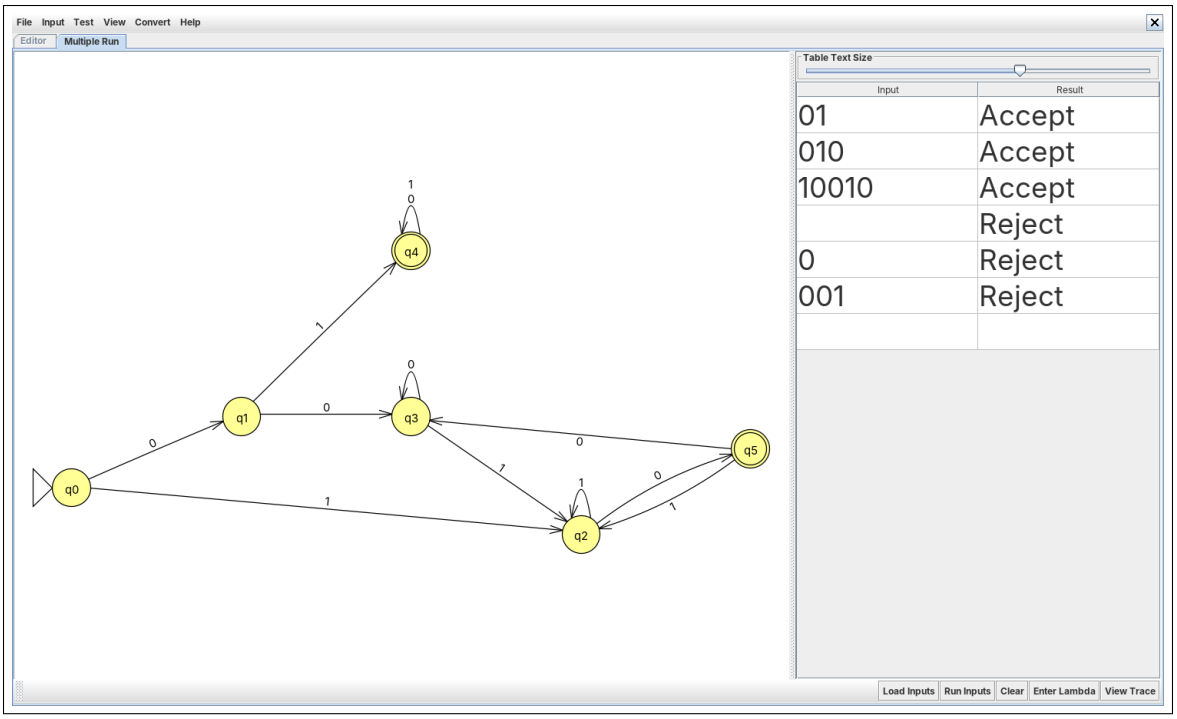


Figura 80. lista3-239 en JFLAP: Multiple Run con las tres cadenas válidas y las tres inválidas de la tabla anterior.

A.2.40. Lista 3 · Ejercicio 240

Enunciado. Expresión regular sobre {0, 1} para: Cadenas de longitud par que contienen 01.

Diseño. La expresión se convierte en AFN-λ con la construcción de Thompson y se verifica contra la definición del lenguaje. El enunciado se resuelve de forma natural con un AFN-λ de 78 estados; el AFD equivalente se obtuvo con la construcción de subconjuntos y la minimización y tiene 6 estados.

Archivo. `automatas/lista3/lista3-240.jff` · $\Sigma = \{0, 1\}$ · $|Q| = 6$

Tabla 146. Tabla de transiciones ($\rightarrow$ inicial, * aceptación, — sin transición).

δ	0	1	Qué recuerda del prefijo leído
$\rightarrow$ q0	q1	q2	activos del AFN: {q76} o {q4,q0,q2,q9,q11,q13,q14,q41,q43,q48,q44,q46}
q1	q3	q4	activos del AFN: {q1,q5,q10,q6,q8,q15,q16,q33,q37,q50,q42,q38,q40,q51,q52} o {q1,q5,q10,q6,q8,q15,q16,q42,q38,q40,q45,q49,q51,q52}
q2	q3	q0	activos del AFN: {q3,q5,q10,q6,q8,q35,q37,q50,q42,q38,q40,q51,q52} o {q3,q5,q10,q6,q8,q42,q38,q40,q47,q49,q51,q52}
q3	q1	q5	activos del AFN: {q4,q0,q2,q7,q11,q13,q14,q39,q43,q48,q44,q46,q53,q54}

δ	0	1	Qué recuerda del prefijo leído
* q4	q5	q5	activos del AFN: {q4,q0,q2,q9,q11,q13,q14,q17,q30,q22,q18,q20,q31,q41,q43,q48,q4 o {q4,q0,q2,q7,q11,q13,q14,q39,q43,q48,q44,q46,q53,q54,q57,q61,q6 o {q4,q0,q2,q9,q11,q13,q14,q41,q43,q48,q44,q46,q59,q61,q66,q62,q6 o {q4,q0,q2,q7,q11,q13,q14,q22,q18,q20,q25,q29,q31,q39,q43,q48,q4 o {q4,q0,q2,q9,q11,q13,q14,q17,q30,q22,q18,q20,q27,q29,q31,q41,q4 o {q4,q0,q2,q9,q11,q13,q14,q22,q18,q20,q27,q29,q31,q41,q43,q48,q4 o {q4,q0,q2,q9,q11,q13,q14,q17,q30,q22,q18,q20,q31,q41,q43,q48,q4 o {q4,q0,q2,q7,q11,q13,q14,q22,q18,q20,q25,q29,q31,q39,q43,q48,q4 o {q4,q0,q2,q9,q11,q13,q14,q22,q18,q20,q27,q29,q31,q41,q43,q48,q4 o {q4,q0,q2,q9,q11,q13,q14,q17,q30,q22,q18,q20,q27,q29,q31,q41,q4
q5	q4	q4	activos del AFN: {q3,q5,q10,q6,q8,q42,q38,q40,q47,q49,q51,q52,q55,q68,q60,q56,q5 o {q1,q5,q10,q6,q8,q15,q16,q19,q23,q28,q24,q26,q42,q38,q40,q45,q4 o {q3,q5,q10,q6,q8,q21,q23,q28,q24,q26,q42,q38,q40,q47,q49,q51,q5 o {q1,q5,q10,q6,q8,q15,q16,q42,q38,q40,q45,q49,q51,q52,q60,q56,q5 o {q3,q5,q10,q6,q8,q42,q38,q40,q47,q49,q51,q52,q55,q68,q60,q56,q5 o {q3,q5,q10,q6,q8,q42,q38,q40,q47,q49,q51,q52,q60,q56,q58,q65,q6 o {q3,q5,q10,q6,q8,q21,q23,q28,q24,q26,q42,q38,q40,q47,q49,q51,q5 o {q1,q5,q10,q6,q8,q15,q16,q19,q23,q28,q24,q26,q42,q38,q40,q45,q4 o {q3,q5,q10,q6,q8,q21,q23,q28,q24,q26,q42,q38,q40,q47,q49,q51,q5 o {q3,q5,q10,q6,q8,q21,q23,q28,q24,q26,q42,q38,q40,q47,q49,q51,q5

Tabla 147. Cadenas de prueba.

Cadena	Longitud	Esperado	Obtenido	Tipo de caso
`01`	2	acepta	acepta	válida
`100010`	6	acepta	acepta	válida
`00001001`	8	acepta	acepta	válida
λ	0	rechaza	rechaza	inválida
`1`	1	rechaza	rechaza	inválida
`001`	3	rechaza	rechaza	inválida

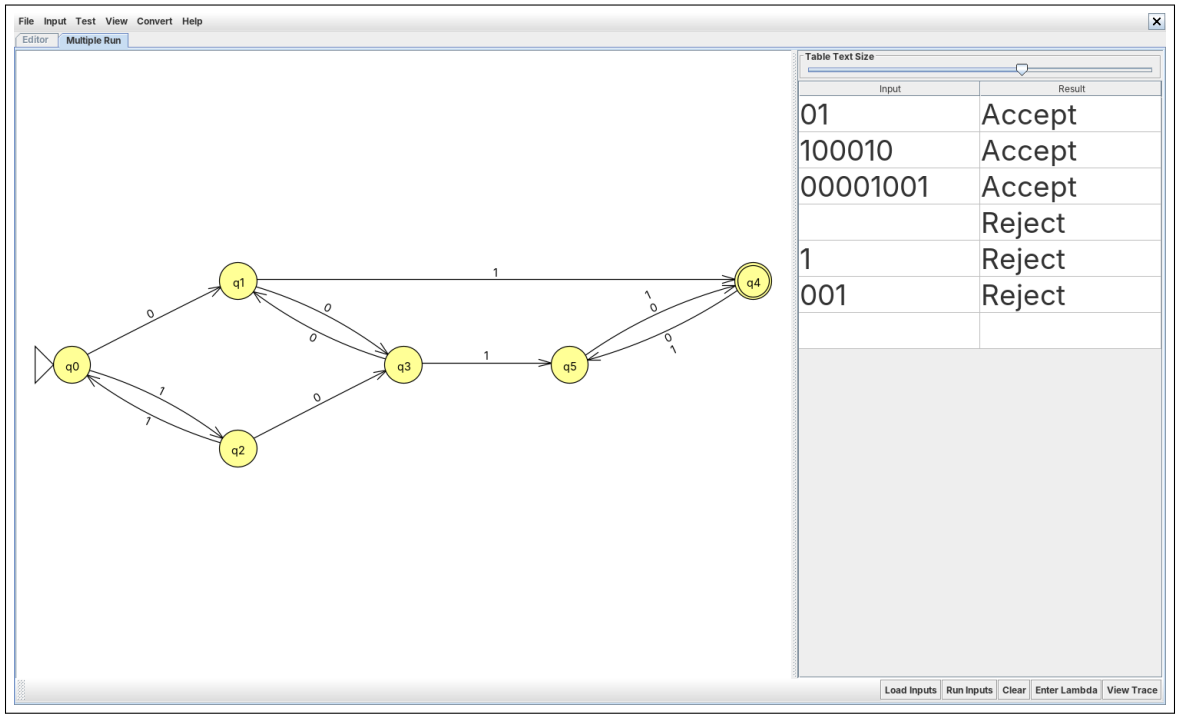


Figura 81. lista3-240 en JFLAP: Multiple Run con las tres cadenas válidas y las tres inválidas de la tabla anterior.

A.2.41. Lista 3 · Ejercicio 254

Enunciado. Expresión regular para el autómata de la tabla 254.

Diseño. Eliminación de estados sobre el autómata generalizado (inicial S y final F nuevos), en el orden de Q.

Archivo. `automatas/lista3/lista3-254.jff` · $\Sigma = \{0, 1\}$ · $|Q| = 2$

Tabla 148. Tabla de transiciones ($\rightarrow$ inicial, * aceptación, — sin transición).

δ	0	1
$\rightarrow$ A	A	B

δ	0	1
* B	A	B

Tabla 149. Cadenas de prueba.

Cadena	Longitud	Esperado	Obtenido	Tipo de caso
`1`	1	acepta	acepta	válida
`01`	2	acepta	acepta	válida
`0001`	4	acepta	acepta	válida
λ	0	rechaza	rechaza	inválida
`0`	1	rechaza	rechaza	inválida
`010`	3	rechaza	rechaza	inválida

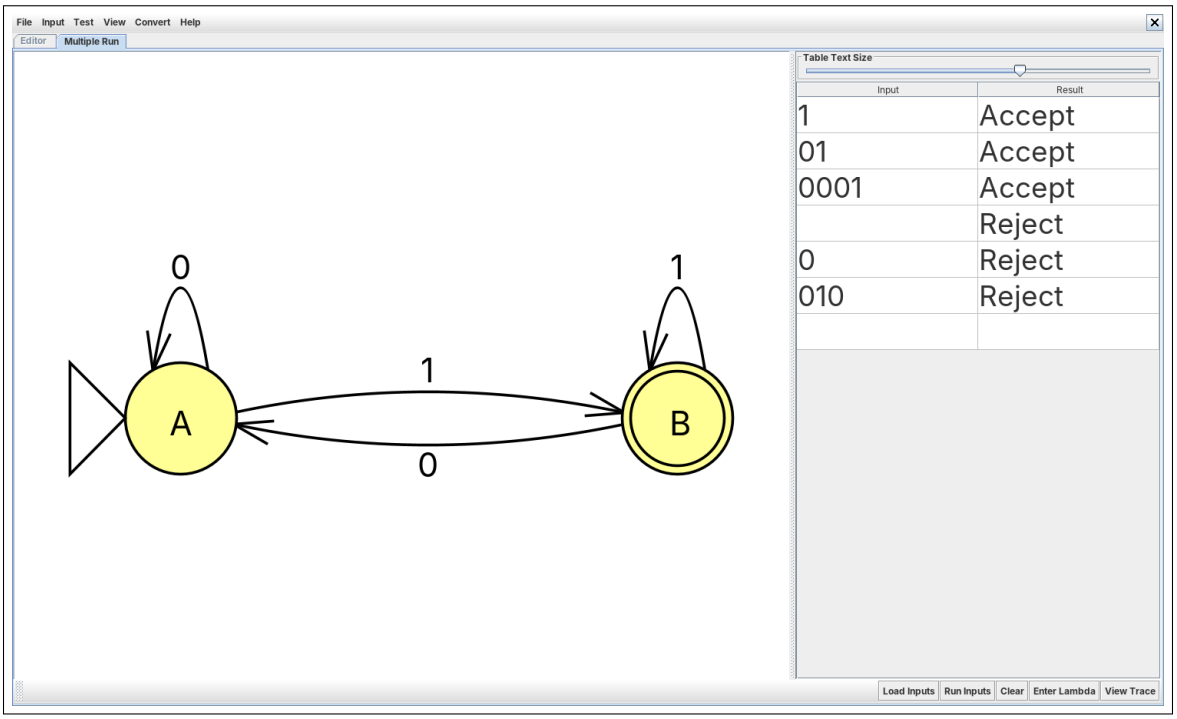


Figura 82. lista3-254 en JFLAP: Multiple Run con las tres cadenas válidas y las tres inválidas de la tabla anterior.

A.2.42. Lista 3 · Ejercicio 255

Enunciado. Expresión regular para el autómata de la tabla 255.

Diseño. Eliminación de estados sobre el autómata generalizado (inicial S y final F nuevos), en el orden de Q.

Archivo. `automatas/lista3/lista3-255.jff` · $\Sigma = \{0, 1\}$ · $|Q| = 2$

Tabla 150. Tabla de transiciones (→ inicial, * aceptación, — sin transición).

δ	0	1
→* A	B	B
B	A	B

Tabla 151. Cadenas de prueba.

Cadena	Longitud	Esperado	Obtenido	Tipo de caso
λ	0	acepta	acepta	válida
`00`	2	acepta	acepta	válida
`1111010`	7	acepta	acepta	válida
`1`	1	rechaza	rechaza	inválida
`01`	2	rechaza	rechaza	inválida
`1101`	4	rechaza	rechaza	inválida

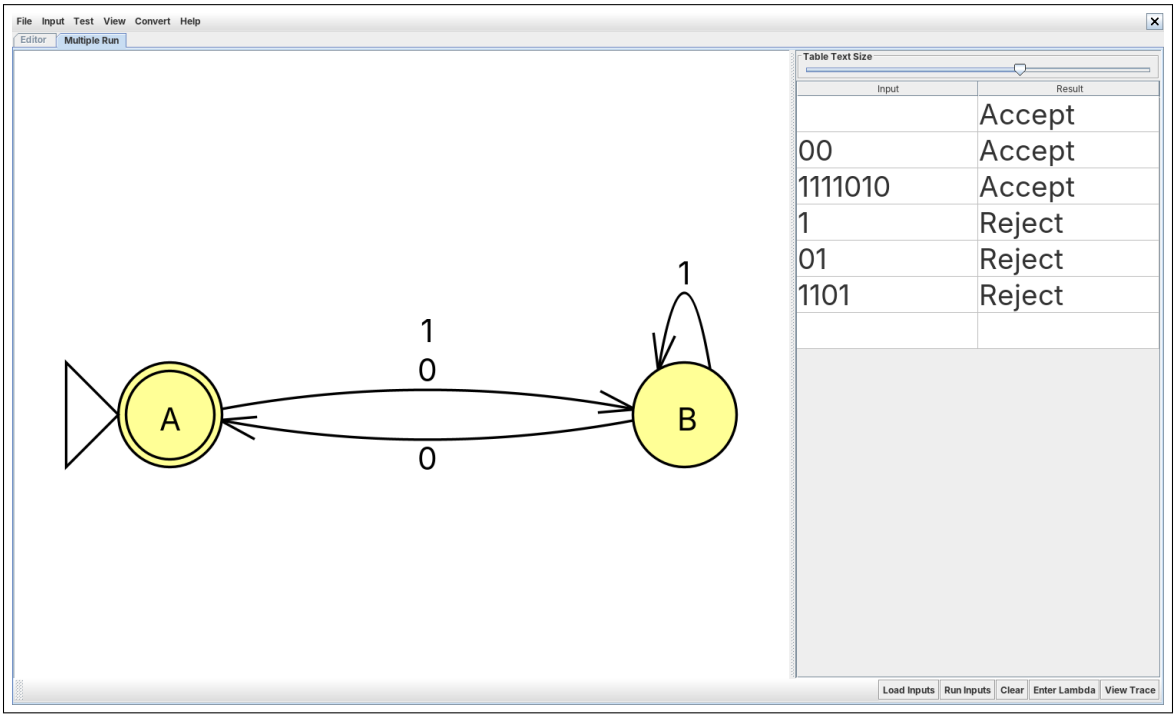


Figura 83. lista3-255 en JFLAP: Multiple Run con las tres cadenas válidas y las tres inválidas de la tabla anterior.

A.2.43. Lista 3 · Ejercicio 256

Enunciado. Expresión regular para el autómata de la tabla 256.

Diseño. Eliminación de estados sobre el autómata generalizado (inicial S y final F nuevos), en el orden de Q.

Archivo. `automatas/lista3/lista3-256.jff` · $\Sigma = \{0, 1, 2\}$ · $|Q| = 2$

Tabla 152. Tabla de transiciones ($\rightarrow$ inicial, * aceptación, — sin transición).

δ	0	1	2
$\rightarrow$ A	B	A	A
* B	B	B	A

Tabla 153. Cadenas de prueba.

Cadena	Longitud	Esperado	Obtenido	Tipo de caso
`0`	1	acepta	acepta	válida
`01`	2	acepta	acepta	válida
`0000`	4	acepta	acepta	válida
λ	0	rechaza	rechaza	inválida
`2`	1	rechaza	rechaza	inválida
`202`	3	rechaza	rechaza	inválida

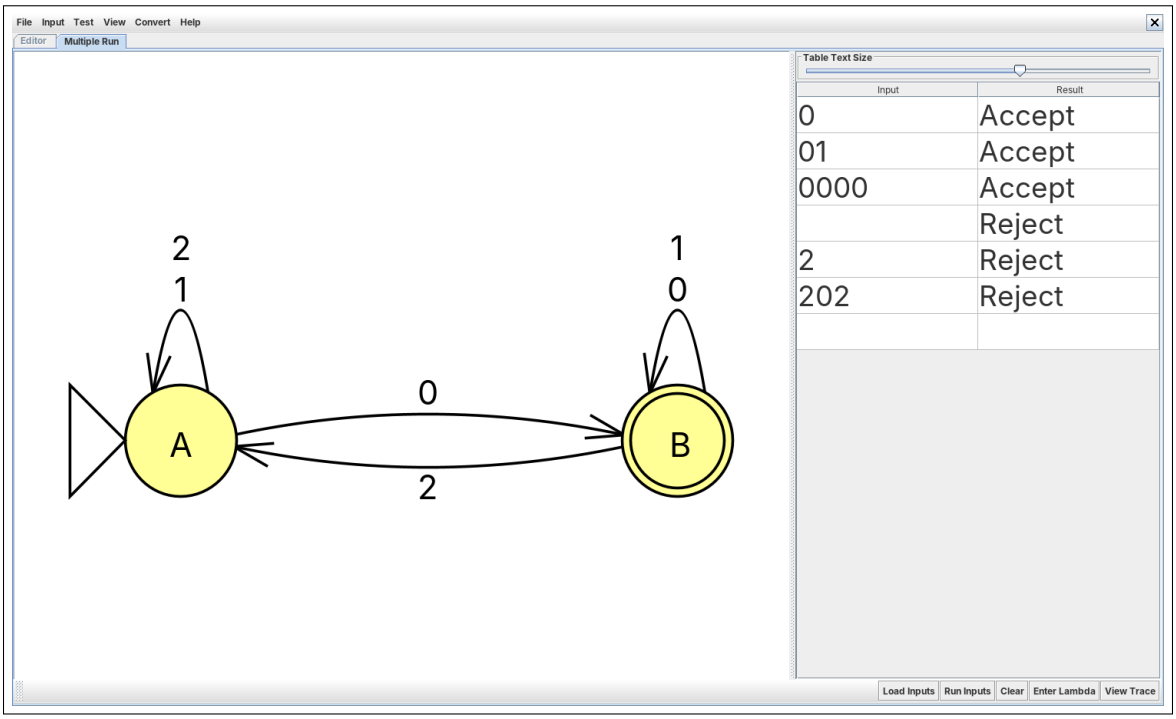


Figura 84. lista3-256 en JFLAP: Multiple Run con las tres cadenas válidas y las tres inválidas de la tabla anterior.

A.2.44. Lista 3 · Ejercicio 257

Enunciado. Expresión regular para el autómata de la tabla 257.

Diseño. Eliminación de estados sobre el autómata generalizado (inicial S y final F nuevos), en el orden de Q.

Archivo. `automatas/lista3/lista3-257.jff` · $\Sigma = \{0, 1\}$ · $|Q| = 3$

Tabla 154. Tabla de transiciones ($\rightarrow$ inicial, * aceptación, — sin transición).

δ	0	1
$\rightarrow$ A	B	C
* B	C	B
* C	A	A

Tabla 155. Cadenas de prueba.

Cadena	Longitud	Esperado	Obtenido	Tipo de caso
`0`	1	acepta	acepta	válida
`00`	2	acepta	acepta	válida
`1101`	4	acepta	acepta	válida
λ	0	rechaza	rechaza	inválida
`001`	3	rechaza	rechaza	inválida
`00010001`	8	rechaza	rechaza	inválida

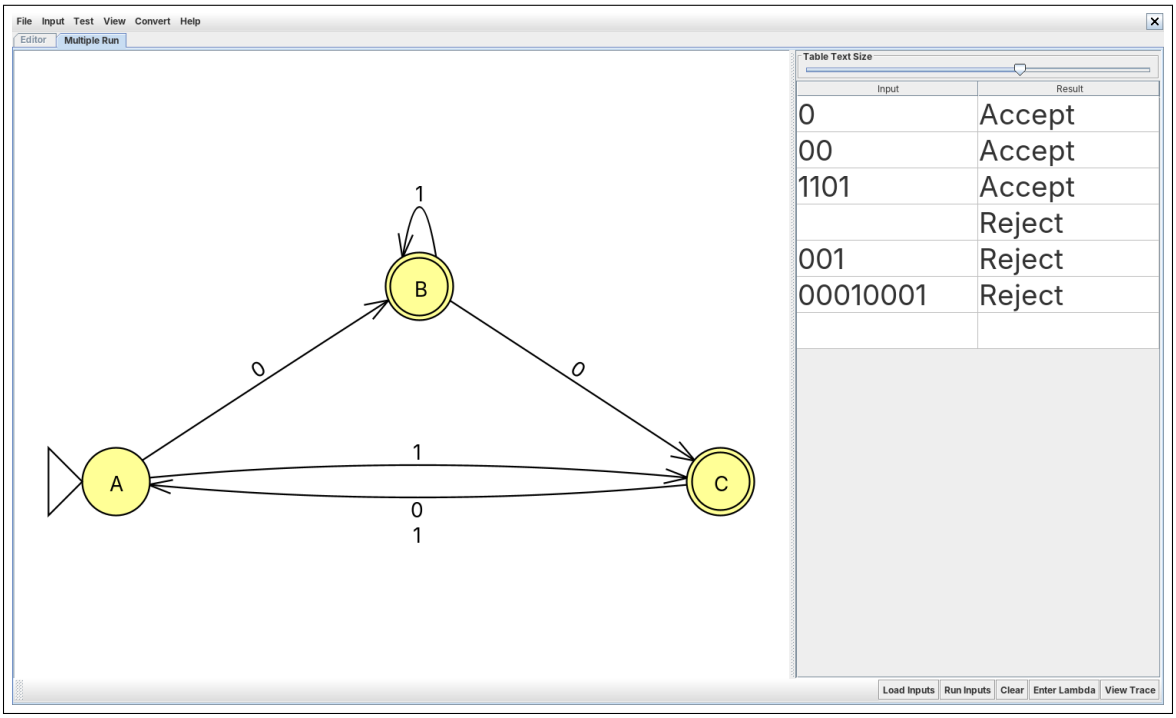


Figura 85. lista3-257 en JFLAP: Multiple Run con las tres cadenas válidas y las tres inválidas de la tabla anterior.

A.2.45. Lista 3 · Ejercicio 258

Enunciado. Expresión regular para el autómata de la tabla 258.

Diseño. Eliminación de estados sobre el autómata generalizado (inicial S y final F nuevos), en el orden de Q.

Archivo. `automatas/lista3/lista3-258.jff` · $\Sigma = \{0, 1\}$ · $|Q| = 3$

Tabla 156. Tabla de transiciones ($\rightarrow$ inicial, * aceptación, — sin transición).

δ	0	1
$\rightarrow$ A	C	C
B	A	C
* C	B	A

Tabla 157. Cadenas de prueba.

Cadena	Longitud	Esperado	Obtenido	Tipo de caso
`1`	1	acepta	acepta	válida
`01100011`	8	acepta	acepta	válida
`1110100010`	10	acepta	acepta	válida
λ	0	rechaza	rechaza	inválida
`0000101`	7	rechaza	rechaza	inválida
`00000100`	8	rechaza	rechaza	inválida

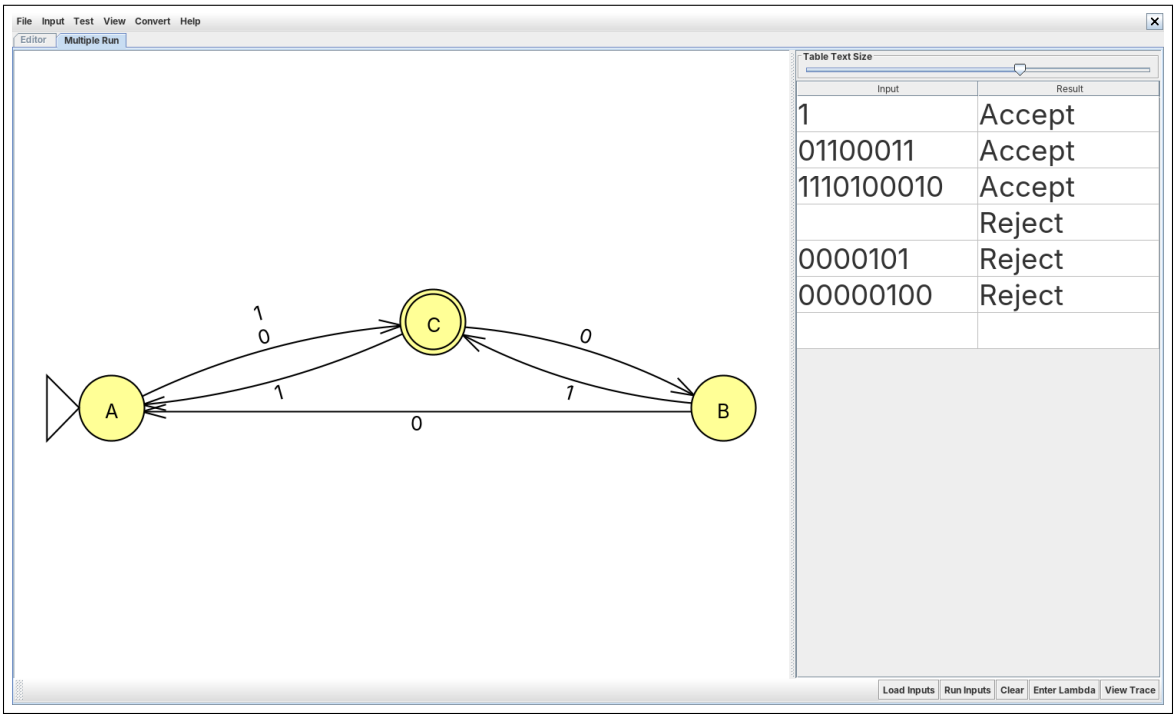


Figura 86. lista3-258 en JFLAP: Multiple Run con las tres cadenas válidas y las tres inválidas de la tabla anterior.

A.2.46. Lista 3 · Ejercicio 260

Enunciado. AFD equivalente al AFN de la tabla 260; describa cómo se obtuvo.

Diseño. Construcción de subconjuntos: se parte de {q0} y, para cada subconjunto nuevo y cada símbolo, se reúnen los destinos; sólo se crean subconjuntos alcanzables. El enunciado se resuelve de forma natural con un AFN de 5 estados; el AFD equivalente se obtuvo con la construcción de subconjuntos y la minimización y tiene 6 estados.

Archivo. `automatas/lista3/lista3-260.jff` · $\Sigma = \{0, 1\}$ · $|Q| = 6$

Tabla 158. Tabla de transiciones ($\rightarrow$ inicial, * aceptación, — sin transición).

δ	0	1	Qué recuerda del prefijo leído
$\rightarrow$ q0	—	q1	activos del AFN: {A}
q1	q2	q3	activos del AFN: {B,C}
* q2	q2	q2	activos del AFN: {A,B,C,D} o {A,B,C,D,E}
* q3	q4	q2	activos del AFN: {A,C,D,E}
q4	q4	q5	activos del AFN: {A,C,E}
q5	q2	q2	activos del AFN: {A,B,C,E}

Tabla 159. Cadenas de prueba.

Cadena	Longitud	Esperado	Obtenido	Tipo de caso
`10`	2	acepta	acepta	válida
`101`	3	acepta	acepta	válida
`10011`	5	acepta	acepta	válida
λ	0	rechaza	rechaza	inválida
`1`	1	rechaza	rechaza	inválida
`011`	3	rechaza	rechaza	inválida

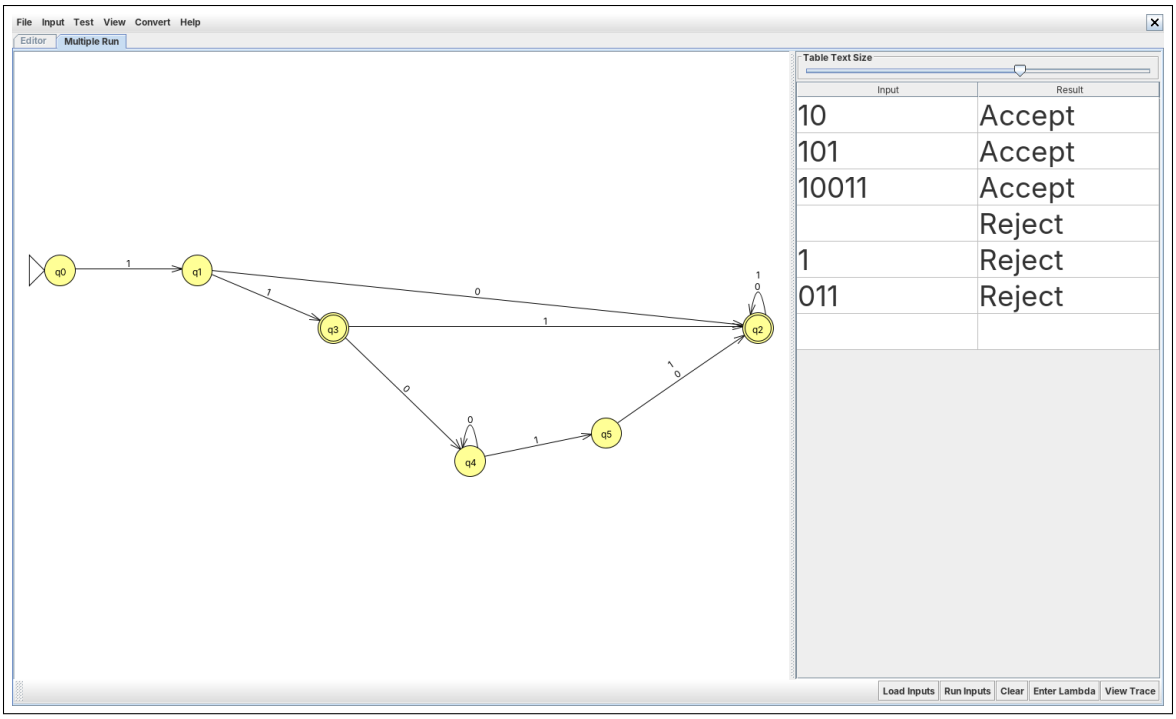


Figura 87. lista3-260 en JFLAP: Multiple Run con las tres cadenas válidas y las tres inválidas de la tabla anterior.

A.2.47. Lista 3 · Ejercicio 261

Enunciado. AFD equivalente al AFN de la figura 261 (transcrito).

Diseño. Construcción de subconjuntos: se parte de {q0} y, para cada subconjunto nuevo y cada símbolo, se reúnen los destinos; sólo se crean subconjuntos alcanzables. El enunciado se resuelve de forma natural con un AFN de 5 estados; el AFD equivalente se obtuvo con la construcción de subconjuntos y la minimización y tiene 3 estados.

Archivo. `automatas/lista3/lista3-261.jff` · $\Sigma = \{0, 1\}$ · $|Q| = 3$

Tabla 160. Tabla de transiciones ($\rightarrow$ inicial, * aceptación, — sin transición).

δ	0	1	Qué recuerda del prefijo leído
$\rightarrow$ q0	q1	q0	activos del AFN: {q0} o {q0,q3} o {q0,q3,q4}
q1	q2	q0	activos del AFN: {q0,q1}
* q2	q2	q0	activos del AFN: {q0,q1,q2}

Tabla 161. Cadenas de prueba.

Cadena	Longitud	Esperado	Obtenido	Tipo de caso
`00`	2	acepta	acepta	válida
`000`	3	acepta	acepta	válida

Cadena	Longitud	Esperado	Obtenido	Tipo de caso
`01000`	5	acepta	acepta	válida
λ	0	rechaza	rechaza	inválida
`0`	1	rechaza	rechaza	inválida
`010`	3	rechaza	rechaza	inválida

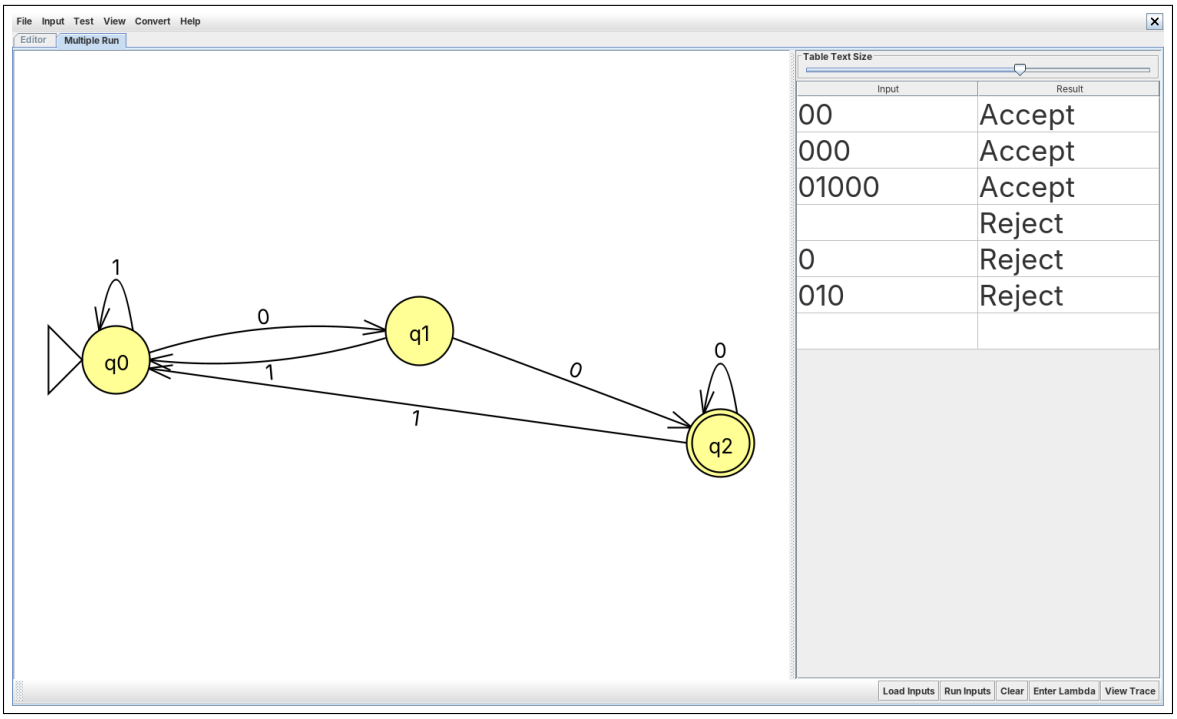


Figura 88. lista3-261 en JFLAP: Multiple Run con las tres cadenas válidas y las tres inválidas de la tabla anterior.

A.2.48. Lista 3 · Ejercicio 262

Enunciado. Diseñe un AFN que acepte {ab, ba} y obtenga un AFD.

Diseño. Construcción de subconjuntos: se parte de {q0} y, para cada subconjunto nuevo y cada símbolo, se reúnen los destinos; sólo se crean subconjuntos alcanzables. El AFN tiene dos ramas desde q0, una por palabra.

Archivo. `automatas/lista3/lista3-262.jff` · $\Sigma = \{a, b\}$ · $|Q| = 5$

Tabla 162. Tabla de transiciones ($\rightarrow$ inicial, * aceptación, — sin transición).

δ	a	b
$\rightarrow$ q0	q1	q3
q1	—	q2
* q2	—	—
q3	q4	—

δ	a	b
* q4	—	—

Tabla 163. Cadenas de prueba.

Cadena	Longitud	Esperado	Obtenido	Tipo de caso
`ab`	2	acepta	acepta	válida
`ba`	2	acepta	acepta	válida
λ	0	rechaza	rechaza	inválida
`a`	1	rechaza	rechaza	inválida
`bba`	3	rechaza	rechaza	inválida
`b`	1	rechaza	rechaza	frontera: justo por debajo de la longitud mínima

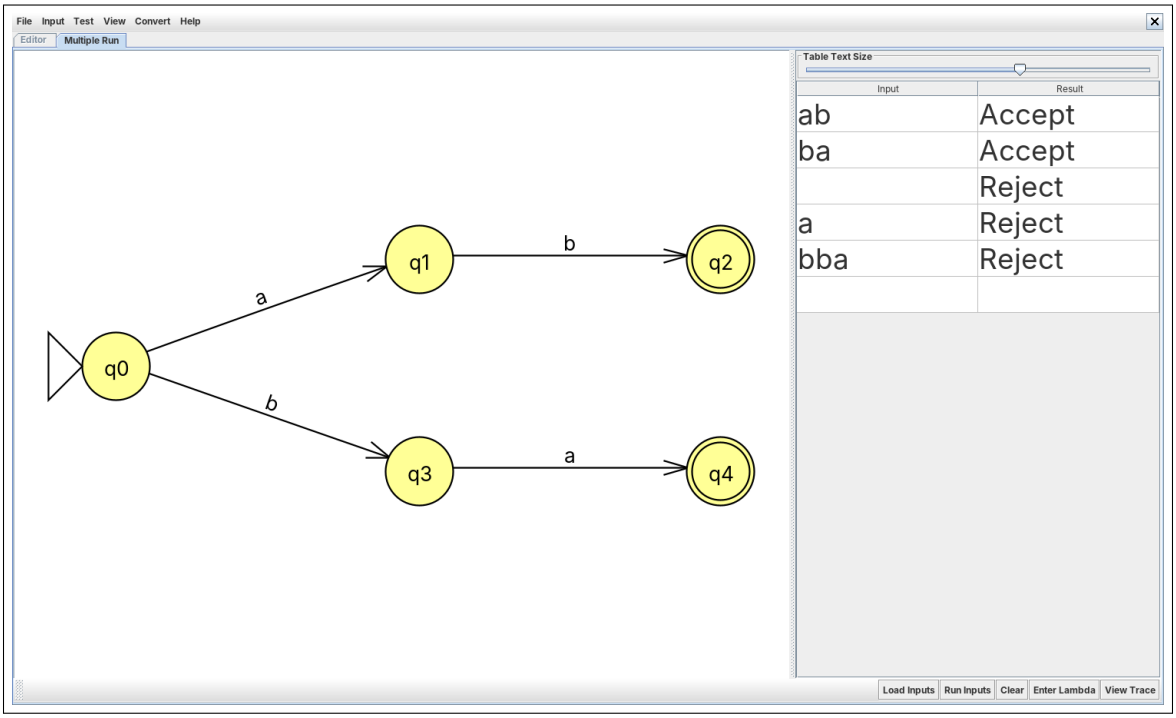


Figura 89. lista3-262 en JFLAP: Multiple Run con las tres cadenas válidas y las tres inválidas de la tabla anterior.

A.2.49. Lista 3 · Ejercicio 263

Enunciado. AFD equivalente al AFN de la figura 263 (transcrito).

Diseño. Construcción de subconjuntos: se parte de {q0} y, para cada subconjunto nuevo y cada símbolo, se reúnen los destinos; sólo se crean subconjuntos alcanzables. El enunciado se resuelve de forma natural con

un AFN de 3 estados; el AFD equivalente se obtuvo con la construcción de subconjuntos y la minimización y tiene 3 estados.

Archivo. `automatas/lista3/lista3-263.jff` · $\Sigma = \{0, 1\}$ · $|Q| = 3$

Tabla 164. Tabla de transiciones ($\rightarrow$ inicial, * aceptación, — sin transición).

δ	0	1	Qué recuerda del prefijo leído
$\rightarrow$ q0	q1	q0	activos del AFN: {q0}
q1	q1	q2	activos del AFN: {q0,q1}
* q2	q1	q0	activos del AFN: {q0,q2}

Tabla 165. Cadenas de prueba.

Cadena	Longitud	Esperado	Obtenido	Tipo de caso
`01`	2	acepta	acepta	válida
`001`	3	acepta	acepta	válida
`00001`	5	acepta	acepta	válida
λ	0	rechaza	rechaza	inválida
`0`	1	rechaza	rechaza	inválida
`100`	3	rechaza	rechaza	inválida

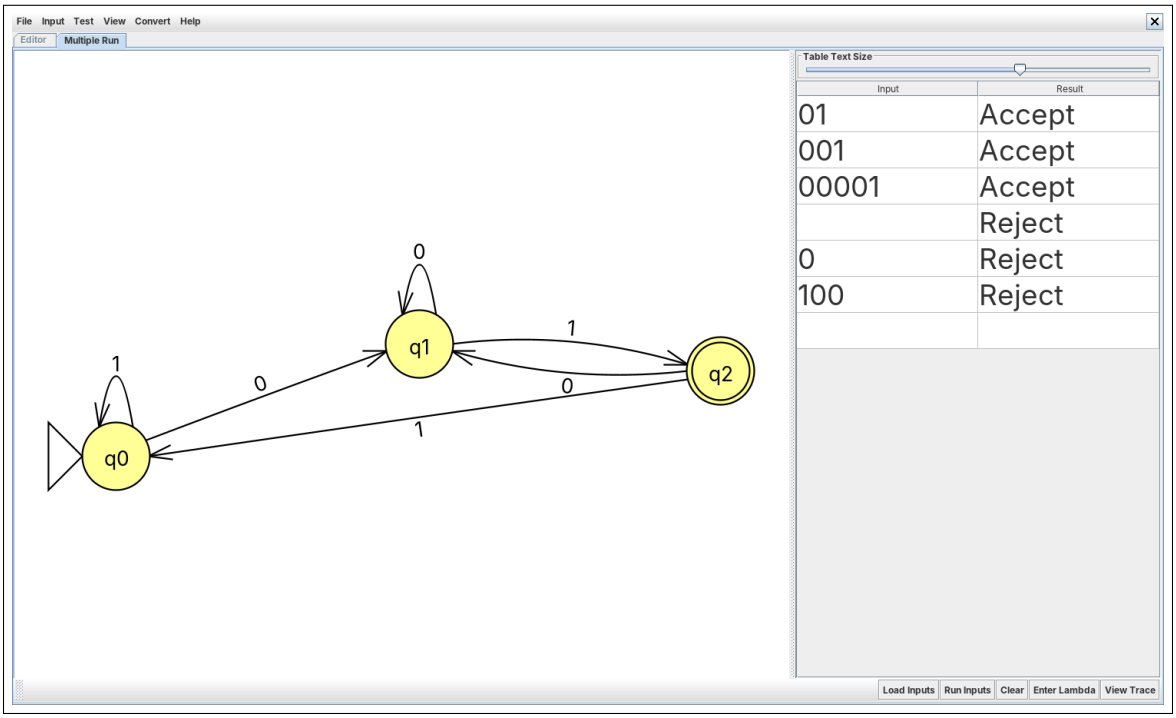


Figura 90. lista3-263 en JFLAP: Multiple Run con las tres cadenas válidas y las tres inválidas de la tabla anterior.